

Lecture Notes on the Physics Foundations of AI: Part I

Chen Lan¹ and Huifang Geng²

Department of Physics, Yantai University, 30 Qingquan Road, Yantai 264005, China

Abstract

These lecture notes introduce the basic ideas and mathematical foundations of artificial intelligence from the viewpoint of physics, with particular emphasis on the deep connection between statistical mechanics and machine learning. Although traditionally regarded as distinct fields—the former studies physical systems with a vast number of microscopic degrees of freedom, while the latter focuses on extracting regularities from data—both deal with complex, high-dimensional, stochastic systems whose macroscopic behavior emerges from many interacting degrees of freedom. As a result, they share a common mathematical language, including probability distributions, entropy, free energy, the partition function, fluctuations, and phase transitions. This framework provides a unified perspective for understanding probabilistic models, loss functions, model complexity, and generalization in machine learning. The work is organized into three parts. The present notes constitute Part I, which begins with probabilistic and statistical descriptions and introduces entropy, free energy, the partition function, and the Boltzmann distribution, showing how these concepts repeatedly appear, under different names, throughout machine learning. Part II will discuss physical models of neural networks, while Part III will focus on optimization and learning dynamics. Together, the three parts establish a continuous connection from statistical description and model structure to the learning process. These notes are intended for readers with a university-level background in physics, mathematics, or related fields, without requiring systematic training in machine learning. Throughout, emphasis is placed on the interplay between physical intuition, mathematical derivation, and computational experiments, rather than on superficial analogies between physics and artificial intelligence.

¹stlanchen@126.com

²genghf@ytu.edu.cn

*...wherever my tracks had been,
my mother's footprints had also been.*

—— **Shi Tiesheng**, *The Temple of Earth and I*

Preface

These lecture notes originated in a course the author began designing in 2023 for physics undergraduates at Yantai University. The course was originally titled “Efficient Physics Computing in Python,” with the goal of bringing modern artificial-intelligence methods such as deep learning into the teaching of computational physics — exploring applications of AI within physics itself (AI for Physics, AI4Phys). As the teaching practice and the course content kept expanding, it gradually turned toward a more fundamental question: could the direction be reversed — could physics, and statistical mechanics in particular, with its concepts and methods, be used instead to understand artificial intelligence itself? The course grew, step by step, into what is now *Physical Foundations of Artificial Intelligence*.

The course runs 18 weeks, with 4 class hours per week: 2 hours of lecture and 2 hours of hands-on lab work. **These notes correspond to the lecture portion of the course:** the lab sessions are built on the physical picture established in these notes, with accompanying programming exercises and numerical experiments that reinforce the lecture material; the two are organized independently, however, and these notes do not include the specific content of the lab sessions.

The core goal of these notes is to build, for students of physics, a bridge connecting statistical mechanics and machine learning. The two are usually regarded as relatively independent fields, but they share a deep commonality in mathematical structure and methodology: both must deal with complex systems in high-dimensional spaces, and both must confront collective phenomena that arise jointly from randomness and a large number of degrees of freedom. Statistical-mechanics concepts such as probability distributions, entropy, free energy, the partition function, fluctuations, and phase transitions can likewise provide a unified language for understanding loss functions, probabilistic models, model complexity, generalization, and learning behavior in machine learning.

These notes are divided into three parts, planned to correspond to three semesters of teaching. The present notes make up Part I, devoted mainly to establishing the basic connection between statistical mechanics and machine learning. We start from probabilistic and statistical descriptions, and progressively introduce entropy, free energy, the partition function, and the Boltzmann distribution, examining how these concepts recur, again and again, in different forms, throughout machine learning. The reader will see that this similarity is not a simple formal analogy, but arises from a mathematical structure shared by the two fields in how they handle complex, high-dimensional, uncertain systems — though the note is equally explicit about identifying the places where the similarity is merely an analogy, and should not be taken as a strict equivalence.

Building on this foundation, Part II will go on to discuss physical models of neural networks,

and Part III will turn to optimization and learning dynamics, studying how a model acquires structure from data through a dynamical process. Together, the three parts form a single logical thread running from statistical description, through model structure, to the learning process, attempting to answer, from the point of view of physics, a basic question:

Why is artificial intelligence able to learn regularities from complex data?

These notes are aimed at readers with a university-level background in physics, but not necessarily any systematic training in machine learning — they are especially well suited for undergraduate and graduate students in physics, mathematics, and related fields. Reading them requires only a basic grounding in calculus, linear algebra, classical mechanics, and thermal physics, and presupposes no background in machine learning whatsoever. The mutual reinforcement of physical intuition, mathematical derivation, and computational experiment is a writing principle that runs throughout the note — the computational-experiment component is carried out mainly through the lab sessions that accompany these notes.

Beyond the core text, each chapter also includes exercises for readers to work through progressively alongside their reading. The exercises are organized into three tiers — Foundational, Intermediate, and Challenge: Foundational exercises consolidate the chapter’s core derivations, and most require only repeating a calculation already demonstrated in the text; Intermediate exercises require applying methods already learned to a new setting, usually calling for an independent, complete derivation; Challenge exercises typically require consulting outside material, or carrying out a simple numerical or programming experiment, and are meant to showcase the broader reach of the ideas involved.

A note on production: the figures throughout these notes were generated with AI assistance, and the English of this edition has likewise been polished with AI assistance. For readers interested in the original Chinese edition, please visit: https://github.com/stpblc/Phys4AI_1.

Contents

Preface	3
1 A Map of the AI Landscape	7
1.1 What Are We Talking About When We Talk About Machine Learning?	8
1.2 Why Statistical Physics, of All Things?	17
1.3 A Guided Tour of Part I	22
1.4 AI Is Also Changing Physics Research Itself	23
2 Starting from Classical Statistical Physics	27
2.1 The Concentration of Macroscopic Probability	28
2.2 The Canonical Ensemble	31
2.3 The One-Dimensional Ising Model	41
3 Probabilistic Models in Machine Learning	55
3.1 The Basic Setup of Supervised Learning	56
3.2 The Basic Assumptions of Probabilistic Modeling	59
3.3 Maximum Likelihood Estimation (MLE)	61
3.4 Maximum A Posteriori Estimation (MAP)	64
4 Loss Functions as Energy Functions	73
4.1 The Core Analogy	74
4.2 Example One: Linear Regression	76
4.3 Example Two: Logistic Regression	82
4.4 Example Three: Softmax Regression	86
5 The Partition Function and Normalization	93
5.1 The Ubiquity of Normalization	94
5.2 Undirected Graphical Models (Markov Random Fields)	95
5.3 The Computational Challenge of the Partition Function	105
5.4 An Overview of Approximation Methods	108
6 The Deep Connection Between Free Energy and Loss Functions	113
6.1 The Physical Derivation of Variational Free Energy	114
6.2 Variational Inference in Machine Learning	118
6.3 The Physics–Machine-Learning Correspondence Table	120

6.4	The Mean-Field Approximation	121
7	The Maximum Entropy Principle	131
7.1	The Maximum Entropy Principle in Statistical Mechanics	133
7.2	Applications of Maximum Entropy in Machine Learning	137
7.3	The Principle of Minimum Cross-Entropy	139
A	Quick Reference of Common Notation and Formulas	155
A.1	Notation for Probability Theory and Mathematical Tools	155
A.2	Notation for Graphs and Markov Random Fields	156
A.3	Key Formulas for Variational Free Energy and the KL Divergence	157
A.4	The Method of Lagrange Multipliers	157
B	Common Probability Distributions and Statistics Table	159
B.1	Common Probability Distributions	159
B.2	General Definitions	160
C	Markov Chains and MCMC Basics	161
C.1	The Stationary Distribution	161
C.2	Detailed Balance	162
C.3	The Metropolis Algorithm	162
C.4	Gibbs Sampling	164
C.5	Ergodicity	164

Chapter 1

A Map of the AI Landscape

*Give me the map there. — Know that we
have divided
In three our kingdom. . .*

— Shakespeare, *King Lear*, Act I, Scene
1, spoken by King Lear

Objectives

- Understand the scope that the term “artificial intelligence” covers today, and where “machine learning” fits within that scope;
- Understand the general structure of a machine-learning problem — input, model, output — that is, the core metaphor of “learning a function”;
- Understand the intuitive meaning of “training” and “parameters,” laying the groundwork for the formal introduction of the likelihood function, parameter estimation, and related concepts in Chapter 3;
- Work through the forward pass of a small multilayer neural network entirely by hand, and understand precisely what “depth” and “activation function” mean, laying the groundwork for Part II of these notes, “Physical Models of Neural Networks”;
- Understand why probability and uncertainty enter machine learning naturally — this is precisely the doorway through which statistical physics is about to enter;
- Understand why it is specifically *statistical physics* — rather than classical mechanics, quantum mechanics, or fluid dynamics — that provides the most natural physical language for understanding machine learning; this judgment rests on a testable rationale, not merely on disciplinary preference;
- Build a map of the overall route taken by Part I of these notes (Chapters 2 through 7), making clear the question each chapter answers, the corresponding physical concept, and how the chapters connect to one another;

- Recognize that, alongside “using physics to understand AI,” there exists an equally important story running in the opposite direction: “using AI to do physics.”

Before we embark on the story of statistical physics meeting machine learning, this chapter sets out to do something that has nothing to do with deriving formulas, yet is equally important: draw a map. The next six chapters will successively introduce combinatorial counting, the Boltzmann distribution, maximum likelihood estimation, loss functions, partition functions, variational free energy, and the maximum entropy principle — concepts that are tightly interconnected, each link leading to the next. But without first knowing what all of this is ultimately trying to answer, it is easy, working purely from the derivations themselves, to see the trees but miss the forest. This chapter contains no formula that needs to be memorized and requires no background in machine learning or programming. Its sole purpose is to give every new term that appears in the following six chapters a clear place in your mind.

1.1 What Are We Talking About When We Talk About Machine Learning?

In recent years the term “artificial intelligence” has become almost inescapable: ChatGPT, which can converse with you, write code, and draft papers; AlphaFold, which predicts a protein’s three-dimensional structure from its amino-acid sequence and is widely regarded as having solved a problem that had stumped biologists for decades; AlphaGo, the Go-playing program that once defeated the world’s top human players; Stable Diffusion, which generates an image from a text description; not to mention self-driving cars, recommendation systems, speech recognition, and more.

Playing chess, painting pictures, driving cars, predicting molecular structures, chatting — the tasks these systems perform could hardly seem more different from one another, and yet all of them are lumped together under the single label “artificial intelligence.”

This tells us something important: **“artificial intelligence” is not a single technology, but the name of an entire, continually expanding field.** If you plunge straight into one particular corner of it (such as the branch this course is about to take up) without first understanding the field’s rough internal structure, confusion is almost guaranteed: which part of “artificial intelligence,” exactly, are these specific derivations trying to solve? This section is meant to unfold that map and take a look at it before we begin in earnest.

■ Think and Check

Of the systems mentioned above — ChatGPT, AlphaFold, AlphaGo, Stable Diffusion — if you had to state their common feature in a single sentence, what would you say?

Solution: A reasonable answer is that none of them had every one of their specific rules programmed in by a human, line by line; instead, each of them “learned” its capabilities from large amounts of data through some particular procedure. Exactly what this “learning” means is precisely what the next few sections will make precise.

The Tree of Artificial Intelligence: Where Does Machine Learning Sit?

In one sentence: the overwhelming majority of artificial intelligence’s practical capability today comes from the branch known as “machine learning.” If we sketch artificial intelligence roughly as a tree (Figure 1.1), the structure looks something like this (this is only one of many possible ways to organize the field — the boundaries are not as sharp as the diagram suggests, but it is enough to give you a first coordinate system to work with):

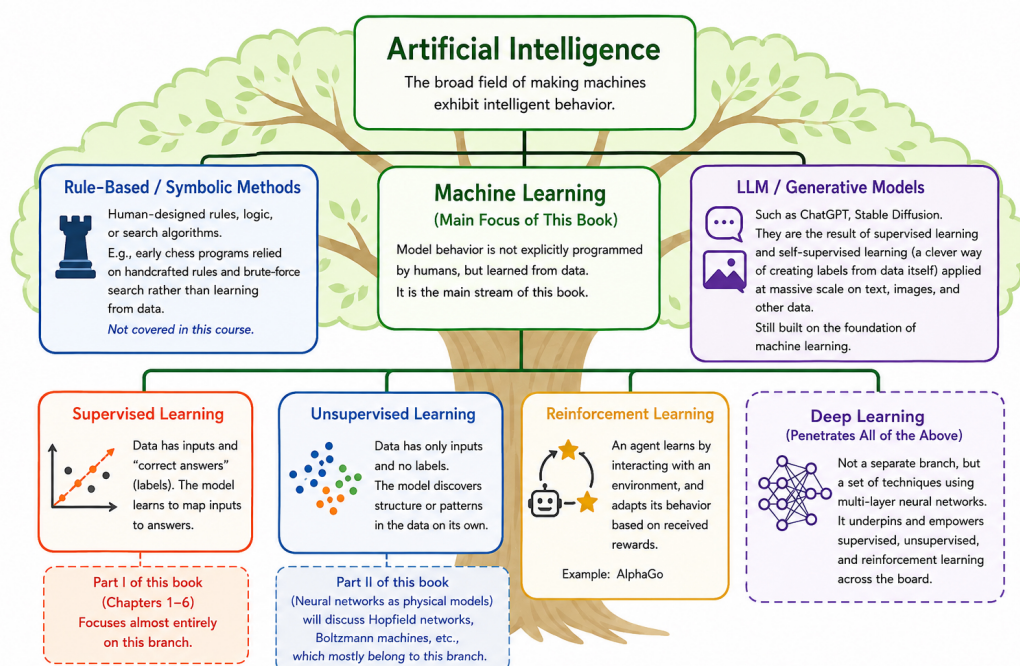


Figure 1.1: A map of artificial intelligence.

- **Artificial Intelligence:** the entire field concerned with making machines exhibit some form of “intelligent behavior.”
 - **Rule-based / symbolic methods:** approaches in which explicit rules, logic, or search algorithms are written by hand in advance. Chess programs from the last century, for example, relied mainly on hand-coded rules and brute-force search rather than “learning” from game data. This category will not be covered in this course.
 - **Machine Learning: the main thread of this book.** Its defining feature is that a model’s behavior is not written directly by a human but “learned” from data (we will make precise what “learned” means shortly). Machine learning itself is usually further divided into:
 - * **Supervised Learning:** the data include both inputs and “correct answers” (labels), and the model’s task is to learn to predict the answer from the input. **Part I of these notes (Chapters 2 through 7) is devoted almost entirely to this branch.**
 - * **Unsupervised Learning:** the data consist of inputs only, with no labels, and the model must discover structure or regularities in the data on its own. The Hopfield

networks and Boltzmann machines discussed in Part II, “Physical Models of Neural Networks,” mostly belong to this branch.

- * **Reinforcement Learning:** a model (called an “agent”) adjusts its behavior through repeated interaction with an environment, guided by the reward signals it receives; AlphaGo is the emblematic example of this branch.
- * **Deep Learning:** not a fourth branch standing alongside the previous three, but rather an entire family of techniques that implement supervised, unsupervised, or reinforcement learning using “multilayer neural networks.” It permeates nearly every corner of machine learning.
- **Large Language Models / Generative Models:** examples include ChatGPT and Stable Diffusion. These are, at their core, the product of supervised learning and “self-supervised learning” (a clever form of supervised learning in which labels are constructed from the data itself) applied at massive scale to enormous quantities of text and image data — they still rest on the foundation of machine learning.

Looking at this diagram (Figure 1.1), you can already be sure of one thing: the material this course is about to cover belongs to the **machine learning** branch, and lies almost entirely within **supervised learning**. This is the particular corner we are about to explore in depth.

■ Looking Ahead

The Hopfield networks and Boltzmann machines discussed in Part II, “Physical Models of Neural Networks,” actually belong to the “unsupervised learning” branch mentioned above — they require no labels; instead, an energy-based model directly inspired by statistical physics learns on its own to “memorize” or “generate” the distribution of the data. This is exactly why Chapter 2 takes such pains to lay a solid foundation in the language of the Boltzmann distribution and the partition function: this language serves not only the supervised learning covered in Part I of the book, but also forms the foundation for Part II.

What Problem Is Machine Learning Actually Solving?

Setting aside the somewhat anthropomorphic word “learning,” almost every machine-learning problem can be reduced to one extremely simple structure:

$$\text{Input} \longrightarrow \text{Model} \longrightarrow \text{Output}$$

Let us look at a few concrete examples (Figure 1.2):

- Input an image $\rightarrow$ model $\rightarrow$ output “cat” (whether the image contains a cat);
- Input a house’s floor area, location, and other information $\rightarrow$ model $\rightarrow$ output the house’s price (this is precisely the example that Chapters 2 and 3 will use repeatedly);
- Input an English sentence $\rightarrow$ model $\rightarrow$ output the corresponding Chinese sentence;

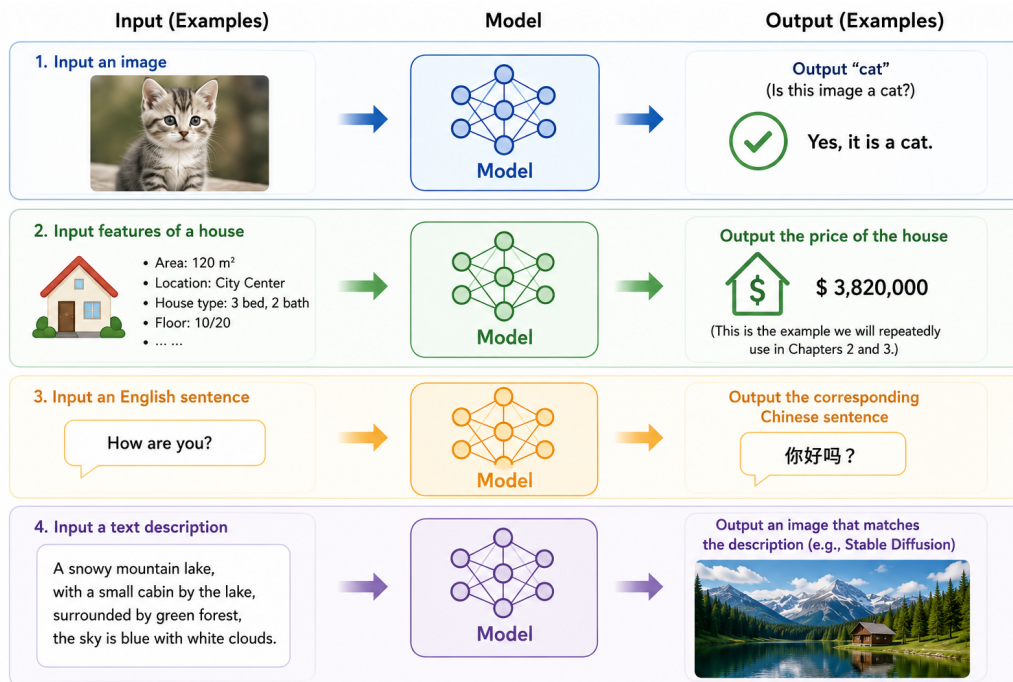


Figure 1.2: Concrete examples of machine learning.

- Input a piece of text description → model → output an image matching the description (as with Stable Diffusion).

These examples appear to have nothing in common, but once we strip away the specific content, their structure is identical: in every case we are given some input x and want the model to produce the output y we expect. Put in the most compact mathematical language, what machine learning does is **learn a function** $y = f(x)$ — but note that the word “learn” here is crucial: this function is not written into the code correspondence by correspondence by a human; rather, it is found automatically by an algorithm from a large number of “input–output” examples. This is precisely the most essential difference between machine learning and traditional programming: traditional programming is “a human tells the machine the rules,” whereas machine learning is “the machine finds the rules for itself from examples.”

■ Think and Check

Determine, for each of the following tasks, what the input is and what the output is:

- (1) determining whether an email is spam;
- (2) determining, from an electrocardiogram (ECG) signal, whether an arrhythmia is present;
- (3) given the opening of a melody, continuing it with the next few notes.

Solution: (1) The input is the content of the email (which can be text, or a set of numerical features extracted from the text), and the output is a categorical label, “spam” or “not spam”; (2) the input is the ECG signal as it varies over time, and the output is again a categorical label; (3) the input is the existing fragment of melody, and the output is the sequence of notes that follows — this example reminds us that an “output” need not be a

simple number or category; it can also be an entire piece of structured content. But the core structure of “predicting an output from an input” remains unchanged.

What Does “Training” Mean?

We said above that “a function is learned from data,” but what exactly does that mean? Let us make it precise using an example that requires no formulas at all and can be understood through intuition alone.

Imagine a sheet of paper in front of you, with a scatter of points on it: the horizontal axis is a house’s floor area, and the vertical axis is the price the house sold for (this is exactly the data that Chapters 2 and 3 will formally use). These points are roughly distributed along a certain direction — the larger the floor area, the higher the price tends to be, on the whole. Now you want to draw a straight line through the middle of these points, as close as possible to all of them.

How would you draw this line? You probably would not get it right on the first try; instead, you would sketch a line, see how far it is from the points, and then make small adjustments to the line’s **slope** and **intercept**, over and over, until the line “looks” like the best fit to the points.

This line can be written in the form $y = ax + b$, where the two numbers a (the slope) and b (the intercept) are the model’s **parameters**. And what you just did — repeatedly adjusting a and b so that the line fits the data points ever more closely — is precisely the plainest possible meaning of the word **training**:

Training means repeatedly adjusting a model’s parameters so that the model’s output moves ever closer to the answer we expect.

A real machine-learning model (such as logistic regression or softmax regression, discussed in later chapters, or the neural networks discussed in Part II) is of course far more complicated than a single straight line, and the number of parameters can grow from a handful to hundreds of millions or more. But “training,” in essence, is exactly the same action you just performed by hand in adjusting the line — except that it must be handed over to a computer and carried out automatically by a more systematic algorithm. This is precisely what Part III of these notes, “Optimization and Dynamics,” is devoted to. The questions of what makes a line “close” to the data, and in which direction the parameters should be adjusted, are exactly what Chapters 2 and 3 are about to answer formally.

Multilayer Neural Networks: From a Single Neuron to Deep Networks

The line $y = ax + b$ above has only two parameters, and its expressive power is extremely limited — no matter how a and b are adjusted, it always traces out a straight line, and can never fit a curve that is visibly bent. Most regularities in the real world (whether an image contains a cat, what a sentence should be translated into) clearly cannot be described by a “straight line.” **Neural networks** are a family of models designed precisely to obtain greater expressive power. Our goal in the rest of this section, without touching the backpropagation algorithm at all, is to have you work through, by hand, exactly what a neural network does in going “from input to

output” — a step called **forward propagation**. This is also a direct stepping stone to Part II, “Physical Models of Neural Networks.”

From a Line to a Neuron

$y = ax + b$ has only a single input x : it predicts the price using only the house’s floor area. But in reality, price obviously depends on more than area alone — the quality of the interior finish usually matters a great deal too. Suppose the quality of the finish is quantified on a scale from 1 to 10, call it x_2 (with floor area still denoted x_1); a natural guess is that each additional point of finish quality adds roughly ¥80,000 to the price, each additional square meter of floor area adds roughly ¥30,000, and there is a base price of, say, ¥300,000, reflecting location and other factors. Put together, the estimated price becomes

$$\text{Price} \approx 3x_1 + 8x_2 + 30 \quad (\text{units: ¥10,000}).$$

For instance, a house with 100 square meters of floor area and a finish-quality rating of 7 has a predicted price of about $3(100) + 8(7) + 30 = 386$, i.e., ¥3.86 million. Compared with $y = ax + b$, the only change in this expression is that the single slope a has become two numbers (3 and 8), one for each of the two different inputs. If the input consists of even more numbers (written as a vector $\mathbf{x} = (x_1, \dots, x_n)$), the most natural generalization of this same example is to weight each of them individually and sum:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w}^T\mathbf{x} + b,$$

where each w_i (the **weight**, corresponding to the 3 and 8 in the example above) and b (the **bias**, corresponding to the 30 above) is a parameter, and the number of parameters has grown from the line’s 2 to $n + 1$. This step, by itself, is still just a direct generalization of a “line” (a hyperplane), and is no more powerful. What actually brings about a change is the next step: passing z through an **activation function** σ to obtain

$$a = \sigma(z).$$

This pairing of “weighted sum + activation function” is called a **neuron**.

The name “neuron” here is not an arbitrary choice. It comes from the biological **nerve cell** (see Figure 1.3): in a real nervous system, a neuron receives signals from other neurons through its dendrites, integrates these signals in some way, and, once certain conditions are met, transmits a signal outward through its axon. Early researchers in artificial intelligence, inspired by this process, tried to describe an analogous “receive–integrate–output” process using an extremely simplified mathematical model. Of course, an artificial neuron is not an exact replica of a biological one. The workings of a real neuron involve intricate electrochemical processes, whereas the mathematical model here extracts only one very crude but extremely important idea from it: **multiple inputs are combined through weighting, and the result is passed through a nonlinear mechanism to produce an output**. It is precisely this simple idea that forms the most basic computational unit of a modern neural network.

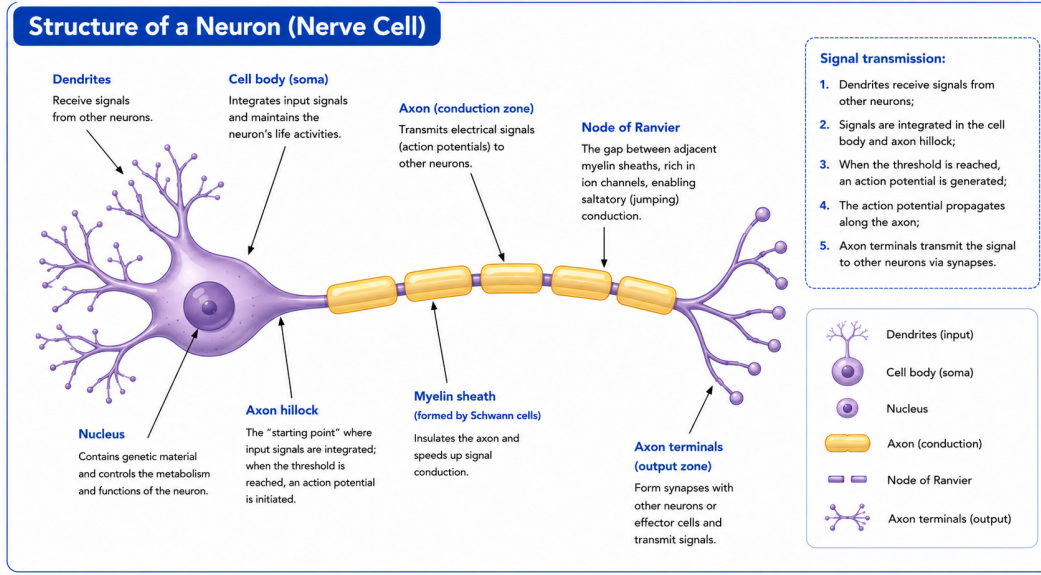


Figure 1.3: A neuron (nerve cell).

■ Why Must There Be an Activation Function?

Later in this section we will prove, using concrete numbers, that if σ is the identity function (i.e., it performs no transformation at all, $a = z$), then no matter how many layers of neurons are stacked, the final result computed is still only a linear function of the input — no different from a (high-dimensional) “line.” The activation function must contain some kind of “bend” for stacking multiple layers to mean anything. This section uses the two most common activation functions:

$$\text{ReLU}(z) = \max(0, z), \quad \sigma_{\text{sigmoid}}(z) = \frac{1}{1 + e^{-z}}.$$

ReLU simply “zeroes out negatives and leaves positives unchanged,” making it the cheapest to compute by hand; sigmoid will make its formal debut in Chapter 4 — readers might want to keep an eye on its specific form and ask themselves whether it looks familiar.

Assembling a Layer, Stacking Multiple Layers

A single neuron can only output one number. Placing several neurons side by side, each independently performing “weighted sum + activation” on the same input $\mathbf{x}$, constitutes one **layer** of a neural network. Taking the output of one layer as the input to the next, and repeating this process, chaining layers together in series, gives a **multilayer neural network** — this is exactly where the word **depth** in “deep learning” comes from: depth is simply the number of layers. In matrix notation, the computation of layer l can be written uniformly as

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \quad (1.1)$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the network’s input, each row of $\mathbf{W}^{(l)}$ is the weight vector of one neuron in that layer, and the activation function σ acts elementwise. The output of the network’s final layer, $\mathbf{a}^{(L)}$, is the model’s prediction $\hat{y}$.

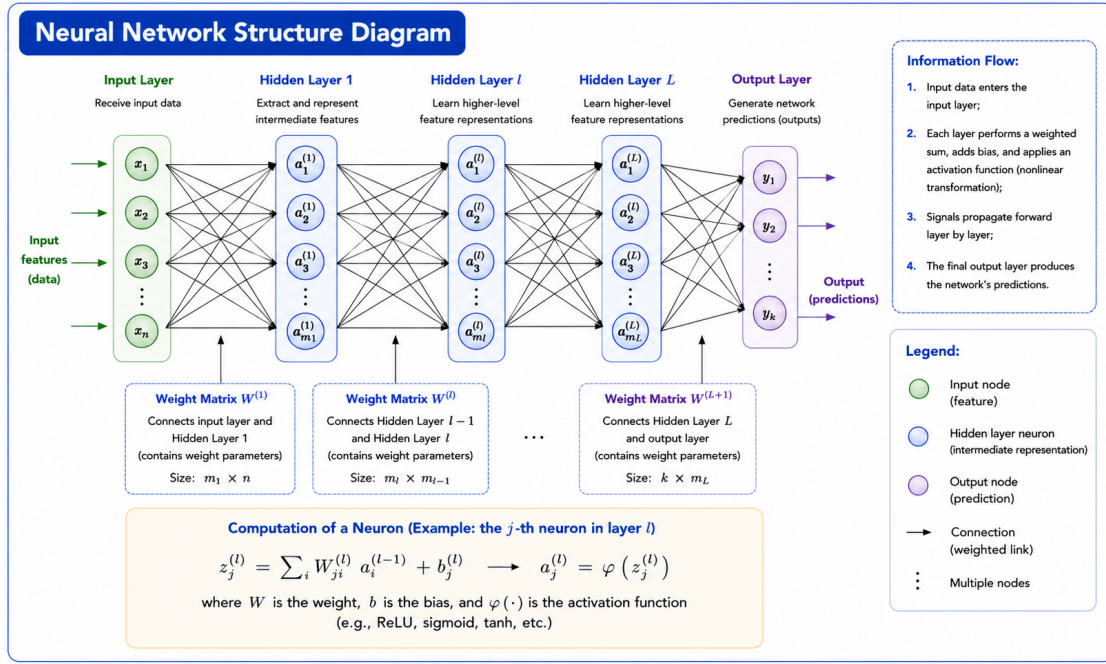


Figure 1.4: A neural network (deep learning).

A Two-Layer Network You Can Compute Entirely by Hand

That is enough abstraction; let us now work through Equation (1.1) from beginning to end using a concrete set of numbers. We design the smallest possible network that already has “depth”: the input is a 2-dimensional vector $\mathbf{x} = (x_1, x_2)$, the hidden layer contains 2 ReLU neurons, and the output layer is a single sigmoid neuron.

Network parameters (set by hand, unrelated to training):

$$\mathbf{W}^{(1)} = \begin{pmatrix} 2 & -1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} -1 \\ -2 \end{pmatrix} \quad \left(\text{i.e., } h_1 : z_1 = 2x_1 - x_2 - 1, \quad h_2 : z_2 = x_1 + x_2 - 2 \right),$$

$$\mathbf{W}^{(2)} = \begin{pmatrix} 3 & -2 \end{pmatrix}, \quad b^{(2)} = 1 \quad \left(\text{i.e., the output layer: } z_{\text{out}} = 3a_1 - 2a_2 + 1 \right).$$

Step 1: substitute $\mathbf{x} = (1, 2)$ and compute the hidden layer.

$$z_1 = 2(1) - 2 - 1 = -1 \quad \implies \quad a_1 = \text{ReLU}(-1) = 0,$$

$$z_2 = 1 + 2 - 2 = 1 \quad \implies \quad a_2 = \text{ReLU}(1) = 1.$$

Step 2: compute the output layer.

$$z_{\text{out}} = 3(0) - 2(1) + 1 = -1 \quad \implies \quad \hat{y} = \sigma_{\text{sigmoid}}(-1) = \frac{1}{1 + e^1} \approx 0.269.$$

■ A Familiar-Looking 0.269

This number should not look unfamiliar — it is exactly the numerical value of the excited-state probability P_1 for the two-level system at $k_B T = 1$ in Table 2.4 of Chapter 2! This is no coincidence: the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$ and the two-level-system probability

formula given by the Boltzmann distribution are, written out, literally the same algebraic expression (differing only in variable-naming convention) — this can be seen directly just by placing the two formulas side by side, with no further proof required. Chapter 4 develops this connection systematically, formally naming it **logistic regression** and deriving the corresponding loss function. Here we let you catch a first glimpse of this connection with your own eyes.

Verifying that the network really is “nonlinear”: trying a different input point.

Take $\mathbf{x} = (3, 1)$: $z_1 = 2(3) - 1 - 1 = 4$, $z_2 = 3 + 1 - 2 = 2$ (both positive, so ReLU does not clip, $a_1 = 4, a_2 = 2$), $z_{\text{out}} = 3(4) - 2(2) + 1 = 9$, $\hat{y} = \sigma_{\text{sigmoid}}(9) \approx 0.9999$. Now take $\mathbf{x} = (0, 0)$: $z_1 = -1, z_2 = -2$ (both clipped to 0 by ReLU), $z_{\text{out}} = 3(0) - 2(0) + 1 = 1$, $\hat{y} = \sigma_{\text{sigmoid}}(1) \approx 0.731$.

The key check: what happens without ReLU? Suppose the hidden layer performs no nonlinear transformation at all ($a_1 = z_1$, $a_2 = z_2$, i.e., the identity function is used in place of ReLU); expanding algebraically directly:

$$\begin{aligned} z_{\text{out}} &= 3z_1 - 2z_2 + 1 = 3(2x_1 - x_2 - 1) - 2(x_1 + x_2 - 2) + 1 \\ &= (6 - 2)x_1 + (-3 - 2)x_2 + (-3 + 4 + 1) = 4x_1 - 5x_2 + 2. \end{aligned}$$

In other words, once the activation function is removed, no matter how many neurons the hidden layer appears to contain or how complicated the parameter layout looks, the two-layer network’s computed z_{out} is always equal to the very same linear function $4x_1 - 5x_2 + 2$ — this is exactly the fact promised in the preliminary remarks: **without a nonlinear activation function, no matter how many layers are stacked, the result is still only a (high-dimensional) straight line**. Table 1.1 places the “collapsed-to-linear” prediction side by side with the actual output of the ReLU network, letting you see directly when the two agree and when they differ:

Table 1.1: With and Without the ReLU Activation Function: A Comparison at Three Input Points

Input (x_1, x_2)	Without activation ($4x_1 - 5x_2 + 2$)	Actual z_{out} (with ReLU)	Same?
(1, 2)	-4	-1	No (h_1 clipped to 0)
(3, 1)	9	9	Yes (both pre-activations positive, no clipping)
(0, 0)	2	1	No (h_1, h_2 both clipped)

Whenever ReLU genuinely “clips” some neuron (its pre-activation value is negative), the network’s output departs from the “collapsed linear model” prediction — this is exactly the expressive power, beyond that of any single straight line (or hyperplane), that depth and nonlinearity together provide.

■ Think and Check

Using the same network (with $\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, b^{(2)}$ as given above), compute by hand the hidden-layer activations a_1, a_2 , the output-layer value z_{out} , and the final prediction $\hat{y}$ for the

input $\mathbf{x} = (1, 0)$.

Solution: $z_1 = 2(1) - 0 - 1 = 1 \Rightarrow a_1 = 1$; $z_2 = 1 + 0 - 2 = -1 \Rightarrow a_2 = \text{ReLU}(-1) = 0$;
 $z_{\text{out}} = 3(1) - 2(0) + 1 = 4$; $\hat{y} = \sigma_{\text{sigmoid}}(4) = 1/(1 + e^{-4}) \approx 0.982$.

What Does This Have to Do with “Training”?

What we have just computed is **output given parameters** (forward propagation). As noted above, “training” means repeatedly adjusting the parameters so that the output approaches the desired answer — for a neural network, the parameters that need adjusting are $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ in every layer, which in this example amount to $2 \times 2 + 2 + 1 \times 2 + 1 = 9$ numbers in total. Nine parameters can no longer be tuned by intuition the way we adjusted the line’s a and b ; a systematic algorithm is needed — this is exactly the problem Part III, “Optimization and Dynamics,” is devoted to solving, and it is also the object that Part II, “Physical Models of Neural Networks,” reinterprets in the language of statistical physics: readers will see that, viewed through the lens of statistical physics, a trained neural network is, at bottom, a system with an “energy” engraved into its weights.

1.2 Why Statistical Physics, of All Things?

In the last section we upgraded “a line” to “a neural network,” but no matter how complicated the model becomes, we have not yet answered a more fundamental question: whether it is a line or a nine-parameter network, both can only produce a **definite number** as their output. Let us set the neural network aside for a moment and return to the very first, simplest example — fitting house prices with a line — to raise this question again.

Suppose you find that two houses with exactly the same floor area do not sell for exactly the same price — one, because of good interior finish and a favorable floor level, sells for more; the other, for other reasons, sells for less. This phenomenon — “the same input, yet not quite the same output” — is nearly ubiquitous in real data.

This means that a line (or any model) can almost never make every data point fall exactly on it — there will always be some “leftover” deviation that cannot be explained by the factors the model has already accounted for. A physicist will immediately think of a familiar term: **noise**¹. And once we accept that real data are pervaded by noise, a natural question arises: should the output a model produces be an “absolutely certain” number, or should it instead carry some

¹“Noise” here does not necessarily mean that the data were genuinely mismeasured by some instrument. More precisely, it refers to whatever randomness or uncertainty in the output, relative to the current model, cannot be fully explained by the input factors the model has already taken into account. In the house-price problem, for instance, even if two houses have the same floor area, their prices may still be affected by small differences in finish, floor level, orientation, and location, or even by incidental factors specific to the buyer and seller. As long as these factors are not included among the model’s inputs, their effect on price will appear, from the model’s point of view, as “noise.” Noise, therefore, is usually not some independently existing mysterious disturbance in nature, but rather **the unexplained portion, relative to a particular model and the information available to it.**

degree of confidence — something like, “this house’s price is most likely ¥2.85 million, but there is some probability it falls between ¥2.70 million and ¥3.00 million”?

The latter view is precisely the fundamental reason probability enters machine learning. And here we run into a question any physicist will find familiar: **how can we describe a macroscopically fairly definite, orderly outcome using a vast number of microscopic, unpredictable details?** This is exactly the core problem statistical physics has grappled with since its birth — no one can track the motion of every one of the 10^{23} molecules in a cup of coffee individually, and yet macroscopic quantities such as the coffee’s temperature and pressure can be predicted and described with precision.

What the next six chapters do is take this already highly mature mathematical language that statistical physics has developed for “how microscopic uncertainty gives rise to macroscopic regularity” — the Boltzmann distribution, the partition function, free energy, entropy, variational methods, the maximum entropy principle — and transplant it, essentially unchanged, into the context of machine learning, using the very same tools to answer the question of how a model should represent the uncertainty in its data. This is why the next chapter abruptly begins discussing a physical model that seems to have nothing whatsoever to do with “artificial intelligence”: a collection of particles randomly distributed on the two sides of a container. It is not really about thermal physics — it is laying the groundwork for the language shared by all six of the chapters that follow.

Having answered “why probability is needed,” we still need to answer a sharper question: physics has many branches — classical mechanics, quantum mechanics, fluid dynamics — all mature theories of “how large numbers of constituent parts give rise to collective behavior.” Why does this book choose statistical mechanics, of all of them? This question deserves a serious answer rather than being waved away — if the answer were simply “because the author’s background is in statistical physics” (which, as it happens, it is not), the book’s methodology would rest on nothing. This choice is justified only if statistical physics genuinely fits the *structure of the machine-learning problem itself* more closely than the alternatives do.

Why Not Classical Mechanics?

Classical mechanics (Newton’s laws, Lagrangian/Hamiltonian mechanics) addresses the following problem: **given the forces or potential energy governing a system, how will the system evolve exactly?** This requires that you already know the rules themselves at the outset — you must know a spring’s stiffness constant before you can compute the trajectory of an oscillator.

But the core difficulty of machine learning is precisely the opposite: we never know the “true rule” to begin with. Given a batch of house-price data, no one tells us the “true functional relationship” between floor area and price; this relationship itself is an object that must be *inferred* from a finite, noisy sample. This is an **epistemological problem** (how should our ignorance about the rule be quantified and updated?), not a **dynamical problem** (given the rule, how does the system evolve?). The entire toolkit of classical mechanics — the Lagrangian, the action, the equations of motion — is built on the premise that “the rule is already known,” and this premise happens not to hold in the central problem of machine learning: learning an

unknown function.

■ Classical Mechanics Is Not Useless — It Simply Answers the Other Half of the Question

One clarification is needed: this does *not* mean classical mechanics has no place at all in this book. Once a model’s “rule” (the shape of its loss function) has been fixed, the **training process itself** — how the parameters evolve as gradient updates proceed step by step — is very much a dynamical problem; gradient descent itself can be viewed as a (dissipative) mechanical system moving within a “potential energy” (the loss function). This is exactly what Part III, “Optimization and Dynamics,” is devoted to. The accurate statement is not “classical mechanics does not apply,” but rather: “classical mechanics answers *how* training proceeds, while statistical physics answers *what the loss function looks like and why it looks that way*” — Part I of these notes deals with the latter question first, because before discussing how to walk downhill along the terrain, one must first make clear where that terrain comes from.

Why Not Quantum Mechanics?

Quantum mechanics also deals with probability, and its mathematical structure — the partition function, $\beta = 1/k_B T$, even the path integral — shares a great deal of formal similarity with statistical physics; historically, the two have indeed been deeply intertwined (the quantum version of statistical physics is built precisely through quantum ensemble theory). So why not simply use the language of quantum mechanics to understand machine learning directly?

The key distinction lies in **where the uncertainty comes from**. Probability in quantum mechanics arises from the **interference of probability amplitudes** — two paths can interfere constructively or destructively, and this is an uncertainty intrinsic to the microscopic world, one that cannot be eliminated simply by “obtaining more information.” Probability in machine learning, by contrast, is purely **epistemic**: the probability that a coin lands heads is, in principle, just a matter of insufficient information on our part; once we know every physical detail of the coin (its mass distribution, the initial conditions of the toss), the outcome is no longer random at all. The “noise” that machine learning deals with — measurement error, unobserved confounding factors, finite samples — all falls into this category, and involves no quantum superposition or interference structure whatsoever. Using the formal apparatus of quantum mechanics (complex probability amplitudes, Hermitian operators) to explain why the cross-entropy loss takes the form it does would not make the argument any clearer; it would only introduce entirely unnecessary additional structure. (Quantum machine learning is a real and actively developing direction, but it addresses a different class of problem — using a quantum computer itself as a computational resource — rather than “understanding classical supervised learning in the language of quantum mechanics.” That is not the subject of these notes.)

Why Not Fluid Dynamics?

The answer here is the most surprising of the three: fluid dynamics (the Navier–Stokes equations and the like) is not, within physics itself, an independent foundational theory standing

alongside statistical mechanics — it is a **consequence** of statistical mechanics in a particular limit. Specifically, starting from the Boltzmann equation, which describes the evolution of the probability distribution of a large number of particles, one can rigorously derive the Navier–Stokes equations, in the limit of frequent collisions and macroscopic length scales far exceeding the molecular mean free path, through the Chapman–Enskog expansion (a systematic multiscale approximation method). The fluid-dynamical variables — density, flow velocity, temperature — are all moments of the Boltzmann distribution over molecular velocities. In other words, fluid dynamics addresses the continuum-limit approximation of statistical mechanics under the specific physical circumstances of “a large number of weakly interacting particles plus local near-equilibrium” — it is not a parallel language capable of replacing statistical mechanics.

This is easy to state in a single sentence, but carrying it out rigorously is difficult enough to deserve an authoritative example. Starting from Newtonian mechanics (a large number of colliding hard-sphere particles) and rigorously proving that a system does indeed produce the Boltzmann equation in an appropriate limit, and, further, deriving the Navier–Stokes equations from it — this is precisely one of the central components of Hilbert’s sixth problem (proposed in 1900: can macroscopic physics be established rigorously, in axiomatic fashion, from microscopic mechanics?), and it went unresolved in full generality for more than a century. In 2026, a body of work by the Chinese mathematician Yu Deng and collaborators rigorously proved, in the Boltzmann–Grad limit, that the microscopic dynamics of a hard-sphere gas does indeed produce the Boltzmann equation over sufficiently long time scales, and further established the connection to the equations of fluid dynamics — work for which Deng was awarded the Fields Medal. The statement that “microscopic laws do not automatically imply macroscopic laws” is not a convenient slogan; it is a genuine problem that took even the most rigorous mathematicians well over a hundred years to gradually resolve.

It should be noted that rigorously deriving macroscopic equations from microscopic particle dynamics, and “training a machine-learning model from data,” discussed in the chapters that follow, are two different mathematical problems, and should not be casually equated simply because both, on the surface, involve “extracting macroscopic regularities from a complex system.” But the two do share the same scientific perspective: when facing a system with a huge number of degrees of freedom, the real task is not to describe every detail individually, but to search for an effective structure capable of organizing those details. Statistical mechanics searches for this structure through probability distributions and thermodynamic potentials; machine learning searches for it through parameterized models and data-driven optimization. It is this shared perspective — not any single specific equation — that this book is really borrowing.

This point also highlights the special position statistical mechanics occupies among these four fields. It should be clarified that this does *not* mean statistical mechanics is “more fundamental” than classical mechanics — quite the opposite: statistical mechanics typically requires, as inputs, a whole additional layer of structure — microscopic dynamics (classical or quantum), ensemble assumptions, equilibrium assumptions, and the thermodynamic limit — and is a theory built *on top of* microscopic dynamics, rather than replacing or superseding it.

What we want to emphasize here is something else: the problem that statistical mechanics addresses — how the probability distribution over a large number of microscopic degrees of freedom gives rise to macroscopic regularities — and the problem that classical mechanics addresses

— given the rules, solve for the deterministic evolution — are problems of two fundamentally different natures (the former epistemological/probabilistic, the latter dynamical). Fluid dynamics (the continuum limit), for its part, is largely a consequence of statistical mechanics in a particular limit.

For the specific purpose of this book — providing an appropriate language for machine learning — statistical mechanics deals with the relationship between probability distributions, fluctuations, and macroscopic regularities, and this happens to match the core structure of machine learning extremely well. But this is a judgment about which theoretical framework best suits this particular class of problem, not a general claim, within the edifice of physics, about which theory is “more fundamental.”

A Testable Criterion

The three comparisons above can be compressed into a single concrete, testable criterion, rather than resting on “a physicist’s intuition”:

Whenever a problem’s core structure takes the form of a competition between energy minimization and entropy maximization, governed by a single scalar temperature parameter, statistical physics is the natural language for it.

Classical mechanics is organized around the principle that the action is stationary (the principle of least action), and involves no entropy at all; quantum mechanics is organized around unitary evolution and measurement, and likewise involves no picture of a “competition between energy and entropy”; fluid dynamics is organized around conservation laws (the continuity equations for mass, momentum, and energy), is itself a consequence of statistical mechanics in a specific limit, and is not organized around “entropy” as a guiding principle either. Nearly every core concept in machine learning, by contrast — regularization (a prior pulls the parameters toward simplicity, corresponding to an “external field”; the data pull the parameters toward complexity, corresponding to flexibility), the trade-off between overfitting and underfitting, the temperature parameter in softmax — repeatedly exhibits this same competition between “fit to the data” and “flexibility.” This is neither a coincidence nor a mere analogy: this competition recurs throughout the book in the guise of a single mathematical object, $F = E - TS$ (or its information-theoretic counterpart), which will be rigorously established rather than left as something the reader must intuit from a handful of analogies — Chapter 2 derives it directly from a physical system; Chapter 6 proves that variational free energy and the ELBO are two statements of the same inequality; Chapter 7 proves that it is exactly the solution given by the maximum entropy principle.

■ Summary

Classical mechanics requires that the rule already be known, whereas machine learning is precisely about *inferring* the rule from data — this is the most fundamental mismatch between the two; quantum-mechanical uncertainty arises from intrinsic interference between probability amplitudes, whereas machine-learning uncertainty is purely epistemic, a matter of insufficient information; fluid dynamics is itself a consequence of statistical mechanics in a particular limit, rather than an alternative standing alongside it. What truly makes

statistical physics the natural language here is not disciplinary prestige, but a concrete structural criterion: the core concepts of machine learning routinely take the form of “a competition between energy and entropy, governed by a single scalar parameter,” and among the major frameworks of classical physics, statistical physics is precisely the branch organized most systematically and directly around this structure (modern extensions such as quantum statistics, non-equilibrium statistical physics, and information thermodynamics likewise deal with the relationship among energy, entropy, and temperature — but these are extensions internal to the statistical-physics framework, not external competitors to it). This is also why the roadmap in the next section can be compressed into a single concrete, checkable table like Table 2.8, rather than a loose collection of analogies.

1.3 A Guided Tour of Part I

With this groundwork in place, we can now lay out the complete route map for Part I of the book (Chapters 2 through 7). Table 1.2 gives an overview; a more detailed “chapter guide” will also reappear at the start of each chapter’s main text.

Table 1.2: Roadmap for Part I of the Book (Chapters 2 through 7)

Chapter	Question this chapter answers	Core concepts introduced
Ch. 2	Why does probability take the exponential form “lower energy, higher probability”?	Microstates/macrostates, the Boltzmann distribution, the partition function, free energy, the one-dimensional Ising model
Ch. 3	Given observed data, how do we estimate the “most reasonable” parameters for a model?	The likelihood function, maximum likelihood estimation (MLE), maximum a posteriori estimation (MAP)
Ch. 4	Where do machine learning’s loss functions (mean squared error, cross-entropy, ...) come from?	Negative log-likelihood = energy + normalization term; linear, logistic, and softmax regression
Ch. 5	Why is “normalization” a genuine computational bottleneck in practical training?	Markov random fields, the Hammersley–Clifford theorem, the positive and negative phases
Ch. 6	What should we do when the partition function cannot be computed?	Variational free energy, the evidence lower bound (ELBO), the mean-field approximation
Ch. 7	Is the Boltzmann distribution a physical assumption, or an inevitable consequence?	The maximum entropy principle, the method of Lagrange multipliers, the principle of minimum cross-entropy

Let us briefly unpack the narrative behind this route. Chapter 2 starts from the most elementary combinatorial counting and arrives at the Boltzmann distribution and the partition function — the shared physical language for the rest of the book. Chapter 3 turns to machine learning,

showing that “estimating parameters from data” is, mathematically, strikingly similar to what Chapter 2 just did. Chapter 4 formally welds the two threads together: it shows that the negative log-likelihood in machine learning and “energy plus the logarithm of the partition function” in statistical physics have exactly the same mathematical structure — “loss function” and “energy function” turn out to be simply two different names for the same mathematical operation, used in two different fields. Chapter 5 points out that the examples we could compute by hand in Chapters 2 through 4 were tractable only because the number of variables was small; once the system grows larger, computing the partition function becomes a genuine bottleneck. Chapter 6 supplies the principal weapon for dealing with this bottleneck — variational methods — and proves that, in physics and in machine learning, it is in fact the same mathematical theorem. Chapter 7 re-derives the Boltzmann distribution from a purely information-theoretic point of view, bringing Part I of the book to a close: physical intuition, the variational principle, and information-theoretic constraints are three independent derivation paths that all converge on the very same mathematical object — and the identity established in Chapter 4, $-\ln P = \text{energy} + \text{normalization term}$, is exactly the bridge that lets the conclusions of all three paths be applied, without modification, to any machine-learning model.

1.4 AI Is Also Changing Physics Research Itself

The entire map we have built up so far points in a single direction: **using physics to understand AI** — using the language of statistical mechanics to give a fresh, clear account of what machine-learning concepts such as the loss function, normalization, and variational inference are “really doing.” This is what the next six chapters of the book set out to do.

But there is another half to this story, running in exactly the opposite direction: **AI is also becoming a tool for physics research itself**, and its impact goes well beyond simply “computing faster.” This other half is not the focus of this book (if it interests you, it is substantial enough to fill an entire course of its own), but as a reader you should at least know that it exists. Roughly speaking, the ways AI is currently being brought into physics research fall into the categories listed in Table 1.3:

■ A Real Example

In early 2026, Harvard physics professor Matthew Schwartz ran an experiment: directing Claude purely through text prompts — without editing any files himself or substituting in formulas he had worked out by hand — he had the AI independently complete a technical calculation in quantum chromodynamics (QCD): the resummation of the “Sudakov shoulder” in the C -parameter distribution. He estimated that this work would have taken a full year had he assigned it to a graduate student to work through under his guidance; this time, a publishable paper was ready within two weeks (Schwartz, arXiv:2601.02484, 2026). The process was not entirely smooth: the AI at one point confidently produced an incorrect foundational formula, sidestepping the real physics calculation, and it was only after repeated questioning and hands-on checking by Schwartz that the genuine derivation emerged. Schwartz’s own assessment was that, by the end of 2025, this generation of large

Table 1.3: Several Typical Ways AI Is Being Brought into Physics Research

Direction	Roughly what it involves
Numerical AI	Using neural networks to directly learn solutions of partial differential equations (e.g., physics-informed neural networks, PINNs; neural operators), replacing or accelerating traditional numerical methods.
Symbolic AI	Automatically discovering closed-form physical laws from data (symbolic regression), rather than merely fitting a “black-box” numerical model.
Formal AI	Using proof assistants such as Lean 4 to turn every step of a physical derivation into a rigorous proof that a computer can check line by line.
Research-agent AI	Having large models, under human supervision, take on concrete research work such as computation, coding, and debugging — see the real example below.
Scientific foundation models	Large models pretrained on vast quantities of scientific data (such as protein structures); AlphaFold is the most famous example.
Autonomous-scientist AI	A longer-term vision: AI autonomously proposes hypotheses, designs and (with robotic assistance) carries out experiments, and analyzes the results, closing the loop.

models had computational ability roughly comparable to that of a **second-year graduate student** — capable of carrying out complex calculations, but still lacking the independent research “taste” needed to judge whether a result is actually correct. This assessment may be, at present, the most honest summary of where AI stands in physics research.

The point of this half of the story is this: the machine-learning concepts you are about to understand through statistical physics are not merely a convenient mnemonic exercise — AI is genuinely, actively entering physicists’ everyday work. And whatever role AI ends up playing in physics research in the future, the person who can see through what it is “actually doing mathematically” will always have more agency than someone who can only call on it. Cultivating exactly that ability is what this entire book is trying to do.

What This Course Wants to Teach You

Having come this far, perhaps it is worth stating something bigger, quite plainly: the real goal of this course is not to turn you into someone who “knows how to call libraries and write code for machine learning” (although understanding this mathematics will make writing code considerably

easier), nor is it merely a broad-strokes “introduction to artificial intelligence.” What it wants to do is more specific, and more ambitious: **to cultivate a way of thinking that sees “learning” through a physicist’s eyes.**

When you see a loss function in the six chapters that follow, I hope you will also see an energy function; when you see the strength of regularization during training, I hope you will also think of temperature; when you ask why a model overfits, I hope you will think of the competition between energy and entropy inside free energy. This double vision is not meant as a showy exercise in drawing analogies everywhere — it is because, as Chapter 7 will prove at its close, these concepts are, in their mathematical structure, literally the same object to begin with. What physics offers for understanding machine learning is not a convenient mnemonic metaphor, but an entire toolkit of mathematics that has already been tested and matured far earlier.

Exercises for This Chapter

Most of the following questions (with the exception of Question 4) require no calculation; they are simply meant to help you organize the material of this chapter for yourself before moving formally into Chapter 2.

1. Beyond the examples listed in this chapter, what other artificial-intelligence applications can you think of that fit the “input $\rightarrow$ model $\rightarrow$ output” structure? Try to identify explicitly what the input and output are in each case.
2. Why do “rule-based methods” (such as traditional chess programs) not need the concept of “training,” while machine-learning methods cannot do without it?
3. This chapter mentioned that statistical physics deals with problems of the type “how do macroscopic regularities arise from a large number of microscopic degrees of freedom.” Can you think of an everyday analogue (not necessarily related to artificial intelligence)?
4. In the worked example in §1.1, if the two ReLU neurons in the hidden layer were replaced by two identity functions (i.e., no nonlinear transformation at all), what would the network degenerate into? Try to re-derive, without looking at the main text, the process by which it “collapses” into a linear function.
5. Look back at Table 1.2: which chapter’s question currently seems the hardest for you to imagine physics being able to help with? Keep this question in mind as you read on, and see whether it has been answered by the time you finish the six chapters.

Chapter 2

Starting from Classical Statistical Physics

Though this be madness, yet there is method in't.

— Shakespeare, *Hamlet*, Act II, Scene 2, spoken by Polonius

Objectives

- Starting from the principle of equal a priori probabilities, understand the combinatorial origin of the Boltzmann distribution and the partition function;
- Master the complete derivation of the canonical ensemble through concrete numerical examples that can be checked by hand;
- Systematically show how the partition function gives rise to thermodynamic quantities such as average energy, free energy, entropy, and heat capacity;
- Apply this framework to the one-dimensional Ising model, obtain an exact solution for an interacting system, and establish a first correspondence with machine-learning models.

■ Chapter Guide

The question this chapter answers: Where does the mathematical structure behind concepts that recur throughout machine learning — “loss function,” “normalization,” “temperature parameter” — actually come from? This chapter presupposes no physics background whatsoever, and derives this mathematical structure from scratch, starting from the most elementary combinatorial counting.

Correspondence with statistical physics: This chapter builds the core language of statistical physics itself — microstates, energy, the Boltzmann distribution, the partition function, free energy, the one-dimensional Ising model — which forms the shared vocabulary

for the five chapters that follow; every other chapter will keep returning to the notation defined here.

Looking ahead: Chapter 3 will show that “parameter estimation” in machine learning follows a chain of logic that is almost a mirror image of this chapter’s; Chapter 4 will prove that this chapter’s energy function is the loss function itself; Chapter 7 will re-derive the Boltzmann distribution obtained in this chapter from a purely information-theoretic point of view — an entirely independent derivation path from the physical intuition used here.

The goal of this chapter is to let the reader understand the physical and mathematical origin of core concepts such as the Boltzmann distribution and the partition function without any prior grounding in statistical physics — rather than committing them to memory as established conclusions. Every derivation starts from a simple model that can be solved completely with nothing more than combinatorial counting, unfolds step by step through a sequence of concrete numerical examples the reader can check independently, and finally arrives at the one-dimensional Ising model — the standard example, in statistical physics, of how to handle an interacting system.

2.1 The Concentration of Macroscopic Probability

Before formally introducing physical concepts such as energy and temperature, this section first discusses a problem that is purely combinatorial — it involves no physical content whatsoever, yet it is the foundation of the entire theory of statistical physics.

The Basic Model

Consider a container divided into a left half and a right half by an imaginary partition, containing N particles. Each particle is independently and equally likely to be in the left half or the right half (probability $1/2$ each, as if flipping a fair coin).

If we are only interested in the macroscopic quantity k , the number of particles in the left half, without distinguishing which specific particles are on the left, what is the probability distribution over the different values k can take?

Here we need to carefully distinguish two levels of “state,” a distinction that will run through the whole of statistical physics:

- **Microstate:** a precise specification of where every single particle is (left or right). For example, when $N = 4$, “particles 1 and 2 on the left, particles 3 and 4 on the right” constitutes one microstate. The N particles have 2^N microstates in total, and since each particle’s left/right choice is independent and equally likely, these microstates are naturally equally probable.
- **Macrostate:** specifies only the total number of particles in the left half, without distinguishing which particular ones. For example, “exactly 2 particles on the left” is one macrostate, corresponding to multiple microstates.

Let $\Omega(k)$ denote the number of microstates corresponding to macrostate k . Since this is equivalent to choosing k out of N particles to place on the left, with the rest remaining on the right,

$$\Omega(k) = \mathcal{C}_N^k \equiv \frac{N!}{k!(N-k)!}, \quad (2.1)$$

the corresponding probability is

$$P(k) = \frac{\Omega(k)}{2^N}. \quad (2.2)$$

A Worked Example: The Case $N = 4$

When $N = 4$, there are $2^4 = 16$ microstates in total, which can be listed one by one (using L and R to denote whether a given particle is on the left or the right):

Table 2.1: Number of Microstates and Probability for Each Macrostate at $N = 4$

Particles on the left k	Number of microstates $\Omega(k) = \mathcal{C}_4^k$	Probability $P(k)$
0 (RRRR)	1	0.0625
1 (LRRR, RLRR, RRLR, RRRL)	4	0.2500
2	6	0.3750
3	4	0.2500
4 (LLLL)	1	0.0625

Check on the total: $1 + 4 + 6 + 4 + 1 = 16 = 2^4$, as expected.

This reveals a key phenomenon: the macrostate “exactly 2 particles on the left” occurs with a probability far higher than that of “all 0 on the left” or “all 4 on the left,” even though every individual microstate has exactly the same probability (each $1/16$). This is not because some physical law favors a “uniform” distribution; it is because the number of microstates realizing the “uniform” macrostate is far greater than the number realizing the macrostate “all particles clustered on one side.”

The Concentration Effect in the Large- N Limit

As N grows, this effect becomes even more pronounced. Table 2.2 and Table 2.3 give the results for $N = 10$ and $N = 20$, respectively (readers may verify any single column for themselves):

Table 2.2: Microstate Counts and Probabilities at $N = 10$

k	0	1	2	3	4	5	6	7	8	9	10
$\Omega(k)$	1	10	45	120	210	252	210	120	45	10	1
$P(k)$	0.001	0.010	0.044	0.117	0.205	0.246	0.205	0.117	0.044	0.010	0.001

This shows that, as N increases, the distribution $P(k)$ becomes ever sharper around the most probable macrostate ($k = N/2$), while the probability of extreme macrostates such as “all particles

Table 2.3: Microstate Counts and Probabilities at $N = 20$ (Selected Values; the Full Distribution Is a Symmetric Bell Curve)

k	0	5	8	9	10	11	12	15	20
$\Omega(k)$	1	15504	125970	167960	184756	167960	125970	15504	1
$P(k)$	$\approx 10^{-6}$	0.015	0.120	0.160	0.176	0.160	0.120	0.015	$\approx 10^{-6}$

clustered on one side” rapidly becomes negligible. When $N \sim 10^{23}$ (the order of magnitude of the number of molecules in a real gas), this sharpening reaches astronomical proportions — the event “the gas in a container spontaneously clusters entirely on one side” would not occur even once over a timescale as long as the age of the universe; this is not because some physical law forbids it, but because the number of microstates capable of realizing that macrostate is vanishingly small.

■ Core Intuition

The probability of a macrostate occurring depends on the number of microstates that can realize it. The “certainty” the macroscopic world displays (for instance, a gas always tending toward a uniform distribution) is, at bottom, the outcome of an enormous number of microscopic possibilities competing against one another in a probabilistic sense: the macrostate supported by the majority of microstates is simply what is actually observed.

From Counting States to the Definition of Entropy

Statistical physics provides a formal measure of “the number of microstates corresponding to a given macrostate.” Boltzmann defined **entropy** as

$$S = k_B \ln \Omega, \quad (2.3)$$

where k_B is the Boltzmann constant (in a purely mathematical discussion, it is common to simply set $k_B = 1$, treating it as a unit-conversion factor). The reason for taking a logarithm is as follows: Ω grows extremely fast (already reaching 184,756 at its maximum when $N = 20$, and an astronomical number when $N \sim 10^{23}$), so working with Ω directly is inconvenient; $\ln \Omega$, on the other hand, grows gently and is additive — when two independent subsystems are combined, their numbers of microstates multiply ($\Omega = \Omega_1 \Omega_2$), which becomes addition after taking the logarithm ($S = S_1 + S_2$), exactly matching the physical picture of entropy as an extensive quantity.

■ Think and Check

Suppose a system consists of two independent parts: the first has $\Omega_1 = 100$ microstates, and the second has $\Omega_2 = 1000$ microstates. How many microstates does the system as a whole have? Is the total entropy (with $k_B = 1$) equal to the sum of the two parts’ entropies?

Solution: The total number of microstates is $\Omega = \Omega_1 \Omega_2 = 10^5$; and $\ln(10^5) = \ln 100 + \ln 1000$, so indeed the two are equal.

Up to this point, the discussion has not involved the concept of “energy” at all. We now assign an energy label to each microstate, and examine how the principle of equal a priori probabilities

evolves once the system can exchange energy with its surroundings.

2.2 The Canonical Ensemble

Posing the Problem

Consider the following scenario: a cup of coffee sitting still on a table, containing on the order of 10^{23} molecules, each in motion, each with its own position and velocity. To fully describe the microstate of this cup of coffee would require specifying all 6 degrees of freedom of every molecule (3 position coordinates and 3 momentum components), for a total of 6×10^{23} numbers — clearly an impossible task.

And yet, we do not need to know these microscopic details in order to predict the system’s macroscopic properties accurately — its temperature, pressure, and heat capacity, for example. This is precisely the core method of statistical physics, and the logic revealed by the “counting the boxes” example in §2.1 — that macroscopic certainty arises from a microscopic probabilistic advantage — already provides the basic idea we need.

The core idea by which statistical physics handles this kind of problem is: rather than asking which specific microstate the system is in, ask for the **probability distribution** over the microstates the system could be in.

Setting Up the Canonical Ensemble

In §2.1, a particle being on the left or the right was completely symmetric, with no preference either way — this corresponds, physically, to the simplest case of a system “unconstrained by its surroundings.” Real physical systems, however, typically must exchange energy with their surroundings: a hot cup of coffee continuously loses heat to the surrounding air, and the more violently its molecules move (the higher its energy), the harder that state is to sustain over time. We therefore need to bring energy into the calculation of probability.

Consider a system (the coffee itself, say) in contact with a “heat bath” at temperature T (the large quantity of surrounding air, for instance). The system can exchange energy with the heat bath, but not particles — this is the basic setup of the **canonical ensemble**.

The system may be in any of a great many different microstates. Let $\mathbf{x}$ denote a particular microstate (this could be a configuration of particle positions and momenta, the “each particle on the left or the right” configuration from §2.1, or a configuration of spin orientations), and let every state $\mathbf{x}$ correspond to a definite energy $E(\mathbf{x})$.

The central question this raises is: what form does the probability $P(\mathbf{x})$ that the system is in state $\mathbf{x}$ take? In §2.1, since there was no energy preference, all microstates were equally probable; once energy is introduced, states of lower energy should intuitively be more “common,” but the precise functional form must be obtained through the derivation below, whose starting point is, once again, the very same principle of equal a priori probabilities.

Deriving the Boltzmann Distribution

Basic assumption (the principle of equal a priori probabilities): for an isolated system with fixed total energy E_{total} , all microstates with energy E_{total} occur with equal probability. This is exactly a generalization of the assumption “every microstate is equally probable” from §2.1: an isolated system is simply restricted to one with fixed energy.

Consider the combination “small system + heat bath” (this is an isolated system):

- Let the small system be in state $\mathbf{x}$, with energy $E(\mathbf{x})$;
- The heat bath’s energy is then $E_R = E_{\text{total}} - E(\mathbf{x})$;
- The number of microstates of the heat bath at energy E_R is $\Omega_R(E_R)$.

The key trick here is exactly the same as the “counting the boxes” technique in §2.1: the small system is fixed in one particular microstate $\mathbf{x}$ (only 1 way to realize it), while the heat bath, subject to the energy constraint E_R , still has $\Omega_R(E_R)$ possible microscopic arrangements. By the principle of equal a priori probabilities, every joint microstate of “system + heat bath” is equally probable, and so the probability that the small system is in state $\mathbf{x}$ is proportional to the number of microstates of the heat bath compatible with it:

$$P(\mathbf{x}) \propto \Omega_R(E_{\text{total}} - E(\mathbf{x})). \quad (2.4)$$

This is the same structure as “the probability that there are exactly k particles on the left is proportional to $\Omega(k) = \mathcal{C}_N^k$ ” in §2.1; only the “macroscopic label” has been replaced by “the small system is in state $\mathbf{x}$,” and “the number of compatible microstates” has been replaced by Ω_R on the heat-bath side.

Using the heat bath’s entropy, defined by $S_R = k_B \ln \Omega_R$ (Equation (2.3)), we obtain

$$\Omega_R(E_R) = e^{S_R(E_R)/k_B}.$$

Because the heat bath is far larger than the small system (it can be thought of as all the air in the room, an almost infinite reservoir of energy compared with a single cup of coffee), we can Taylor-expand $S_R(E_R)$ around $E_R = E_{\text{total}}$. Recall the Taylor expansion of a single-variable function,

$$g(a + \delta) = g(a) + \delta g'(a) + \frac{1}{2} \delta^2 g''(a) + \dots,$$

Taking $g = S_R$, $a = E_{\text{total}}$, $\delta = -E(\mathbf{x})$ (since $E_R = E_{\text{total}} - E(\mathbf{x})$), and substituting term by term gives

$$S_R(E_{\text{total}} - E(\mathbf{x})) = S_R(E_{\text{total}}) - E(\mathbf{x}) \left. \frac{\partial S_R}{\partial E_R} \right|_{E_R=E_{\text{total}}} + \frac{1}{2} E(\mathbf{x})^2 \left. \frac{\partial^2 S_R}{\partial E_R^2} \right|_{E_R=E_{\text{total}}} - \dots \quad (2.5)$$

Why can we keep only the first-order term? The first-order term $\partial S_R / \partial E_R$ is an *intensive quantity* — by a thermodynamic relation we are about to use, it equals $1/T$, which does not depend on the size (particle number) of the heat bath, so the first-order term itself is $O(1)$ and cannot be dropped. The second-order term $\partial^2 S_R / \partial E_R^2$ behaves differently, however: readers

will already have learned, in an introductory thermodynamics course, that the heat capacity $C_R = \partial E_R / \partial T$ is an *extensive quantity*, proportional to the number of particles N_R the heat bath contains; differentiating $\partial S_R / \partial E_R = 1/T$ gives

$$\frac{\partial^2 S_R}{\partial E_R^2} = \frac{\partial}{\partial E_R} \left(\frac{1}{T} \right) = -\frac{1}{T^2} \frac{\partial T}{\partial E_R} = -\frac{1}{T^2 C_R},$$

and since $C_R \propto N_R$, this term is a small quantity of order $O(1/N_R)$ (this implicitly relies on a few conditions that are usually automatically satisfied, but worth stating explicitly: the heat bath is in an ordinary thermodynamic phase, its heat capacity grows linearly with particle number as an extensive quantity should, and the system is not near a critical point — near a critical point, fluctuations grow anomalously, the heat capacity’s dependence on N_R departs from simple linear scaling, and the second-order term may no longer be negligible). The entire second-order term in Equation (2.5) is therefore

$$\frac{1}{2} E(\mathbf{x})^2 \frac{\partial^2 S_R}{\partial E_R^2} \sim O\left(\frac{E(\mathbf{x})^2}{N_R}\right),$$

which vanishes strictly in the limit where the heat bath becomes infinitely large ($N_R \rightarrow \infty$, while the small system’s energy $E(\mathbf{x})$ remains finite), and so can be dropped; by the same reasoning, third- and higher-order terms decay even faster. This is exactly what the concept of a “heat bath” means: it must be large enough that exchanging a small amount of energy with the small system does not change its own temperature (i.e., the first-order term $1/T$ stays fixed) — and this is precisely why we only need to keep the Taylor expansion to first order.

Using the thermodynamic relation $\partial S / \partial E = 1/T$ (T being the temperature), we have

$$S_R(E_{\text{total}} - E(\mathbf{x})) \approx S_R(E_{\text{total}}) - \frac{E(\mathbf{x})}{T}.$$

Substituting into the probability expression (2.4):

$$P(\mathbf{x}) \propto e^{S_R(E_{\text{total}})/k_B} \cdot e^{-E(\mathbf{x})/(k_B T)}.$$

The first factor is the same for every state (it does not depend on $\mathbf{x}$, only on the heat bath’s total energy) and can be absorbed into the normalization constant. Defining the inverse temperature $\beta \equiv 1/(k_B T)$, we finally obtain

$$\boxed{P(\mathbf{x}) = \frac{1}{Z} e^{-\beta E(\mathbf{x})}} \quad (2.6)$$

This is the **Boltzmann distribution**: the lower a microstate’s energy, the greater its probability; every additional increment of $k_B T$ in energy reduces the probability by roughly a factor of $1/e$. The temperature parameter controls how quickly this decay proceeds — the higher the temperature, the more weakly the equilibrium distribution suppresses high-energy states; the lower the temperature, the more the equilibrium distribution concentrates around the state of lowest energy.

Strictly speaking, the above is a property of the **equilibrium distribution** itself, and is not directly a statement about dynamics: claims like “the system finds it easier to explore high-energy regions” implicitly assume some specific stochastic dynamical process (such as Langevin

dynamics, the Metropolis algorithm, or Glauber dynamics); the equilibrium distribution $P(\mathbf{x})$ alone cannot, by itself, tell us the specific “rate of exploration.” Nonetheless, in these common sampling algorithms, raising the temperature does indeed tend to make high-energy regions easier to access, which is why this intuition gets borrowed repeatedly, both for the “temperature parameter” in §4.4 and for “simulated annealing” in later chapters.

This is mathematically completely parallel to the phenomenon in §2.1 that “the larger N is, the sharper the distribution becomes”: here, $1/T$ (the inverse temperature) plays the role that N (the particle number) played before.

■ Clearing Up a Question

Why does probability decay exponentially with energy, rather than in some other functional form? The answer is contained in the derivation above: entropy is a quantity on a logarithmic scale ($S = k_B \ln \Omega$), and once the thermodynamic relation “the derivative of entropy with respect to energy equals $1/T$ ” is exponentiated, it naturally converts a linear dependence on energy into an exponential one. This is not an arbitrary convention; it is the inevitable consequence of the principle of equal a priori probabilities combined with the basic relations of thermodynamics.

The Partition Function and Its Physical Meaning

The normalization constant Z is called the **partition function**:

$$Z = \sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})} \quad (2.7)$$

For a continuous system, the sum is replaced by an integral accordingly:

$$Z = \int e^{-\beta E(\mathbf{x})} d\mathbf{x}.$$

The term “partition” comes from the fact that this function describes how energy weight is “partitioned” among the different microstates: $e^{-\beta E(\mathbf{x})}$ can be understood as the share of weight allotted to state $\mathbf{x}$, and Z is simply the sum of the weights of all states. This also clarifies the connection between the “counting the boxes” example of §2.1 and the present discussion: if every state has the same energy ($E(\mathbf{x}) = 0$), then $Z = \sum_{\mathbf{x}} 1$ is simply the total number of microstates, and we recover the purely combinatorial counting problem we started with. Once energy is introduced, Z becomes a kind of **weighted count**: each state is no longer simply counted as 1, but weighted by its own Boltzmann factor $e^{-\beta E(\mathbf{x})}$.

■ History: From Boltzmann’s Counting to Gibbs’s Partition Function

The order followed in this section — first doing purely combinatorial counting (§2.1), then introducing energy and a heat bath to obtain the Boltzmann distribution — is not an arbitrary pedagogical choice made by this book; it roughly retraces the actual historical order of discovery. By the 1870s, Boltzmann had already used combinatorial counting to establish a distribution $\propto e^{-\alpha E}$ for a dilute gas, and had connected entropy to the logarithm

of the number of microstates (which he called the H -function). But it was J. Willard Gibbs, in his 1902 book *Elementary Principles in Statistical Mechanics* (Yale University), who systematized this picture into the general framework of “a system in contact with a heat bath,” and who explicitly defined the partition function Z itself as an independent object of study. It was in that book that the **ensemble** — a large number of hypothetical copies of a system, each in a different microstate — was first used systematically to define the partition function, from which every thermodynamic quantity could then be derived; this constitutes the original version of this section’s narrative, “the partition function as a generating function.” This is also why the partition function is denoted Z in much of the literature — from the German *Zustandssumme*, literally “sum over states.”

The partition function carries a dual physical meaning: first, as a normalization constant, it guarantees $\sum_{\mathbf{x}} P(\mathbf{x}) = 1$; second, as a generating function, every thermodynamic quantity can be derived from it (as shown below). We first work through the simplest possible concrete example to make the calculation of the Boltzmann distribution and the partition function fully explicit, before deriving the thermodynamic quantities in the general case.

A Worked Example: The Two-Level System

Consider a system with only two microstates — an atom with just a ground state and a single excited state, say, or a molecular switch with only “on” and “off” configurations. Let:

- State 0 (ground state): energy $E_0 = 0$;
- State 1 (excited state): energy $E_1 = \varepsilon$ ($\varepsilon > 0$).

The partition function is the sum over the two states:

$$Z = e^{-\beta E_0} + e^{-\beta E_1} = 1 + e^{-\beta \varepsilon}. \quad (2.8)$$

The probability of each state is

$$P_0 = \frac{1}{Z} = \frac{1}{1 + e^{-\beta \varepsilon}}, \quad P_1 = \frac{e^{-\beta \varepsilon}}{Z} = \frac{e^{-\beta \varepsilon}}{1 + e^{-\beta \varepsilon}}. \quad (2.9)$$

Taking $\varepsilon = 1$ (that is, using the excitation energy itself as the unit of energy), the specific values obtained by varying the temperature $k_B T$ are listed in Table 2.4 (where $\langle E \rangle = P_1 \varepsilon$ is the system’s average energy):

Taking the row $k_B T = 1$ as an example: $Z = 1 + e^{-1} = 1 + 0.3679 = 1.3679$, $P_1 = 0.3679/1.3679 = 0.2689$, matching the value in the table.

Table 2.4 reveals two limits that will recur repeatedly:

- **Low-temperature limit** ($k_B T \rightarrow 0$, i.e., $\beta \rightarrow \infty$): $P_0 \rightarrow 1$, $P_1 \rightarrow 0$; the system is almost certain to be in the ground state, its lowest-energy state. This is a direct manifestation of the phenomenon in §2.1 that the distribution sharpens and suppresses extreme macrostates as the relevant parameter grows — except that here the direction of “extremity” is reversed: at low temperature, the system stops precisely at the extreme state of lowest energy, because the heat bath no longer supplies enough energy to let the system reach the excited state.

Table 2.4: Numerical Results for the Two-Level System ($\varepsilon = 1$) at Different Temperatures

$k_B T$	$\beta = 1/(k_B T)$	Z	P_0 (ground-state prob.)	P_1 (excited-state prob.)	$\langle E \rangle$
0.2	5.00	1.007	0.993	0.007	0.007
0.5	2.00	1.135	0.881	0.119	0.119
1.0	1.00	1.368	0.731	0.269	0.269
2.0	0.50	1.607	0.622	0.378	0.378
5.0	0.20	1.819	0.550	0.450	0.450
10.0	0.10	1.905	0.525	0.475	0.475

- **High-temperature limit** ($k_B T \rightarrow \infty$, i.e., $\beta \rightarrow 0$): $e^{-\beta\varepsilon} \rightarrow 1$, so $P_0 \rightarrow 1/2$, $P_1 \rightarrow 1/2$; the two states tend toward equal probability — reproducing exactly the situation in §2.1 in which “with no energy preference, every microstate is equally probable”: high temperature effectively washes out the effect of energy.

■ Think and Check

If the excited-state energy is changed to $\varepsilon = 5$ (a larger level spacing), at the same $k_B T = 1$, should P_1 increase or decrease relative to its value of 0.269 at $\varepsilon = 1$?

Solution: A larger level spacing relative to $k_B T$ means a relatively higher energetic cost to transition to the excited state, so P_1 should decrease. Substituting $\beta\varepsilon = 5$: $P_1 = e^{-5}/(1 + e^{-5}) \approx 0.0067$, indeed dropping sharply.

Simple as its structure is, this two-level system is the physical prototype for logistic regression and binary classification, discussed later in the book: a system with only two possible “labels,” whose label probabilities are determined by exactly the Boltzmann factor of some energy difference. This thread will be picked up again in the book’s later chapter on the softmax function.

Deriving Thermodynamic Quantities from the Partition Function

Let us now return to the general case and examine how the partition function Z “generates” all the macroscopic quantities of interest. To keep the derivation concrete, every new formula will immediately be checked numerically against the two-level system above. Before doing so, let us formally define two notations we will use repeatedly in this section — **expectation value** and **variance**.

■ Preliminary Definitions: Expectation Value and Variance

For a system taking values in a set of states $\{\mathbf{x}\}$ and governed by a probability distribution $P(\mathbf{x})$, the **expectation value** (also called the “average,” often written $\langle A \rangle$ in physics) of any state-dependent physical quantity $A(\mathbf{x})$ (for example, the energy $E(\mathbf{x})$) is defined as the probability-weighted sum of that quantity over all states:

$$\langle A \rangle \equiv \sum_{\mathbf{x}} P(\mathbf{x}) A(\mathbf{x}).$$

In particular, taking $A(\mathbf{x}) = E(\mathbf{x})$ gives the average energy $\langle E \rangle$ in (1) below; taking $A(\mathbf{x}) = E(\mathbf{x})^2$ gives what we write as $\langle E^2 \rangle = \sum_{\mathbf{x}} P(\mathbf{x}) E(\mathbf{x})^2$ — the expectation value of the quantity “energy squared” (not the square of the average energy).

The **variance** of a quantity A is defined as the mean squared deviation from its own average, characterizing how strongly it fluctuates:

$$\text{Var}(A) \equiv \langle (A - \langle A \rangle)^2 \rangle.$$

This definition has a more computationally convenient equivalent form, $\text{Var}(A) = \langle A^2 \rangle - \langle A \rangle^2$, which we now prove (simply treat $\langle A \rangle$ as an ordinary constant and expand the square):

$$\begin{aligned} \text{Var}(A) &= \langle A^2 - 2A\langle A \rangle + \langle A \rangle^2 \rangle \\ &= \langle A^2 \rangle - 2\langle A \rangle \cdot \langle A \rangle + \langle A \rangle^2 \quad (\text{since } \langle A \rangle \text{ is a constant and can be pulled out of the sum}) \\ &= \langle A^2 \rangle - \langle A \rangle^2. \end{aligned}$$

This section will use this equivalent relation directly in deriving the heat capacity in (5).

(1) Average energy. The system’s average energy is

$$\langle E \rangle = \sum_{\mathbf{x}} P(\mathbf{x}) E(\mathbf{x}) = \frac{1}{Z} \sum_{\mathbf{x}} E(\mathbf{x}) e^{-\beta E(\mathbf{x})}.$$

Notice that differentiating with respect to β automatically produces a factor of $E(\mathbf{x})$:

$$\frac{\partial}{\partial \beta} e^{-\beta E(\mathbf{x})} = -E(\mathbf{x}) e^{-\beta E(\mathbf{x})},$$

and therefore

$$\sum_{\mathbf{x}} E(\mathbf{x}) e^{-\beta E(\mathbf{x})} = -\frac{\partial}{\partial \beta} \sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})} = -\frac{\partial Z}{\partial \beta},$$

from which

$$\langle E \rangle = -\frac{1}{Z} \frac{\partial Z}{\partial \beta} = -\frac{\partial \ln Z}{\partial \beta}. \quad (2.10)$$

This is a compact result: the average energy equals minus the derivative of the log-partition function with respect to β . All one needs is Z itself (no knowledge of any individual state’s probability is required); taking its logarithm and differentiating immediately yields the average energy — this is exactly the essence of the “generating function” method: Z is, in effect, a compressed archive, and differentiating its logarithm is how you “unpack” it.

■ Numerical Check

In the two-level system, $\ln Z = \ln(1 + e^{-\beta \varepsilon})$, and differentiating gives

$$-\frac{\partial \ln Z}{\partial \beta} = \frac{\varepsilon e^{-\beta \varepsilon}}{1 + e^{-\beta \varepsilon}} = \varepsilon P_1,$$

which agrees exactly with the directly computed $\langle E \rangle = P_1 \varepsilon$ (since $E_0 = 0$ contributes

nothing).

(2) Free energy. Define the **Helmholtz free energy**

$$F = -k_B T \ln Z = -\frac{1}{\beta} \ln Z. \quad (2.11)$$

We now use the definition of the free energy to rewrite the average energy in terms of entropy, listing every step of the algebra individually.

Step 1: express $\ln Z$ in terms of F . From the definition (2.11), $F = -\frac{1}{\beta} \ln Z$; multiplying both sides by $-\beta$ gives

$$\ln Z = -\beta F.$$

Step 2: substitute into the average-energy formula (2.10).

$$\langle E \rangle = -\frac{\partial \ln Z}{\partial \beta} = -\frac{\partial(-\beta F)}{\partial \beta} = \frac{\partial(\beta F)}{\partial \beta}.$$

Step 3: differentiate βF with respect to β , using the product rule (treating $F = F(T(\beta))$ as a function of β):

$$\frac{\partial(\beta F)}{\partial \beta} = \underbrace{\frac{\partial \beta}{\partial \beta}}_{=1} \cdot F + \beta \cdot \frac{\partial F}{\partial \beta} = F + \beta \frac{\partial F}{\partial \beta}.$$

Therefore

$$\langle E \rangle = F + \beta \frac{\partial F}{\partial \beta}. \quad (2.12)$$

Step 4: convert $\partial F/\partial \beta$ to $\partial F/\partial T$. By the chain rule,

$$\frac{\partial F}{\partial \beta} = \frac{\partial F}{\partial T} \cdot \frac{\partial T}{\partial \beta}.$$

Here $\partial T/\partial \beta$ must be obtained from $\beta = 1/(k_B T)$. Differentiating $\beta(T)$ directly:

$$\frac{\partial \beta}{\partial T} = \frac{\partial}{\partial T} \left(\frac{1}{k_B T} \right) = -\frac{1}{k_B T^2}.$$

Then, using the **inverse function rule** (if $y = f(x)$ is invertible, then $dx/dy = 1/(dy/dx)$):

$$\frac{\partial T}{\partial \beta} = \frac{1}{\partial \beta / \partial T} = \frac{1}{-1/(k_B T^2)} = -k_B T^2.$$

(Under the mathematical convention $k_B = 1$, this is simply $\partial T/\partial \beta = -T^2$.) Substituting into the chain rule:

$$\frac{\partial F}{\partial \beta} = -k_B T^2 \frac{\partial F}{\partial T}. \quad (2.13)$$

Step 5: substitute Equation (2.13) into Equation (2.12), and simplify using $\beta = 1/(k_B T)$:

$$\langle E \rangle = F + \beta \left(-k_B T^2 \frac{\partial F}{\partial T} \right) = F - \beta k_B T^2 \frac{\partial F}{\partial T} = F - \underbrace{(\beta k_B T)}_{=1} T \frac{\partial F}{\partial T},$$

finally giving (independent of the specific value of k_B):

$$\langle E \rangle = F - T \frac{\partial F}{\partial T}.$$

(3) Entropy. In thermodynamics, entropy is defined by $S = -\partial F / \partial T$, so $\langle E \rangle = F + TS$, which rearranges to

$$F = \langle E \rangle - TS. \quad (2.14)$$

This relation shows that free energy is the outcome of a competition between energy and entropy: at low temperature ($T \rightarrow 0$), $F \approx \langle E \rangle$, and the system tends to minimize energy; at high temperature ($T \rightarrow \infty$), the entropy term dominates, and the system tends to maximize entropy (disorder).

■ Numerical Check

For the two-level system at $k_B T = 1$ ($\beta = 1$), $\ln Z = \ln 1.3679 = 0.3133$, so $F = -k_B T \ln Z = -0.3133$; and $\langle E \rangle = 0.2689$, so from Equation (2.14) the entropy is $S = (\langle E \rangle - F)/T = (0.2689 - (-0.3133))/1 = 0.5822$ (in units of k_B). This value lies between “the system is entirely confined to the ground state” ($S = 0$) and the maximum entropy $S = \ln 2 \approx 0.693$ corresponding to “both states equally probable” — consistent with the physical picture of a system that is partly determined and partly uncertain at an intermediate temperature.

(4) The statistical expression for entropy. Entropy can also be computed directly from the probability distribution:

$$S = -k_B \sum_{\mathbf{x}} P(\mathbf{x}) \ln P(\mathbf{x}). \quad (2.15)$$

This is a generalization of Equation (2.3) ($S = k_B \ln \Omega$): if all Ω states are equally probable, $P(\mathbf{x}) = 1/\Omega$, and substituting into the above recovers $S = k_B \ln \Omega$ (readers may check this for themselves). Once energy is introduced and the probabilities are no longer uniform, this “information-entropy” expression continues to hold, and it is exactly the central object in the book’s later discussion of the maximum entropy principle.

Substituting the Boltzmann distribution (2.6):

$$\begin{aligned} S &= -k_B \sum_{\mathbf{x}} P(\mathbf{x}) [-\beta E(\mathbf{x}) - \ln Z] \\ &= k_B \beta \sum_{\mathbf{x}} P(\mathbf{x}) E(\mathbf{x}) + k_B \ln Z \sum_{\mathbf{x}} P(\mathbf{x}) \\ &= k_B \beta \langle E \rangle + k_B \ln Z \\ &= \frac{\langle E \rangle}{T} + k_B \ln Z. \end{aligned}$$

Since $F = -k_B T \ln Z$ implies $k_B \ln Z = -F/T$, we have

$$S = \frac{\langle E \rangle - F}{T},$$

which precisely confirms Equation (2.14).

(5) Heat capacity. The heat capacity is defined as

$$C = \frac{\partial \langle E \rangle}{\partial T}.$$

Using Equation (2.10) and the chain rule:

$$C = \frac{\partial \langle E \rangle}{\partial \beta} \frac{\partial \beta}{\partial T} = -\frac{1}{k_B T^2} \frac{\partial \langle E \rangle}{\partial \beta}.$$

where

$$\frac{\partial \langle E \rangle}{\partial \beta} = -\frac{\partial^2 \ln Z}{\partial \beta^2},$$

and

$$\begin{aligned} \frac{\partial^2 \ln Z}{\partial \beta^2} &= \frac{\partial}{\partial \beta} \left(\frac{1}{Z} \frac{\partial Z}{\partial \beta} \right) \\ &= \frac{1}{Z} \frac{\partial^2 Z}{\partial \beta^2} - \frac{1}{Z^2} \left(\frac{\partial Z}{\partial \beta} \right)^2. \end{aligned}$$

Using

$$\frac{\partial Z}{\partial \beta} = -\sum_{\mathbf{x}} E(\mathbf{x}) e^{-\beta E(\mathbf{x})}, \quad \frac{\partial^2 Z}{\partial \beta^2} = \sum_{\mathbf{x}} E^2(\mathbf{x}) e^{-\beta E(\mathbf{x})},$$

we obtain

$$\begin{aligned} \frac{\partial^2 \ln Z}{\partial \beta^2} &= \frac{1}{Z} \sum_{\mathbf{x}} E^2(\mathbf{x}) e^{-\beta E(\mathbf{x})} - \left(\frac{1}{Z} \sum_{\mathbf{x}} E(\mathbf{x}) e^{-\beta E(\mathbf{x})} \right)^2 \\ &= \langle E^2 \rangle - \langle E \rangle^2 = \text{Var}(E). \end{aligned}$$

Therefore

$$\boxed{C = \frac{1}{k_B T^2} \text{Var}(E)}. \quad (2.16)$$

This result shows that the heat capacity is proportional to the variance of the energy fluctuations — an important conclusion in statistical physics: a macroscopic response function (heat capacity) is directly tied to microscopic fluctuations (the variance of energy). This is a concrete instance of the **fluctuation–dissipation theorem**.

■ Numerical Check

For the two-level system, E^2 takes only the value 0 (with probability P_0) or ε^2 (with probability P_1), so $\langle E^2 \rangle = P_1 \varepsilon^2$, while $\langle E \rangle^2 = (P_1 \varepsilon)^2$. At $k_B T = 1$, $\varepsilon = 1$, $\text{Var}(E) = P_1 - P_1^2 =$

$P_1(1 - P_1) = 0.2689 \times 0.7311 = 0.1966$, so $C = \text{Var}(E)/T^2 = 0.1966$ (here $T = 1$, in units of k_B). One can check that $\text{Var}(E) = P_0 P_1 \epsilon^2$ takes exactly the form of the variance formula $p(1 - p)$ for a Bernoulli distribution — this is no coincidence: the two-level system is, at its core, a single Bernoulli trial in which “temperature determines the probability of success.”

■ Summary

Once the partition function $Z(\beta) = \sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})}$ has been written down, the average energy follows from a first derivative of $\ln Z$, the heat capacity follows from its second derivative (expressed in the form of a variance), and the free energy follows from $F = -k_B T \ln Z$, from which the entropy can then be derived in turn. This is the whole content of the statement “the partition function is a generating function.”

2.3 The One-Dimensional Ising Model

The two-level system, though intuitive, has an important limitation: it contains only a single particle, with no interaction between particles. What makes real systems genuinely interesting — and what makes the neural networks and Markov random fields discussed later in the book interesting — is precisely the presence of interactions among a large number of units. To make the abstract concepts above concrete in a setting with interactions, this section examines the one-dimensional Ising model in detail — the most classic, simplest model in statistical physics for handling interacting systems, and also the direct prototype for the Hopfield networks and Boltzmann machines discussed later.

Definition of the Model

Consider N spins arranged in a chain, each spin s_i taking one of two values, up ($s_i = +1$) or down ($s_i = -1$) — this can be viewed as a generalization of the “two boxes, left and right” model in §2.1: neighboring spins will interact in a way that favors aligning in the same direction, and there is an external magnetic field acting on every spin. The system’s energy function is

$$E(\{s_i\}) = -J \sum_{i=1}^{N-1} s_i s_{i+1} - h \sum_{i=1}^N s_i, \quad (2.17)$$

where $J > 0$ is the interaction strength (ferromagnetic coupling: pairs of aligned spins contribute negative energy, making them more stable), and h is the strength of the external magnetic field ($h > 0$ favors spins pointing up).

The Physical Meaning of the Energy Function

A few typical configurations help illustrate how this energy function behaves.

Case one: all spins up ($s_i = +1, \forall i$):

$$E = -J(N - 1) - hN,$$

when $J, h > 0$, this is one of the lowest-energy (most stable) configurations.

Case two: all spins down ($s_i = -1, \forall i$):

$$E = -J(N-1) + hN,$$

when $h = 0$, this configuration has the same energy as case one, reflecting the system's symmetry.

Case three: complete disorder (spins take random values): On average, the probability that neighboring spins are aligned is about $1/2$, so

$$\langle s_i s_{i+1} \rangle \approx 0, \quad \langle s_i \rangle \approx 0, \quad \langle E \rangle \approx 0.$$

Case four: a single spin flip: Suppose the k -th spin has $s_k = -1$ ($1 < k < N$, i.e., not at either end of the chain), with every other spin equal to $+1$. We now compute the energy of this configuration step by step.

Step 1: count the interaction term $\sum_i s_i s_{i+1}$. The whole chain has $N-1$ neighboring bonds in total. Of these:

- The two bonds directly connected to the k -th spin, (s_{k-1}, s_k) and (s_k, s_{k+1}) , each have one end equal to $+1$ and the other equal to -1 (since only s_k is flipped), so each bond contributes $s_i s_{i+1} = (+1)(-1) = -1$; together the two bonds contribute $2 \times (-1) = -2$;
- The remaining $(N-1)-2 = N-3$ bonds have both ends equal to $+1$ (unaffected by the flip), each contributing $s_i s_{i+1} = (+1)(+1) = +1$, for a total contribution of $(N-3) \times (+1) = N-3$.

Therefore

$$\sum_{i=1}^{N-1} s_i s_{i+1} = (N-3) + 2 \times (-1) = N-3-2 = N-5.$$

Step 2: count the external-field term $\sum_i s_i$. Apart from the k -th spin, which is -1 , the remaining $N-1$ spins are all $+1$, so

$$\sum_{i=1}^N s_i = (N-1) \times (+1) + 1 \times (-1) = N-2.$$

Step 3: substitute into the energy function (2.17):

$$E = -J \sum_i s_i s_{i+1} - h \sum_i s_i = -J(N-5) - h(N-2) = -JN + 5J - hN + 2h.$$

Rewriting this in a form that is easy to compare with the “all spins up” case ($E_{\uparrow\uparrow\ldots\uparrow} = -J(N-1) - hN = -JN + J - hN$):

$$E = [-JN + J - hN] + 4J + 2h = -J(N-1) + 4J - hN + 2h.$$

Compared with the all-up configuration, the energy has increased by

$$\Delta E = E - E_{\uparrow\uparrow\ldots\uparrow} = 4J + 2h,$$

and this is exactly the energetic cost of creating a single “domain wall”: $4J$ comes from the two ferromagnetic bonds that are “broken” (each bond costs $2J$, since it goes from $-J$ to $+J$), and $2h$ comes from the Zeeman-energy cost of a single spin pointing against the external field (going from $-h$ to $+h$).

The Ising Chain at $N = 2$

Before treating general N , let us first, following the same approach used in §2.1 and for the two-level system, completely solve the case $N = 2$ (only two spins, one interaction “bond”). Take $h = 0$, so the energy simplifies to $E = -Js_1s_2$. There are only $2^2 = 4$ microstates, which we can list one by one:

Table 2.5: Microstates and Boltzmann Weights for the $N = 2$ Ising Chain ($h = 0$)

State (s_1, s_2)	Energy $E = -Js_1s_2$	Boltzmann weight $e^{-\beta E}$
$(+1, +1)$	$-J$	$e^{\beta J}$
$(+1, -1)$	$+J$	$e^{-\beta J}$
$(-1, +1)$	$+J$	$e^{-\beta J}$
$(-1, -1)$	$-J$	$e^{\beta J}$

The partition function is the sum of the four weights:

$$Z = 2e^{\beta J} + 2e^{-\beta J} = 4 \cosh(\beta J). \quad (2.18)$$

Taking $J = 1$, the specific values (Table 2.6) are:

Table 2.6: Numerical Results for the $N = 2$ Ising Chain ($J = 1$) at Different Temperatures

$k_B T$	βJ	Weights $(++, +-, -+, --)$	$Z = 4 \cosh(\beta J)$	Probabilities $(++, +-, -+, --)$
0.2	5.0	(148.41, 0.0067, 0.0067, 148.41)	296.84	(0.500, 0.000, 0.000, 0.500)
1.0	1.0	(2.718, 0.368, 0.368, 2.718)	6.172	(0.440, 0.060, 0.060, 0.440)
5.0	0.2	(1.221, 0.819, 0.819, 1.221)	4.080	(0.299, 0.201, 0.201, 0.299)

Table 2.6 reveals the following physical picture:

- **Low temperature** ($k_B T = 0.2$): the two “aligned” states $(++)$ and $(--)$ take up almost the entire probability (roughly 50% each), while the “anti-aligned” states are strongly suppressed — a direct, strict manifestation, at low temperature, of the rule that $J > 0$ favors neighboring spins aligning in the same direction.
- **High temperature** ($k_B T = 5.0$): the probabilities of the four states gradually approach $1/4$ (a uniform distribution); the preference imposed by the interaction is once again “washed out” by high temperature, the same mechanism by which the two-level system tends toward equal probability at high temperature.
- The intermediate temperature ($k_B T = 1.0$) lies in a crossover region between the two.

■ Think and Check

Can you verify that, at every temperature in the table above, $P(++) = P(--)$ and $P(+-) = P(-+)$? What symmetry of the system does this property come from?

Solution: Flipping every spin simultaneously ($s_i \rightarrow -s_i$) leaves the energy $E = -Js_1s_2$ unchanged, since the product of signs is unaffected. This is the up-down symmetry the system possesses when $h = 0$, and it guarantees that paired states must have equal probability.

An Exact Formula for a Finite Open Chain

The solution above for $N = 2$ generalizes directly to any finite N , with no approximation required. Consider an “open chain” (not closed at the ends, with $N - 1$ neighboring bonds in total); the partition function is

$$Z = \sum_{s_1, \dots, s_N = \pm 1} \exp \left(\beta J \sum_{i=1}^{N-1} s_i s_{i+1} \right).$$

The key to solving this is to sum over the spins one at a time. Expanding the sum in the exponent as a product of terms:

$$Z = \sum_{s_1, \dots, s_N = \pm 1} \prod_{i=1}^{N-1} e^{\beta J s_i s_{i+1}} = \sum_{s_1} \sum_{s_2} \dots \sum_{s_N} e^{\beta J s_1 s_2} e^{\beta J s_2 s_3} \dots e^{\beta J s_{N-1} s_N}.$$

Notice that, among the factors being summed, only the last one, $e^{\beta J s_{N-1} s_N}$, contains s_N ; we can therefore sum over s_N first, treating the remaining factors as constants to be pulled out:

$$\sum_{s_N = \pm 1} e^{\beta J s_{N-1} s_N} = e^{\beta J s_{N-1}} + e^{-\beta J s_{N-1}}.$$

Since $s_{N-1} = \pm 1$, this expression evaluates to the same number regardless of the value of s_{N-1} :

$$\sum_{s_N = \pm 1} e^{\beta J s_{N-1} s_N} = e^{\beta J} + e^{-\beta J} = 2 \cosh(\beta J),$$

(this can be checked directly by substituting $s_{N-1} = +1$ or $s_{N-1} = -1$: in either case, $\{e^{\beta J s_{N-1}}, e^{-\beta J s_{N-1}}\}$ simply swaps the order of the two numbers $\{e^{\beta J}, e^{-\beta J}\}$, leaving the sum unchanged). Once we have summed over s_N , the remaining expression becomes

$$Z = \sum_{s_1} \dots \sum_{s_{N-1}} e^{\beta J s_1 s_2} \dots e^{\beta J s_{N-2} s_{N-1}} \times 2 \cosh(\beta J),$$

which is exactly the same summation problem we started with, except that the number of spins has decreased from N to $N - 1$, with an extra overall factor $2 \cosh(\beta J)$.

Inductive argument. Let $Z(m)$ denote the partition function of an open chain of m spins and $m - 1$ bonds (summed over $s_1, \dots, s_m$). The calculation above shows that

$$Z(m) = 2 \cosh(\beta J) \cdot Z(m - 1), \quad m \geq 2,$$

that is, summing out one spin at a time contributes an independent factor of $2 \cosh(\beta J)$ each time, and this recursion relation holds regardless of the specific value of m (the derivation at

every step is identical; only the variable name changes, from s_N to s_{N-1} , to s_{N-2} , and so on). Applying this recursion repeatedly, working down from $m = N$ to $m = 1$:

$$Z(N) = 2 \cosh(\beta J) \cdot Z(N-1) = (2 \cosh(\beta J))^2 Z(N-2) = \dots = (2 \cosh(\beta J))^{N-1} Z(1).$$

Finally, $Z(1)$ represents a single free spin s_1 left over, with no bonds at all; the sum is then simply $\sum_{s_1=\pm 1} 1 = 2$ (each value has weight 1, since there is no bond contributing an exponential factor). Substituting this in gives a result that holds exactly for any finite N :

$$Z_{\text{open chain}}(N) = 2(2 \cosh(\beta J))^{N-1}. \quad (2.19)$$

We can check: at $N = 2$, $Z = 2(2 \cosh \beta J)^1 = 4 \cosh(\beta J)$, matching Equation (2.18). Taking $N = 3$, $k_B T = 1$ (i.e., $J = 1, \beta = 1$) and substituting into Equation (2.19): $Z = 2 \times (2 \cosh 1)^2 = 2 \times (2 \times 1.5431)^2 = 2 \times (3.0862)^2 = 2 \times 9.524 = 19.05$; listing out all $2^3 = 8$ microstates directly and summing them term by term gives the same value, 19.05 — readers can use this as a self-check.

■ Note

This trick — summing one variable at a time, with each step yielding the same factor, independent of the variables that came before — is exactly the seed of what later chapters will call **belief propagation** and **variable elimination**: in inference on graphical models in machine learning, this is, at bottom, the very same mathematical operation under different names.

The Transfer-Matrix Method and Periodic Boundary Conditions

The open-chain formula is exact, but when the chain length is taken arbitrarily large and we want to study its bulk properties, the boundary effects at the two ends make the formula look less symmetric than it could be. Physicists typically switch instead to **periodic boundary conditions**, joining the two ends of the chain into a ring (that is, setting $s_{N+1} = s_1$). The advantage of doing this is that every spin on the chain occupies an exactly equivalent position, with no special endpoints, which makes it much better suited to extracting the universal behavior in the thermodynamic limit ($N \rightarrow \infty$); and when N is large enough, the choice of boundary condition affects macroscopic thermodynamic quantities only through a negligible boundary correction, so the two boundary conditions give results that converge to the same answer.

Define the transfer matrix $\mathbf{T}$:

$$T_{s,s'} = e^{\beta J s s'}, \quad s, s' \in \{-1, +1\}, \quad (2.20)$$

explicitly,

$$\mathbf{T} = \begin{pmatrix} e^{\beta J} & e^{-\beta J} \\ e^{-\beta J} & e^{\beta J} \end{pmatrix},$$

where the first row/column corresponds to $s = +1$ and the second to $s = -1$. It is not hard to see that every entry of this matrix is exactly one of the four Boltzmann weights we already computed in the $N = 2$ example above; the transfer matrix simply arranges them into matrix form so that longer chains can be handled.

The partition function can be written as

$$Z = \sum_{\{s_i\}} \prod_{i=1}^{N-1} e^{\beta J s_i s_{i+1}} = \sum_{s_1, \dots, s_N} T_{s_1, s_2} T_{s_2, s_3} \cdots T_{s_{N-1}, s_N}.$$

Under periodic boundary conditions ($s_{N+1} = s_1$),

$$Z = \sum_{s_1} (T^N)_{s_1, s_1} = \text{Tr}(\mathbf{T}^N). \quad (2.21)$$

The trace of a matrix is exactly the sum of all its eigenvalues, which is precisely the power of the transfer-matrix method: it converts a summation problem with 2^N terms into a linear-algebra problem of finding the eigenvalues of a 2×2 matrix.

Solving for the Eigenvalues and Eigenvectors of the Transfer Matrix

The eigenvalues λ satisfy the characteristic equation $\det(\mathbf{T} - \lambda \mathbf{I}) = 0$. Substituting the explicit form of $\mathbf{T}$:

$$\det \begin{pmatrix} e^{\beta J} - \lambda & e^{-\beta J} \\ e^{-\beta J} & e^{\beta J} - \lambda \end{pmatrix} = 0 \implies (e^{\beta J} - \lambda)^2 - (e^{-\beta J})^2 = 0.$$

This is a difference-of-squares equation in $(e^{\beta J} - \lambda)$, and can be solved directly by taking square roots:

$$e^{\beta J} - \lambda = \pm e^{-\beta J} \implies \lambda = e^{\beta J} \mp e^{-\beta J}.$$

This yields two eigenvalues:

$$\lambda_+ = e^{\beta J} + e^{-\beta J} = 2 \cosh(\beta J), \quad \lambda_- = e^{\beta J} - e^{-\beta J} = 2 \sinh(\beta J). \quad (2.22)$$

λ_+ is exactly the same $2 \cosh(\beta J)$ that kept appearing in the term-by-term summation above — this is no coincidence: the two methods (summing term by term, and diagonalizing the transfer matrix) are, at bottom, two different ways of organizing the same calculation.

The corresponding eigenvectors can be obtained directly by substituting into $(\mathbf{T} - \lambda_{\pm} \mathbf{I})\mathbf{v} = 0$. Taking λ_+ as an example:

$$\begin{pmatrix} e^{\beta J} - \lambda_+ & e^{-\beta J} \\ e^{-\beta J} & e^{\beta J} - \lambda_+ \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \implies -e^{-\beta J} v_1 + e^{-\beta J} v_2 = 0 \implies v_1 = v_2,$$

(here we used $e^{\beta J} - \lambda_+ = -e^{-\beta J}$, from the definition (2.22)) so the (normalized) eigenvector corresponding to λ_+ is

$$\mathbf{v}_+ = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Entirely analogously, substituting λ_- gives $-e^{-\beta J} v_1 - e^{-\beta J} v_2 = 0$, i.e., $v_1 = -v_2$, with corresponding eigenvector

$$\mathbf{v}_- = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

These two eigenvectors are, respectively, analogous to the “symmetric” and “antisymmetric” superpositions of the spin configuration; they are orthogonal, and form an orthonormal basis of $\mathbb{R}^2$ — a fact we will use directly when deriving the correlation function below. Thus

$$Z = \lambda_+^N + \lambda_-^N.$$

For large N , since $\lambda_+ > |\lambda_-|$ (when $J > 0$), the largest eigenvalue λ_+ dominates ($(\lambda_-/\lambda_+)^N \rightarrow 0$):

$$Z \approx \lambda_+^N = (e^{\beta J} + e^{-\beta J})^N = (2 \cosh(\beta J))^N.$$

Compared with the open-chain formula $Z_{\text{open chain}} = 2(2 \cosh \beta J)^{N-1}$, this differs only by a prefactor that does not grow with N (namely, the ratio of 2 to $(2 \cosh \beta J)^N / (2 \cosh \beta J)^{N-1} = 2 \cosh \beta J$); once we take per-site averaged quantities (such as the “free energy per site”) in the limit $N \rightarrow \infty$, this difference is divided out by N and vanishes entirely — confirming the earlier claim that boundary effects are negligible in the thermodynamic limit.

The free energy per site is therefore

$$f = \frac{F}{N} = -\frac{1}{\beta N} \ln Z \approx -\frac{1}{\beta} \ln(2 \cosh(\beta J)). \quad (2.23)$$

The High- and Low-Temperature Limits

High-temperature limit ($T \rightarrow \infty$, $\beta \rightarrow 0$): Using the Taylor expansion of $\cosh$ near $x = 0$, $\cosh x = 1 + \frac{x^2}{2!} + \dots$, and keeping only up to second order as $\beta J \rightarrow 0$:

$$\cosh(\beta J) \approx 1 + \frac{(\beta J)^2}{2} \quad \implies \quad 2 \cosh(\beta J) \approx 2 \left[1 + \frac{(\beta J)^2}{2} \right].$$

Substituting into Equation (2.23), and using the product rule for logarithms, $\ln(ab) = \ln a + \ln b$:

$$f = -\frac{1}{\beta} \ln(2 \cosh(\beta J)) \approx -\frac{1}{\beta} \ln \left(2 \left[1 + \frac{(\beta J)^2}{2} \right] \right) = -\frac{1}{\beta} \left[\ln 2 + \ln \left(1 + \frac{(\beta J)^2}{2} \right) \right].$$

As $\beta \rightarrow 0$, $\ln \left(1 + \frac{(\beta J)^2}{2} \right) \rightarrow 0$, and it vanishes faster than $\ln 2$ (which is a constant, independent of β), so keeping only the leading term:

$$f \approx -\frac{1}{\beta} \ln 2 = -k_B T \ln 2.$$

The corresponding entropy can be obtained by differentiating $f \approx -k_B T \ln 2$ directly:

$$s = -\frac{\partial f}{\partial T} = -\frac{\partial}{\partial T} (-k_B T \ln 2) = k_B \ln 2 \cdot \frac{\partial T}{\partial T} = k_B \ln 2,$$

which is exactly the entropy of N independent spins: at high temperature the spins are completely disordered, and the interaction is negligible — consistent with the result in the $N = 2$ numerical example that “at high temperature, the four states tend toward equal probability, $1/4$ ”: each spin independently contributes an entropy of $\ln 2$, exactly the generalization of the “no preference, equal probability” case in §2.1.

Low-temperature limit ($T \rightarrow 0$, $\beta \rightarrow \infty$): As $\beta J \rightarrow \infty$, $e^{-\beta J} \rightarrow 0$, so

$$\cosh(\beta J) = \frac{e^{\beta J} + e^{-\beta J}}{2} \approx \frac{e^{\beta J}}{2} \implies 2 \cosh(\beta J) \approx e^{\beta J}.$$

Substituting into Equation (2.23):

$$f = -\frac{1}{\beta} \ln(2 \cosh(\beta J)) \approx -\frac{1}{\beta} \ln e^{\beta J} = -\frac{1}{\beta} \cdot \beta J = -J,$$

which is exactly the ground-state energy (all spins aligned, each bond contributing $-J$). At low temperature the system is “frozen” into its lowest-energy state, a conclusion that echoes both the result $P_0 \rightarrow 1$ as $T \rightarrow 0$ in the two-level system, and the phenomenon, in the $N = 2$ numerical example, that the aligned states take up the entire probability at low temperature.

The Correlation Function

Define the spin–spin correlation function

$$C(r) = \langle s_i s_{i+r} \rangle - \langle s_i \rangle \langle s_{i+r} \rangle.$$

At $h = 0$, symmetry implies $\langle s_i \rangle = 0$ (consistent with the up-down symmetry verified in the “Think and Check” box above), so

$$C(r) = \langle s_i s_{i+r} \rangle.$$

We now derive the full transfer-matrix expression for $\langle s_i s_{i+r} \rangle$, from which Equation (2.24) follows. Since, under periodic boundary conditions, every site on the chain is equivalent (translational invariance), we may take $i = 1$ without loss of generality.

Step 1: write the sum as a contraction of matrix products. By the definition of expectation value,

$$\langle s_1 s_{1+r} \rangle = \frac{1}{Z} \sum_{s_1, \dots, s_N = \pm 1} s_1 s_{1+r} T_{s_1 s_2} T_{s_2 s_3} \cdots T_{s_N s_1}.$$

Cut this entire closed path, running from s_1 back to s_1 , into two segments at s_{1+r} : the first segment runs from s_1 to s_{1+r} in r steps, and the second runs from s_{1+r} back to s_1 in $N - r$ steps. Summing out the intermediate spins $s_2, \dots, s_r$ and $s_{r+2}, \dots, s_N$ one at a time, and using the definition of matrix multiplication $(T^m)_{a,b} = \sum_{c_1, \dots, c_{m-1}} T_{a,c_1} T_{c_1,c_2} \cdots T_{c_{m-1},b}$, the two segments contract, respectively, to matrix elements of T^r and T^{N-r} :

$$\langle s_1 s_{1+r} \rangle = \frac{1}{Z} \sum_{s_1, s_{1+r}} s_1 (T^r)_{s_1, s_{1+r}} s_{1+r} (T^{N-r})_{s_{1+r}, s_1}.$$

Step 2: introduce the spin-operator matrix $\mathbf{S}$. Define the diagonal matrix

$$\mathbf{S} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (\text{i.e., } S_{s,s'} = s \delta_{s,s'}),$$

whose effect is to multiply each spin state by its own value $s = \pm 1$. Since $\mathbf{S}$ is diagonal, $s_1 (T^r)_{s_1, s_{1+r}} s_{1+r} = S_{s_1, s_1} (T^r)_{s_1, s_{1+r}} S_{s_{1+r}, s_{1+r}} = (\mathbf{S} T^r \mathbf{S})_{s_1, s_{1+r}}$, and so

$$\langle s_1 s_{1+r} \rangle = \frac{1}{Z} \sum_{s_1, s_{1+r}} (\mathbf{S} T^r \mathbf{S})_{s_1, s_{1+r}} (T^{N-r})_{s_{1+r}, s_1} = \frac{1}{Z} \text{Tr}(\mathbf{S} \mathbf{T}^r \mathbf{S} \mathbf{T}^{N-r}),$$

where the last step uses the definition of the trace of a matrix product, $\text{Tr}(AB) = \sum_{a,b} A_{a,b} B_{b,a}$.

Step 3: evaluate this trace in the eigenbasis of the transfer matrix. The trace does not depend on the choice of basis, so we may instead evaluate it in the eigenbasis $\{\mathbf{v}_+, \mathbf{v}_-\}$ of $\mathbf{T}$ (already found above). In this basis $\mathbf{T}$ is the diagonal matrix $\text{diag}(\lambda_+, \lambda_-)$; and $\mathbf{S}$ acts as $\mathbf{S}\mathbf{v}_+ = \mathbf{v}_-$, $\mathbf{S}\mathbf{v}_- = \mathbf{v}_+$ (readers can verify this directly by substituting $\mathbf{v}_\pm = \frac{1}{\sqrt{2}}(1, \pm 1)^\top$: $\mathbf{S}\mathbf{v}_+ = \frac{1}{\sqrt{2}}(1, -1)^\top = \mathbf{v}_-$), i.e., $\mathbf{S}$ swaps the two eigenvectors in the eigenbasis, with matrix representation

$$\mathbf{S} \longrightarrow \mathbf{X} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (\text{in the eigenbasis}).$$

Writing $D_1 = \text{diag}(\lambda_+^r, \lambda_-^r)$, $D_2 = \text{diag}(\lambda_+^{N-r}, \lambda_-^{N-r})$, we compute $\mathbf{X}D_1\mathbf{X}D_2$ step by step:

$$\begin{aligned} \mathbf{X}D_1 &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \lambda_+^r & 0 \\ 0 & \lambda_-^r \end{pmatrix} = \begin{pmatrix} 0 & \lambda_-^r \\ \lambda_+^r & 0 \end{pmatrix}, \\ (\mathbf{X}D_1)\mathbf{X} &= \begin{pmatrix} 0 & \lambda_-^r \\ \lambda_+^r & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} \lambda_-^r & 0 \\ 0 & \lambda_+^r \end{pmatrix}, \\ (\mathbf{X}D_1\mathbf{X})D_2 &= \begin{pmatrix} \lambda_-^r & 0 \\ 0 & \lambda_+^r \end{pmatrix} \begin{pmatrix} \lambda_+^{N-r} & 0 \\ 0 & \lambda_-^{N-r} \end{pmatrix} = \begin{pmatrix} \lambda_-^r \lambda_+^{N-r} & 0 \\ 0 & \lambda_+^r \lambda_-^{N-r} \end{pmatrix}. \end{aligned}$$

Taking the trace (summing the diagonal entries):

$$\text{Tr}(\mathbf{S}\mathbf{T}^r\mathbf{S}\mathbf{T}^{N-r}) = \lambda_+^{N-r}\lambda_-^r + \lambda_+^r\lambda_-^{N-r}.$$

Step 4: substitute $Z = \lambda_+^N + \lambda_-^N$ and take the thermodynamic limit. Thus

$$\langle s_1 s_{1+r} \rangle = \frac{\lambda_+^{N-r}\lambda_-^r + \lambda_+^r\lambda_-^{N-r}}{\lambda_+^N + \lambda_-^N}.$$

Dividing numerator and denominator by λ_+^N :

$$\langle s_1 s_{1+r} \rangle = \frac{\left(\frac{\lambda_-}{\lambda_+}\right)^r + \left(\frac{\lambda_-}{\lambda_+}\right)^{N-r}}{1 + \left(\frac{\lambda_-}{\lambda_+}\right)^N}.$$

When $J > 0$ and the temperature is finite, $0 < \lambda_-/\lambda_+ = \tanh(\beta J) < 1$; fixing r and taking $N \rightarrow \infty$, $(\lambda_-/\lambda_+)^N \rightarrow 0$, and $(\lambda_-/\lambda_+)^{N-r} \rightarrow 0$ as well (since $N-r \rightarrow \infty$). The denominator tends to 1, the second term in the numerator tends to 0, leaving only the first term:

$$C(r) = \langle s_1 s_{1+r} \rangle \xrightarrow{N \rightarrow \infty} \left(\frac{\lambda_-}{\lambda_+}\right)^r = \left(\frac{e^{\beta J} - e^{-\beta J}}{e^{\beta J} + e^{-\beta J}}\right)^r = [\tanh(\beta J)]^r. \quad (2.24)$$

This fully proves Equation (2.24).

The definition of the correlation length. Physically, it is conventional to write the decay of the correlation function in exponential form, $C(r) \sim e^{-r/\xi}$, where ξ is called the correlation

length, characterizing the spatial scale over which correlations “propagate.” Rewriting the power-law form in Equation (2.24) as an exponential (using $a^r = e^{r \ln a}$):

$$C(r) = [\tanh(\beta J)]^r = e^{r \ln \tanh(\beta J)} = e^{-r/\xi},$$

Equating the coefficients in the two exponents, $\ln \tanh(\beta J) = -1/\xi$, and solving (noting that, since $0 < \tanh(\beta J) < 1$, $\ln \tanh(\beta J) < 0$, so we must take the absolute value to ensure $\xi > 0$):

$$\xi = -\frac{1}{\ln |\tanh(\beta J)|}. \quad (2.25)$$

Taking $J = 1$ and substituting specific values (Table 2.7):

Table 2.7: Correlation Length ξ as a Function of Temperature ($J = 1$)

$k_B T$	βJ	$\tanh(\beta J)$	ξ (correlation length, in lattice sites)
0.3	3.33	0.998	393
1.0	1.00	0.762	3.7
3.0	0.33	0.322	0.9

Table 2.7 shows that at high temperature (the last row of the table), $\beta J \ll 1$, $\tanh(\beta J) \approx \beta J$, and the correlation length is less than a single lattice site — spins are almost completely uncorrelated with one another.

At low temperature (the first row of the table), $\beta J \gg 1$, and we can go further and give an analytic approximation for ξ . In this regime $e^{-\beta J} \ll e^{\beta J}$, so dividing the numerator and denominator of $\tanh(\beta J) = \frac{e^{\beta J} - e^{-\beta J}}{e^{\beta J} + e^{-\beta J}}$ by $e^{\beta J}$:

$$\tanh(\beta J) = \frac{1 - e^{-2\beta J}}{1 + e^{-2\beta J}}.$$

Using $e^{-2\beta J} \ll 1$ when $\beta J \gg 1$, applying a first-order approximation to the denominator, $\frac{1}{1 + e^{-2\beta J}} \approx 1 - e^{-2\beta J}$, and substituting while keeping only first-order small terms:

$$\tanh(\beta J) \approx (1 - e^{-2\beta J})(1 - e^{-2\beta J}) \approx 1 - 2e^{-2\beta J}.$$

Substituting into the correlation-length formula (2.25), and using $\ln(1 - x) \approx -x$ (for $x \ll 1$; here $x = 2e^{-2\beta J}$):

$$\ln \tanh(\beta J) \approx \ln(1 - 2e^{-2\beta J}) \approx -2e^{-2\beta J},$$

so

$$\xi = -\frac{1}{\ln \tanh(\beta J)} \approx \frac{1}{2e^{-2\beta J}} = \frac{1}{2}e^{2\beta J}.$$

The correlation length grows exponentially, potentially reaching hundreds of lattice sites — at that point, the orientation of a single spin “propagates” out to spins far away, driving the whole chain toward coordinated alignment. This phenomenon, that “the lower the temperature, the longer the correlation length,” is the prototype of the critical phenomena and correlation structure seen in many physical systems (and, as we will discuss later, of long-range dependencies in deep neural networks).

The Connection to Machine Learning

The one-dimensional Ising model already contains every element that will recur repeatedly in machine learning:

1. **The energy function** $E(\{s_i\})$: penalizes configurations that fail to satisfy a constraint;
2. **The Boltzmann distribution** $P \propto e^{-\beta E}$: strikes a balance between minimizing energy and exploring the space of states;
3. **The partition function** Z : a normalization constant, but one that is typically very difficult to compute — once the number of spins is large and the interaction graph becomes complex, Z no longer admits a closed-form solution the way the one-dimensional chain does; this computational challenge will be discussed in detail in later chapters of this book;
4. **Interaction**: dependencies between neighboring units;
5. **Temperature**: the parameter controlling the system’s randomness.

As we will see in later chapters, Hopfield networks and Boltzmann machines are exactly direct generalizations of the one-dimensional Ising model: the simple rule on a one-dimensional chain, “neighboring spins interact,” is generalized to “any two neurons may be connected, with some weight”; and every concept checked, one by one, in this section — energy, the partition function, the Boltzmann distribution, correlation length — will carry over, essentially unchanged, into the context of machine learning.

■ Summary

This section unfolded along a complete path: starting from a purely combinatorial “counting the boxes” problem (§2.1), we obtained, with no physical assumptions at all, the core intuition that macroscopic certainty arises from a microscopic probabilistic advantage; introducing energy and a heat bath, the same principle of equal a priori probabilities naturally led to the Boltzmann distribution and the partition function (§2.2), and the logarithm of the partition function was shown to generate every thermodynamic quantity — average energy, free energy, entropy, heat capacity. Finally, we applied this toolkit to a system with genuine interactions — the one-dimensional Ising model (this section) — obtaining exact solutions for the partition function, the correlation function, and the correlation length, and observing for the first time the theme of “disordered at high temperature, ordered at low temperature,” a theme that will recur repeatedly. The language established in this section — energy, the Boltzmann distribution, the partition function, temperature — makes up the entire basic vocabulary needed to connect statistical physics with machine learning in the chapters that follow.

A Table Waiting to Be Filled In: What the Rest of This Book Will Do

Not a single machine-learning term has appeared anywhere in this chapter — that is deliberate: we first lay a solid foundation for the physical language itself, before asking what it can carry.

But by this point, the reader already has the entire left-hand column of vocabulary in Table 2.8. The right-hand column is still empty, marked only with “where it will be revealed”; what the next five chapters do, in the end, amounts to filling in that right-hand column one row at a time — every time a row gets filled in, it will be explicitly flagged in the main text, and readers are welcome to flip back here at any time to check on the progress.

Table 2.8: The Book’s Master Correspondence Table — Seeds Planted in Chapter 2 (to Be Filled In Row by Row in Later Chapters)

Statistical physics (established in this chapter)	Machine-learning counterpart	Where it will be revealed
Microstate $\mathbf{x}$	<i>(to be filled in)</i>	Ch. 4
Energy function $E(\mathbf{x})$	<i>(to be filled in)</i>	Ch. 4
Boltzmann distribution $P(\mathbf{x}) \propto e^{-\beta E(\mathbf{x})}$	<i>(to be filled in)</i>	Ch. 4
Partition function Z	<i>(to be filled in)</i>	Ch. 4–5
Free energy $F = \langle E \rangle - TS$	<i>(to be filled in)</i>	Ch. 6
Inverse temperature $\beta = 1/(k_B T)$	<i>(to be filled in)</i>	Ch. 4
Entropy S	<i>(to be filled in)</i>	Ch. 6–7

This table is not decoration — it is a promise that can be checked. If, by the end of Part I of the book (the table at the end of Chapter 7, Table 7.2), any single entry listed here under “where it will be revealed” has not been fulfilled, then this book has failed to deliver on its own narrative, and the reader is entitled to ask why.

Exercises

Foundational

Exercise 2.1: The Partition Function and Average Energy

Consider a two-state system with energy levels $E_1 = 0$ and $E_2 = \epsilon$.

- Write down the partition function $Z(\beta)$;
- Compute the average energy $\langle E \rangle$ as a function of temperature T ;
- Sketch how $\langle E \rangle$ varies with T . What is the value of $\langle E \rangle$ in the high- and low-temperature limits?
- Compute the heat capacity $C = \frac{\partial \langle E \rangle}{\partial T}$, and find the temperature at which C is maximized.

$$\text{Hint: } \langle E \rangle = \frac{\epsilon}{e^{\beta\epsilon} + 1}.$$

Intermediate

Exercise 2.2: An Exact Solution of the One-Dimensional Ising Model

Consider a one-dimensional Ising model with N spins (periodic boundary conditions):

$$E = -J \sum_{i=1}^N s_i s_{i+1}, \quad s_i \in \{-1, +1\} \quad (\text{convention: } s_{N+1} = s_1).$$

- Write down the transfer matrix $\mathbf{T}$, with entries $T_{s,s'} = e^{\beta J s s'}$;
- Prove that the partition function can be written as $Z = \text{Tr}(\mathbf{T}^N)$;
- Compute the eigenvalues of the transfer matrix;
- In the thermodynamic limit $N \rightarrow \infty$, derive the free energy (per site) $f = -\frac{1}{\beta} \lim_{N \rightarrow \infty} \frac{1}{N} \ln Z$;
- Compute the average energy (per site) and the heat capacity.

Challenge

Exercise 2.3: Landauer's Principle and the Physical Limits of Computation

- Investigate Landauer's principle: erasing 1 bit of information requires dissipating at least $k_B T \ln 2$ of energy;
- Estimate: at room temperature ($T = 300$ K), what is the minimum energy, in joules, required to erase 1 bit?
- A modern CPU transistor switch dissipates roughly 10^{-15} J per switching event — how many times larger is this than the Landauer limit?
- Estimate the theoretical minimum energy required to train a large language model (such as GPT-3, with roughly 10^{11} parameters), assuming each parameter requires 32 bits of storage;
- The human brain has roughly 10^{11} neurons and 10^{15} synapses, and consumes about 20 W of power. Estimate the “computational efficiency” of the human brain (synaptic operations per joule), and compare it with a modern computer.

Exercise 2.4: An Interdisciplinary Project: Physics-Inspired Optimization Algorithms

Design and test a physics-inspired optimization algorithm.

- (a) **Simulated annealing:** implement a simulated-annealing algorithm to optimize a non-convex function (such as the Ackley function or the Rastrigin function);
- (b) **Annealing schedules:** compare different “temperature decay” schedules — linear $T(t) = T_0 - \alpha t$, exponential $T(t) = T_0 e^{-\alpha t}$, logarithmic $T(t) = T_0 / \ln(1 + t)$;
- (c) **Physical analogy:** visualize the optimization process as diffusion over an energy landscape — what role does temperature play?
- (d) **Comparison with gradient descent:** in what situations does simulated annealing outperform gradient descent? When does it fall short?
- (e) **Extension:** investigate the basic principles of “quantum annealing” — how does it differ from classical simulated annealing?

Chapter 3

Probabilistic Models in Machine Learning

Trifles light as air

Are to the jealous confirmations strong

As proofs of holy writ.

— Shakespeare, *Othello*, Act III, Scene 3, spoken by Iago

Objectives

- Understand the essential difference between regression and classification problems in supervised learning, and the advantages of the probabilistic perspective over the traditional function-fitting perspective;
- Through concrete worked examples that can be verified by hand, master how to construct a likelihood function and derive maximum likelihood estimation (MLE);
- Understand the limitation that MLE tends to give overconfident estimates with small samples, and master maximum a posteriori estimation (MAP), obtained by introducing a prior distribution;
- Establish the correspondence between regularization (such as L^2 regularization / ridge regression) and prior distributions, and connect it to the analogy with an “external field” in statistical physics.

■ Chapter Guide

The question this chapter answers: Given a batch of observed data, how do we estimate the “most reasonable” parameters for a parameterized model? How are the two most basic estimation methods in machine learning — maximum likelihood estimation (MLE) and maximum a posteriori estimation (MAP) — derived from the first principles of probability

theory?

Correspondence with statistical physics: This chapter’s chain of logic, “observed data $\rightarrow$ likelihood function $\rightarrow$ parameter estimation,” is almost a mirror image of Chapter 2’s chain, “the principle of equal a priori probabilities $\rightarrow$ the Boltzmann distribution $\rightarrow$ the partition function”; the role played by the “prior” in MAP will be reinterpreted in Chapter 4 as a physical “external field.”

Looking ahead: Chapter 4 will prove that this chapter’s operation of “taking the negative log-likelihood as the loss” is exactly Chapter 2’s energy function, carried over unchanged into machine learning; Chapter 7’s principle of minimum cross-entropy will generalize this chapter’s intuition — that “the prior pulls the estimate toward itself” — from a single parameter to the choice of an entire probability distribution.

In Chapter 2, we unfolded the discussion along this chain of logic: the principle of equal a priori probabilities $\rightarrow$ the Boltzmann distribution $\rightarrow$ the partition function $\rightarrow$ thermodynamic quantities. This chapter unfolds along a path that is almost a mirror image of that one: observed data $\rightarrow$ likelihood function $\rightarrow$ parameter estimation. The mathematical operations along this path are strikingly similar to those of the previous chapter — in both cases, we take the logarithm of a “product” or “sum,” and then find an extremum. If some step in the derivation feels familiar, that is no coincidence — it is exactly one of the central threads running through this book.

3.1 The Basic Setup of Supervised Learning

Problem Description

Consider a set of training data:

$$\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\},$$

where:

- $\mathbf{x}^{(n)} \in \mathbb{R}^d$ is the **feature vector** (input) of the n -th sample;
- $y^{(n)}$ is the corresponding **label** (output):
 - Regression problems: $y^{(n)} \in \mathbb{R}$;
 - Classification problems: $y^{(n)} \in \{1, 2, \dots, K\}$.

Goal: learn a function $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ such that, for a new input $\mathbf{x}$, it accurately predicts $y = f(\mathbf{x})$.

Regression and Classification: Two Concrete Examples

The notation above is fairly abstract; let us use two small, concrete datasets to fully clarify the distinction between “regression” and “classification” — a distinction that, at bottom, comes down to whether the label $y^{(n)}$ is a continuous numerical value or a discrete category.

Example 1 (Regression): Predicting House Prices from Floor Area. Suppose we have collected the sale records of 5 properties, with the feature $x^{(n)}$ being the floor area (in square meters) and the label $y^{(n)}$ being the sale price (in units of ¥10,000):

Table 3.1: Example Data for House-Price Prediction (a Regression Problem)

Sample n	Floor area $x^{(n)}$ (m ²)	Sale price $y^{(n)}$ (¥10,000)
1	50	150
2	70	205
3	90	260
4	110	310
5	130	365

Here $y^{(n)}$ can take any real value (150, 205, 260, ..., and could in principle be a number like 208.5), and these values carry a clear **ordering and notion of distance**: “¥3.10 million” is ¥500,000 more expensive than “¥2.60 million,” and this “¥500,000” has real meaning. This is exactly the defining feature of a regression problem: the label is a continuous quantity on which arithmetic operations can be performed. Given a new floor area (say $x = 100$), the model needs to output a specific price prediction (say, around ¥2.85 million).

Example 2 (Classification): Determining Whether an Email Is Spam. The feature $\mathbf{x}^{(n)}$ can be a vector — for example, [the number of times the word “free” appears in the email, the number of links it contains, whether the sender’s address is unfamiliar] — and the label $y^{(n)} \in \{0, 1\}$, where 1 denotes spam and 0 denotes a legitimate email:

Table 3.2: Example Data for Spam Detection (a Classification Problem)

Sample n	Occurrences of “free”	Number of links	Unfamiliar sender?	Label $y^{(n)}$
1	3	5	Yes	1 (spam)
2	0	1	No	0 (legitimate)
3	2	4	Yes	1 (spam)
4	0	0	No	0 (legitimate)

Here the labels 0 and 1 are merely **category codes**, not numbers on which arithmetic operations can meaningfully be performed — “spam” minus “legitimate email” has no meaning; if the labels were replaced with words like “cat/dog,” the essence of the problem would be entirely unchanged. This is exactly the defining feature of a classification problem: the label is a discrete category, and (in the general case) no ordering or distance relationship exists between categories. If the number of categories $K > 2$ (as in handwritten-digit recognition, where $y \in \{0, 1, \dots, 9\}$), the reasoning is exactly the same, except that the choice must be made among more categories.

■ Clearing Up a Question

The line between regression and classification can sometimes blur. For instance, “predicting a student’s final grade band: excellent/good/pass/fail” looks, on the surface, like several discrete categories (apparently a classification problem), but these bands carry a natural ordering (excellent > good > pass > fail). Problems of this kind are sometimes treated specially as **ordinal classification**, sitting somewhere between pure regression and unordered classification. When facing a concrete problem, it often helps to first ask: “can the labels be subtracted from one another — is there an ordering relationship between them?” This is usually a good guide to which class of methods to use.

■ Think and Check

Determine whether each of the following two prediction tasks is a regression or a classification problem: (1) predicting how many people are speaking in an audio clip (0 people, 1 person, 2 people, ...); (2) predicting whether the animal in an image is a cat, a dog, or a bird.

Solution: (1) Although the number of speakers is an integer, it carries a clear ordering and arithmetic meaning (“3 people” speak the equivalent of 2 more people’s worth than “1 person”), so it can, in principle, be handled with a regression approach; but **directly applying**, without modification, the ordinary regression this chapter is about to derive — which assumes Gaussian noise (see §4.2) — is not ideal, since a Gaussian regression’s predictions can take any real value, and can easily produce a meaningless result like $\hat{y} < 0$ for a “count of people.” A more natural choice is to build the counting structure of $y \in \{0, 1, 2, \dots\}$ directly into the probabilistic model — for example, Poisson regression (assuming $y|\mathbf{x} \sim \text{Poisson}(\lambda(\mathbf{x}))$) or negative binomial regression; this is exactly the entire methodology of “choose a distributional form for the noise/randomness → write down the likelihood → take the negative logarithm” (the subject of Chapter 4), applied specifically to count data, simply replacing “Gaussian noise” with “Poisson noise.” (2) There is no ordering or magnitude relationship whatsoever among “cat, dog, bird” — this is a standard three-class classification problem.

From Function Fitting to Probabilistic Modeling

The traditional view: machine learning is the search for a “best-fitting” function.

The probabilistic view: the data are generated by some unknown probability distribution, and the goal of learning is to estimate that distribution (or its parameters).

The difference between these two views is easiest to appreciate through the example of weather forecasting. A traditional-view forecast would give a definite judgment directly: “it will rain tomorrow.” A probabilistic-view forecast would instead say: “there is a 70% chance of rain tomorrow.” The latter clearly carries more information — it gives not only the most likely outcome, but also how confident that judgment is. This is exactly the core advantage of probabilistic modeling over pure function fitting.

The advantages of the probabilistic view show up chiefly in the following respects:

1. **Quantifying uncertainty:** a probabilistic model gives not just a predicted value, but also

a degree of confidence;

2. **Naturally handling noise:** real-world data are pervasively noisy, and the probabilistic framework naturally incorporates a noise model;
3. **A natural interpretation of regularization:** a prior distribution corresponds to regularization;
4. **A unified framework:** regression, classification, and clustering can all be handled uniformly within the probabilistic framework.

■ Physical Analogy

In the language of probabilistic modeling, the model parameters θ play exactly the role that the coupling strength J and the external field h play in the Ising model — they are not themselves “data,” but parameters that determine the **shape** of the entire probability distribution. Adjusting J, h changes which spin configurations the Ising chain favors; adjusting θ changes which values of y the model assigns higher probability to. The process of learning is, at bottom, exactly the process of adjusting these parameters so that the distribution better matches the observed data.

■ Think and Check

Consider a binary classification problem — determining whether an email is spam. The output under the “traditional view” is “yes” or “no”; the output under the “probabilistic view” might instead be “the probability of being spam is 0.93.” Why is the probabilistic output more useful to the designer of an email client?

Solution: Consider the two cases “probability 0.51” and “probability 0.99” — both correspond to the same “yes” judgment, but the degree of trust one should place in that judgment, and whether it should be handed off to the user for review, ought to differ. A pure “yes/no” output erases this distinction entirely.

3.2 The Basic Assumptions of Probabilistic Modeling

■ Preliminary Definitions: Conditional Probability and Independence

This chapter makes extensive use of notation of the form $P(y|\mathbf{x}, \theta)$, with a vertical bar; let us formally clarify its meaning here. For two (possibly related) random quantities A, B , the **conditional probability** of A given B is defined as

$$P(A|B) \equiv \frac{P(A, B)}{P(B)} \quad (\text{requiring } P(B) > 0),$$

where $P(A, B)$ denotes the **joint probability** that A and B occur together. Intuitively, $P(A|B)$ answers the question: “once we already know that B has occurred, how should the probability of A occurring be re-evaluated?” For the $P(y|\mathbf{x}, \theta)$ appearing in Equation (3.1), we can regard both $\mathbf{x}$ (the input features) and θ (the model parameters) as “given condi-

tions”; $P(y|\mathbf{x}, \boldsymbol{\theta})$ describes precisely the probability distribution of the label y under these conditions — this is exactly the same usage of conditional probability found in an introductory thermal-physics course, except that the “condition” here is replaced by the features and the parameters.

Two random quantities A, B are said to be **mutually independent** if knowing the value of B does not change the probability distribution of A at all, that is,

$$P(A|B) = P(A) \iff P(A, B) = P(A)P(B),$$

(the two conditions are equivalent, which can be verified directly by substituting into the definition of conditional probability: if $P(A|B) = P(A)$, substituting into the definition $P(A|B) = P(A, B)/P(B)$ gives $P(A, B) = P(A)P(B)$, and conversely). This is exactly the precise meaning of the i.i.d. assumption in Equation (3.2) — that “the joint probability equals the product of the individual probabilities.”

Assume the data are generated from a parameterized conditional probability distribution:

$$y \sim P(y|\mathbf{x}, \boldsymbol{\theta}), \tag{3.1}$$

where $\boldsymbol{\theta}$ denotes the model parameters (in the physical analogy, these correspond to the system’s “external parameters”).

The independent and identically distributed (i.i.d.) assumption: assume that all data points are drawn independently from the same distribution:

$$P(\mathcal{D}|\boldsymbol{\theta}) = \prod_{n=1}^N P(y^{(n)}|\mathbf{x}^{(n)}, \boldsymbol{\theta}). \tag{3.2}$$

This “product” structure already appeared in Chapter 2: recall the “counting the boxes” discussion in §2.1 — if a system consists of two independent subsystems, the number of microstates of the combined system is the **product** of the two subsystems’ microstate counts ($\Omega = \Omega_1\Omega_2$), and correspondingly the entropy is the **sum** of the two subsystems’ entropies ($S = S_1 + S_2$, because a logarithm has been taken). The logic here is exactly the same: if N data points are generated independently, the probability of the event “the entire dataset is observed” is the product of the probabilities of each data point occurring individually; once we take the logarithm of this (which we do below), the product turns into a sum. This is not a mathematical coincidence, but two independent applications, in physics and in machine learning, of the same general rule: “independence $\Rightarrow$ joint probabilities multiply $\Rightarrow$ logarithms add.”

This assumption often does not hold exactly in reality (time-series data, for example, have correlations, much as neighboring spins in the Ising model are not independent), but it greatly simplifies the analysis, and is the default starting point for the great majority of basic machine-learning models.

■ Think and Check

In the one-dimensional Ising model discussed in §2.3, are neighboring spins s_i and s_{i+1} independent? If $J \neq 0$, can the joint probability $P(s_i, s_{i+1})$ be written as the product $P(s_i)P(s_{i+1})$?

Solution: No — precisely because $J \neq 0$ introduces a correlation, the joint distribution of the two spins cannot be factored into the product of their individual marginal distributions; this is exactly why the correlation function $C(r) = \langle s_i s_{i+r} \rangle$, solved specifically in §2.3, is not identically zero. **Under the specific analogy of “treating each data point as a spin, and treating the coupling between data points as analogous to the Ising coupling J ,”** the i.i.d. assumption corresponds precisely to “assuming $J = 0$ ” — that is, within this particular analogical framework, there is no interaction between data points.

It is worth emphasizing that this is only **an analogy**, not a general mathematical equivalence between i.i.d. and $J = 0$: the strict meaning of i.i.d. is that the joint probability in Equation (3.2) can be factored into a product, and this property does not itself depend on whether there exists some parameter that can be likened to an “Ising coupling.” Real data can perfectly well possess various kinds of structure without violating the i.i.d. assumption (for example, the inputs $\mathbf{x}^{(n)}$ can be correlated with one another, as long as the labels’ conditional distributions $P(y^{(n)}|\mathbf{x}^{(n)}, \boldsymbol{\theta})$ are mutually independent), and data can also violate i.i.d. without there being any natural “ J ” to correspond to it (for example, hierarchically structured data).

Keeping “analogy” and “strict equivalence” clearly distinct is itself an instance of a principle this book emphasizes repeatedly: an analogy is not a metaphor, but a mathematical structure whose scope of applicability must be clearly stated.

3.3 Maximum Likelihood Estimation (MLE)

Before giving the general definition, let us first fully lay out the core logic of MLE through a concrete example that can be verified by hand, and only then treat the general case.

Estimating a Coin’s Bias

Setup: a coin lands heads with some unknown constant probability $p \in [0, 1]$ ($p = 0.5$ corresponds to a “fair coin,” but the true value of p is not known in advance). The coin is tossed independently N times, yielding k heads and $N - k$ tails. **Question:** based on these observations, what is the most reasonable estimate of p ?

This setup is exactly the machine-learning counterpart of the “two-level system” in §2.2: treat “heads” and “tails” as two possible “states,” with each toss an independent observation. The probability of a single toss can be written uniformly as

$$P(x|p) = p^x(1-p)^{1-x}, \quad x = \begin{cases} 1 & \text{heads} \\ 0 & \text{tails} \end{cases} \quad (3.3)$$

This distribution is called the **Bernoulli distribution**. Substituting $x = 1$ gives $P = p$, and substituting $x = 0$ gives $P = 1 - p$, as expected.

Using the i.i.d. assumption of Equation (3.2), the likelihood of observing this entire set of outcomes (k heads and $N - k$ tails, in a known specific order) is

$$\mathcal{L}(p) = \prod_{i=1}^N P(x_i|p) = p^k(1-p)^{N-k}. \quad (3.4)$$

Substituting specific numbers. Take $N = 10$, with $k = 7$ heads observed. Substituting a few candidate values of p into Equation (3.4), we compare which p makes this set of observations “least surprising”:

Table 3.3: The Likelihood Function $\mathcal{L}(p) = p^7(1-p)^3$ for Different Values of p ($N = 10$, $k = 7$)

p	$\mathcal{L}(p) = p^7(1-p)^3$
0.1	7.29×10^{-8}
0.3	7.50×10^{-5}
0.5	9.77×10^{-4}
0.7	2.22×10^{-3}
0.9	4.78×10^{-4}

Table 3.3 shows that $p = 0.7$ (that is, the observed **frequency** of heads, $k/N = 7/10$) achieves the maximum value of the likelihood function in the table. This is no coincidence — we now prove it rigorously using calculus.

Analytic solution. Taking the logarithm of the likelihood (the log-likelihood):

$$\ell(p) = \ln \mathcal{L}(p) = k \ln p + (N - k) \ln(1 - p).$$

Differentiating with respect to p and setting the result to zero:

$$\frac{d\ell}{dp} = \frac{k}{p} - \frac{N - k}{1 - p} = 0.$$

Rearranging:

$$k(1 - p) = (N - k)p \implies k - kp = Np - kp \implies k = Np,$$

from which

$$\boxed{p_{\text{MLE}} = \frac{k}{N}}. \quad (3.5)$$

This result is exactly the most naive intuition: use the observed frequency to estimate the probability. In this simplest possible example, MLE reduces the seemingly abstract method of “maximum likelihood” to something everyone already knows — and this is also a useful way to understand MLE in general: however complicated the model, MLE is, at bottom, always a generalization of “estimation by frequency.”

■ A Note on Computation

Taking the logarithm does more than turn a product into a sum that is easy to differentiate — it also matters a great deal for numerical computation. Imagine $N = 100$ tosses: even near the maximum-likelihood point (say $p = 0.5$), the value of the likelihood function itself is as small as $0.5^{100} \approx 7.9 \times 10^{-31}$ — a number already pushing against the lower limit representable by floating-point arithmetic on a computer; increasing N further would cause **numerical underflow**, with the computer simply rounding the value to zero, making it impossible to compare different values of p . By contrast, the log-likelihood $100 \ln 0.5 \approx -69.3$ is a quantity of ordinary numerical size, and computation with it is far more stable. This is exactly why nearly all practical machine-learning code optimizes at the level of the log-likelihood (or the negative log-likelihood loss), rather than manipulating the likelihood function directly. This trick belongs to the same idea as using $\ln Z$ rather than Z itself to extract thermodynamic quantities in §2.2 — “take a logarithm in exchange for numerical stability and additivity.”

■ Think and Check

If this coin is tossed $N = 3$ times and comes up **heads all three times** ($k = 3$), then $p_{\text{MLE}} = 3/3 = 1$ — the model asserts that “this coin can never possibly land tails.” Is this conclusion believable? Where does the problem lie?

Solution: This question is not well answered by a formula on its own; we defer it to §3.4 — maximum a posteriori estimation (MAP) is introduced precisely to address the problem that “with a small sample, MLE tends to give overconfident, extreme conclusions.”

The General Case: The Likelihood Function

Let us now carry over, unchanged, the treatment we just applied to the coin, and generalize it to a general supervised-learning problem. Given a dataset $\mathcal{D}$ and parameters θ , the **likelihood function** is defined as

$$\mathcal{L}(\theta) = P(\mathcal{D}|\theta) = \prod_{n=1}^N P(y^{(n)}|\mathbf{x}^{(n)}, \theta). \quad (3.6)$$

It is worth noting that here the dataset $\mathcal{D}$ is treated as fixed, and θ as the variable. The likelihood function is not a probability density (its integral over θ need not equal 1); rather, it measures “the likelihood of observing the dataset $\mathcal{D}$, given the parameters θ .” The coin example above, with $\mathcal{L}(p) = p^k(1-p)^{N-k}$, is exactly the special case of this general formula when $P(y|\mathbf{x}, \theta)$ is taken to be a Bernoulli distribution and the parameter $\theta = p$.

The Principle of Maximum Likelihood

Maximum Likelihood Estimation (MLE) seeks the parameters that maximize the likelihood function:

$$\theta^* = \arg \max_{\theta} \mathcal{L}(\theta). \quad (3.7)$$

Intuitively, this principle selects the parameters that make “the data actually observed the most likely to occur” — exactly the same thing as “picking the p that maximizes $\mathcal{L}(p)$ ” from the coin’s likelihood table above, except that here $\boldsymbol{\theta}$ is generally a high-dimensional vector, which can no longer simply be compared by tabulation, and instead requires calculus (differentiation) or numerical optimization (gradient descent) to locate the maximum.

The Log-Likelihood

Since the likelihood function has a product structure, taking its logarithm turns it into a sum:

$$\ell(\boldsymbol{\theta}) = \ln \mathcal{L}(\boldsymbol{\theta}) = \sum_{n=1}^N \ln P(y^{(n)} | \mathbf{x}^{(n)}, \boldsymbol{\theta}). \quad (3.8)$$

Since the logarithm is a monotonically increasing function, maximizing $\ell(\boldsymbol{\theta})$ is equivalent to maximizing $\mathcal{L}(\boldsymbol{\theta})$.

Maximizing the log-likelihood is equivalent to minimizing the **negative log-likelihood** (NLL):

$$\boxed{\mathcal{L}(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta}) = -\sum_{n=1}^N \ln P(y^{(n)} | \mathbf{x}^{(n)}, \boldsymbol{\theta}).} \quad (3.9)$$

This is exactly the “loss function” of machine learning. (Following the usual convention, we use $\mathcal{L}$ to denote both the “likelihood function” and the “loss function” — readers can tell them apart from context, and by whether a minus sign is present; it is the negative log-likelihood that needs to be **minimized**.) In the chapters that follow, when we discuss loss functions, we will see that nearly every common loss function — mean squared error, cross-entropy — can be **derived** from this negative-log-likelihood formula, simply by choosing a specific distributional form for $P(y|\mathbf{x}, \theta)$, rather than being constructed out of thin air.

3.4 Maximum A Posteriori Estimation (MAP)

The earlier example — “three tosses, three heads, MLE gives $p = 1$ ” — exposes a problem: **when the amount of data is small, MLE tends to give overconfident estimates, detached from common sense, at the extremes.** If, before tossing the coin, we already had reason to believe some prior knowledge along the lines of “the great majority of coins are close to fair,” could we incorporate this prior knowledge into the estimation process, making the result more “robust”? This is exactly the problem MAP is designed to solve.

Sometimes we hold prior knowledge or preferences about the parameters. In the Bayesian framework, this is introduced through a **prior distribution** over the parameters, $P(\boldsymbol{\theta})$.

Deriving Bayes’ theorem. We want to know how the probability distribution over the parameters $\boldsymbol{\theta}$ should be updated after seeing the data $\mathcal{D}$ — that is, we want $P(\boldsymbol{\theta}|\mathcal{D})$. This only requires applying the definition of conditional probability (see the “Preliminary Definitions” box earlier in this chapter) to the pair of random quantities $(\boldsymbol{\theta}, \mathcal{D})$ twice: once conditioning on $\mathcal{D}$, and once conditioning on $\boldsymbol{\theta}$. By definition,

$$P(\boldsymbol{\theta}|\mathcal{D}) = \frac{P(\boldsymbol{\theta}, \mathcal{D})}{P(\mathcal{D})}, \quad P(\mathcal{D}|\boldsymbol{\theta}) = \frac{P(\boldsymbol{\theta}, \mathcal{D})}{P(\boldsymbol{\theta})}.$$

The joint probability $P(\boldsymbol{\theta}, \mathcal{D})$ appearing in these two expressions is the same quantity; rewriting the second expression as $P(\boldsymbol{\theta}, \mathcal{D}) = P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})$ and substituting into the first gives **Bayes' theorem**:

$$P(\boldsymbol{\theta}|\mathcal{D}) = \frac{P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})}{P(\mathcal{D})}. \quad (3.10)$$

Each term here has a well-defined name: $P(\boldsymbol{\theta}|\mathcal{D})$ is called the **posterior distribution** (our belief about $\boldsymbol{\theta}$ after seeing the data); $P(\mathcal{D}|\boldsymbol{\theta})$ is exactly the likelihood function of Equation (3.6); $P(\boldsymbol{\theta})$ is called the **prior distribution** (our belief about $\boldsymbol{\theta}$ before seeing the data); and $P(\mathcal{D})$ is called the **evidence** — it does not depend on $\boldsymbol{\theta}$, and serves only as a normalization factor.

■ The Same Symbol, Playing a Different Role in Different Probability Objects

A mistake beginners very easily make here is treating $\boldsymbol{\theta}$ and $\mathcal{D}$ in $P(\mathcal{D}|\boldsymbol{\theta})$ and $P(\boldsymbol{\theta}|\mathcal{D})$ as two variables on “equal footing,” without noticing that the roles on either side of the vertical bar have actually swapped:

In the likelihood function $P(\mathcal{D}|\boldsymbol{\theta})$, $\boldsymbol{\theta}$ is a **given, fixed parameter** (treat it as a known constant plugged in), and $\mathcal{D}$ is the object this probability distribution actually describes — this is “if the parameter is $\boldsymbol{\theta}$, how likely is it that we would see this particular data?”

In the posterior distribution $P(\boldsymbol{\theta}|\mathcal{D})$, the roles are completely reversed: $\mathcal{D}$ becomes the **given, fixed data** (already observed, no longer varying), and $\boldsymbol{\theta}$ is now the object this probability distribution actually describes — this is “having seen this data, how likely is each possible value of the parameter?”

The same $\boldsymbol{\theta}$ is a background condition in $P(\mathcal{D}|\boldsymbol{\theta})$, yet becomes the random variable itself in $P(\boldsymbol{\theta}|\mathcal{D})$. Whatever sits to the right of the vertical bar “|” is always the side that is *given and held fixed*; whatever sits to the left is the side the probability distribution *actually describes* — when reading expressions of this kind, it is often easier to first ask who is fixed on the right of the bar, rather than starting from the left.

Maximum a posteriori estimation (MAP) seeks the parameters that maximize the posterior probability:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} P(\boldsymbol{\theta}|\mathcal{D}). \quad (3.11)$$

Substituting Equation (3.10):

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \frac{P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})}{P(\mathcal{D})}.$$

A general property of $\arg \max$: if $g(\boldsymbol{\theta}) = c \cdot f(\boldsymbol{\theta})$, where $c > 0$ is a constant that does not depend on $\boldsymbol{\theta}$, then g and f attain their maximum at the same $\boldsymbol{\theta}$ — because, for $c > 0$, multiplying by c does not change the relative ordering of the function's values anywhere, i.e., $f(\boldsymbol{\theta}_1) > f(\boldsymbol{\theta}_2) \Leftrightarrow c f(\boldsymbol{\theta}_1) > c f(\boldsymbol{\theta}_2)$, and so $\arg \max_{\boldsymbol{\theta}} g(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$. Here $P(\mathcal{D})$ does not depend on $\boldsymbol{\theta}$ (it is the result of integrating/summing over all possible $\boldsymbol{\theta}$, a fixed number), so taking $c = 1/P(\mathcal{D}) > 0$ lets us drop it from the $\arg \max$:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} [P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})].$$

Taking the logarithm (the logarithm is monotonically increasing and does not change the location of the $\arg \max$, as already noted when deriving the log-likelihood in §3.3):

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} [\ln P(\mathcal{D}|\boldsymbol{\theta}) + \ln P(\boldsymbol{\theta})],$$

or, equivalently, minimizing

$$\boxed{\mathcal{L}(\boldsymbol{\theta}) = -\ln P(\mathcal{D}|\boldsymbol{\theta}) - \ln P(\boldsymbol{\theta})}. \quad (3.12)$$

Physical analogy:

- $-\ln P(\mathcal{D}|\boldsymbol{\theta})$: the data term, analogous to “energy”;
- $-\ln P(\boldsymbol{\theta})$: the prior term, analogous to an “external field.”

The prior distribution $P(\boldsymbol{\theta})$ corresponds, in machine learning, to **regularization**.

A Worked Example with the Coin: How the Prior Corrects an Overconfident Estimate

Let us continue with the coin example and work through MAP concretely, directly answering the question left open at the end of the previous subsection. Choose, for p , a prior known as the **Beta distribution**:

$$P(p) \propto p^{\alpha-1} (1-p)^{\beta-1}. \quad (3.13)$$

■ Preliminary Note: The Meaning of the “ $\propto$ ” Notation

The symbol “ $\propto$ ” (proportional to) in Equation (3.13) indicates that only the part of the right-hand side that depends on p has been written out, and a normalization constant $C(\alpha, \beta)$, which does not depend on p , has been omitted; written out in full, this should be $P(p) = C(\alpha, \beta) p^{\alpha-1} (1-p)^{\beta-1}$, where $C(\alpha, \beta)$ is uniquely determined by the normalization condition $\int_0^1 P(p) dp = 1$. This plays exactly the same role as the partition function Z in §2.2: in $P(\mathbf{x}) = \frac{1}{Z} e^{-\beta E(\mathbf{x})}$, the part $e^{-\beta E(\mathbf{x})}$ depends on the state $\mathbf{x}$, while $1/Z$ is exactly the normalization constant that does not depend on $\mathbf{x}$. We can safely drop this constant because — as noted above in “a general property of $\arg \max$ ” — it does not affect the location of the $\arg \max$ (or $\arg \min$): whatever the specific value of $C(\alpha, \beta)$, the p that maximizes $P(p|\mathcal{D})$ is the same. The derivation below continues to make use of this fact, and only recovers the normalization constant at the very last step, if needed at all.

The shape of this prior happens to also be a product of powers of p — this is not an arbitrary choice, but because it “matches” the form of the Bernoulli likelihood (mathematically, this is called a **conjugate prior**), so that the posterior distribution remains in the same family, making the calculation very clean. Taking $\alpha = \beta$ makes the prior symmetric about $p = 0.5$; the larger α, β are, the more strongly the prior believes p should be close to 0.5.

The posterior is proportional to the likelihood times the prior:

$$P(p|\mathcal{D}) \propto p^k (1-p)^{N-k} \cdot p^{\alpha-1} (1-p)^{\beta-1} = p^{k+\alpha-1} (1-p)^{N-k+\beta-1}. \quad (3.14)$$

This has exactly the same form as the earlier MLE derivation, except that the exponents now carry the two additional terms $\alpha - 1$ and $\beta - 1$. We now fully repeat the three steps used in §3.3 — “take the logarithm, differentiate, set the result to zero” — simply replacing $p^k(1-p)^{N-k}$ with $p^{k+\alpha-1}(1-p)^{N-k+\beta-1}$ from Equation (3.14).

Step 1: take the logarithm. Since the normalization constant omitted in Equation (3.14) does not depend on p , taking the logarithm contributes only a negligible additive constant:

$$\ln P(p|\mathcal{D}) = (k + \alpha - 1) \ln p + (N - k + \beta - 1) \ln(1 - p) + \text{const.}$$

Step 2: differentiate with respect to p and set the result to zero.

$$\frac{d}{dp} \ln P(p|\mathcal{D}) = \frac{k + \alpha - 1}{p} - \frac{N - k + \beta - 1}{1 - p} = 0.$$

Step 3: solve for p . Multiplying both sides by $p(1-p)$:

$$(k + \alpha - 1)(1 - p) = (N - k + \beta - 1)p.$$

Expanding the left-hand side:

$$(k + \alpha - 1) - (k + \alpha - 1)p = (N - k + \beta - 1)p.$$

Moving all terms containing p to the right-hand side and combining:

$$k + \alpha - 1 = (N - k + \beta - 1)p + (k + \alpha - 1)p = [(N - k + \beta - 1) + (k + \alpha - 1)]p.$$

Combining like terms inside the brackets, $N - k$ and k cancel: $(N - k + \beta - 1) + (k + \alpha - 1) = N + \alpha + \beta - 2$, so

$$k + \alpha - 1 = (N + \alpha + \beta - 2)p,$$

solving,

$$\boxed{p_{\text{MAP}} = \frac{k + \alpha - 1}{N + \alpha + \beta - 2}} \quad (3.15)$$

An illuminating interpretation is to understand α, β as “virtual observation counts.” Look at the structure of the formula above — it is almost identical to $p_{\text{MLE}} = k/N$, except that the numerator has an extra $\alpha - 1$ added, and the denominator an extra $\alpha + \beta - 2$. This can be understood as: **the prior is equivalent to having already “virtually” observed $\alpha - 1$ heads and $\beta - 1$ tails before the actual coin tosses**, and these virtual observations join the real data in the “counting.” The more real data there is, the smaller the share these virtual observations occupy in the total, and the weaker the prior’s influence; the less data there is, the larger the relative weight of the virtual observations, and the more visibly the prior pulls the estimate.

Substituting specific numbers to verify this intuition (taking a relatively weak prior $\alpha = \beta = 2$, corresponding to having “virtually” observed 1 head and 1 tail each; and, for comparison, a stronger prior $\alpha = \beta = 5$):

Table 3.4 illustrates two phenomena worth remembering:

Table 3.4: Comparing the MLE and MAP Estimates

Data (N, k)	$p_{\text{MLE}} = k/N$	$p_{\text{MAP}}(\alpha = \beta = 2)$	$p_{\text{MAP}}(\alpha = \beta = 5)$
$N = 3, k = 3$ (all heads)	1.000	0.800	0.636
$N = 10, k = 7$	0.700	0.667	0.611
$N = 100, k = 70$	0.700	0.696	0.685

1. **A small sample with an extreme observation** (the row $N = 3, k = 3$): MLE gives the implausible result $p = 1$, while MAP pulls it back to 0.8 (weak prior) or 0.636 (strong prior) — this is exactly the answer to the question raised at the end of §3.3: prior knowledge (“most coins are close to fair”) successfully prevents the model from drawing the overconfident conclusion, from just 3 observations, that “this coin can never possibly land tails.”
2. **A large sample** (the row $N = 100, k = 70$): regardless of how strong the prior is, MAP’s result is already very close to MLE’s 0.700 — once the real data are plentiful enough, the share occupied by “virtual observations” in the total count becomes negligible, and **the influence of the prior is “swamped” by the data**. This is a concrete instance of a conclusion often cited in Bayesian statistics: once there is enough data, the specific choice of prior ceases to matter.

■ Think and Check

The prior $\alpha = \beta = 5$ in the table, compared with $\alpha = \beta = 2$, always pulls the estimate closer to 0.5 given the same data. Does this mean “the stronger the prior, the better”? If we actually have no idea whether the coin is biased, what risk does choosing a very strong prior carry?

Solution: If the prior is at odds with the facts — for example, if this coin genuinely is biased, with a true p of 0.9 — an overly strong prior will keep “pulling” the estimate away from the true value, and even a large amount of data will struggle to correct it. The strength of a prior needs to match one’s actual confidence in that prior knowledge; this is itself a trade-off to be made during modeling, not a matter of “the stronger, the safer.”

From Virtual Counts to L^2 Regularization: Another Common Prior

The Beta prior is a classic example in the setting of a discrete probability parameter; in the setting of a continuous parameter (such as the weights $\boldsymbol{\theta} \in \mathbb{R}^d$ of linear regression), the most common prior is the Gaussian distribution:

$$P(\boldsymbol{\theta}) \propto \exp\left(-\frac{\lambda}{2}\|\boldsymbol{\theta}\|^2\right). \quad (3.16)$$

Written out in full, this should be $P(\boldsymbol{\theta}) = C(\lambda) \exp(-\frac{\lambda}{2}\|\boldsymbol{\theta}\|^2)$, where $C(\lambda)$ is a normalization constant that does not depend on $\boldsymbol{\theta}$. Substituting this into the $-\ln P(\boldsymbol{\theta})$ term of the general

MAP objective (3.12):

$$\begin{aligned}
 -\ln P(\boldsymbol{\theta}) &= -\ln \left[C(\lambda) \exp \left(-\frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 \right) \right] \\
 &= -\ln C(\lambda) - \ln \exp \left(-\frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 \right) \quad (\text{using } \ln(ab) = \ln a + \ln b) \\
 &= \underbrace{-\ln C(\lambda)}_{\text{a constant, independent of } \boldsymbol{\theta}} + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 \quad (\text{using } \ln e^x = x).
 \end{aligned}$$

Substituting into Equation (3.12), and dropping $-\ln C(\lambda)$ according to the principle that “a constant does not affect $\arg \min$ ” (perfectly symmetric to the earlier property “concerning $\arg \max$ ”; we simply need to flip the direction of the inequality):

$$\mathcal{L}(\boldsymbol{\theta}) = -\ln P(\mathcal{D}|\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2. \quad (3.17)$$

This is exactly **ridge regression**, also known as **weight decay**. It is the continuous version of the same logic as the “virtual count” intuition for the Beta prior above: the Gaussian prior “virtually” pulls $\boldsymbol{\theta}$ toward $\mathbf{0}$ (since a Gaussian distribution has its highest probability density at $\boldsymbol{\theta} = \mathbf{0}$), with the strength of this pull controlled by λ — the larger λ is, the more it is as if there were more “virtual evidence,” and the less the model dares to let the parameters stray far from zero. This is exactly the mathematical essence of how regularization prevents overfitting: **using prior knowledge to counterbalance the overconfidence that arises when data are scarce** — precisely the same phenomenon observed earlier in the coin example.

■ Summary

Maximum likelihood estimation (MLE) answers the question “which set of parameters makes the observed data seem the least surprising,” and is, at bottom, a generalization of frequency estimation; when the amount of data is small, it can give overconfident, extreme results. **Maximum a posteriori estimation (MAP)** builds on MLE by introducing a prior, which is equivalent to “mixing in” a certain amount of virtual prior evidence with the data; the less data there is, the greater the prior’s influence, and the more data there is, the more the prior’s influence gets “swamped.” Whether the prior is a Beta prior or a Gaussian prior, regularization can be understood as “virtual observations” and the real data jointly determining the final estimate.

At this point, the book has laid out, separately, the logic of “from energy to probability” in statistical physics, and the logic of “from data to parameter estimation” in machine learning. The next chapter will formally unify these two chains of logic — readers will see that the negative log-likelihood loss is exactly the energy function, presented in a different form.

Exercises

Foundational

Exercise 3.1: MLE and MAP for Another Coin

Still the “coin toss” setup from §3.3: the probability the coin lands heads is an unknown constant $p \in [0, 1]$. This time, the coin is tossed independently $N = 16$ times, and $k = 11$ heads are observed.

- Write down the likelihood function $L(p)$, and, following the differentiation steps in §3.3, find p_{MLE} ;
- Take a $\text{Beta}(\alpha = 3, \beta = 3)$ prior, and, following the formula in §3.4, find p_{MAP} , and state whether it has been pulled toward 0.5 or away from 0.5 relative to p_{MLE} ;
- If the observed proportion stays the same but the sample size grows tenfold ($N = 160$, $k = 110$), recompute p_{MAP} under the same prior $\alpha = \beta = 3$, and compare its gap from $p_{\text{MLE}} = k/N$ with the previous case — has it grown larger or smaller?
- Combining the numerical results of (b) and (c), use the “virtual observation count” intuition (§3.4) to explain: why does a larger sample size weaken the prior’s influence on the MAP estimate?

Hint: $p_{\text{MLE}} = k/N$, $p_{\text{MAP}} = \frac{k + \alpha - 1}{N + \alpha + \beta - 2}$.

Exercise 3.2: Maximum Likelihood Estimation for the Gaussian Distribution

Let $x^{(1)}, \dots, x^{(N)}$ be N samples drawn independently from a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, with both the mean μ and the variance σ^2 unknown. The probability density of the Gaussian distribution is

$$P(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

- Following the steps used for the coin example in §3.3 (first write the likelihood, then take the logarithm, then differentiate), write down the log-likelihood $\ell(\mu, \sigma^2)$;
- Take the partial derivative with respect to μ and set it to zero, solving for μ_{MLE} ;
- Take the partial derivative with respect to σ^2 and set it to zero (treat σ^2 as a single variable to differentiate with respect to; there is no need to split it into a derivative with respect to σ), solving for σ_{MLE}^2 ;
- (Reflection) σ_{MLE}^2 is computed by substituting in μ_{MLE} , which was estimated from the same set of data — could this cause σ_{MLE}^2 to systematically underestimate the true variance? (Hint: this intuition is correct; σ_{MLE}^2 is a biased estimator, and an unbiased estimator requires replacing the denominator N with $N - 1$; if you are curious, try substituting two concrete data samples with $N = 2$ into both formulas and see how much they differ.)

$$\text{Hint: } \mu_{\text{MLE}} = \frac{1}{N} \sum_n x^{(n)}, \sigma_{\text{MLE}}^2 = \frac{1}{N} \sum_n (x^{(n)} - \mu_{\text{MLE}})^2.$$

Intermediate

■ A Note on Notation

The exercises below begin using the notation $\mathbb{E}[\cdot]$ for expectation value, which is exactly the same operation as the notation $\langle \cdot \rangle$ used conventionally in physics in Chapter 2 (a probability-weighted sum or integral over all possible values); it is simply that the machine-learning literature is more accustomed to the former. The rest of this book will use whichever notation fits the context.

Exercise 3.3: Fisher Information and the Cramér–Rao Bound

For a parameterized probability distribution $P(x; \theta)$:

- Define the Fisher information $I(\theta) = \mathbb{E} \left[\left(\frac{\partial \ln P}{\partial \theta} \right)^2 \right]$;
- Prove that $I(\theta) = -\mathbb{E} \left[\frac{\partial^2 \ln P}{\partial \theta^2} \right]$ (assuming differentiation and integration can be interchanged);
- State the Cramér–Rao inequality: for any unbiased estimator $\hat{\theta}$ of the parameter θ , its variance satisfies $\text{Var}(\hat{\theta}) \geq \frac{1}{I(\theta)}$;
- For the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, compute the Fisher information with respect to μ and σ^2 ;
- For the Bernoulli distribution $\text{Ber}(p)$, compute the Fisher information with respect to p .

Exercise 3.4: MAP with a Gaussian Prior: A Continuous Version of “Virtual Counts”

§3.4 used a Beta prior to handle MAP estimation for the coin’s bias p , a *discrete* parameter. This problem applies the same logic to a *continuous* parameter. Let the data $x^{(1)}, \dots, x^{(N)}$ be independently and identically distributed as $\mathcal{N}(\mu, \sigma^2)$, where the noise variance σ^2 is a known constant (it does not need to be estimated), and only the mean μ is unknown. Choose a Gaussian prior for μ ,

$$P(\mu) \propto \exp \left(-\frac{(\mu - \mu_0)^2}{2\sigma_0^2} \right),$$

where μ_0, σ_0^2 are a pre-specified prior mean and prior variance — the smaller σ_0^2 is, the more strongly the prior pulls μ toward μ_0 (this plays exactly the same role as larger α, β giving a stronger prior in §3.4).

- Following the steps used for the coin example in §3.4 (take the logarithm, differentiate,

set the result to zero), derive μ_{MAP} , and show that the result can be written as

$$\mu_{\text{MAP}} = \frac{N\sigma_0^2 \bar{x} + \sigma^2 \mu_0}{N\sigma_0^2 + \sigma^2}, \quad \bar{x} \equiv \frac{1}{N} \sum_{n=1}^N x^{(n)};$$

- (b) Verify that $\mu_{\text{MAP}} \rightarrow \bar{x}$ (i.e., the μ_{MLE} of the previous exercise) as $N \rightarrow \infty$;
- (c) Verify that $\mu_{\text{MAP}} \rightarrow \bar{x}$ also holds as $\sigma_0^2 \rightarrow \infty$ (i.e., when the prior is completely “uninformative”);
- (d) Rewrite μ_{MAP} as a weighted average, $\mu_{\text{MAP}} = w\bar{x} + (1 - w)\mu_0$, find the weight w , and explain the meaning of w using the language of “virtual observation count” from §3.4 — which quantity here plays the role of the “virtual sample size”?

Challenge

Exercise 3.5: Natural Gradient Descent

For a parameterized family of distributions $\{P_\theta(x)\}$:

- (a) Define the Fisher information matrix $F_{ij}(\theta) = \mathbb{E}_{P_\theta} \left[\frac{\partial \ln P_\theta}{\partial \theta_i} \frac{\partial \ln P_\theta}{\partial \theta_j} \right]$;
- (b) Prove that, for an infinitesimal change in the parameters, the second-order approximation of the KL divergence is

$$D_{KL}(P_\theta \| P_{\theta+d\theta}) \approx \frac{1}{2} \sum_{i,j} F_{ij}(\theta) d\theta_i d\theta_j;$$

- (c) Define the natural gradient $\tilde{\nabla}_\theta \mathcal{L} = F^{-1}(\theta) \nabla_\theta \mathcal{L}$;
- (d) For the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, compute the Fisher information matrix;
- (e) For logistic regression, $P(y = 1|x) = \sigma(\theta^T x)$, what is the relationship between the Fisher information matrix and the Hessian matrix?
- (f) Discuss the advantage of natural gradient descent over ordinary gradient descent: why is the natural gradient the steepest direction under the “Riemannian geometry of parameter space”?

Chapter 4

Loss Functions as Energy Functions

*Fair is foul, and foul is fair:
Hover through the fog and filthy air.*

— Shakespeare, *Macbeth*, Act I, Scene
1, the Three Witches

Objectives

- Understand the mathematical correspondence between loss functions and energy functions — the unifying perspective that “a probabilistic model is a Boltzmann distribution”;
- Through three concrete examples — linear regression, logistic regression, and softmax regression — master the standard procedure for deriving a loss function: “choose a noise distribution → write down the likelihood → take the negative log-likelihood”;
- Derive the closed-form expressions for the mean-squared-error and cross-entropy losses, and understand the algebraic and geometric meaning of the normal equation;
- Understand how a single temperature parameter uniformly controls both a regression model’s sensitivity to error and a classification model’s degree of certainty.

■ Chapter Guide

The question this chapter answers: Where do the loss functions of machine learning — mean squared error, cross-entropy — that look, on the surface, like “ad hoc engineering choices” actually come from? Is there a unified method for deriving them systematically, rather than inventing them one at a time by trial and error?

Correspondence with statistical physics: This chapter establishes the single most important analogy in the whole book — that the Boltzmann distribution $P(\mathbf{x}) \propto e^{-\beta E(\mathbf{x})}$ and the probabilistic model $P(y|\mathbf{x}, \boldsymbol{\theta}) \propto e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})}$ share exactly the same mathematical structure; “loss function” is simply “energy function” under a different name (the precise sense in which this holds, and the precise conditions required, are spelled out below in “An important caveat

about the ‘energy’ row” — please do not stop at this one-sentence intuitive summary).

Looking ahead: Chapter 5 will point out that a question this chapter deliberately sidesteps — how to compute the partition function Z in general — is in fact the single greatest difficulty in training real models; the “temperature” parameter that appears in this chapter’s treatment of softmax will reappear repeatedly in Chapters 6 and 7, playing the same role of “controlling the balance between certainty and flexibility.”

Chapter 2 built the physical chain “energy $\rightarrow$ Boltzmann distribution $\rightarrow$ partition function”; Chapter 3 built the machine-learning chain “data $\rightarrow$ likelihood function $\rightarrow$ parameter estimation.” This chapter’s task is to unify these two chains: the loss functions of machine learning that look “ad hoc” (mean squared error, cross-entropy, ...) can, in fact, all be systematically derived from a single unified operation — “choose a specific distributional form for the probabilistic model, then take the negative log-likelihood” — with no additional constructive tricks required.

4.1 The Core Analogy

We can now establish the first major analogy. Let us first pin down the most basic correspondence: the **microstate** $\mathbf{x}$ of statistical physics (a spin configuration, a configuration of particle positions) corresponds, in machine learning, to the **feature vector** $\mathbf{x}$, or the **model’s latent variables/configuration** — the two are simply different names for the same role, “the specific state the system currently occupies,” and the values involved can be either discrete or continuous. This is exactly what the first row of Chapter 2’s Table 2.8 promised to fill in; comparing the two expressions below immediately shows how the next few rows are filled in as well:

Table 4.1: The Core Analogy Between Statistical Physics and Machine Learning — Filling In the First Four Rows of Chapter 2’s Table 2.8

Statistical physics	Machine learning
Microstate $\mathbf{x}$	Feature vector / latent variable $\mathbf{x}$ (or model configuration)
Boltzmann distribution $P(\mathbf{x}) \propto e^{-\beta E(\mathbf{x})}$	Probabilistic model $P(y \mathbf{x}, \boldsymbol{\theta}) \propto e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})}$
Energy function $E(\mathbf{x})$ (the bare quantity inside the exponent)	Loss $\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})$ (the bare quantity inside the exponent, same $\mathcal{L}$ as the row below)
Inverse temperature $\beta = 1/(k_B T)$	Usually taken as $\beta = 1$, or used as a regularization parameter
Partition function $Z = \sum e^{-\beta E}$	Normalization constant $Z(\boldsymbol{\theta}) = \sum_y e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})}$
Minimizing free energy $F = E - TS$	Minimizing loss $\mathcal{L} = -\ln P$

An important caveat about the “energy” row: the pairing of “energy function $E(\mathbf{x})$ ” with “loss $\mathcal{L}$ ” in the table above matches **the role each plays inside its own exponent**, not a claim that “the bare energy in physics is numerically equal to the complete negative log-likelihood in machine learning.” In physics, $-\ln P(\mathbf{x}) = \beta E(\mathbf{x}) + \ln Z$ (the partition function

Z is, in general, not equal to 1 — it is a genuine, present quantity), so $E(\mathbf{x})$ itself is *not* equal to $-\ln P(\mathbf{x})$; the reason the machine-learning side can write $\mathcal{L} = -\ln P$ directly is that $\mathcal{L}$ is **by definition** the *complete* negative log-likelihood, already absorbing the normalization term (the remark box following “Key Insight” below makes this precise). In short: “ $E(\mathbf{x})$ ” corresponds to the **role $\mathcal{L}$ plays inside the exponent**, not to a claim that $\mathcal{L}$ itself must always equal the bare, unnormalized physical energy.

Against Chapter 2’s seed table: the five rows “microstate,” “Boltzmann distribution,” “energy function,” “inverse temperature,” and “partition function” have now all been formally filled in; the “free energy” row still awaits Chapter 6 — the last row of the present table, “minimizing free energy = minimizing loss,” is for now only a formal analogy offered in advance, and Chapter 6 will put this analogy on solid footing using the rigorous correspondence between variational free energy and the evidence lower bound.

Key insight:

$$\boxed{\text{Loss function} \leftrightarrow \text{Energy function}} \quad (4.1)$$

The precise meaning of this statement is: **for any probabilistic model $P(y|\mathbf{x}, \boldsymbol{\theta})$, provided it is written in Boltzmann form,**

$$P(y|\mathbf{x}, \boldsymbol{\theta}) = \frac{1}{Z(\mathbf{x}, \boldsymbol{\theta})} e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})}, \quad Z(\mathbf{x}, \boldsymbol{\theta}) = \sum_y e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})}, \quad (4.2)$$

then “maximizing the likelihood” (the subject of §3.3) automatically becomes equivalent to “minimizing $\mathcal{L}$ ” — and this $\mathcal{L}$, in its mathematical structure, plays **exactly the same role** that the energy function $E(\mathbf{x})$ plays in §2.2: higher probability corresponds to smaller $\mathcal{L}$, lower probability to larger $\mathcal{L}$. Conversely, this also means that **any energy function automatically defines a probabilistic model, together with its associated loss function** — this is exactly what the next three sections demonstrate concretely: given a “physical system” (that is, a specific distributional form for the noise), the loss function is **uniquely derived** from that distribution, rather than being “invented” by an engineer through trial and error.

■ A Note on Notation: “Loss $\mathcal{L}$ ” and “Energy E ” Are Not Always the Same Object

The $\mathcal{L}$ in Equation (4.2) is the **complete negative log-likelihood**, $\mathcal{L} := -\ln P$ — by this definition, $Z(\mathbf{x}, \boldsymbol{\theta}) = \sum_y e^{-\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})} = \sum_y P(y|\mathbf{x}, \boldsymbol{\theta}) \equiv 1$ holds identically for every $\boldsymbol{\theta}$ (this is a direct consequence of probability normalization, not an assumption requiring separate verification), so “maximizing the likelihood $\Leftrightarrow$ minimizing $\mathcal{L}$ ” is a tautology that holds unconditionally.

But starting from §4.4, another piece of notation will also appear — “energy” E , referring to the **bare score** produced by the model (for example, $E_k = -\boldsymbol{\theta}_k^T \mathbf{x}$ in softmax), which **does not include** the normalization term $\ln Z$. In general $\mathcal{L} = E + \ln Z$; only when $\ln Z$ does not depend on $\boldsymbol{\theta}$ (for example, in linear regression, where the noise variance σ is a fixed hyperparameter not involved in the optimization) do E and $\mathcal{L}$ differ by a constant independent of $\boldsymbol{\theta}$, in which case “minimizing E ” and “minimizing $\mathcal{L}$ ” are equivalent optimization problems; once Z (as in softmax) depends explicitly on $\boldsymbol{\theta}$, E and $\mathcal{L}$ are two distinct quantities that cannot be substituted for one another. The last row of the table

above, “loss function $\leftrightarrow$ energy function,” always refers to the **complete** $\mathcal{L}$; readers should watch, in the concrete examples that follow, for whether the “energy” E under discussion still carries an additional $\ln Z$ term.

■ Think and Check

By the correspondence in Table 4.1: if the loss $\mathcal{L}(y, \mathbf{x}, \boldsymbol{\theta})$ the model assigns to some sample is very large, does the model regard this sample as “extremely unlikely to occur” or “extremely likely to occur”? Is this the same question as “is the Boltzmann probability of a very-high-energy microstate large or small?” from §2.2?

Solution: From $P \propto e^{-\mathcal{L}}$, the larger $\mathcal{L}$ is, the closer $e^{-\mathcal{L}}$ gets to 0, and the smaller the probability; a large loss $\Leftrightarrow$ the model regards the sample as “extremely unlikely to occur” — this is exactly the same mathematical structure as “a high-energy state has low Boltzmann probability,” recurring across two different fields.

Below, we work through this “from distribution to loss function” derivation procedure in full, using three concrete examples.

4.2 Example One: Linear Regression

Model Setup

Assume the true relationship is

$$y = \boldsymbol{\theta}^T \mathbf{x} + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise (the symbol “ $\sim$ ” is read “is distributed as”: $\epsilon \sim \mathcal{N}(0, \sigma^2)$ means ϵ is a random variable whose values are drawn from a normal distribution with mean 0 and variance σ^2 — this differs from “ $\epsilon = \mathcal{N}(0, \sigma^2)$ ”; ϵ is not equal to some distribution, but is sampled from it). This implies

$$y \mid \mathbf{x}, \boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}^T \mathbf{x}, \sigma^2),$$

with probability density¹

$$P(y \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \boldsymbol{\theta}^T \mathbf{x})^2}{2\sigma^2}\right). \quad (4.3)$$

¹The line “ $y \mid \mathbf{x}, \boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}^T \mathbf{x}, \sigma^2)$ ” only declares which family of distributions y (conditional on given $\mathbf{x}, \boldsymbol{\theta}$) belongs to, and what its parameters are; it does not directly give a formula one can compute with. $P(y \mid \mathbf{x}, \boldsymbol{\theta})$ is the concrete function corresponding to that declaration — substituting a value of y into it yields an actual number (a probability density). The two statements say the same thing: the former is concise, while the latter is the explicit formula needed for the differentiation that follows, which is why the two lines appear back to back.

Deriving the Negative Log-Likelihood

The log-likelihood is (using $\ln(ab) = \ln a + \ln b$ and $\ln e^x = x$, applying $\ln$ to the product structure of the probability density (4.3))

$$\begin{aligned}\ln P(y|\mathbf{x}, \boldsymbol{\theta}) &= \ln \frac{1}{\sqrt{2\pi\sigma^2}} + \ln \exp\left(-\frac{(y - \boldsymbol{\theta}^T \mathbf{x})^2}{2\sigma^2}\right) \\ &= -\frac{1}{2} \ln(2\pi\sigma^2) - \frac{(y - \boldsymbol{\theta}^T \mathbf{x})^2}{2\sigma^2}.\end{aligned}$$

For the entire dataset:

$$\begin{aligned}\ell(\boldsymbol{\theta}) &= \sum_{n=1}^N \ln P(y^{(n)}|\mathbf{x}^{(n)}, \boldsymbol{\theta}) \\ &= -\frac{1}{2\sigma^2} \sum_{n=1}^N (y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)})^2 - \frac{N}{2} \ln(2\pi\sigma^2).\end{aligned}$$

The negative log-likelihood (the loss function) is

$$\mathcal{L}(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta}) = \frac{1}{2\sigma^2} \sum_{n=1}^N (y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)})^2 + \text{const.}$$

Dropping the constant term and the overall constant factor gives the **Mean Squared Error** (MSE):

$$\boxed{\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{n=1}^N (y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)})^2.} \quad (4.4)$$

The most important point to emphasize in this derivation is that mean squared error was not “designed” as a loss function; it was **automatically derived**, via the negative log-likelihood, from the physical assumption that “the noise follows a Gaussian distribution.” If the noise instead follows a different distribution (as in the two examples that follow), the resulting loss function will be entirely different — the specific form of a loss function is, in essence, a direct reflection of the assumption about “how the data are contaminated by noise.”

Physical Interpretation

Viewing the MSE as an energy function:

$$E(\boldsymbol{\theta}) = \frac{1}{2} \sum_{n=1}^N (y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)})^2.$$

- Each prediction error $(y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)})^2$ is analogous to the potential energy of a “spring”;
- Minimizing the energy corresponds to finding the equilibrium position of all the “springs” simultaneously;

- The noise standard deviation σ is analogous to “temperature”: large $\sigma \rightarrow$ the system is insensitive to error (high temperature, large fluctuations allowed); small $\sigma \rightarrow$ the system is sensitive to error (low temperature, exact fitting required).

Let us verify this “ σ is analogous to temperature” claim by substituting concrete numbers: suppose some sample has a residual $y - \theta^T \mathbf{x} = 5$ (¥10,000), and compute the probability density corresponding to this residual at $\sigma = 2$ (low temperature, the model is “strict”) and at $\sigma = 10$ (high temperature, the model is “lenient”):

Table 4.2: The Probability Density for a Residual of 5 (¥10,000) at Different Values of σ

σ	Probability density $P(y \mathbf{x}, \theta)$ at a residual of 5
2 (low temperature)	0.0088
10 (high temperature)	0.0352

The very same “¥50,000 error” looks quite improbable to the “low-temperature” (strict) model (a small probability density), while it seems entirely unremarkable to the “high-temperature” (lenient) model (a probability density 4 times larger) — this is exactly the rule from §2.2, that “temperature controls a system’s sensitivity to the level of energy,” showing up unchanged in a regression problem.

Fitting the House-Price Data

Let us now actually fit the house-price data of Table 3.1, working the normal equation $\theta^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ of Equation (4.5) all the way through. Take the model $\hat{y} = \theta_0 + \theta_1 x$ (θ_0 is the intercept term, equivalent to augmenting $\mathbf{x}$ with an extra component that is always 1).

Step 1: write down the design matrix and the label vector. Augmenting each sample’s feature to $(1, x^{(n)})$ (1 corresponds to the intercept term):

$$\mathbf{X} = \begin{pmatrix} 1 & 50 \\ 1 & 70 \\ 1 & 90 \\ 1 & 110 \\ 1 & 130 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 150 \\ 205 \\ 260 \\ 310 \\ 365 \end{pmatrix}.$$

Step 2: compute $\mathbf{X}^T \mathbf{X}$. Computing term by term via matrix multiplication ($N = 5$):

$$(\mathbf{X}^T \mathbf{X})_{11} = \sum_n 1 \cdot 1 = 5, \quad (\mathbf{X}^T \mathbf{X})_{12} = (\mathbf{X}^T \mathbf{X})_{21} = \sum_n 1 \cdot x^{(n)} = \sum_n x^{(n)},$$

$$(\mathbf{X}^T \mathbf{X})_{22} = \sum_n (x^{(n)})^2.$$

Substituting the data from the table, $\sum_n x^{(n)} = 50 + 70 + 90 + 110 + 130 = 450$, $\sum_n (x^{(n)})^2 = 50^2 + 70^2 + 90^2 + 110^2 + 130^2 = 2500 + 4900 + 8100 + 12100 + 16900 = 44500$, so

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 5 & 450 \\ 450 & 44500 \end{pmatrix}.$$

Step 3: compute $\mathbf{X}^T \mathbf{y}$. Entirely analogously,

$$(\mathbf{X}^T \mathbf{y})_1 = \sum_n y^{(n)}, \quad (\mathbf{X}^T \mathbf{y})_2 = \sum_n x^{(n)} y^{(n)}.$$

$\sum_n y^{(n)} = 150 + 205 + 260 + 310 + 365 = 1290$; $\sum_n x^{(n)} y^{(n)} = 50 \times 150 + 70 \times 205 + 90 \times 260 + 110 \times 310 + 130 \times 365 = 7500 + 14350 + 23400 + 34100 + 47450 = 126800$, so

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 1290 \\ 126800 \end{pmatrix}.$$

Step 4: invert the 2×2 matrix $\mathbf{X}^T \mathbf{X}$. Using the 2×2 matrix inversion formula $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$, first compute the determinant:

$$\det(\mathbf{X}^T \mathbf{X}) = 5 \times 44500 - 450 \times 450 = 222500 - 202500 = 20000,$$

so

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{20000} \begin{pmatrix} 44500 & -450 \\ -450 & 5 \end{pmatrix}.$$

Step 5: substitute into $\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$.

$$\begin{aligned} \theta_0^* &= \frac{1}{20000} (44500 \times 1290 - 450 \times 126800) = \frac{57,405,000 - 57,060,000}{20000} = \frac{345,000}{20000} = 17.25, \\ \theta_1^* &= \frac{1}{20000} (-450 \times 1290 + 5 \times 126800) = \frac{-580,500 + 634,000}{20000} = \frac{53,500}{20000} = 2.675. \end{aligned}$$

So we obtain

$$\theta_0^* = 17.25, \quad \theta_1^* = 2.675.$$

That is: the model believes that for every additional 1 m^2 of floor area, the price rises by about ¥26,750, plus a fixed base price of about ¥172,500. Substituting these parameters back in, and comparing the predicted values with the true values:

Table 4.3: The Linear-Regression Fit to the House-Price Data

Sample n	True price $y^{(n)}$	Predicted price $\hat{y}^{(n)} = \theta_0^* + \theta_1^* x^{(n)}$	Residual
1	150	151.0	-1.0
2	205	204.5	+0.5
3	260	258.0	+2.0
4	310	311.5	-1.5
5	365	365.0	0.0

The corresponding “energy” (the MSE loss, not divided by N) is $E(\boldsymbol{\theta}^*) = \frac{1}{2} \sum (\text{residual})^2 = 3.75$ — this is **the lowest energy achieved by any choice of (θ_0, θ_1)** . For comparison, take an arbitrary, “seemingly reasonable” set of parameters, $\theta_0 = 0, \theta_1 = 2.5$ (simply assuming “price

is 2.5 times floor area, with no base price”); the resulting predicted residuals are 25, 30, 35, 35, 40, giving an energy of $E = 2787.5$ — more than 700 times larger than the energy of the optimal solution. This concretely illustrates the substance of “minimizing energy”: **among all possible ways of describing the data, picking out the one whose total “sense of mismatch” (the sum of squared residuals) is smallest.**

■ Think and Check

If the noise standard deviation is switched from “low temperature” $\sigma = 2$ to “high temperature” $\sigma = 10$, will the optimal solution θ_0^*, θ_1^* change?

Solution: No. Looking at $\mathcal{L}(\theta) = \frac{1}{2\sigma^2} \sum (y - \theta^T \mathbf{x})^2 + \text{const}$, σ is only an overall scaling constant, and does not affect the answer to the question “which θ minimizes the sum.” σ affects only how steep the loss surface is (which in turn affects the numerical behavior of the optimization and the resulting confidence intervals), not the location of the optimum itself — this point will be used later when optimization algorithms are discussed.

The Analytic Solution

Before differentiating, let us first establish a rule of vector calculus that will be used repeatedly (both in this section and in the gradient derivation for logistic regression in §4.3).

■ Preliminary Rule: $\partial(\theta^T \mathbf{x})/\partial \theta = \mathbf{x}$

Viewing $\theta^T \mathbf{x} = \sum_{i=1}^d \theta_i x_i$ as a multivariate function of $\theta = (\theta_1, \dots, \theta_d)$, take the partial derivative with respect to the j -th component:

$$\frac{\partial}{\partial \theta_j} \sum_{i=1}^d \theta_i x_i = x_j$$

(because only the term $i = j$ in the sum contains θ_j ; every other term has zero derivative with respect to θ_j). Arranging the results for the d components $j = 1, \dots, d$ into a vector gives

$$\frac{\partial(\theta^T \mathbf{x})}{\partial \theta} = \mathbf{x}.$$

Differentiating with respect to θ and setting the result to zero. Using the rule above together with the chain rule:

$$\frac{\partial}{\partial \theta} (y^{(n)} - \theta^T \mathbf{x}^{(n)})^2 = 2(y^{(n)} - \theta^T \mathbf{x}^{(n)}) \cdot \frac{\partial}{\partial \theta} (y^{(n)} - \theta^T \mathbf{x}^{(n)}) = 2(y^{(n)} - \theta^T \mathbf{x}^{(n)}) \cdot (-\mathbf{x}^{(n)}),$$

summing over Equation (4.4)’s $\mathcal{L} = \frac{1}{2} \sum_n (\dots)^2$ and using the expression above (the leading $\frac{1}{2}$ exactly cancels the factor of 2 produced by differentiation):

$$\frac{\partial \mathcal{L}}{\partial \theta} = - \sum_{n=1}^N (y^{(n)} - \theta^T \mathbf{x}^{(n)}) \mathbf{x}^{(n)} = 0.$$

Rearranging (splitting the sum into two parts and moving terms):

$$\sum_{n=1}^N y^{(n)} \mathbf{x}^{(n)} = \sum_{n=1}^N (\boldsymbol{\theta}^T \mathbf{x}^{(n)}) \mathbf{x}^{(n)}.$$

In each term $(\boldsymbol{\theta}^T \mathbf{x}^{(n)}) \mathbf{x}^{(n)}$ on the right-hand side, $\boldsymbol{\theta}^T \mathbf{x}^{(n)} = (\mathbf{x}^{(n)})^T \boldsymbol{\theta}$ is a scalar, and a scalar can be freely reordered with a vector:

$$(\boldsymbol{\theta}^T \mathbf{x}^{(n)}) \mathbf{x}^{(n)} = \mathbf{x}^{(n)} [(\mathbf{x}^{(n)})^T \boldsymbol{\theta}] = [\mathbf{x}^{(n)} (\mathbf{x}^{(n)})^T] \boldsymbol{\theta},$$

where the last step uses the associativity of matrix multiplication ($\mathbf{x}^{(n)}$ is a $d \times 1$ column vector, $(\mathbf{x}^{(n)})^T$ a $1 \times d$ row vector, $\mathbf{x}^{(n)} (\mathbf{x}^{(n)})^T$ a $d \times d$ matrix, and multiplying by $\boldsymbol{\theta}$ again gives a $d \times 1$ column vector — the dimensions are self-consistent throughout). Thus

$$\sum_{n=1}^N y^{(n)} \mathbf{x}^{(n)} = \left[\sum_{n=1}^N \mathbf{x}^{(n)} (\mathbf{x}^{(n)})^T \right] \boldsymbol{\theta}.$$

Define the design matrix $\mathbf{X} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}]^T \in \mathbb{R}^{N \times d}$ (the n -th row is $(\mathbf{x}^{(n)})^T$) and the label vector $\mathbf{y} = [y^{(1)}, \dots, y^{(N)}]^T \in \mathbb{R}^N$. Checking term by term against the definition of matrix multiplication: $(\mathbf{X}^T \mathbf{X})_{ij} = \sum_n X_{ni} X_{nj} = \sum_n (\mathbf{x}^{(n)})_i (\mathbf{x}^{(n)})_j$, which is exactly the (i, j) -th entry of $\sum_n \mathbf{x}^{(n)} (\mathbf{x}^{(n)})^T$, so $\mathbf{X}^T \mathbf{X} = \sum_n \mathbf{x}^{(n)} (\mathbf{x}^{(n)})^T$; similarly $(\mathbf{X}^T \mathbf{y})_i = \sum_n X_{ni} y^{(n)} = \sum_n (\mathbf{x}^{(n)})_i y^{(n)}$, i.e., $\mathbf{X}^T \mathbf{y} = \sum_n y^{(n)} \mathbf{x}^{(n)}$. The equation above can therefore be written as

$$\mathbf{X}^T \mathbf{y} = \mathbf{X}^T \mathbf{X} \boldsymbol{\theta}.$$

If $\mathbf{X}^T \mathbf{X}$ is invertible, left-multiplying both sides by $(\mathbf{X}^T \mathbf{X})^{-1}$ gives the solution

$$\boxed{\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.} \quad (4.5)$$

This is the famous **normal equation**; the hand-computed house-price example above (Table 4.3) is exactly a concrete instance of this formula for $d = 2$ (including the intercept) and $N = 5$.

Geometric Interpretation

In N -dimensional space, $\mathbf{y}$ is a vector, and $\mathbf{X}\boldsymbol{\theta}$ spans a d -dimensional subspace (the column space). The optimal $\boldsymbol{\theta}^*$ is exactly the one for which $\mathbf{X}\boldsymbol{\theta}^*$ is the orthogonal projection of $\mathbf{y}$ onto that subspace. The residual vector $\mathbf{y} - \mathbf{X}\boldsymbol{\theta}^*$ is perpendicular to the column space, that is,

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}^*) = 0.$$

Expanding the left-hand side by distributivity, $\mathbf{X}^T \mathbf{y} - \mathbf{X}^T \mathbf{X} \boldsymbol{\theta}^* = 0$, i.e., $\mathbf{X}^T \mathbf{y} = \mathbf{X}^T \mathbf{X} \boldsymbol{\theta}^*$ — exactly the normal equation of Equation (4.5). “The optimal solution makes the residual orthogonal to the column space” and “the optimal solution makes the loss function’s gradient vanish” turn out to be two ways of saying the very same thing.

4.3 Example Two: Logistic Regression

Model Setup

For a binary classification problem, $y \in \{0, 1\}$, modeled as

$$P(y = 1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \mathbf{x}), \quad (4.6)$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is the **sigmoid function** (or logistic function).

Properties of the Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}.$$

Properties:

1. **Range:** $\sigma(z) \in (0, 1)$, suitable for representing a probability.
2. **Symmetry:** $\sigma(-z) = 1 - \sigma(z)$.

Proof: reduce $\sigma(-z)$ and $1 - \sigma(z)$ separately to the same form. First, $\sigma(-z)$: substitute $-z$ for z in the definition, then multiply numerator and denominator by e^{-z} :

$$\sigma(-z) = \frac{1}{1 + e^z} = \frac{1}{1 + e^z} \cdot \frac{e^{-z}}{e^{-z}} = \frac{e^{-z}}{e^{-z} + e^{-z}e^z} = \frac{e^{-z}}{e^{-z} + 1},$$

using $e^{-z}e^z = 1$. Now $1 - \sigma(z)$: simply put over a common denominator:

$$1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}.$$

Both reduce to the same expression, $\frac{e^{-z}}{1 + e^{-z}}$, so

$$\sigma(-z) = \frac{e^{-z}}{1 + e^{-z}} = 1 - \sigma(z). \quad \blacksquare$$

3. **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Proof: view $\sigma(z) = (1 + e^{-z})^{-1}$ as a composite function, and use the power rule together with the chain rule:

$$\begin{aligned} \sigma'(z) &= \frac{d}{dz}(1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) \quad (\text{power rule } (u^{-1})' = -u^{-2}u', \ u = 1 + e^{-z}) \\ &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \quad (\text{since } \frac{d}{dz}e^{-z} = -e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}. \end{aligned}$$

The first factor above is exactly $\sigma(z)$; the second factor, $\frac{e^{-z}}{1 + e^{-z}}$, is exactly the $1 - \sigma(z)$ already computed in the proof of the previous property, so

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)). \quad \blacksquare$$

4. **Limiting behavior:** $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$; $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$; $\sigma(0) = 1/2$.

Substituting Concrete Values

The properties above are fairly abstract; let us substitute a set of concrete values of z directly, to display the shape of the sigmoid function’s “S-curve” and its derivative together at a glance:

Table 4.4: Numerical Values of the Sigmoid Function and Its Derivative

z	−4	−2	−1	0	1	2	4
$\sigma(z)$	0.018	0.119	0.269	0.500	0.731	0.881	0.982
$\sigma'(z) = \sigma(1 - \sigma)$	0.018	0.105	0.197	0.250	0.197	0.105	0.018

Two phenomena stand out: first, $\sigma(z)$ changes fastest near $z = 0$ (the region where the “decision boundary” is most sensitive), and quickly flattens out toward 0 or 1 away from $z = 0$; second, the derivative $\sigma'(z)$ attains its maximum value of 0.25 exactly at $z = 0$, decaying symmetrically on both sides — this “bell-shaped” derivative curve is exactly the source of the “vanishing gradient” phenomenon that will be encountered repeatedly when discussing neural-network training later in the book: when $|z|$ is large, $\sigma'(z)$ is nearly zero, and gradient descent can learn almost nothing.

The Complete Probabilistic Model

$$P(y|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})^y [1 - \sigma(\boldsymbol{\theta}^T \mathbf{x})]^{1-y}. \quad (4.7)$$

This expression unifies the two cases $y = 0$ and $y = 1$: when $y = 1$, $P(y = 1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$; when $y = 0$, $P(y = 0|\mathbf{x}, \boldsymbol{\theta}) = 1 - \sigma(\boldsymbol{\theta}^T \mathbf{x})$.

This is exactly the direct generalization of the Bernoulli distribution $P(x|p) = p^x(1 - p)^{1-x}$ (Equation (3.3)) from §3.3 — the difference is that there, the coin’s “bias” p was a fixed constant, whereas here the “bias” $\sigma(\boldsymbol{\theta}^T \mathbf{x})$ depends on the input features $\mathbf{x}$, and varies as $\mathbf{x}$ varies. Logistic regression is, in essence, a Bernoulli model “whose bias is determined by the features.”

Deriving the Negative Log-Likelihood

Taking the logarithm:

$$\ln P(y|\mathbf{x}, \boldsymbol{\theta}) = y \ln \sigma(\boldsymbol{\theta}^T \mathbf{x}) + (1 - y) \ln [1 - \sigma(\boldsymbol{\theta}^T \mathbf{x})].$$

Using properties of the sigmoid:

$$\ln \sigma(z) = \ln \frac{1}{1 + e^{-z}} = -\ln(1 + e^{-z}), \quad \ln[1 - \sigma(z)] = \ln \frac{e^{-z}}{1 + e^{-z}} = -z - \ln(1 + e^{-z}).$$

Substituting and simplifying step by step:

$$\begin{aligned}
 \ln P(y|\mathbf{x}, \boldsymbol{\theta}) &= -y \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}) + (1 - y) [-\boldsymbol{\theta}^T \mathbf{x} - \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}})] \\
 &= -y \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}) - (1 - y) \boldsymbol{\theta}^T \mathbf{x} - (1 - y) \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}) \\
 &= -(1 - y) \boldsymbol{\theta}^T \mathbf{x} - \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}) \\
 &= y \boldsymbol{\theta}^T \mathbf{x} - \boldsymbol{\theta}^T \mathbf{x} - \ln(1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}).
 \end{aligned}$$

Using the identity $\ln(1 + e^{-z}) = \ln(e^z + 1) - z$ (proof: $1 + e^{-z} = e^{-z}(e^z + 1)$; taking the logarithm of both sides, $\ln(1 + e^{-z}) = \ln e^{-z} + \ln(e^z + 1) = -z + \ln(e^z + 1)$, which is exactly the identity):

$$\ln P(y|\mathbf{x}, \boldsymbol{\theta}) = y \boldsymbol{\theta}^T \mathbf{x} - \ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}}).$$

Therefore, the negative log-likelihood (the cross-entropy loss) for a single sample is

$$\ell(y, \mathbf{x}, \boldsymbol{\theta}) = -\ln P(y|\mathbf{x}, \boldsymbol{\theta}) = -y \boldsymbol{\theta}^T \mathbf{x} + \ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}}).$$

For the entire dataset:

$$\boxed{\mathcal{L}(\boldsymbol{\theta}) = \sum_{n=1}^N \left[-y^{(n)} \boldsymbol{\theta}^T \mathbf{x}^{(n)} + \ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}) \right]}. \quad (4.8)$$

This is called the **cross-entropy loss**, or **log loss**.

Physical Interpretation

Define the “energy”

$$E^{(n)}(\boldsymbol{\theta}) = -y^{(n)} \boldsymbol{\theta}^T \mathbf{x}^{(n)} + \ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}).$$

The first term, $-y^{(n)} \boldsymbol{\theta}^T \mathbf{x}^{(n)}$: if $y^{(n)} = 1$ and $\boldsymbol{\theta}^T \mathbf{x}^{(n)}$ is large (a strong positive prediction), the energy is low (good); if $y^{(n)} = 1$ but $\boldsymbol{\theta}^T \mathbf{x}^{(n)}$ is small, the energy is high (poor). The second term, $\ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}})$, plays a role analogous to the “partition function,” guaranteeing that the probability is normalized.

Computing the Gradient

Computing the gradient (needed for optimization). Differentiating the two terms of Equation (4.8) separately: the first term, $-y^{(n)} \boldsymbol{\theta}^T \mathbf{x}^{(n)}$, gives $-y^{(n)} \mathbf{x}^{(n)}$ directly, using the “preliminary rule” $\partial(\boldsymbol{\theta}^T \mathbf{x})/\partial \boldsymbol{\theta} = \mathbf{x}$ (§4.2); the second term, $\ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}})$, is a composite function, requiring the chain rule: treating $u = 1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}$ as an intermediate variable,

$$\frac{\partial}{\partial \boldsymbol{\theta}} \ln(1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}) = \frac{1}{1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}} \cdot \frac{\partial}{\partial \boldsymbol{\theta}} (1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}) = \frac{1}{1 + e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}}} \cdot e^{\boldsymbol{\theta}^T \mathbf{x}^{(n)}} \cdot \mathbf{x}^{(n)},$$

where the last step again uses the preliminary rule ($\partial e^{\theta^T \mathbf{x}} / \partial \theta = e^{\theta^T \mathbf{x}} \cdot \partial(\theta^T \mathbf{x}) / \partial \theta = e^{\theta^T \mathbf{x}} \mathbf{x}$). Adding the two terms:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{n=1}^N \left[-y^{(n)} \mathbf{x}^{(n)} + \frac{e^{\theta^T \mathbf{x}^{(n)}}}{1 + e^{\theta^T \mathbf{x}^{(n)}}} \mathbf{x}^{(n)} \right].$$

Notice that $\frac{e^{\theta^T \mathbf{x}}}{1 + e^{\theta^T \mathbf{x}}}$ is exactly $\sigma(\theta^T \mathbf{x})$: dividing numerator and denominator by $e^{\theta^T \mathbf{x}}$,

$$\frac{e^{\theta^T \mathbf{x}}}{1 + e^{\theta^T \mathbf{x}}} = \frac{1}{e^{-\theta^T \mathbf{x}} + 1} = \sigma(\theta^T \mathbf{x}),$$

substituting gives

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \theta} &= \sum_{n=1}^N [\sigma(\theta^T \mathbf{x}^{(n)}) - y^{(n)}] \mathbf{x}^{(n)} \\ &= \sum_{n=1}^N [\hat{y}^{(n)} - y^{(n)}] \mathbf{x}^{(n)}, \end{aligned}$$

where $\hat{y}^{(n)} = \sigma(\theta^T \mathbf{x}^{(n)})$ is the predicted probability.

This is a compact result: the gradient is exactly “the prediction error times the input,” a form that is exactly the same as linear regression’s gradient, even though the two loss functions are entirely different.

A Worked Example: One Step of Gradient Descent

Let us use the simplest possible single-feature, single-sample example to work through this “prediction error times input” gradient formula from beginning to end. Let the feature $x = 2$ (a scalar), the current parameter $\theta = 0.5$, and the true label $y = 1$.

Step 1: compute the current prediction.

$$z = \theta x = 0.5 \times 2 = 1.0, \quad \hat{y} = \sigma(z) = \sigma(1.0) = 0.731.$$

The model currently believes the probability that “ $y = 1$ ” is 0.731, while the true label is indeed 1 — the direction is correct, but the confidence is not yet high enough.

Step 2: compute the gradient.

$$\frac{\partial \mathcal{L}}{\partial \theta} = (\hat{y} - y)x = (0.731 - 1) \times 2 = -0.538.$$

The gradient is negative, meaning: to decrease the loss (bringing the prediction closer to the true value 1), θ should **increase** (since gradient descent “subtracts” the gradient times the learning rate).

Step 3: update by one step, with learning rate 0.1.

$$\theta_{\text{new}} = \theta - 0.1 \times (-0.538) = 0.5 + 0.054 = 0.554.$$

Step 4: verify the effect. Recompute the prediction with the new θ : $z_{\text{new}} = 0.554 \times 2 = 1.108$, $\hat{y}_{\text{new}} = \sigma(1.108) = 0.752$. The predicted probability rose from 0.731 to 0.752, indeed moving toward the true label $y = 1$ — this single hand-computed step is exactly what happens in every iteration of every neural-network training run, except that in real training θ is a vector with thousands or millions of dimensions, requiring a computer rather than pencil and paper.

■ Think and Check

Return to the spam-classification example in Table 3.2 of §3.1. If the model computes $\theta^T \mathbf{x} = 3$ (a fairly large positive number) for some email, then $\sigma(3) \approx 0.953$, and the email is predicted to be spam with high probability. If this email’s true label is actually “legitimate” ($y = 0$), can we determine the sign of the gradient $(\hat{y} - y)\mathbf{x}$ directly from the symmetry $\sigma(-z) = 1 - \sigma(z)$ of the sigmoid alone, and say which direction the parameters will be adjusted?

Solution: $\hat{y} = 0.953$, $y = 0$, so the gradient $= (0.953 - 0)\mathbf{x} = 0.953\mathbf{x}$, with the same sign as $\mathbf{x}$; gradient descent will adjust θ in the direction that **decreases** $\theta^T \mathbf{x}$, i.e., lowering the predicted probability that this email is spam — this is exactly the mathematical expression of the intuition that “the further off a prediction is, the harder the correction.”

4.4 Example Three: Softmax Regression

Model Setup

For a K -class classification problem, $y \in \{1, 2, \dots, K\}$. Define a weight vector θ_k for each class k , and model

$$P(y = k | \mathbf{x}, \{\theta_j\}_{j=1}^K) = \frac{e^{\theta_k^T \mathbf{x}}}{\sum_{j=1}^K e^{\theta_j^T \mathbf{x}}}. \quad (4.9)$$

This expression is called the **softmax function**, the generalization of the sigmoid function to the multi-class case.

The Physical Interpretation of Softmax

Equation (4.9) is exactly the Boltzmann distribution of statistical physics: “energy” $E_k = -\theta_k^T \mathbf{x}$, “partition function” $Z = \sum_{j=1}^K e^{\theta_j^T \mathbf{x}}$, probability $P(y = k) = e^{-E_k}/Z$. The probability of class k is exponentially related to its “negative energy” $\theta_k^T \mathbf{x}$: the lower the energy ($\theta_k^T \mathbf{x}$ larger), the higher the probability.

Concrete Numbers for Three-Class Classification

Take $K = 3$, and suppose some input $\mathbf{x}$ has raw scores (i.e., $\theta_k^T \mathbf{x}$) on the three classes of

$$z_1 = 2.0, \quad z_2 = 1.0, \quad z_3 = -0.5.$$

Step 1: compute the exponentials: $e^{z_1} = 7.389$, $e^{z_2} = 2.718$, $e^{z_3} = 0.607$. **Step 2:** sum to get the partition function: $Z = 7.389 + 2.718 + 0.607 = 10.714$. **Step 3:** normalize to obtain the probabilities:

Table 4.5: Numerical Results for the Three-Class Softmax Example

Class k	Raw score z_k	e^{z_k}	Probability $P(y = k)$
1	2.0	7.389	0.690
2	1.0	2.718	0.254
3	-0.5	0.607	0.057

The class with the highest score, class 1, receives the largest share of probability (69%), but this is not “winner-take-all” — class 2 still retains about a quarter of the probability. This is exactly softmax’s advantage over “simply picking the class with the highest score” (hard argmax): it retains information about the model’s uncertainty regarding the runner-up options, rather than arbitrarily giving a single answer.

Specific numerical values of the cross-entropy loss: if the true label happens to be the highest-probability class, 1, the loss is $\mathcal{L} = -z_1 + \ln Z = -2.0 + 2.372 = 0.372$ (small, the model “guessed correctly,” at low cost); if the true label is actually the lowest-probability class, 3, the loss is $\mathcal{L} = -z_3 + \ln Z = 0.5 + 2.372 = 2.872$ (much larger, the model “guessed wrong,” at high cost). One can verify that these two numbers exactly equal $-\ln P(y = k)$: $-\ln(0.690) = 0.372$, $-\ln(0.057) = 2.872$, matching precisely.

The Temperature Parameter and the Control of Certainty

We have already seen repeatedly, in §2.2, how “temperature” controls a system’s transition between “exploring every possibility” and “locking onto the lowest-energy state.” This same effect is present in softmax as well — simply introduce a scaling factor β (corresponding to the inverse temperature) in the exponent:

$$P_\beta(y = k) = \frac{e^{\beta z_k}}{\sum_j e^{\beta z_j}}. \quad (4.10)$$

Using the scores $z = (2.0, 1.0, -0.5)$ from above, taking three different values of β :

Table 4.6: Softmax Output at Different Inverse Temperatures β

β	Probabilities (P_1, P_2, P_3)
0.2 (high temperature)	(0.412, 0.338, 0.250)
1.0 (baseline)	(0.690, 0.254, 0.057)
5.0 (low temperature)	(0.993, 0.007, 0.000)

At **high temperature** ($\beta = 0.2$), the probabilities of the three classes are almost “smoothed out” into a uniform distribution, and the model behaves almost like a “random guess”; at **low temperature** ($\beta = 5.0$), the distribution nearly collapses entirely onto the highest-scoring class, 1, approximately equivalent to simply taking the argmax — this is, once again, exactly the “uniform at high temperature, frozen at low temperature” rule from §2.2 (the two-level system) and §2.3 (the one-dimensional Ising model), recurring here. In practice, this β (sometimes written

$1/\tau$, with τ called the **temperature parameter**) is often used to control the “degree of certainty” of a classifier’s or generative model’s output: the lower the temperature, the more conservative the output, concentrated on the most likely option; the higher the temperature, the more diverse the output, more inclined to try suboptimal options.

One-Hot Encoding

For convenience, encode the label $y \in \{1, \dots, K\}$ as a vector $\mathbf{y} \in \{0, 1\}^K$:

$$\mathbf{y} = [y_1, y_2, \dots, y_K],$$

where $y_k = 1$ if $y = k$, and $y_k = 0$ otherwise. For example, when $K = 3$: class 1 corresponds to $\mathbf{y} = [1, 0, 0]$, class 2 to $\mathbf{y} = [0, 1, 0]$, and class 3 to $\mathbf{y} = [0, 0, 1]$.

The Cross-Entropy Loss

The log-likelihood is

$$\ln P(\mathbf{y}|\mathbf{x}, \{\boldsymbol{\theta}_k\}) = \sum_{k=1}^K y_k \ln P(y = k|\mathbf{x}, \{\boldsymbol{\theta}_k\})$$

(since $\mathbf{y}$ is a one-hot vector, only one term in this sum is nonzero — the term corresponding to the true class k^* , $y_{k^*} = 1$, with every other $y_k = 0$). Substituting the softmax definition (4.9) and taking the logarithm ($\ln$ applied to a quotient, $\ln(a/b) = \ln a - \ln b$, then using $\ln e^x = x$):

$$\begin{aligned} \ln P(\mathbf{y}|\mathbf{x}, \{\boldsymbol{\theta}_k\}) &= \sum_{k=1}^K y_k \left[\boldsymbol{\theta}_k^T \mathbf{x} - \ln \sum_{j=1}^K e^{\boldsymbol{\theta}_j^T \mathbf{x}} \right] \\ &= \sum_{k=1}^K y_k \boldsymbol{\theta}_k^T \mathbf{x} - \left(\sum_{k=1}^K y_k \right) \ln \sum_{j=1}^K e^{\boldsymbol{\theta}_j^T \mathbf{x}}, \end{aligned}$$

This step splits the sum into two parts: the first, $\sum_k y_k \boldsymbol{\theta}_k^T \mathbf{x}$, is kept term by term (since every term depends on k); in the second, $\ln \sum_j e^{\boldsymbol{\theta}_j^T \mathbf{x}}$ does not depend on the summation index k , and can be pulled out as a common factor, producing an extra factor $\sum_k y_k$. Since $\mathbf{y}$ is a one-hot vector (exactly one of its K components is 1, the rest 0), $\sum_k y_k = 1$, and this term simplifies directly to $\ln \sum_j e^{\boldsymbol{\theta}_j^T \mathbf{x}}$ itself:

$$\ln P(\mathbf{y}|\mathbf{x}, \{\boldsymbol{\theta}_k\}) = \sum_{k=1}^K y_k \boldsymbol{\theta}_k^T \mathbf{x} - \ln \sum_{j=1}^K e^{\boldsymbol{\theta}_j^T \mathbf{x}}.$$

The negative log-likelihood (the cross-entropy loss) is

$$\boxed{\mathcal{L}(\{\boldsymbol{\theta}_k\}) = - \sum_{k=1}^K y_k \boldsymbol{\theta}_k^T \mathbf{x} + \ln \sum_{j=1}^K e^{\boldsymbol{\theta}_j^T \mathbf{x}}.} \quad (4.11)$$

For the entire dataset:

$$\mathcal{L} = \sum_{n=1}^N \left[- \sum_{k=1}^K y_k^{(n)} \boldsymbol{\theta}_k^T \mathbf{x}^{(n)} + \ln \sum_{j=1}^K e^{\boldsymbol{\theta}_j^T \mathbf{x}^{(n)}} \right].$$

The Analogy with Free Energy

Let us compare this loss function with statistical physics: define the “energy” $E_y := -\theta_y^T \mathbf{x}$ (consistent with the definition in “The Physical Interpretation of Softmax” at the start of this section); then the first term of Equation (4.11), $-\sum_k y_k \theta_k^T \mathbf{x}$, is exactly the energy E_y of the true class (since $\mathbf{y}$ is one-hot, the sum leaves only the term for the true class k^*); the second term, $\ln \sum_{j=1}^K e^{\theta_j^T \mathbf{x}} = \ln \sum_j e^{-E_j}$, is exactly the log-partition function $\ln Z$. Strictly, then,

$$\mathcal{L} = E_y + \ln Z. \quad (4.12)$$

A notational trap requires particular care here: Chapter 2 defined the Helmholtz free energy as $F = -\frac{1}{\beta} \ln Z$ (Equation (2.11)), which, in the units $\beta = 1$ used uniformly in this chapter, becomes $F = -\ln Z$ — that is, $\ln Z = -F$, and *not* $\ln Z = F$. Substituting into Equation (4.12), the correct correspondence is

$$\boxed{\mathcal{L} = E_y + \ln Z = E_y - F.} \quad (4.13)$$

That is, “loss = energy – free energy,” not “energy + free energy” — a difference of a single sign, but with entirely different physical meaning: the free energy $F = \langle E \rangle - TS$ is itself typically negative (since $\ln Z > 0$ holds in most cases), and the loss $\mathcal{L}$ is the energy *minus* a negative F , giving something that is added to, and is consistent with the fact that the loss (being a negative log-likelihood) is always non-negative; if one mistakenly wrote $\mathcal{L} = E + F$, then whenever $\ln Z > 0$ this would give $\mathcal{L} = E - \ln Z$, directly contradicting the explicit expansion of Equation (4.11). Readers can verify this for themselves using the numbers from “A Worked Example: Concrete Numbers for Three-Class Classification” above: $Z = 10.714$, $\ln Z = 2.372$, $F = -\ln Z = -2.372$; when the true label is class 1, $E_1 = -z_1 = -2.0$, and substituting into $\mathcal{L} = E_1 - F = -2.0 - (-2.372) = 0.372$ agrees exactly with the previously computed $\mathcal{L} = 0.372$; whereas using the incorrect formula $\mathcal{L} = E_1 + F = -2.0 + (-2.372) = -4.372$ is clearly wrong.

This form will reappear later, when Boltzmann machines are discussed. **A note on terminology:** the “energy” E_y in this example (the bare negative score, $-\theta_y^T \mathbf{x}$) and the complete loss $\mathcal{L}$ defined in §4.1 ($-\ln P$ itself) are objects at two different levels — $\mathcal{L} = E_y + \ln Z$, and the two coincide only when $\ln Z$ can be treated as a constant independent of θ (as in the case of linear regression, where the noise variance σ is fixed and does not vary with θ). In softmax, Z depends explicitly on the entire set $\{\theta_j\}$, so E_y and $\mathcal{L}$ are not equal — conflating these two concepts is exactly the easiest mistake to make in this section.

■ Summary

Linear regression, logistic regression, and softmax regression appear, on the surface, to be three different models, but their loss functions all follow the very same derivation procedure: **choose a distributional form for the noise/randomness** → **write down the likelihood** → **take the negative logarithm**. Gaussian noise produces mean squared error; the Bernoulli distribution produces binary cross-entropy; the categorical distribution produces multi-class cross-entropy. The next chapter directly confronts a difficulty this framework has, until now, deliberately sidestepped: just how hard the partition function Z is to compute in general.

Exercises

Foundational

Exercise 4.1: From Gaussian Noise to Mean Squared Error

Give a detailed derivation of why the assumption of Gaussian noise leads to the mean-squared-error loss function.

- Write down the probabilistic model assuming $y = f(\mathbf{x}; \boldsymbol{\theta}) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$;
- For a single data point $(\mathbf{x}, y)$, write down the log-likelihood $\ln P(y|\mathbf{x}, \boldsymbol{\theta})$;
- For N data points, write down the total log-likelihood;
- Prove that maximizing the log-likelihood is equivalent to minimizing the mean squared error;
- Discuss the role of σ : what effect does a very small or very large σ have on learning?

Exercise 4.2: Properties of the Sigmoid Function

Prove the following properties of the sigmoid function $\sigma(z) = \frac{1}{1 + e^{-z}}$:

- $\sigma(-z) = 1 - \sigma(z)$;
- $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$;
- $\sigma(z) + \sigma(-z) = 1$;
- $\ln \sigma(z) = -\ln(1 + e^{-z})$ and $\ln[1 - \sigma(z)] = -z - \ln(1 + e^{-z})$.

Exercise 4.3: Properties of Softmax

For the softmax function $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$:

- Prove that $\sum_k p_k = 1$;
- Prove that every $p_k \in (0, 1)$;
- Compute $\frac{\partial p_k}{\partial z_i}$, distinguishing the cases $i = k$ and $i \neq k$;
- Prove that $\frac{\partial p_k}{\partial z_k} = p_k(1 - p_k)$ and $\frac{\partial p_k}{\partial z_i} = -p_k p_i$ (when $i \neq k$).

Hint: use $\frac{\partial}{\partial z_i} \ln \sum_j e^{z_j} = \frac{e^{z_i}}{\sum_j e^{z_j}} = p_i$.

Intermediate

Exercise 4.4: Laplace Noise and the L_1 Loss

§4.2 derived the mean-squared-error loss from the assumption that the noise ϵ follows a Gaussian distribution. Now change the assumed noise distribution, and work through the complete procedure “choose a noise distribution $\rightarrow$ write down the likelihood $\rightarrow$ take the negative log-likelihood” once again. Let $y = \boldsymbol{\theta}^T \mathbf{x} + \epsilon$, where ϵ follows a **Laplace distribution**:

$$P(\epsilon) = \frac{1}{2b} \exp\left(-\frac{|\epsilon|}{b}\right), \quad b > 0.$$

- Write down the conditional probability density $P(y|\mathbf{x}, \boldsymbol{\theta})$;
- For the dataset $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, following the steps of §4.2, give a complete derivation of the negative-log-likelihood loss function $L(\boldsymbol{\theta})$;
- Prove that this loss function is proportional to the **sum of absolute errors** (i.e., the L_1 loss): $L(\boldsymbol{\theta}) = \frac{1}{b} \sum_{n=1}^N |y^{(n)} - \boldsymbol{\theta}^T \mathbf{x}^{(n)}| + \text{const}$;
- The Laplace distribution has “heavier tails” than the Gaussian (that is, a higher probability of extreme values far from the mean). Using this fact, explain why the L_1 loss (corresponding to Laplace noise) is less sensitive than mean squared error (corresponding to Gaussian noise) to outliers in the training data — this is exactly the basic idea behind “robust regression” in statistics.

Challenge

Exercise 4.5: Generalized Linear Models and the Exponential Family

This chapter used three concrete examples — Gaussian, Bernoulli, and categorical — and, in each case, applied the very same procedure: choose a distributional form for the noise/randomness $\rightarrow$ write down the likelihood $\rightarrow$ take the negative log-likelihood $\rightarrow$ obtain the loss function. This is no coincidence — all three distributions belong to the **exponential family** of distributions in statistics, whose general form is

$$P(y|\eta) = h(y) \exp(\eta y - A(\eta)),$$

where η is called the **natural parameter**, and $A(\eta)$ the **log-partition function**.

- Look up (or derive for yourself) how the Gaussian distribution (with fixed variance σ^2) and the Bernoulli distribution can both be written in the form above, giving the corresponding $\eta, h(y), A(\eta)$ in each case; for the Bernoulli distribution, show that η is exactly the “log-odds,” $\ln \frac{p}{1-p}$, that appeared in §4.3;
- A **Generalized Linear Model** (GLM) assumes that the natural parameter is a linear function of the features, $\eta = \boldsymbol{\theta}^T \mathbf{x}$. Prove that the resulting negative-log-likelihood loss

is exactly the unified form of this chapter’s linear-regression and logistic-regression loss functions;

- (c) Prove that an exponential-family distribution satisfies $\mathbb{E}[y] = A'(\eta)$ (the derivative of the log-partition function gives the mean). Is this result, in its mathematical structure, the same fact as the derivation $\langle E \rangle = -\partial \ln Z / \partial \beta$ from Chapter 2? Explain your reasoning;
- (d) The Poisson distribution (commonly used to model count data, such as “the number of clicks a webpage receives per day”) also belongs to the exponential family. Try to derive the negative-log-likelihood loss function corresponding to “Poisson regression,” and explain what kind of data it is suited to, compared with mean squared error and cross-entropy.

Chapter 5

The Partition Function and Normalization

*O God, I could be bounded in a nutshell
And count myself a king of infinite space.*

— Shakespeare, *Hamlet*, Act II, Scene
2, spoken by Hamlet

Objectives

- Understand the ubiquity of normalization in probabilistic models, and recognize that examples already computed earlier — the Boltzmann distribution, softmax — are, at bottom, all instances of the same “partition function” problem;
- Master the basic language of Markov Random Fields (MRFs) — cliques, potential functions, and the local and global Markov properties — and understand how the Hammersley–Clifford theorem equates local conditional independence with a global Gibbs factorization;
- Recognize the fundamental difficulty that partition-function computation grows exponentially with the number of variables, and understand how this difficulty concretely shows up in normalization, marginalization, and parameter learning (the “positive phase minus negative phase” formula);
- Build an initial overall picture of approximation methods — Monte Carlo sampling, variational methods, belief propagation, the renormalization group.

■ Chapter Guide

The question this chapter answers: Why is “normalization” a genuine computational difficulty in the practical training of probabilistic models, rather than merely a formal technicality? How does the language of Markov random fields (graphical models) help us organize

and understand this difficulty?

Correspondence with statistical physics: This chapter generalizes Chapter 2’s one-dimensional Ising model to a general Markov random field; the Hammersley–Clifford theorem shows that “a graph structure of local interactions” and “energy factored by clique” are, in essence, the very same thing. The gradient formula for parameter learning (“positive phase minus negative phase”) comes directly from differentiating $\ln Z(\boldsymbol{\theta})$ — a direct extension of the technique, from Chapter 2, of using $\ln Z$ to generate thermodynamic quantities.

Looking ahead: Chapter 6 will show how, when Z cannot be computed exactly, variational methods replace “exact computation” with “searching for a sufficiently good approximation” — a problem an optimization algorithm can solve, rather than a dead end.

Over the previous three chapters, we have, without quite realizing it, repeatedly computed the very same object: the two-level system’s $Z = 1 + e^{-\beta\varepsilon}$ (§2.2), the $N = 2$ Ising chain’s $Z = 4 \cosh(\beta J)$ (§2.3), and softmax’s $Z = \sum_j e^{z_j}$ (§4.4) — every one of these is a “partition function,” merely appearing in a different guise in a different field. This chapter confronts a question we have deliberately sidestepped until now: the examples above could be computed by hand only because the number of variables was small (2 spins, 3 classes); once the number of variables grows to the scale of a real problem, how hard does computing the partition function become? Where does this difficulty come from, and how should it be dealt with?

5.1 The Ubiquity of Normalization

Normalization is an indispensable step in any probabilistic model. Whether it is the Boltzmann distribution of statistical physics or softmax in machine learning, a normalization constant (a partition function) is always required:

$$P(\mathbf{x}) = \frac{1}{Z} f(\mathbf{x}), \quad (5.1)$$

where $Z = \sum_{\mathbf{x}} f(\mathbf{x})$ or $Z = \int f(\mathbf{x}) d\mathbf{x}$.

■ Updating the Correspondence Table: the Partition-Function Row, from “What It Is” to “How Hard It Is”

Both Chapter 2’s seed table (Table 2.8) and Chapter 4 have already filled in the “partition function Z ” row once: Z is the normalization constant. This chapter does not repeat that point, but instead dives into the *flip side* of that row — filling in the table is easy; computing it is hard. What this chapter sets out to explain is why this seemingly mundane normalization step becomes the hardest part of the entire framework once the number of variables grows even moderately large.

Let us look back at several instances of Z we have already computed by hand:

The number of variables in each of the three examples above is small (1, 2, and 3 “units”), and the summation involves at most a handful of terms — easily written out in full and computed by eye. **The real difficulty was never “what is the partition function”; it is that, once**

Table 5.1: Partition Functions Already Computed in Previous Chapters

Source	System	Partition function Z
§2.2	Two-level system	$Z = 1 + e^{-\beta\epsilon}$
§2.3	$N = 2$ Ising chain	$Z = 4 \cosh(\beta J)$
§4.4	Three-class softmax	$Z = \sum_{j=1}^3 e^{z_j}$

the number of variables grows, the number of terms in the sum spirals out of control at an exponential rate — exactly the problem this chapter turns to next.

5.2 Undirected Graphical Models (Markov Random Fields)

Why Do We Need Graphical Models? The Combinatorial Explosion of the Joint Distribution

Let there be N random variables $X_1, \dots, X_N$, each taking K discrete values. With no structural assumptions whatsoever, the joint distribution $P(X_1, \dots, X_N)$ requires $K^N - 1$ independent parameters to fully specify (the minus one is due to the normalization constraint).

For example, $N = 4$ binary variables ($K = 2$) require $2^4 - 1 = 15$ parameters — still tolerable; but at $N = 100$, $2^{100} \approx 1.3 \times 10^{30}$ — a figure that, although far smaller than the estimated number of atoms in the observable universe (on the order of 10^{80}), is still far beyond what any real computer could store or exhaustively process term by term: even treating every atom on Earth (roughly 10^{50} atoms) as a single storage unit, storing this entire probability table would still fall short by 20 orders of magnitude; a parameter count of this size can neither be stored nor be estimated from any realistically sized finite dataset.

Laying out this rate of growth explicitly makes it easier to grasp intuitively:

Table 5.2: The Growth of the Total Number of States, 2^N , with the Number of Variables N

N (number of binary variables)	2^N
10	1.0×10^3 (a thousand)
20	1.0×10^6 (a million)
30	1.1×10^9 (a billion)
60	1.2×10^{18}
100	1.3×10^{30}

Going from $N = 10$ to $N = 100$, the input size grows only tenfold, yet the number of states swells from “a thousand” to “ 10^{30} ” — this is exactly the sibling phenomenon of what the $\Omega(k)$ table in the “counting the boxes” example of §2.1 displayed: there, it was “the number of microstates corresponding to a macrostate” growing exponentially with N ; here, it is “the total number of states of the entire system” growing exponentially with N — fundamentally the very same combinatorial explosion.

This predicament is no stranger to statistical physicists: it is exactly the probabilistic version of the “many-body problem.” The joint distribution of N spins requires, in principle, 2^N numbers to describe, but physics tells us that the overwhelming majority of microscopic detail is irrelevant to macroscopic behavior — what genuinely matters is the **structure of local interactions**: for instance, a given spin couples directly only to its nearest neighbors, not to every other spin in the system. The **graphical model** is exactly the tool that translates this physical intuition into precise mathematical language: the nodes of the graph represent random variables, edges represent “direct dependence,” and the absence of an edge means two variables have no direct interaction — any correlation between them (if it exists at all) must be “relayed” through other variables on the graph.

From Markov Chains to Markov Fields: Generalizing “Locality” from Time to Graph Structure

Recall the definition of a one-dimensional Markov chain: for a sequence of random variables ordered in time, $X_1, X_2, \dots, X_T$ (for example, “today’s weather $\rightarrow$ tomorrow’s weather $\rightarrow$ the day after tomorrow’s weather”), the Markov property is

$$P(X_{t+1} \mid X_t, X_{t-1}, \dots, X_1) = P(X_{t+1} \mid X_t), \quad (5.2)$$

that is, “the future depends only on the present, not on the past.” This is a locality assumption **along the time direction**: each variable is directly related only to the one variable immediately adjacent to it in time.

The **Markov Random Field** (MRF) does the very same thing, except that the “time axis” is replaced by an arbitrary graph structure — locality is no longer a matter of temporal succession, but of adjacency on the graph. Let $G = (V, E)$ be a graph, with node set $V = \{1, \dots, N\}$ corresponding to random variables $\{X_i\}$, and edge set E encoding direct-dependence relations. For a node i , define its **neighbor set** $N(i) = \{j : (i, j) \in E\}$. The **local Markov property** satisfied by an MRF is

$$X_i \perp X_{V \setminus (\{i\} \cup N(i))} \mid X_{N(i)}, \quad (5.3)$$

that is, “once conditioned on all of i ’s graph neighbors, X_i is conditionally independent of every other variable in the graph.” For example, in a graph of the form $A-B-C$, with B also connected to D , A needs to know only B , and need not concern itself with C, D : $P(A \mid B, C, D) = P(A \mid B)$.

Graph-Theoretic Language: Separating Sets and Conditional Independence

An undirected graph provides an intuitive geometric picture of conditional-independence relations: **separation on the graph corresponds to conditional independence in probability**. Let $A, B, S \subseteq V$ be three mutually disjoint subsets of nodes; if every path in the graph from any node in A to any node in B must pass through S , then S is said to **separate** A and B . The **global Markov property** of an MRF asserts:

$$S \text{ separates } A, B \text{ on the graph} \implies X_A \perp X_B \mid X_S. \quad (5.4)$$

The simplest example is still the chain $A-B-C$: B separates A, C , so $X_A \perp X_C \mid X_B$ — once B is given, A and C carry no information about each other whatsoever. This is exactly the most precise mathematical expression of the physical intuition that “no edge means no direct correlation.”

It is worth noting (a point beginners easily overlook) that an MRF actually possesses three Markov properties of increasing strength — **pairwise**, **local**, and **global**. The definition of separation gives directly “global $\Rightarrow$ local $\Rightarrow$ pairwise”; but the reverse direction (pairwise $\Rightarrow$ global) does not hold automatically — it requires the additional condition that the joint distribution be **everywhere positive** ($P(\mathbf{x}) > 0$ for every configuration $\mathbf{x}$) (Pearl, 1988). This positivity condition will be encountered again below, in the discussion of the Hammersley–Clifford theorem — it is not an incidental technicality, but the genuine premise underlying the claim that “local independence is equivalent to a global energy factorization.”

Cliques: the Basic Unit the Energy Function Actually “Sees”

The edges of a graph tell us “who is directly related to whom,” but the object the energy function actually acts on is the **clique**.

Definition 5.1: Clique

A subset of nodes $c \subseteq V$ in a graph G is called a **clique** if every pair of nodes in c is connected by an edge (i.e., c forms a complete subgraph of G). If adding any further node would destroy this property, c is called a **maximal clique**.

A triangle $A-B-C$ (three edges, every pair connected) is a 3-clique; the chain $A-B-C$ (only the two edges $A-B$ and $B-C$, with no $A-C$) is **not** a clique — its maximal cliques are the two edges $\{A, B\}$ and $\{B, C\}$.

Potential functions are defined only on (maximal) cliques, not on the entire variable set, and this is exactly the fundamental mechanism by which an MRF reduces parameter complexity: rather than describing the joint interaction among all N variables with a single $\psi(x_1, \dots, x_N)$ (which would require K^N parameters), it is factored into a product of local potential functions $\{\psi_c(\mathbf{x}_c)\}_{c \in \mathcal{C}}$, each ψ_c depending only on the small number of variables within one clique — this is exactly the key step that brings us back from “combinatorial explosion” to “tractable.”

The Hammersley–Clifford Theorem: Local Markov Property $\iff$ Global Gibbs Factorization

This is the central theorem of MRF theory: it rigorously equates two things that appear, on the surface, to be quite different — “a local conditional-independence assumption on a graph” and “an energy function factored by clique.”

Theorem 5.1: The Hammersley–Clifford Theorem

Let $\mathbf{X} = (X_1, \dots, X_N)$ take values in a finite set, with joint distribution **everywhere positive**: $P(\mathbf{x}) > 0$ for every $\mathbf{x}$. Then P satisfies the Markov property with respect to graph G **if and only if** there exists a family of positive **clique potential functions** $\{\psi_c(\mathbf{x}_c) > 0\}_{c \in \mathcal{C}}$ ($\mathcal{C}$ being the set of all maximal cliques of G) such that

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{x}_c) \quad (5.5)$$

holds (Hammersley & Clifford, 1971; see also Besag, 1974).

■ **From “Potential” to “Energy”: A Notational Conversion That Must Be Done Correctly**

Equation (5.5) is a **product** form; ψ_c itself is required to be strictly positive but is not required to satisfy any additive relation. If we want to rewrite it in the exponential Boltzmann form conventional in statistical physics, $P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})}$, we **cannot** directly set $E(\mathbf{x}) = \sum_c \psi_c(\mathbf{x}_c)$ — that would conflate “a factor inside a product” with “a term added inside an exponent,” which is not even dimensionally consistent: taking the logarithm of the right-hand side should give $-E(\mathbf{x}) = \sum_c \ln \psi_c(\mathbf{x}_c)$, so the correct definition is

$$E(\mathbf{x}) := - \sum_{c \in \mathcal{C}} \ln \psi_c(\mathbf{x}_c), \quad (5.6)$$

that is, “take the negative logarithm of each clique potential, then add them up,” rather than “add the potential functions themselves directly.” Substituting back confirms $\prod_c \psi_c(\mathbf{x}_c) = \prod_c e^{-\ln \psi_c(\mathbf{x}_c) \cdot (-1)} = e^{\sum_c \ln \psi_c(\mathbf{x}_c)} = e^{-E(\mathbf{x})}$, so Equation (5.5) is strictly equivalent to the Boltzmann form $P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})}$. The following subsection, “The Energy-Function Representation,” adopts precisely the definition in Equation (5.6).

Proof sketch. *The easy direction (Gibbs factorization $\Rightarrow$ Markov property):* the intuition here is thoroughly physical. If $P(\mathbf{x}) \propto \prod_c \psi_c(\mathbf{x}_c)$, then when computing the conditional distribution $P(x_i \mid \mathbf{x}_{-i})$, every clique-potential function that does not contain node i is independent of x_i , and cancels completely between numerator and denominator; only cliques containing i (i.e., depending only on i and its neighbors) survive. So the conditional distribution of x_i depends only on the neighbors $N(i)$ — exactly the local Markov property.

The hard direction (Markov property $\Rightarrow$ Gibbs factorization): the core technique of the standard proof (Brook’s lemma) is: fix a reference configuration $\mathbf{x}^0$, and expand $\ln[P(\mathbf{x})/P(\mathbf{x}^0)]$ in an inclusion–exclusion (Möbius-inversion-like) expansion, organized by “which coordinates of $\mathbf{x}$ differ from $\mathbf{x}^0$.” The Markov property guarantees that, in this expansion, every term involving a subset of coordinates that is **not a clique** (does not form a complete subgraph) necessarily has coefficient exactly zero; the surviving terms recombine exactly by maximal clique — note that the object being expanded here is $\ln P$ itself, so each surviving term is directly the clique energy $\phi_c(\mathbf{x}_c) = -\ln \psi_c(\mathbf{x}_c)$ in the sense of Equation (5.6); summing gives the form

$E(\mathbf{x}) = \sum_c \phi_c(\mathbf{x}_c) = -\sum_c \ln \psi_c(\mathbf{x}_c)$, consistent with the product form $\prod_c \psi_c(\mathbf{x}_c) = \prod_c e^{-\phi_c(\mathbf{x}_c)}$ of Equation (5.5). ■

■ Clearing Up a Question

Why can the positivity condition not be omitted? If some configurations have probability strictly zero, the proof of the “hard direction” breaks down — Moussouris (1974) constructed a concrete counterexample: a distribution satisfying the global Markov property that nonetheless cannot be written in Gibbs form factored by clique, with the root cause being precisely that it sets the probability of some otherwise-permitted configuration to zero. This is a reminder that the seemingly obvious physical intuition of deriving “a global energy factorization” from “local constraints” is, in its rigorous form, actually contingent on a premise that is frequently overlooked — and this is exactly where the true value of formal reasoning lies (whether in a mathematical proof or in a proof system such as Lean): exposing the hidden assumptions lurking behind what seems “obvious.”

For a statistical physicist, the significance of this theorem can be summarized in a single sentence: **any Gibbs measure containing only local interactions must have a conditional-independence structure corresponding to some graph; conversely, any (everywhere positive) distribution satisfying the intuitive property “conditionally independent given its neighbors” can necessarily be written in Gibbs form, as a product of local potential functions.** Locality (sparse connectivity of the graph) and the additivity of energy (factorization by clique) are, in essence, two ways of saying the same thing.

The Energy-Function Representation

For an undirected graphical model, define the **energy function**, following Equation (5.6), as

$$E(\mathbf{x}) = -\sum_{c \in \mathcal{C}} \ln \psi_c(\mathbf{x}_c),$$

where $\mathcal{C}$ is the set of all maximal cliques of the graph, $\mathbf{x}_c$ the variables within clique c , and $\psi_c(\mathbf{x}_c) > 0$ the potential function defined on clique c . In practice, this is often turned around: one directly **assigns** each clique a “clique energy” $\phi_c(\mathbf{x}_c) := -\ln \psi_c(\mathbf{x}_c)$ (which may be positive or negative, with no need to take a logarithm), so that $E(\mathbf{x}) = \sum_c \phi_c(\mathbf{x}_c)$ — the one-dimensional Ising example below adopts exactly this equivalent form of “directly specifying energies, rather than specifying potential functions and then taking logarithms.” The joint probability distribution is

$$\boxed{P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})},} \quad (5.7)$$

with partition function

$$Z = \sum_{\mathbf{x}} e^{-E(\mathbf{x})}.$$

This is exactly the form used in statistical physics — and, by the Hammersley–Clifford theorem, this is not a coincidental resemblance, but the **unique** form that “local Markov property” mathematically permits.

Revisiting a Case Study: The One-Dimensional Ising Model Is a Chain-Shaped MRF

We can now rigorously fold the one-dimensional Ising model of §2.3 into the language of MRFs. Viewing each spin s_i as a graph node, and connecting an edge between neighboring spins, gives the chain-shaped graph

$$s_1 - s_2 - s_3 - \cdots - s_N.$$

The maximal cliques of this chain are exactly the neighboring spin pairs $\{s_i, s_{i+1}\}$ (if $h \neq 0$, single nodes $\{s_i\}$ are also cliques, corresponding to the external-field term). Following the “directly specify the clique energy” convention of the previous subsection, take the clique energies to be

$$\phi_{i,i+1}(s_i, s_{i+1}) = -\beta J s_i s_{i+1}, \quad \phi_i(s_i) = -\beta h s_i,$$

(equivalently, the corresponding clique potential functions are $\psi_{i,i+1} = e^{-\phi_{i,i+1}} = e^{\beta J s_i s_{i+1}}$, $\psi_i = e^{\beta h s_i}$, both everywhere positive, satisfying the requirements of the Hammersley–Clifford theorem). Substituting into $E(\mathbf{s}) = \sum_c \phi_c$ immediately recovers the energy function of Equation (2.17), $E = -J \sum_i s_i s_{i+1} - h \sum_i s_i$ (here the clique energy has already absorbed the factor of β , corresponding to the difference in convention between “energy” in the MRF convention and “ $\beta \times$ energy” in the physics convention).

This also explains why the transfer-matrix method works so well: the clique structure of a chain-shaped graph (only nearest-neighbor pairs) is exactly why the transfer matrix $T_{s,s'} = e^{\beta J s s'}$ (Equation (2.20)) is able to write the partition function as the matrix product $\text{Tr}(\mathbf{T}^N)$ (Equation (2.21)) — each step of the transfer corresponds exactly to one clique-potential function. From the point of view of graphical models, **the transfer-matrix method is, in essence, “clique-to-clique” elimination performed along the chain structure of an MRF** — exactly the one-dimensional special case of the belief-propagation idea we will meet later.

At the same time, the Markov property $s_i \perp \{s_j : |j - i| > 1\} \mid s_{i-1}, s_{i+1}$ corresponds, physically, to a simple fact: the i -th spin directly feels only the local field of its nearest neighbors, and the influence of spins farther away must be “relayed” step by step through the intervening spins — this is exactly the graphical-model origin of the exponential decay of the correlation function with distance (Equation (2.24)), and of the existence of a finite correlation length ξ (Equation (2.25)), in the one-dimensional chain.

The Partition Function and Marginal Probabilities of a Three-Node MRF

Let us directly reuse the $N = 3$ open-chain Ising example already worked out in §2.3 ($J = 1$, $k_B T = 1$, i.e., $\beta J = 1$, $h = 0$), and fully compute “marginalization” — an operation of central importance in inference problems — from start to finish. The energy is $E(s_1, s_2, s_3) = -J(s_1 s_2 + s_2 s_3)$, and the Boltzmann weight is $e^{-\beta E} = e^{\beta J(s_1 s_2 + s_2 s_3)}$. Listing out all $2^3 = 8$ states one by one:

Adding all 8 weights in Table 5.3 gives the partition function (checked against the result of the transfer-matrix method in §2.3):

$$Z = 2 \times 7.389 + 4 \times 1 + 2 \times 0.135 = 14.778 + 4 + 0.270 = 19.048 \approx 19.05.$$

Table 5.3: All Microstates and Weights of the $N = 3$ Open-Chain Ising Model ($\beta J = 1$, $h = 0$)

(s_1, s_2, s_3)	$s_1 s_2 + s_2 s_3$	Weight $e^{\beta J(s_1 s_2 + s_2 s_3)}$
$(+, +, +)$	2	$e^2 \approx 7.389$
$(+, +, -)$	0	$e^0 = 1$
$(+, -, +)$	-2	$e^{-2} \approx 0.135$
$(+, -, -)$	0	$e^0 = 1$
$(-, +, +)$	0	$e^0 = 1$
$(-, +, -)$	-2	$e^{-2} \approx 0.135$
$(-, -, +)$	0	$e^0 = 1$
$(-, -, -)$	2	$e^2 \approx 7.389$

Now consider a new question: **if we only care about the orientation of the first spin s_1 , regardless of the specific values of s_2, s_3 , what is the probability that $s_1 = +1$?** This is the **marginal probability** $P(s_1)$, computed by separately summing the weights of every state with $s_1 = +1$, and dividing by the full Z :

$$P(s_1 = +1) = \frac{1}{Z} \sum_{s_2, s_3} e^{-\beta E(+1, s_2, s_3)}.$$

From Table 5.3, $s_1 = +1$ corresponds to the first four rows, whose weights sum to $7.389 + 1 + 0.135 + 1 = 9.524$, giving

$$P(s_1 = +1) = \frac{9.524}{19.048} = 0.500.$$

By the up–down symmetry (already verified in the “Think and Check” box of §2.3), this result is not surprising; the asymmetry between the two ends of the chain does not affect it either, since flipping every spin in the entire chain leaves the energy unchanged when $h = 0$.

More interesting is the conditional probability. If we have **observed** $s_1 = +1$, does the probability that s_3 (the other end of the chain, across s_2) equals $+1$ change as a result? First compute the joint probability: $s_1 = +1$ and $s_3 = +1$ corresponds to the first and third rows of Table 5.3 ($(+, +, +)$ and $(+, -, +)$), whose weights sum to $7.389 + 0.135 = 7.524$, so

$$P(s_1 = +1, s_3 = +1) = \frac{7.524}{19.048} = 0.395,$$

then, by the definition of conditional probability (Chapter 3’s “Preliminary Definitions”: $P(A|B) = P(A, B)/P(B)$):

$$P(s_3 = +1 | s_1 = +1) = \frac{P(s_1 = +1, s_3 = +1)}{P(s_1 = +1)} = \frac{0.395}{0.500} = 0.790.$$

Compared with the unconditional $P(s_3 = +1) = 0.5$, once $s_1 = +1$ is observed, the probability that $s_3 = +1$ jumps to 0.79 — **even though s_1 and s_3 are not directly connected on the graph** (there is no edge between them; s_2 sits between them), correlated information still “propagates” across through s_2 . This is exactly the concrete numerical confirmation of the point emphasized at the start of this section: “no edge does not mean no correlation, only that the

correlation must be relayed along a path on the graph”; and this is precisely why computing something as seemingly “single-variable” as $P(s_1 = +1)$ still requires summing over the entire joint distribution (here, all 8 states) — exactly a miniature prototype, at the smallest possible scale, of the “partition-function computational difficulty” that §5.3 is about to discuss: even with only 3 variables, marginalization already requires traversing the entire state space.

■ Think and Check

In the example above, what should $P(s_2 = +1)$ (the marginal probability of the middle spin) equal? There is no need to recompute it — a symmetry argument suffices.

Solution: Also 0.5 — the entire chain, at $h = 0$, has the symmetry that “flipping every spin simultaneously leaves the energy unchanged,” and this symmetry holds for a spin at any position on the chain, not only at the two ends.

An MRF Model for Image Segmentation

Consider a simple image-segmentation problem: given an image, classify every pixel as “foreground” or “background.” Let the label of pixel i be $x_i \in \{0, 1\}$ (0 = background, 1 = foreground), and let the labels of all pixels form the vector $\mathbf{x} = (x_1, \dots, x_N)$.

The pixels are arranged on a two-dimensional grid, and each pixel is connected to its four neighbors above, below, left, and right — this is the common **4-connected MRF** of image processing, whose maximal cliques are single nodes (corresponding to the data term) and neighboring pixel pairs (corresponding to the smoothness term). Following the “directly specify the clique energy” convention of §5.2, the energy function consists of two parts:

Data term: measures how consistent a label is with the observation,

$$\phi_i^{\text{data}}(x_i) = -\ln P(\text{observation}_i \mid x_i).$$

For example, if pixel i ’s color is c_i , one might model $P(c_i \mid x_i = 1) = \mathcal{N}(c_i; \mu_{\text{fg}}, \sigma_{\text{fg}}^2)$ (the foreground color distribution) and $P(c_i \mid x_i = 0) = \mathcal{N}(c_i; \mu_{\text{bg}}, \sigma_{\text{bg}}^2)$ (the background color distribution).

Smoothness term: encourages neighboring pixels to take the same label,

$$\phi_{ij}^{\text{smooth}}(x_i, x_j) = \begin{cases} 0 & \text{if } x_i = x_j, \\ \lambda & \text{if } x_i \neq x_j, \end{cases}$$

where $\lambda > 0$ is a penalty parameter. The total energy is

$$E(\mathbf{x}) = \sum_i \phi_i^{\text{data}}(x_i) + \sum_{(i,j) \in \mathcal{E}} \phi_{ij}^{\text{smooth}}(x_i, x_j)$$

(equivalently, the corresponding clique potential functions are $\psi_i^{\text{data}} = e^{-\phi_i^{\text{data}}} = P(\text{observation}_i \mid x_i)$, $\psi_{ij}^{\text{smooth}} = e^{-\phi_{ij}^{\text{smooth}}}$, both everywhere positive).

Physical analogy: the data term ϕ_i^{data} is analogous to an “external magnetic field” (the observed data drives each pixel’s label); the smoothness term $\phi_{ij}^{\text{smooth}}$ is analogous to “ferromagnetic coupling” (neighboring pixels favor the same orientation).

This is exactly the form of the Ising model in an external magnetic field — the only difference is that image segmentation uses a two-dimensional grid graph rather than a one-dimensional chain, corresponding to a two-dimensional Ising model, rather than the one-dimensional chain solved exactly by the transfer matrix in §2.3. This also hints at an important fact about complexity: **a one-dimensional chain can be solved exactly, but MRFs in two or more dimensions generally have no analytic solution** — exactly the graph-theoretic root of the partition-function computational difficulty that §5.3 is about to discuss.

MRFs vs. Bayesian Networks: Undirected vs. Directed

There is another major family of graphical models — the **Bayesian Network** (BN), which encodes dependency relations using a directed acyclic graph (DAG) rather than an undirected graph. Comparing the two makes the distinctive features of MRFs clearer:

Table 5.4: Comparing Bayesian Networks and Markov Random Fields

	Bayesian Network (BN)		Markov Random Field (MRF)
Type of graph	Directed	acyclic graph (DAG)	Undirected graph
Relation encoded	A causal generative process		A symmetric interaction
Factorization	$P(\mathbf{x}) = \prod_i P(x_i \mid \text{pa}(i))$ (conditional probability tables, self-normalizing)		$P(\mathbf{x}) = \frac{1}{Z} \prod_c \psi_c(\mathbf{x}_c)$ (clique potentials, require the partition function Z)
Local component	Conditional Table (CPT)	Probability	Potential
Typical example	“Rain $\rightarrow$ wet grass” (a causal chain)		“Rain — wet grass” (correlated, direction not presumed)

The essential difference between the two is this: the directed edges of a Bayesian network encode, **first and foremost**, the factorization of the joint distribution into conditional probabilities, $P(x_1, \dots, x_N) = \prod_i P(x_i \mid x_{\text{pa}(i)})$ — this factorization is itself only a mathematical bookkeeping device, and does not automatically amount to a causal relationship. The very same joint distribution can often be factored using DAGs with different edge directions (for example, the joint distribution $P(A, B)$ of two variables can be written either as $P(A)P(B \mid A)$ or as $P(B)P(A \mid B)$). Only once additional causal-modeling assumptions are layered on top (such as the causal Markov condition, the absence of hidden confounders, and so on — the specialized subject matter of causal-inference theory) can an arrow be given the meaning “the cause comes first, and the effect follows.”

By contrast, the edges of an MRF express only the **existence of an interaction**, presuming no direction and involving no choice of factorization order. This is exactly why equilibrium statistical physics (in which interactions are instantaneous and symmetric — in the spirit of Newton’s third law) is naturally described by MRFs/Gibbs measures rather than Bayesian networks; whereas when a system involves a genuine process of temporal evolution (such as hidden Markov

models, or the reverse denoising chain of a generative diffusion model, where the sequence of time itself naturally supplies a direction), directed graphical models become the more natural choice instead — a contrast that will resurface later, when generative models are discussed.

It is also worth noting: a Bayesian network’s conditional probability tables are automatically normalized ($\sum_{x_i} P(x_i \mid \text{pa}(i)) = 1$), so **no partition function is needed**; this is exactly the flip side of why an MRF must introduce a normalization constant Z , and consequently faces the computational difficulty that §5.3 will discuss — this is precisely the root cause.

The MRF Family: A Unified Picture

Once the language of “graph + clique potentials + partition function” is in place, it becomes clear that a considerable number of models in statistical physics and probabilistic machine learning that appear, on the surface, to be quite different are, in fact, all special cases of the very same MRF framework:

- **Ising model / Potts model:** nodes are discrete spin variables, cliques are nearest-neighbor pairs, and the potential function is the coupling term $-Js_i s_j$;
- **Boltzmann Machine:** nodes are binary neurons, the graph is fully connected, and cliques are arbitrary pairs of neurons;
- **Restricted Boltzmann Machine (RBM):** a bipartite graph between a visible layer and a hidden layer, with cliques existing only on cross-layer edges (no connections within a layer);
- **Conditional Random Field (CRF):** an MRF defined conditional on given observations $\mathbf{y}$ (such as input features), commonly used for sequence labeling;
- **Image segmentation / stereo vision:** a pixel-grid graph, data term + smoothness term;
- In addition, error-correcting codes, lattice models of protein folding, and the like are, in essence, also MRFs with a particular graph structure and clique potentials.

In a single sentence: **the MRF is the precise counterpart, in the language of probabilistic graphical models, of the logical chain in statistical physics “local interaction $\rightarrow$ Hamiltonian $\rightarrow$ Boltzmann distribution.”** The graph encodes “who directly interacts with whom,” the clique potential functions encode “the strength of the interaction,” and the Hammersley–Clifford theorem guarantees the strict equivalence between this local characterization and the global Gibbs distribution — exactly why the Ising model, Boltzmann machines, conditional random fields, and other models that appear to come from entirely different fields belong, in essence, to the very same mathematical family; this point will be developed concretely in the later chapters discussing Hopfield networks, Boltzmann machines, and RBMs.

5.3 The Computational Challenge of the Partition Function

Computational Complexity

For a system with N variables, each taking K possible values, the partition function requires summing over K^N states:

$$Z = \sum_{x_1=1}^K \sum_{x_2=1}^K \cdots \sum_{x_N=1}^K e^{-E(x_1, \dots, x_N)}.$$

For example: $N = 100$ binary variables ($K = 2$) correspond to $2^{100} \approx 10^{30}$ states; an image with $N = 256 \times 256$ pixels, each with 2 possible labels, corresponds to 2^{65536} states. A computation of this magnitude is astronomical — even the fastest supercomputer could not exhaust it.

Why Is the Partition Function So Important?

Despite being computationally difficult, the partition function is indispensable for many tasks.

- (1) **Normalizing probabilities.** Computing any probability $P(\mathbf{x})$ requires knowing Z .
- (2) **Marginalization.** Computing the marginal probability $P(x_i)$:

$$P(x_i) = \sum_{x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_N} P(\mathbf{x}) = \frac{1}{Z} \sum_{\mathbf{x}_{-i}} e^{-E(\mathbf{x})},$$

also requires knowing Z for normalization.

(3) **Learning (parameter estimation).** If the energy function depends on parameters $\boldsymbol{\theta}$, i.e., $E(\mathbf{x}; \boldsymbol{\theta})$, then the partition function also depends on the parameters: $Z(\boldsymbol{\theta}) = \sum_{\mathbf{x}'} e^{-E(\mathbf{x}'; \boldsymbol{\theta})}$. Maximum likelihood estimation requires computing the gradient of the log-likelihood with respect to $\boldsymbol{\theta}$.

Step 1: split $\ln P$ into two terms. Taking the logarithm of Equation (5.7)'s $P(\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} e^{-E(\mathbf{x}; \boldsymbol{\theta})}$ ($\ln(a/b) = \ln a - \ln b$, $\ln e^x = x$):

$$\ln P(\mathbf{x}; \boldsymbol{\theta}) = -E(\mathbf{x}; \boldsymbol{\theta}) - \ln Z(\boldsymbol{\theta}).$$

Differentiating with respect to $\boldsymbol{\theta}$, term by term:

$$\frac{\partial}{\partial \boldsymbol{\theta}} \ln P(\mathbf{x}; \boldsymbol{\theta}) = -\frac{\partial E(\mathbf{x}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} - \frac{\partial \ln Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}. \quad (5.8)$$

The first term is already in the form we want; the second, $\partial \ln Z / \partial \boldsymbol{\theta}$, still needs to be expanded separately — this is exactly the same operation as differentiating $\ln Z$ with respect to β to “generate” the average energy in Chapter 2, §2.2, except that here the variable being differentiated with respect to is the parameter $\boldsymbol{\theta}$.

Step 2: expand $\partial \ln Z / \partial \boldsymbol{\theta}$. Using $\partial(\ln Z) / \partial \boldsymbol{\theta} = (1/Z) \partial Z / \partial \boldsymbol{\theta}$, first compute $\partial Z / \partial \boldsymbol{\theta}$: differentiating $Z(\boldsymbol{\theta}) = \sum_{\mathbf{x}'} e^{-E(\mathbf{x}'; \boldsymbol{\theta})}$ term by term (the chain rule, $\partial e^{-E(\mathbf{x}'; \boldsymbol{\theta})} / \partial \boldsymbol{\theta} = -\frac{\partial E(\mathbf{x}'; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} e^{-E(\mathbf{x}'; \boldsymbol{\theta})}$):

$$\frac{\partial Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_{\mathbf{x}'} \left(-\frac{\partial E(\mathbf{x}'; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \right) e^{-E(\mathbf{x}'; \boldsymbol{\theta})}.$$

Dividing both sides by Z , and substituting $e^{-E(\mathbf{x}';\boldsymbol{\theta})}/Z = P(\mathbf{x}';\boldsymbol{\theta})$ (Equation (5.7)) inside the sum:

$$\frac{\partial \ln Z(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \frac{1}{Z} \frac{\partial Z}{\partial \boldsymbol{\theta}} = - \sum_{\mathbf{x}'} P(\mathbf{x}';\boldsymbol{\theta}) \frac{\partial E(\mathbf{x}';\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = - \left\langle \frac{\partial E}{\partial \boldsymbol{\theta}} \right\rangle, \quad (5.9)$$

where the last step uses the definition of expectation value (Chapter 2’s “Preliminary Definitions”): summing over all states, weighted by probability, is exactly the expectation $\langle \cdot \rangle$.

Step 3: substitute back into Equation (5.8). Substituting Equation (5.9), and noting that it itself carries a minus sign, which multiplies against the minus sign of “ $-\partial \ln Z / \partial \boldsymbol{\theta}$ ” in Equation (5.8), so the two minus signs cancel to give a plus sign:

$$\frac{\partial}{\partial \boldsymbol{\theta}} \ln P(\mathbf{x};\boldsymbol{\theta}) = - \frac{\partial E(\mathbf{x};\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} - \left(- \left\langle \frac{\partial E}{\partial \boldsymbol{\theta}} \right\rangle \right) = - \frac{\partial E(\mathbf{x};\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} + \left\langle \frac{\partial E}{\partial \boldsymbol{\theta}} \right\rangle.$$

The gradient of the log-likelihood is

$$\boxed{\frac{\partial \ln P(\mathbf{x};\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = - \frac{\partial E(\mathbf{x};\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} + \left\langle \frac{\partial E}{\partial \boldsymbol{\theta}} \right\rangle.} \quad (5.10)$$

This formula carries a profound physical meaning: the first term lowers the energy of the observed data (the “**positive phase**”); the second term raises the average energy over all states (the “**negative phase**”). Computing the second term (an expectation) requires summing or sampling over every possible state, and this is exactly the principal difficulty in training a Boltzmann machine.

Understanding the “Positive Phase” and “Negative Phase” with the $N = 2$ Ising Chain

To make the abstract “positive phase minus negative phase” concrete, let us work it out explicitly using the $N = 2$ Ising chain of §2.3 ($h = 0$, a single bond, $E = -Js_1s_2$). Treat the coupling strength J as the parameter θ to be learned; the energy is $E(s_1, s_2; J) = -Js_1s_2$, so $\partial E / \partial J = -s_1s_2$. Substituting into Equation (5.10):

$$\frac{\partial \ln P}{\partial J} = s_1^{\text{data}} s_2^{\text{data}} - \langle s_1 s_2 \rangle_{\text{model}}.$$

A direct derivation of $\langle s_1 s_2 \rangle_{\text{model}}$. This $N = 2$ system has only two “alignment patterns”: aligned ($s_1 s_2 = +1$, corresponding to states $(+, +)$ and $(-, -)$) and anti-aligned ($s_1 s_2 = -1$, corresponding to $(+, -)$ and $(-, +)$). Using the Boltzmann weights already computed in Table 2.5 of §2.3, the two aligned states each have weight $e^{\beta J}$, and the two anti-aligned states each have weight $e^{-\beta J}$, so

$$\langle s_1 s_2 \rangle_{\text{model}} = \frac{(+1) \cdot 2e^{\beta J} + (-1) \cdot 2e^{-\beta J}}{2e^{\beta J} + 2e^{-\beta J}} = \frac{e^{\beta J} - e^{-\beta J}}{e^{\beta J} + e^{-\beta J}} = \tanh(\beta J).$$

(This result agrees exactly with the value the correlation-function formula (2.24) gives at $r = 1$, derived in §2.3 for an infinitely long periodic chain in the thermodynamic limit — but the $N = 2$ system here has only a single bond and involves no thermodynamic limit at all; it is the exact

result of directly taking an expectation over this small system. The two different derivation paths simply coincide here, rather than one depending on the other.)

Substituting concrete numbers: let the current parameter be $J = 0.5$ ($\beta = 1$), and suppose the observed data is exactly $(s_1, s_2) = (+1, +1)$ (the “positive phase” contributes $s_1^{\text{data}} s_2^{\text{data}} = 1$):

$$\begin{aligned}\langle s_1 s_2 \rangle_{\text{model}} &= \tanh(0.5) = 0.462 \quad (\text{the “negative phase”}), \\ \frac{\partial \ln P}{\partial J} &= 1 - 0.462 = 0.538 > 0.\end{aligned}$$

The gradient is positive, meaning that increasing J raises the log-likelihood of the observed data — entirely consistent with intuition: the data show the two spins to be aligned, and the current $J = 0.5$ is not yet large enough for the model to sufficiently favor the aligned configuration, so the learning algorithm will continue to increase J .

When does this process stop? When the gradient is exactly zero, that is, when the “positive phase” and “negative phase” are equal:

$$s_1^{\text{data}} s_2^{\text{data}} = \langle s_1 s_2 \rangle_{\text{model}}.$$

If the training data consist of more than one sample, and the **empirical average** over multiple observations is $\langle s_1 s_2 \rangle_{\text{data}}$, the convergence condition becomes $\tanh(\beta J^*) = \langle s_1 s_2 \rangle_{\text{data}}$. As a concrete example: if the data give $\langle s_1 s_2 \rangle_{\text{data}} = 0.6$ (neighboring spins align noticeably more often than not), solving gives

$$\beta J^* = \operatorname{arctanh}(0.6) = \frac{1}{2} \ln \frac{1.6}{0.4} = \frac{1}{2} \ln 4 = \ln 2 \approx 0.693$$

(here $\operatorname{arctanh}(0.6) = \ln 2$ is a coincidence convenient for hand-checking: $\tanh(\ln 2) = \frac{e^{\ln 2} - e^{-\ln 2}}{e^{\ln 2} + e^{-\ln 2}} = \frac{2 - 0.5}{2 + 0.5} = \frac{1.5}{2.5} = 0.6$, matching exactly).

■ Clearing Up a Question

This is an extremely important conclusion, one that will reappear repeatedly in the later chapters discussing Boltzmann machines: **the maximum-likelihood solution of an MRF is, in essence, exactly the requirement that the statistic the model predicts (here, the spin correlation $\langle s_1 s_2 \rangle$) precisely matches the empirical statistic in the training data**, a property called “moment matching.” The “positive phase” comes from the real data, and the “negative phase” comes from the distribution the model itself generates; training is exactly the process of repeatedly adjusting the parameters until the statistical regularities the model “imagines” align completely with the statistical regularities “actually observed” in the data. Training a Boltzmann machine is difficult precisely because, for a complex network, the “negative phase” — the model’s own expectation value — has no closed-form formula (unlike the clean $\tanh(\beta J)$ here), and must instead be approximated through sampling — exactly one of the central topics of the later chapters discussing Boltzmann-machine training.

5.4 An Overview of Approximation Methods

Since exact computation of the partition function is, in general, infeasible, approximation methods are required. These methods have corresponding versions in both statistical physics and machine learning:

Table 5.5: Approximation Methods for the Partition Function: Correspondence Between Statistical Physics and Machine Learning

Method	Statistical physics	Machine learning
Mean-field approximation	Ignore fluctuations, use the average value instead	The mean-field approximation in variational inference
Monte Carlo sampling	The Metropolis algorithm, Gibbs sampling	MCMC, Gibbs sampling, HMC
Variational methods	The Bogoliubov inequality	Variational autoencoders (VAE), variational Bayes
Belief propagation	Transfer matrix, the Bethe approximation	The sum-product algorithm, loopy BP
Renormalization group	Coarse-graining, phase-transition analysis	Understanding hierarchical structure in deep learning

The mean-field approximation will be developed systematically in Chapter 6; the MCMC and Gibbs sampling in the “Monte Carlo sampling” row rely, for their correctness, on a body of theory concerning Markov chains themselves — stationary distributions, detailed balance, ergodicity — which this chapter does not develop further. Appendix C, “Markov Chains and MCMC Basics,” gives the minimal theoretical background needed to understand this row, as well as for the Gibbs-sampling implementation exercise at the end of this chapter.

The next chapter discusses variational methods in detail.

■ Summary

The partition function Z is nothing more than a sum of a few terms in small-scale problems (as this chapter has repeatedly computed by hand), but the number of states grows **exponentially** with the number of variables, which makes exact computation of Z completely infeasible, at real-world scale, for **general high-dimensional graphical models with complex loop structure**. This difficulty is not something “more computing power” can solve — even harnessing the computational resources of the entire universe would not suffice to count all 2^{100} states at $N = 100$.

It is worth emphasizing that “generally infeasible” does not mean “infeasible everywhere”: §5.2 already showed that a chain-shaped graph (the one-dimensional Ising model) allows Z to be computed exactly to the end, via the transfer matrix, and this is no accident — whenever a graph has low **treewidth** (chains and trees themselves are the special case of treewidth 1), variable elimination/belief propagation can compute Z exactly in polynomial

time; what genuinely makes the partition function intractable is the situation where a graph contains a large number of loops, with treewidth growing together with the system size (for example, two-dimensional and higher Ising models, or dense Boltzmann machines) — and it is precisely in this case that no known polynomial-time algorithm exists. The “approximation methods” discussed below are aimed exactly at this genuinely difficult class of situations.

The reason the partition function cannot be sidestepped is that it simultaneously controls normalization, marginal probabilities, and — through the elegant formula “positive phase minus negative phase” — the gradient for parameter learning. The next chapter confronts this difficulty head-on: by introducing the **variational free energy**, it replaces the (for a general graph) impossible task of “exactly computing Z ” with the task of “searching for a sufficiently good approximating distribution” — a task an optimization algorithm can actually solve.

Exercises

Foundational

Exercise 5.1: The $N = 3$ Open-Chain Ising Model at Stronger Coupling

The “marginalization” worked example of §5.2 examined the $N = 3$ open-chain Ising model ($J = 1$, $k_B T = 1$, i.e., $\beta J = 1$, $h = 0$). Now take a stronger coupling, $\beta J = 2$ (all other settings unchanged), and repeat the same calculation.

- Following the format of Table 5.3, list all $2^3 = 8$ microstates (s_1, s_2, s_3) and their Boltzmann weights $e^{\beta J(s_1 s_2 + s_2 s_3)}$;
- Find the partition function Z , and check the result against Chapter 2’s open-chain formula $Z_{\text{open chain}}(N) = 2(2 \cosh(\beta J))^{N-1}$;
- Find the marginal probability $P(s_1 = +1)$ and the conditional probability $P(s_3 = +1 \mid s_1 = +1)$;
- In the main text, at $\beta J = 1$, we computed $P(s_3 = +1 \mid s_1 = +1) = 0.790$. At the stronger coupling $\beta J = 2$, do you expect this conditional probability to move closer to 1 or closer to 0.5? First give a physically intuitive prediction, then verify it using your result from (c).

Exercise 5.2: Cliques and Conditional Independence on a Tree Graph

Consider 5 binary random variables A, B, C, D, E , forming the following undirected graph: B has an edge to each of A, C, D ; D additionally has an edge to E (that is, $A-B-C$ forms one “branch,” $B-D-E$ forms another “branch,” and the whole graph contains no loop).

- Write down all the maximal cliques of this graph (the definition of a clique in §5.2);

- (b) By the local Markov property (5.3): once conditioned on all of B 's neighbors, which variables in the graph is A conditionally independent of? Once conditioned on all of D 's neighbors, which variables is E conditionally independent of?
- (c) Using the concept of “separation” from the global Markov property (the “Graph-Theoretic Language” subsection of §5.2): does $\{B, D\}$ separate A and E ? Determine, from this, whether $X_A \perp X_E \mid X_B, X_D$ holds;
- (d) This graph contains no loop, and has treewidth 1 (the concept mentioned in the summary of §5.4). Even if each of $A, \dots, E$ can take many values (say $K = 10$), can the partition function Z still be computed exactly? Why (hint: the transfer-matrix method for the one-dimensional Ising chain in §5.2's “Revisiting a Case Study” applies equally to a tree graph, except that the matrices must be multiplied separately along each branch of the tree)?

Intermediate

Exercise 5.3: Learning the Coupling Strength and the External Field Simultaneously

The “positive phase minus negative phase” worked example in §5.3 treated only the case $h = 0$, learning only the coupling strength J . Now consider the same $N = 2$ Ising chain, but with an external field $h \neq 0$, and energy

$$E(s_1, s_2; J, h) = -Js_1s_2 - h(s_1 + s_2),$$

treating J and h *simultaneously* as parameters to be learned.

- (a) Write down the partition function $Z(J, h)$ (summing over all 4 states $(s_1, s_2) \in \{\pm 1\}^2$), and find the expressions for $\langle s_1s_2 \rangle_{\text{model}}$ and $\langle s_1 + s_2 \rangle_{\text{model}}$ in terms of J, h ;
- (b) Using Equation (5.10) (the log-likelihood gradient formula), write down the “positive phase minus negative phase” expressions for $\partial \ln P / \partial J$ and $\partial \ln P / \partial h$ separately;
- (c) Suppose the training data give empirical statistics $\langle s_1s_2 \rangle_{\text{data}} = 0.6$, $\langle s_1 + s_2 \rangle_{\text{data}} = 0.4$; write down (there is no need to solve for numerical values — just set up) the system of self-consistency equations that determines the optimal J^*, h^* ;
- (d) Discuss: why do we get a *system of equations* here, rather than being able to solve directly for a closed form for J , as in the main text's $h = 0$ case? What does this imply for training a Boltzmann machine with a larger scale and more parameters?

Exercise 5.4: Verifying the “Easy Direction” of Hammersley–Clifford with Concrete Numbers

The “Proof Sketch” paragraph of §5.2 gave a general argument for the direction “Gibbs factorization $\Rightarrow$ local Markov property”; this problem grounds that abstract argument in

concrete numbers, using the $N = 3$ open-chain Ising model (§5.2’s “A Worked Example: Three-Node MRF,” $\beta J = 1$, the weights already computed in Table 5.3).

- (a) Using the weights in Table 5.3, compute the specific numerical values of the two conditional probabilities $P(s_1 = +1 \mid s_2 = +1, s_3 = +1)$ and $P(s_1 = +1 \mid s_2 = +1, s_3 = -1)$;
- (b) Compare the two values from (a): are they equal? What does this tell us (hint: does the local Markov property hold — that is, is s_1 really conditionally independent of s_3 once s_2 is given)?
- (c) Starting directly from the energy function $E(s_1, s_2, s_3) = -J(s_1 s_2 + s_2 s_3)$, prove algebraically that, for *any* fixed value of s_3 , $P(s_1 \mid s_2, s_3)$ does not depend on the value of s_3 (hint: split $e^{-\beta E}$ into two factors according to their dependence on s_1 , find which factor does not involve s_1 , and see how it cancels between numerator and denominator of the conditional probability);
- (d) Match the algebraic manipulation in (c) to the statement, in the “Proof Sketch” Remark, that “every clique-potential function not containing node i cancels completely between numerator and denominator” — here, which specific term is “the clique potential not containing node s_1 ”?

Challenge

Exercise 5.5: Gibbs Sampling and Image Segmentation

§5.2 used a simplified image-segmentation MRF model (data term + smoothness term) to illustrate the application of MRFs in computer vision, but did not show how to actually solve this model. Before attempting this problem, it is recommended to first read Appendix C, “Markov Chains and MCMC Basics,” for the theoretical background needed to understand “why repeated sampling eventually converges to the target distribution.”

- (a) **Implementation:** implement the Gibbs sampling algorithm in code, for the energy function $E(\mathbf{x}) = \sum_i \phi_i^{\text{data}}(x_i) + \sum_{(i,j)} \phi_{ij}^{\text{smooth}}(x_i, x_j)$ defined in the main text, updating one pixel at a time by sampling from its conditional probability $P(x_i \mid \mathbf{x}_{-i})$, iterating repeatedly until convergence;
- (b) **Experiment:** test your implementation on a simple synthetic image (for example, a white circular foreground + black background, with Gaussian noise added), and observe how the segmentation result evolves, as the number of iterations increases, from “noise dominated entirely by the data term” toward being regularized by the “smoothness term”;
- (c) **Parameter sweep:** vary the smoothness-term strength λ , and discuss what problems arise when λ is too large or too small;
- (d) **Further reading:** the “Think and Check” box of §5.2 noted that the proof of the “local $\Rightarrow$ global” direction of the Hammersley–Clifford theorem depends on the distribution

being everywhere positive, and mentioned that Moussouris (1974) constructed a counterexample. Look up this material, understand the construction of this counterexample, and explain why it depends precisely on the fact that “certain configurations have probability zero.”

Chapter 6

The Deep Connection Between Free Energy and Loss Functions

By indirections find directions out.

— Shakespeare, *Hamlet*, Act II, Scene 1, spoken by Polonius

Objectives

- Master the derivation of variational free energy, prove that it is a strict upper bound on the true free energy, and understand its equivalence to a statement about the KL divergence;
- Master the derivation of the evidence lower bound (ELBO) in machine learning, and understand its strict mathematical correspondence with variational free energy in statistical physics;
- Verify the variational principle through concrete worked examples, and understand the principle that “a larger variational family gives a more accurate approximation,” together with the reliability guarantee provided by the variational lower bound;
- Master the derivation of the mean-field approximation and its self-consistency equation, and, through the example of the one-dimensional Ising model, understand why the mean-field approximation can give qualitatively incorrect conclusions in low-dimensional systems.

■ Chapter Guide

The question this chapter answers: When the partition function cannot be computed exactly, physics and machine learning each independently developed the very same method for coping with this — variational inference. Are these really the same mathematical theorem? If so, what is the core proof they share?

Correspondence with statistical physics: The variational-free-energy bound $F[Q] \geq F_{\text{true}}$ and the machine-learning ELBO inequality $\mathcal{L}[Q] \leq \ln P(\mathbf{v})$ are two statements of exactly the same inequality, sharing the same proof based on the non-negativity of the KL divergence; the mean-field approximation corresponds to the factorization assumption that “the variables are mutually independent,” and its derivation uses nothing more than the ordinary method of Lagrange multipliers, not some new tool requiring the calculus of variations (see the “preliminary note” in §6.4).

Looking ahead: Chapter 7 will revisit the Boltzmann distribution that runs throughout this book once more, this time from the point of view of the maximum entropy principle — an independent derivation path running parallel to this chapter’s variational argument, marking the completion of Part I’s theoretical framework.

Chapter 5 showed that the partition function Z cannot be computed exactly at real-world scale, and that this brings marginalization and the gradient of parameter learning to a standstill alike. This chapter presents the solution jointly adopted by statistical physics and machine learning — **variational methods**: rather than forcing an exact computation of Z , give up on “exactness” and instead search, within a family of tractable distributions, for an approximation “close enough” to the true distribution. In physics this idea is called “variational free energy”; in machine learning it is called the “Evidence Lower BOund” (ELBO) — the two are, in fact, two names, in two different fields, for the very same mathematical theorem.

6.1 The Physical Derivation of Variational Free Energy

Motivation

When the partition function $Z = \sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})}$ cannot be computed exactly, we would like to find a computable upper or lower bound for it. The core idea of the variational method is: introduce a simple trial distribution $Q(\mathbf{x})$, and approximate the true distribution $P(\mathbf{x})$ by optimizing Q .

The Definition of Variational Free Energy

For a trial distribution $Q(\mathbf{x})$, define the **variational free energy**:

$$F[Q] = \langle E(\mathbf{x}) \rangle_Q - TS[Q], \quad (6.1)$$

where $\langle E(\mathbf{x}) \rangle_Q = \sum_{\mathbf{x}} Q(\mathbf{x}) E(\mathbf{x})$ is the expectation value of the energy, and $S[Q] = -k_B \sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x})$ is the entropy of the distribution Q (to simplify notation, we set $k_B = 1$ below, so $\beta = 1/T$).

Proof of the Variational Principle

Theorem 6.1: Variational Free Energy Is an Upper Bound on the True Free Energy

For any trial distribution $Q(\mathbf{x})$,

$$F[Q] \geq F_{\text{true}} = -T \ln Z,$$

with equality if and only if $Q(\mathbf{x}) = P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})/T}$.

Proof. Before proving the main theorem, let us first establish a basic fact that will be used repeatedly: the **non-negativity of the KL divergence** (the Gibbs inequality).

■ Preliminary Tools: Jensen's Inequality and the Gibbs Inequality

If f is a convex function (i.e., the line segment connecting any two points on the graph of f never lies below f itself — intuitively, the graph “curves upward”), **Jensen's inequality** asserts that, for any random quantity X ,

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$$

(“averaging first, then applying the function” is no smaller than “applying the function first, then averaging,” because a convex function lifts up any “spread-out” points). $-\ln$ is a convex function (since $(-\ln x)'' = 1/x^2 > 0$), so applying Jensen's inequality to $-\ln$ gives $\mathbb{E}[-\ln X] \geq -\ln \mathbb{E}[X]$, i.e., $\mathbb{E}[\ln X] \leq \ln \mathbb{E}[X]$.

Using this, we prove $D_{KL}(Q\|P) = \sum_{\mathbf{x}} Q(\mathbf{x}) \ln \frac{Q(\mathbf{x})}{P(\mathbf{x})} \geq 0$: write the sum as an expectation with respect to the distribution Q , and let the random quantity $X = P(\mathbf{x})/Q(\mathbf{x})$ (where $\mathbf{x}$ is drawn according to Q); then

$$D_{KL}(Q\|P) = -\sum_{\mathbf{x}} Q(\mathbf{x}) \ln \frac{P(\mathbf{x})}{Q(\mathbf{x})} = -\mathbb{E}_Q[\ln X] \geq -\ln \mathbb{E}_Q[X],$$

where the last step uses $\mathbb{E}[\ln X] \leq \ln \mathbb{E}[X]$, just derived from Jensen's inequality. And

$$\mathbb{E}_Q[X] = \sum_{\mathbf{x}} Q(\mathbf{x}) \cdot \frac{P(\mathbf{x})}{Q(\mathbf{x})} = \sum_{\mathbf{x}} P(\mathbf{x}) = 1,$$

(in the terms summed over $\mathbf{x}$, $Q(\mathbf{x})$ cancels exactly, leaving $P(\mathbf{x})$ summed over every state, which equals 1 by normalization), so $-\ln \mathbb{E}_Q[X] = -\ln 1 = 0$, and substituting into the inequality above gives

$$D_{KL}(Q\|P) \geq 0,$$

with equality if and only if Jensen's inequality holds with equality, which requires $X = P(\mathbf{x})/Q(\mathbf{x})$ to be constant almost everywhere with respect to Q ; combined with the normalization $\mathbb{E}_Q[X] = 1$, this constant can only be 1, i.e., $P(\mathbf{x}) = Q(\mathbf{x})$ (for every $\mathbf{x}$ with $Q(\mathbf{x}) > 0$). ■

(The exercise “Non-negativity of the KL Divergence” at the end of this chapter asks the reader to independently redo this proof, as reinforcement.)

With the tool $D_{KL}(Q\|P) \geq 0$ in hand, we can now prove the main theorem. Define the KL divergence

$$D_{KL}(Q\|P) = \sum_{\mathbf{x}} Q(\mathbf{x}) \ln \frac{Q(\mathbf{x})}{P(\mathbf{x})}. \quad (6.2)$$

Substituting $P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})/T}$ and expanding:

$$\begin{aligned} D_{KL}(Q\|P) &= \sum_{\mathbf{x}} Q(\mathbf{x}) \ln \frac{Q(\mathbf{x})}{\frac{1}{Z} e^{-E(\mathbf{x})/T}} \\ &= \sum_{\mathbf{x}} Q(\mathbf{x}) \left[\ln Q(\mathbf{x}) - \ln \left(\frac{1}{Z} e^{-E(\mathbf{x})/T} \right) \right] \quad (\text{using } \ln(a/b) = \ln a - \ln b) \\ &= \sum_{\mathbf{x}} Q(\mathbf{x}) \left[\ln Q(\mathbf{x}) - \ln \frac{1}{Z} - \ln e^{-E(\mathbf{x})/T} \right] \quad (\text{using } \ln(ab) = \ln a + \ln b \text{ with a minus sign}) \\ &= \sum_{\mathbf{x}} Q(\mathbf{x}) \left[\ln Q(\mathbf{x}) + \ln Z + \frac{E(\mathbf{x})}{T} \right] \quad (\text{using } \ln(1/Z) = -\ln Z \text{ and } \ln e^{-E/T} = -E/T) \\ &= \sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x}) + \frac{1}{T} \sum_{\mathbf{x}} Q(\mathbf{x}) E(\mathbf{x}) + \ln Z \sum_{\mathbf{x}} Q(\mathbf{x}) \\ &= -S[Q] + \frac{1}{T} \langle E \rangle_Q + \ln Z. \end{aligned}$$

Therefore

$$D_{KL}(Q\|P) = \frac{1}{T} [\langle E \rangle_Q - TS[Q]] + \ln Z = \frac{1}{T} F[Q] + \ln Z.$$

From the just-proved $D_{KL}(Q\|P) \geq 0$, we get $\frac{1}{T} F[Q] + \ln Z \geq 0$, that is,

$$F[Q] \geq -T \ln Z = F_{\text{true}}.$$

■

The significance of the variational principle:

1. $F[Q]$ provides an upper bound on the true free energy;
2. Minimizing $F[Q]$ is equivalent to minimizing $D_{KL}(Q\|P)$, i.e., to making Q approximate P ;
3. The optimal Q can be searched for within a family of simple distributions, thereby bypassing the difficulty of directly computing Z .

Verifying the Variational Principle with the Two-Level System

To see concretely what the abstract inequality $F[Q] \geq F_{\text{true}}$ actually means, let us verify it explicitly using the two-level system of §2.2 ($E_0 = 0$, $E_1 = \varepsilon = 1$, $k_B T = 1$). Recall the exact solution computed there:

$$P_0 = 0.731, \quad P_1 = 0.269, \quad F_{\text{true}} = -T \ln Z = -0.313.$$

Now, **suppose we do not know the exact solution**, and are only permitted to search within a family of simple variational distributions: take $Q(0) = q$, $Q(1) = 1 - q$ ($q \in (0, 1)$ is the sole variational parameter), and substitute directly into the definition to compute

$$F[Q] = \langle E \rangle_Q - TS[Q] = (1 - q)\varepsilon - T[-q \ln q - (1 - q) \ln(1 - q)] :$$

Table 6.1: The Two-Level-System Variational Example: $F[Q]$ and D_{KL} at Different Values of q

q	$\langle E \rangle_Q$	$S[Q]$	$F[Q]$	$D_{KL}(Q\ P) = F[Q] - F_{\text{true}}$
0.300	0.700	0.611	0.089	0.402
0.500	0.500	0.693	-0.193	0.120
0.600	0.400	0.673	-0.273	0.040
0.700	0.300	0.611	-0.311	0.002
0.731	0.269	0.582	-0.313	0.000
0.800	0.200	0.500	-0.300	0.013
0.900	0.100	0.325	-0.225	0.088

Table 6.1 turns the statement “variational free energy is an upper bound on the true free energy” into visible numbers: **no matter what value q takes, $F[Q]$ is never less than -0.313 ; and precisely when q equals the exact solution $P_0 = 0.731$, $F[Q]$ exactly “bottoms out,” equaling F_{true} , with the corresponding KL divergence dropping to zero.** In other words, “minimizing $F[Q]$ within the variational family,” if carried out thoroughly enough (searching over every possible q), automatically recovers the exact solution — this is because the variational family here, $\{Q(0) = q, Q(1) = 1 - q : q \in (0, 1)\}$, happens to cover **every** possible distribution of the two-level system, with no approximating simplification made at all.

■ Clearing Up a Question

In real applications (such as §6.4), the variational family is usually not this “exhaustive” — we deliberately restrict the form of Q (for instance, forcing different variables to be mutually independent), and in that case the optimal Q^* can no longer equal P exactly; $D_{KL}(Q^*\|P) > 0$ becomes an irreducible “approximation error.” But no matter how restricted the variational family is, the inequality $F[Q] \geq F_{\text{true}}$ always holds — and this is exactly what makes the variational method reliable as a “quality-guaranteed approximation” method: we may not know exactly what F_{true} is, but we can be certain that whatever answer $F[Q]$ gives is never more “optimistic” than the true value.

■ Think and Check

In Table 6.1, $q = 0.5$ (i.e., assuming the two states are equally probable) gives $F[Q] = -0.193$, noticeably larger than $F_{\text{true}} = -0.313$. Is this “overestimate” mainly because the energy expectation $\langle E \rangle_Q$ is too high at $q = 0.5$, or because the entropy $S[Q]$ is too low? Solution: Comparing the rows $q = 0.5$ and $q = 0.731$ — the entropy is actually *higher* at $q = 0.5$ ($0.693 > 0.582$; a uniform distribution has the highest entropy to begin with). What

actually “costs” the model is the energy: $q = 0.5$ assigns too much probability to the excited state ($1 - q = 0.5$, while the true P_1 is only 0.269), causing $\langle E \rangle_Q = 0.5$ to be far higher than the true 0.269. This shows that the error in $F[Q]$ need not come only from the entropy being insufficiently “concentrated”; an inaccurate estimate of the energy term is equally an important source of error, and which term dominates in any given case requires case-by-case analysis.

6.2 Variational Inference in Machine Learning

Problem Setup

In machine learning we frequently confront models involving **latent variables**. Let $\mathbf{v}$ be the visible variables (observed data), $\mathbf{h}$ the latent variables (underlying structure), and $\boldsymbol{\theta}$ the model parameters, with joint probability $P(\mathbf{v}, \mathbf{h}; \boldsymbol{\theta})$. There are two goals: **inference** — given $\mathbf{v}$, compute the posterior $P(\mathbf{h}|\mathbf{v}; \boldsymbol{\theta})$; and **learning** — estimate the parameters $\boldsymbol{\theta}$ from the data $\{\mathbf{v}^{(n)}\}$. Both involve marginalization:

$$P(\mathbf{v}; \boldsymbol{\theta}) = \sum_{\mathbf{h}} P(\mathbf{v}, \mathbf{h}; \boldsymbol{\theta}),$$

a sum that is typically intractable, sharing its origin with the difficulty of computing the partition function.

The Evidence Lower Bound (ELBO)

Introduce a variational distribution $Q(\mathbf{h})$ to approximate the posterior $P(\mathbf{h}|\mathbf{v})$. Define the **Evidence Lower Bound** (ELBO):

$$\mathcal{L}[Q] = \sum_{\mathbf{h}} Q(\mathbf{h}) \ln \frac{P(\mathbf{v}, \mathbf{h})}{Q(\mathbf{h})}. \quad (6.3)$$

■ A Notational Reminder: $\mathcal{L}$ ’s Third Meaning

Chapter 3 used $\mathcal{L}$ to denote the **likelihood function** (larger is better); Chapter 4 used the very same symbol for the **loss function** (the negative log-likelihood, smaller is better — both chapters already flagged this once there). Here, $\mathcal{L}[Q]$ is a third, entirely independent meaning — the **evidence lower bound**, likewise larger is better, but it is a *functional* of the variational distribution Q , not the value of a function at a point. All three share the same letter only because they all derive from the initial of “likelihood/loss”; there is no deeper mathematical connection. Please always distinguish square brackets $\mathcal{L}[\cdot]$ (this section’s notation) from round brackets $\mathcal{L}(\cdot)$ (Chapters 3 and 4’s notation).

Expanding:

$$\begin{aligned}\mathcal{L}[Q] &= \sum_{\mathbf{h}} Q(\mathbf{h}) [\ln P(\mathbf{v}, \mathbf{h}) - \ln Q(\mathbf{h})] \\ &= \sum_{\mathbf{h}} Q(\mathbf{h}) \ln P(\mathbf{v}, \mathbf{h}) + H[Q(\mathbf{h})],\end{aligned}$$

where $H[Q] = -\sum_{\mathbf{h}} Q(\mathbf{h}) \ln Q(\mathbf{h})$ is the entropy.

The Relation Between the ELBO and the Log-Evidence

Theorem 6.2: The ELBO Is a Lower Bound on the Log-Evidence

For any variational distribution $Q(\mathbf{h})$,

$$\mathcal{L}[Q] \leq \ln P(\mathbf{v}),$$

with equality if and only if $Q(\mathbf{h}) = P(\mathbf{h}|\mathbf{v})$.

Proof. First, we prove the identity

$$\ln P(\mathbf{v}) = \mathcal{L}[Q] + D_{KL}(Q(\mathbf{h}) \| P(\mathbf{h}|\mathbf{v})). \quad (6.4)$$

By the definition (6.3):

$$\begin{aligned}\mathcal{L}[Q] &= \sum_{\mathbf{h}} Q(\mathbf{h}) \ln \frac{P(\mathbf{v}, \mathbf{h})}{Q(\mathbf{h})} \\ &= \sum_{\mathbf{h}} Q(\mathbf{h}) \ln \frac{P(\mathbf{h}|\mathbf{v})P(\mathbf{v})}{Q(\mathbf{h})} \quad (\text{Chapter 3's "Preliminary Definitions": conditional probability } P(\mathbf{h}|\mathbf{v})) \\ &= \sum_{\mathbf{h}} Q(\mathbf{h}) [\ln P(\mathbf{v}) + \ln P(\mathbf{h}|\mathbf{v}) - \ln Q(\mathbf{h})] \\ &= \ln P(\mathbf{v}) \sum_{\mathbf{h}} Q(\mathbf{h}) + \sum_{\mathbf{h}} Q(\mathbf{h}) \ln \frac{P(\mathbf{h}|\mathbf{v})}{Q(\mathbf{h})} \\ &= \ln P(\mathbf{v}) - D_{KL}(Q(\mathbf{h}) \| P(\mathbf{h}|\mathbf{v})),\end{aligned}$$

where the last step uses the fact that $Q(\mathbf{h})$, being a probability distribution, satisfies the normalization $\sum_{\mathbf{h}} Q(\mathbf{h}) = 1$, so the first term becomes $\ln P(\mathbf{v}) \sum_{\mathbf{h}} Q(\mathbf{h}) = \ln P(\mathbf{v}) \cdot 1 = \ln P(\mathbf{v})$, giving the constant term directly; the second term is exactly the negative of the KL-divergence definition (6.2). Rearranging gives Equation (6.4). From $D_{KL} \geq 0$ (already proved in §6.1), we immediately get $\mathcal{L}[Q] \leq \ln P(\mathbf{v})$. ■

The strategy of variational inference:

1. **Inference:** given model parameters θ and data $\mathbf{v}$, search for the optimal Q^* within a variational family $\mathcal{Q}$:

$$Q^*(\mathbf{h}) = \arg \max_{Q \in \mathcal{Q}} \mathcal{L}[Q];$$

2. **Learning:** maximize the expected ELBO with respect to the parameters θ :

$$\theta^* = \arg \max_{\theta} \sum_n \mathcal{L}[Q^{(n)}],$$

where $Q^{(n)}$ is the variational distribution associated with data point $\mathbf{v}^{(n)}$.

■ Think and Check

Referring back to the identity (6.4): if, during some training run, the ELBO $\mathcal{L}[Q]$ keeps rising, does that necessarily mean the true log-likelihood $\ln P(\mathbf{v})$ (the quantity we actually want to maximize) is also rising?

Solution: Not necessarily. Equation (6.4) is an **identity**, and $\ln P(\mathbf{v})$ itself does not change with Q (Q is merely an auxiliary variational distribution we introduced); so if the parameters θ are held fixed and only Q is being optimized, a rising ELBO can only mean that D_{KL} is falling (Q is getting closer to the true posterior), while $\ln P(\mathbf{v})$ itself stays unchanged.

If θ is being optimized at the same time, more care is needed: in general, moving from $(Q_{\text{old}}, \theta_{\text{old}})$ to $(Q_{\text{new}}, \theta_{\text{new}})$ such that $\mathcal{L}[Q_{\text{new}}, \theta_{\text{new}}] > \mathcal{L}[Q_{\text{old}}, \theta_{\text{old}}]$ **does not, by itself**, imply $\ln P(\mathbf{v}; \theta_{\text{new}}) \geq \ln P(\mathbf{v}; \theta_{\text{old}})$ — because the gap $D_{KL}(Q \| P(\mathbf{h}|\mathbf{v}; \theta))$ itself also changes together with θ ; the increase in the ELBO might simply have widened the gap, rather than having actually raised $\ln P(\mathbf{v})$.

What genuinely guarantees monotonicity is the specific operating sequence of the **standard EM algorithm**:

1. **E-step:** set Q to the **exact posterior under the current parameters**, $Q_{\text{new}}(\mathbf{h}) = P(\mathbf{h} | \mathbf{v}; \theta_{\text{old}})$ — at this point, the KL gap in Equation (6.4) is exactly zero, i.e., $\mathcal{L}[Q_{\text{new}}, \theta_{\text{old}}] = \ln P(\mathbf{v}; \theta_{\text{old}})$ (the gap is fully closed);
2. **M-step:** holding this Q_{new} fixed, perform an exact maximization of the ELBO with respect to θ : $\theta_{\text{new}} = \arg \max_{\theta} \mathcal{L}[Q_{\text{new}}, \theta]$.

Only then does $\ln P(\mathbf{v}; \theta_{\text{new}}) \geq \mathcal{L}[Q_{\text{new}}, \theta_{\text{new}}] \geq \mathcal{L}[Q_{\text{new}}, \theta_{\text{old}}] = \ln P(\mathbf{v}; \theta_{\text{old}})$ hold (the first step uses Equation (6.4) together with $D_{KL} \geq 0$; the second step uses the maximization property of the M-step), rigorously proving that the evidence never decreases. This is exactly the precise version of the theoretical foundation behind the “learning” step of §6.2: **only when Q is reset to the exact posterior at every step** does a rising ELBO automatically guarantee that the evidence does not decrease; if only an approximate E-step is performed (Q stays within some variational family, rather than being the exact posterior), monotonicity is no longer an automatic mathematical consequence, and must instead be analyzed separately for the specific variational family in question.

6.3 The Physics–Machine-Learning Correspondence Table

Let us explicitly compare statistical physics’s variational free energy with machine learning’s ELBO — this also formally fills in the “free energy” row of Chapter 2’s seed table (Table 2.8),

along with the first half of the “entropy” row (the other half is left for Chapter 7’s maximum entropy principle):

Table 6.2: The Correspondence Between Variational Free Energy and the ELBO — Filling In the “Free Energy” Row of Chapter 2’s Table 2.8

Statistical physics	Machine learning
True distribution $P(\mathbf{x}) = \frac{1}{Z}e^{-\beta E(\mathbf{x})}$	Posterior distribution $P(\mathbf{h} \mathbf{v})$
Trial distribution $Q(\mathbf{x})$	Variational distribution $Q(\mathbf{h})$
Variational free energy $F[Q] = \langle E \rangle_Q - TS[Q]$	Negative ELBO $-\mathcal{L}[Q] = -\langle \ln P(\mathbf{v}, \mathbf{h}) \rangle_Q - H[Q]$
Energy $E(\mathbf{x})$	Negative log-joint-probability $-\ln P(\mathbf{v}, \mathbf{h})$
Entropy $S[Q]$	Entropy $H[Q]$
Minimizing $F[Q]$	Maximizing $\mathcal{L}[Q]$
$F[Q] \geq -T \ln Z$	$\mathcal{L}[Q] \leq \ln P(\mathbf{v})$
KL divergence $D_{KL}(Q\ P) = [F[Q] - F_{\text{true}}]/T$	KL divergence $D_{KL}(Q\ P(\cdot \mathbf{v})) = \ln P(\mathbf{v}) - \mathcal{L}[Q]$

The core insight:

$$\boxed{\text{Minimizing free energy (physics)} \iff \text{Maximizing the ELBO (learning)}}. \quad (6.5)$$

This is not merely a mathematical analogy, but the application of the very same variational principle in two different fields.

6.4 The Mean-Field Approximation

Mean-field theory is an important special case of the variational method, widely used in both statistical physics and machine learning.

The Mean-Field Assumption

Assume the variational distribution factorizes into independent single-variable distributions:

$$Q(\mathbf{x}) = \prod_{i=1}^N Q_i(x_i). \quad (6.6)$$

This assumption ignores correlations between variables, but greatly simplifies the calculation.

Deriving the Mean-Field Equations

The goal is to minimize the variational free energy (or maximize the ELBO), subject to the constraint that every Q_i is normalized:

$$\min_{Q_1, \dots, Q_N} F[Q] \quad \text{s.t.} \quad \sum_{x_i} Q_i(x_i) = 1, \quad \forall i.$$

■ Preliminary Note: This Is Actually an Ordinary Lagrange-Multiplier Problem

At first glance, “taking a variation with respect to the function $Q_i(\cdot)$ ” looks as though it requires the new machinery of the calculus of variations (functional analysis). But as long as x_i takes values in a finite set (e.g., $x_i \in \{1, \dots, K\}$, as is always the case in this book — spins ± 1 , category labels, and so on), Q_i is not really an “abstract function” at all — it is simply a vector of K real numbers, $Q_i(1), \dots, Q_i(K)$, subject to the single constraint that they sum to 1. “Taking a variation with respect to $Q_i(\cdot)$ and setting it to zero” is therefore simply the ordinary calculus problem of **finding an extremum of finitely many real variables subject to a constraint** — exactly the same technique used for constrained-extremum problems such as “maximize the area of a rectangle for a fixed perimeter” in calculus, or the method of Lagrange multipliers used for holonomic constraints in classical mechanics, except that the number of variables has grown from 2 to K . Everything below uses only ordinary partial-derivative notation $\partial/\partial Q_i(x_i)$, with no new concepts introduced.

One more key fact, used throughout, is needed here: since $Q_i(1), \dots, Q_i(K)$ are K **mutually independent** real variables (like K independent coordinate axes in a Cartesian system),

$$\frac{\partial Q_i(x'_i)}{\partial Q_i(k)} = \delta_{x'_i, k} = \begin{cases} 1 & x'_i = k, \\ 0 & x'_i \neq k, \end{cases}$$

that is, the derivative is nonzero only when x'_i happens to equal exactly the value k we are differentiating with respect to. This is the same fact as $\partial y_a / \partial y_b = \delta_{ab}$ for multivariate functions.

Holding all other Q_j ($j \neq i$) fixed, and treating only $Q_i(1), \dots, Q_i(K)$ as the K real variables to be solved for, construct the Lagrangian

$$\Lambda = \mathcal{L}[Q] + \mu_i \left(\sum_{k=1}^K Q_i(k) - 1 \right),$$

where μ_i is the Lagrange multiplier for the normalization constraint, and $\mathcal{L}[Q]$ is the ELBO-form objective, equivalent to the free energy (up to a sign and a constant):

$$\mathcal{L}[Q] = \sum_{\mathbf{x}} Q(\mathbf{x}) \ln P(\mathbf{x}) + \sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x})^{-1} = \sum_{\mathbf{x}} Q(\mathbf{x}) \ln P(\mathbf{x}) - \sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x}),$$

where the second term uses $\ln Q(\mathbf{x})^{-1} = -\ln Q(\mathbf{x})$. And from $Q(\mathbf{x}) = \prod_j Q_j(x_j)$ we have $\ln Q(\mathbf{x}) = \sum_j \ln Q_j(x_j)$ (the logarithm of a product equals the sum of the logarithms), so the second term above is simply the negative entropy of the distribution $Q(\mathbf{x})$ itself, with no need to expand it further into the form $\sum_k \ln Q_k(x_k)$.

We now compute $\partial \Lambda / \partial Q_i(k)$ term by term, for a fixed $k \in \{1, \dots, K\}$, and set it to zero.

The derivative of the first term (the energy term), $\sum_{\mathbf{x}} Q(\mathbf{x}) \ln P(\mathbf{x})$. Using the product rule and $Q(\mathbf{x}) = Q_i(x_i) \prod_{j \neq i} Q_j(x_j)$:

$$\frac{\partial Q(\mathbf{x})}{\partial Q_i(k)} = \frac{\partial Q_i(x_i)}{\partial Q_i(k)} \prod_{j \neq i} Q_j(x_j) = \delta_{x_i, k} \prod_{j \neq i} Q_j(x_j)$$

(the remaining factors $Q_j(x_j)$ ($j \neq i$) do not depend on $Q_i(k)$, and their derivative is zero; the above uses the key fact from the “preliminary note”). So, summing over all $\mathbf{x}$, $\delta_{x_i,k}$ “filters” the sum over x_i down to the single term $x_i = k$, while the remaining variables $\mathbf{x}_{-i}$ are summed as usual:

$$\frac{\partial}{\partial Q_i(k)} \sum_{\mathbf{x}} Q(\mathbf{x}) \ln P(\mathbf{x}) = \sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \ln P(x_i=k, \mathbf{x}_{-i}). \quad (6.7)$$

The derivative of the second term (the entropy term), $-\sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x})$. This term is one layer more complicated than the first: $Q_i(k)$ appears simultaneously in the weight $Q(\mathbf{x})$ and in the logarithm $\ln Q(\mathbf{x})$, so we must expand it into two parts using the product rule:

$$\begin{aligned} \frac{\partial}{\partial Q_i(k)} [Q(\mathbf{x}) \ln Q(\mathbf{x})] &= \underbrace{\frac{\partial Q(\mathbf{x})}{\partial Q_i(k)}}_{=\delta_{x_i,k} \prod_{j \neq i} Q_j(x_j)} \ln Q(\mathbf{x}) + Q(\mathbf{x}) \cdot \underbrace{\frac{\partial \ln Q(\mathbf{x})}{\partial Q_i(k)}}_{=\frac{1}{Q(\mathbf{x})} \frac{\partial Q(\mathbf{x})}{\partial Q_i(k)}} \\ &= \delta_{x_i,k} \left[\prod_{j \neq i} Q_j(x_j) \right] \ln Q(\mathbf{x}) + \delta_{x_i,k} \prod_{j \neq i} Q_j(x_j) \\ &= \delta_{x_i,k} \left[\prod_{j \neq i} Q_j(x_j) \right] [\ln Q(\mathbf{x}) + 1]. \end{aligned}$$

Summing over all $\mathbf{x}$, $\delta_{x_i,k}$ again filters x_i down to k :

$$\frac{\partial}{\partial Q_i(k)} \sum_{\mathbf{x}} Q(\mathbf{x}) \ln Q(\mathbf{x}) = \sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] [\ln Q(x_i=k, \mathbf{x}_{-i}) + 1].$$

Now expand the bracket using $\ln Q(x_i=k, \mathbf{x}_{-i}) = \ln Q_i(k) + \sum_{j \neq i} \ln Q_j(x_j)$ (again, “the logarithm of a product equals the sum of the logarithms”):

$$\begin{aligned} &= \ln Q_i(k) \underbrace{\sum_{\mathbf{x}_{-i}} \prod_{j \neq i} Q_j(x_j)}_{=1} + \underbrace{\sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \sum_{j \neq i} \ln Q_j(x_j)}_{\equiv C \text{ (a constant, independent of } k)}} + \underbrace{\sum_{\mathbf{x}_{-i}} \prod_{j \neq i} Q_j(x_j)}_{=1} \\ &= \ln Q_i(k) + C + 1. \end{aligned}$$

The above uses the fact that each Q_j ($j \neq i$) is separately normalized ($\sum_{x_j} Q_j(x_j) = 1$), so the sum of their product over all of $\mathbf{x}_{-i}$ equals 1¹; and $C = \sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \sum_{j \neq i} \ln Q_j(x_j)$ does not depend on k at all (the sum has already integrated out x_i , and k never appears), making it a pure constant — this is exactly the precise origin behind the earlier shorthand “+const.”

Combine the three terms, and set $\partial \Lambda / \partial Q_i(k) = 0$. Adding the energy term (6.7), the entropy term (with a minus sign), and the constraint term μ_i :

$$\sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \ln P(x_i=k, \mathbf{x}_{-i}) - [\ln Q_i(k) + C + 1] + \mu_i = 0.$$

¹A direct generalization of the fact that a product of several independent normalized distributions sums to 1: for two variables, $\sum_{x_j, x_l} Q_j(x_j) Q_l(x_l) = [\sum_{x_j} Q_j(x_j)] [\sum_{x_l} Q_l(x_l)] = 1 \times 1 = 1$, and likewise for more variables.

Solving for $\ln Q_i(k)$:

$$\ln Q_i(k) = \sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \ln P(x_i=k, \mathbf{x}_{-i}) + \underbrace{(\mu_i - C - 1)}_{\text{a constant, independent of } k}.$$

Exponentiating both sides, the k -independent constant term $e^{\mu_i - C - 1}$ becomes, as a whole, a proportionality constant (whose value is determined afterward by the normalization condition $\sum_k Q_i(k) = 1$, with no need to solve for μ_i separately):

$$Q_i(k) \propto \exp \left(\sum_{\mathbf{x}_{-i}} \left[\prod_{j \neq i} Q_j(x_j) \right] \ln P(x_i=k, \mathbf{x}_{-i}) \right).$$

The sum inside the exponent on the right, in the expectation-value notation given by Chapter 2's "Preliminary Definitions," is exactly the expectation value of $\ln P(\mathbf{x})$ with respect to the joint distribution in which every variable other than x_i is independently distributed according to its own Q_j . Renaming k back to the general notation x_i gives the optimal Q_i^* :

$$\boxed{Q_i^*(x_i) \propto \exp \left(\mathbb{E}_{\prod_{j \neq i} Q_j} [\ln P(\mathbf{x})] \right)}. \quad (6.8)$$

Physical interpretation: each variable x_i feels a "mean field" coming from all the other variables, and $Q_i(x_i)$ is exactly the Boltzmann distribution within this mean field.

Solving Iteratively

The mean-field equations are coupled: Q_i depends on all the Q_j ($j \neq i$), and must be solved by iteration:

1. Initialize every Q_i (for example, to a uniform distribution);
2. Repeat until convergence: for each i , hold the other Q_j ($j \neq i$) fixed, and update Q_i ;
3. Output the final $Q = \prod_i Q_i$.

This process is similar to the coordinate-descent algorithm.

The Mean-Field Solution of the Ising Model

Consider the Ising model in an external magnetic field:

$$E(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i.$$

Under the mean-field approximation, assume $Q(\{s_i\}) = \prod_i Q_i(s_i)$, and define the magnetization of each spin, $m_i = \langle s_i \rangle_{Q_i}$. Let us now substitute the general formula (6.8) into this specific energy function, and derive the explicit form of Q_i^* .

Step 1: write down the part of $\ln P(\mathbf{s})$ that depends on s_i . From $P(\mathbf{s}) = e^{-\beta E(\mathbf{s})}/Z$, $\ln P(\mathbf{s}) = -\beta E(\mathbf{s}) - \ln Z = \beta J \sum_{\langle k,l \rangle} s_k s_l + \beta h \sum_k s_k - \ln Z$. Within the coupling term $\sum_{\langle k,l \rangle} s_k s_l$,

the terms containing s_i are exactly s_i paired once with each of its neighbors $j \in \mathcal{N}(i)$, i.e., $s_i \sum_{j \in \mathcal{N}(i)} s_j$; the part of the external-field term containing s_i is $h s_i$. Grouping every term that does not contain s_i into a constant (unaffected when taking the expectation over s_i), we get

$$\ln P(\mathbf{s}) = \beta s_i \left[J \sum_{j \in \mathcal{N}(i)} s_j + h \right] + (\text{terms not containing } s_i).$$

Step 2: take the expectation over every spin except s_i . By Equation (6.8), we need to compute $\mathbb{E}_{\prod_{j \neq i} Q_j}[\ln P(\mathbf{s})]$, that is, take the expectation of each s_j ($j \neq i$) in the expression above with respect to its own mean-field distribution Q_j . Expectation is a linear operation, so simply replace each s_j directly by its expectation value $\langle s_j \rangle_{Q_j} = m_j$ (the “terms not containing s_i ,” taken as a whole constant, remains, after taking the expectation, a constant unrelated to s_i):

$$\mathbb{E}_{\prod_{j \neq i} Q_j}[\ln P(\mathbf{s})] = \beta s_i \left[J \sum_{j \in \mathcal{N}(i)} m_j + h \right] + \text{const} = \beta s_i h_i^{\text{eff}} + \text{const},$$

where we have defined the **effective field**

$$h_i^{\text{eff}} = h + J \sum_{j \in \mathcal{N}(i)} m_j, \quad (6.9)$$

that is, the “field” actually felt by each spin i is the sum of the external field h and the “averaged” internal field contributed, through the coupling J , by the magnetizations m_j of all its neighbors — exactly the origin of the name “mean field”: the real fluctuations of the neighboring spins are replaced by their respective average values m_j .

Step 3: substitute into Equation (6.8), and recognize the two-level system. Thus

$$Q_i^*(s_i) \propto \exp(\beta s_i h_i^{\text{eff}}), \quad s_i = \pm 1,$$

and this is exactly the form of the “two-level system” of §2.2 (here the two states are $s_i = +1$ and $s_i = -1$, with energy difference given by $-h_i^{\text{eff}}$ and $+h_i^{\text{eff}}$), already handled repeatedly in Chapter 4’s $N = 2$ Ising-chain example — a spin sitting in an “effective external field” h_i^{eff} , giving a Boltzmann distribution. After normalization,

$$Q_i^*(s_i = +1) = \frac{e^{\beta h_i^{\text{eff}}}}{e^{\beta h_i^{\text{eff}}} + e^{-\beta h_i^{\text{eff}}}}, \quad Q_i^*(s_i = -1) = \frac{e^{-\beta h_i^{\text{eff}}}}{e^{\beta h_i^{\text{eff}}} + e^{-\beta h_i^{\text{eff}}}}.$$

By exactly the same calculation used when deriving $\langle s_1 s_2 \rangle_{\text{model}} = \tanh(\beta J)$ in Chapter 4 (simply replacing J with h_i^{eff}), the magnetization is

$$m_i = \langle s_i \rangle_{Q_i^*} = (+1)Q_i^*(+1) + (-1)Q_i^*(-1) = \frac{e^{\beta h_i^{\text{eff}}} - e^{-\beta h_i^{\text{eff}}}}{e^{\beta h_i^{\text{eff}}} + e^{-\beta h_i^{\text{eff}}}} = \tanh(\beta h_i^{\text{eff}}).$$

In a uniform system (where every m_i is the same), write $m_i = m$; then in Equation (6.9), $h^{\text{eff}} = h + zJm$ (where $z = |\mathcal{N}(i)|$ is the number of neighbors per spin; in a uniform system, every neighbor’s m_j equals the same m , so $\sum_{j \in \mathcal{N}(i)} m_j = zm$). Then $m_i = \tanh(\beta h_i^{\text{eff}})$ itself depends on m (through h^{eff}), giving the **self-consistency equation**:

$$m = \tanh(\beta h^{\text{eff}}) = \tanh[\beta(h + zJm)]. \quad (6.10)$$

It is called “self-consistent” because the sought-after m appears on both sides of the equation: on the one hand m determines the effective field h^{eff} , and on the other hand it is also the magnetization the two-level system gives under exactly this effective field — both roles must be satisfied simultaneously.

Phase transition. At $h = 0$, the equation simplifies to $m = \tanh(\beta z J m)$: at high temperature ($\beta z J < 1$) only the trivial solution $m = 0$ exists; at low temperature ($\beta z J > 1$) a nonzero solution $m \neq 0$ appears (spontaneous magnetization). The critical temperature is

$$T_c = \frac{zJ}{k_B}. \quad (6.11)$$

This is the phase-transition point predicted by mean-field theory. Although not exact (it ignores fluctuations), it captures the essence of the phase transition.

Numerically Solving the Self-Consistency Equation

Take the coordination number of a one-dimensional chain, $z = 2$ (each spin has exactly two neighbors, left and right), and $J = 1$; mean-field theory then predicts $T_c = zJ = 2$. The self-consistency equation $m = \tanh(\beta z J m) = \tanh(2m/T)$ can be solved directly, numerically, by **fixed-point iteration**: take any initial guess m_0 , repeatedly substitute $m_{n+1} = \tanh(2m_n/T)$, and continue until the value no longer changes. The results are as follows:

Table 6.3: Numerical Solutions of the One-Dimensional-Chain Mean-Field Self-Consistency Equation ($z = 2$, $J = 1$)

T	$\beta z J = 2/T$	m (converged value)
3.0	0.667	0.000
2.5	0.800	0.000
1.5	1.333	0.776
1.0	2.000	0.958
0.5	4.000	0.999

The numerical results agree exactly with the criterion above: for $T > T_c = 2$, no matter what the initial guess is, the iteration always converges back to $m = 0$ (the paramagnetic phase, no spontaneous magnetization); for $T < T_c$, the iteration converges to a nonzero m , and the lower T is, the closer the magnetization gets to the saturation value 1 (the ferromagnetic phase, spins almost entirely aligned).

Why Mean-Field Theory Fails in One Dimension

There is a point here that is well worth examining closely. Mean-field theory gives a **clear finite-temperature phase transition** for the one-dimensional chain considered here ($z = 2$): $T_c = 2$. But §2.3 already **exactly solved** the one-dimensional Ising chain using the transfer matrix, and the conclusion there was that the correlation length $\xi(T)$ (Equation (2.25)) is finite

at **any** finite temperature, diverging only as $T \rightarrow 0$ — in other words, **the one-dimensional Ising chain has no genuine phase transition at any finite temperature!** Mean-field theory’s prediction here is **qualitatively inconsistent** with the exact solution.

Where does the problem lie? Recall the core assumption of the mean-field approximation, $Q(\{s_i\}) = \prod_i Q_i(s_i)$: it treats each spin as an **independent** variable, replacing “the true fluctuations of the surrounding spins” with the numerical value “the average magnetization of the surrounding spins,” which, in essence, erases the destructive effect fluctuations have on the system. In higher dimensions (for example, a three-dimensional lattice, $z = 6$), each spin has more neighbors, and the destructive effect of fluctuations is relatively weakened by being “shared out among multiple neighbors,” so mean-field theory in higher dimensions typically gives qualitatively correct, if quantitatively crude, predictions; but in one dimension ($z = 2$, the fewest possible neighbors), fluctuation effects are strong enough to **destroy long-range order at any finite temperature** — and this physical effect is exactly what the mean-field approximation completely ignores.

■ Summary

This is a lesson worth remembering whenever using any approximation method: the variational method guarantees the strict validity of the **inequality** $F[Q] \geq F_{\text{true}}$, but does not guarantee that the approximate solution gets the **qualitative behavior** right (such as “does a phase transition exist”) — this depends on how well the chosen variational family (here, the factorization assumption that “the variables are mutually independent”) captures the key physical mechanism of the real system. Before using any approximation method, it is equally important to understand under which circumstances it fails, and under which circumstances it works.

Exercises

Foundational

Exercise 6.1: Non-Negativity of the KL Divergence

Prove that, for any two probability distributions P and Q ,

$$D_{KL}(P\|Q) = \sum_x P(x) \ln \frac{P(x)}{Q(x)} \geq 0,$$

with equality if and only if $P = Q$.

Hint: Use Jensen’s inequality: for a convex function f , $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$; note that $-\ln$ is convex.

Intermediate

Exercise 6.2: The EM Algorithm for Variational Inference

For a model with latent variables, $P(\mathbf{v}, \mathbf{h}; \boldsymbol{\theta})$:

- Write down the expression for the ELBO, $\mathcal{L}[Q, \boldsymbol{\theta}]$;
- Prove that $\ln P(\mathbf{v}; \boldsymbol{\theta}) = \mathcal{L}[Q, \boldsymbol{\theta}] + D_{KL}(Q \| P(\mathbf{h}|\mathbf{v}; \boldsymbol{\theta}))$;
- E-step:** holding $\boldsymbol{\theta}$ fixed, maximize $\mathcal{L}[Q, \boldsymbol{\theta}]$ with respect to Q , and prove that the optimum is $Q^* = P(\mathbf{h}|\mathbf{v}; \boldsymbol{\theta})$;
- M-step:** holding Q^* fixed, maximize $\mathcal{L}[Q^*, \boldsymbol{\theta}]$ with respect to $\boldsymbol{\theta}$, and write down the objective function;
- Prove that each iteration of the EM algorithm increases (or leaves unchanged) the log-likelihood $\ln P(\mathbf{v}; \boldsymbol{\theta})$.

Exercise 6.3: The Mean-Field Ising Model

Consider the Ising model in an external magnetic field:

$$E(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i,$$

on a d -dimensional cubic lattice, where each spin has $2d$ neighbors.

- Write down the self-consistency equation under the mean-field approximation;
- In the case $h = 0$, find the critical temperature T_c of the phase transition;
- For $d = 1$ (a one-dimensional chain), what is the T_c predicted by mean-field theory? Compare it with the exact result ($T_c = 0$);
- For $d = 2, 3$, what is the T_c predicted by mean-field theory?
- Discuss the limitations of the mean-field approximation: why does it overestimate T_c ?

Challenge

Exercise 6.4: The Information Bottleneck Theory

The information-bottleneck objective function is

$$\max_{P(t|x)} [I(T; Y) - \beta I(T; X)],$$

where T is the representation, X the input, and Y the output.

- Write down the definitions of the mutual informations $I(T; Y)$ and $I(T; X)$;

- (b) Using the calculus of variations, prove that the optimal distribution $P^*(t|x)$ satisfies

$$P^*(t|x) = \frac{P(t)}{Z(x, \beta)} \exp\left(\beta \mathbb{E}_{P(y|x)}[\ln P(y|t)]\right),$$

where $Z(x, \beta)$ is a normalization constant;

- (c) Prove that this leads to the self-consistency equations

$$P(t) = \sum_x P(t|x)P(x), \quad P(y|t) = \frac{1}{P(t)} \sum_x P(y|x)P(t|x)P(x);$$

- (d) Discuss the limiting cases $\beta \rightarrow 0$ and $\beta \rightarrow \infty$;
- (e) Compare the information bottleneck with autoencoders: how does the VAE loss function relate to the information-bottleneck objective?

Exercise 6.5: Phase Transitions and Learning

Consider a simplified learning scenario: a two-layer neural network learning a binary classification task.

- (a) Modeling: suppose the training data come from a “teacher” network, which a “student” network attempts to learn. Treat the student’s generalization error ϵ_g as an “order parameter”;
- (b) How does ϵ_g vary with the number of training samples N (analogous to temperature)? Is there a critical sample size N_c at which the student network jumps abruptly from “unable to generalize” to “successfully generalizing”?
- (c) Results from the literature: for certain simple models, the generalization error does indeed exhibit phase-transition behavior; discuss the physical mechanism behind this kind of phase transition;
- (d) In real deep learning, how can the “double descent” phenomenon (as model complexity increases, performance first overfits and then generalizes better again) be understood in the language of phase transitions?
- (e) Reflect: can the “emergent abilities” of deep learning (emergence — for example, large language models suddenly acquiring few-shot learning ability) be viewed as a kind of phase transition?

Chapter 7

The Maximum Entropy Principle

*I love your Majesty
According to my bond, no more nor less.*

— Shakespeare, *King Lear*, Act I, Scene
1, spoken by Cordelia

Objectives

- Master the complete derivation of the Boltzmann distribution from the maximum entropy principle via the method of Lagrange multipliers, and understand that this is an entirely independent derivation path from physical intuition;
- Through two concrete worked examples — a die and a two-level system — verify that the two utterly different logical routes, “given constraints, maximize the entropy” and “the system exchanges energy with a heat bath,” arrive at exactly the same destination;
- Understand the equivalence between the maximum entropy model (the log-linear model) and softmax regression;
- Master the principle of minimum cross-entropy as a generalization of the maximum entropy principle, and understand its consistency with the logic, from MAP estimation, that “the prior pulls the estimate toward itself.”

■ Chapter Guide

The question this chapter answers: The previous chapters have already, from more than one mutually independent direction, repeatedly arrived at the very same Boltzmann distribution. Is this merely a coincidence, or is there a deeper reason behind it? Does there exist a derivation path that depends on no physical intuition whatsoever, proceeding purely from information theory?

Correspondence with statistical mechanics: The maximum entropy principle shows that the Boltzmann distribution is not a physical assumption at all, but the inevitable con-

sequence of the information-theoretic axiom “among all distributions satisfying the known constraints, choose the one that makes the fewest additional assumptions”; the “statistical-mechanics/machine-learning correspondence table” at the end of this chapter is a compressed summary of the argument chain running through all six chapters of the book, not a table meant to be memorized by rote.

Looking ahead: Part I ends here. Part II of these notes, “Physical Models of Neural Networks,” will apply the entire mathematical language built up here — energy, the Boltzmann distribution, the partition function, variational methods — to concrete neural-network models such as Hopfield networks and Boltzmann machines; Part III, “Optimization and Dynamics,” will return to the question raised back in Chapter 0 but left unanswered ever since: exactly how should the business of “repeatedly adjusting the parameters” during training be carried out systematically?

By this point in the book, the Boltzmann distribution has already appeared repeatedly, arrived at through **two mutually independent derivations**: §2.2 derived it from the physical picture of “the system exchanging energy with a heat bath”; §6.1 derived it again from an optimization point of view, through an entropy-regularized variational objective function. (The observation in §4.1, that “the loss function is the energy function,” is of a different character: it holds identically for *any* positive probability distribution — simply take $E(\mathbf{x}) \equiv -\ln P(\mathbf{x})$, and $P(\mathbf{x}) = e^{-E(\mathbf{x})}$ follows automatically. This is a notational identity, not a conclusion requiring proof in a physical or information-theoretic sense. But it is precisely this identity that lets the conclusions of the two derivation paths above be carried over, unchanged, to any machine-learning probabilistic model, rather than staying confined to purely physical systems — it is a bridge, not a third independent source.)

What this chapter sets out to demonstrate is a **third, independent derivation path**, depending on no physical intuition whatsoever, proceeding purely from information theory: a single information-theoretic principle — “among all distributions satisfying the known constraints, choose the one that makes the fewest additional assumptions” — is enough to **derive** the Boltzmann distribution.

Three mutually independent roads converging on the same destination is not a coincidence; it is telling us that **once we accept “the maximum entropy principle” as an axiom of inference under incomplete information**, the Boltzmann distribution may not be a physical assumption at all, but the inevitable consequence of this inferential axiom acting together with the average-energy constraint — though this inevitability itself depends on accepting the maximum entropy principle as a starting point in the first place. It is a methodological axiom about “how to make inferences when information is insufficient” (Jaynes’s view), not a theorem that can be independently proved from within probability theory itself; whether this axiom should be accepted as the “uniquely reasonable” principle of inference remains debated to this day in statistics and philosophy, and this book takes no position on that question, presenting only the mathematical structure of this derivation path itself.

7.1 The Maximum Entropy Principle in Statistical Mechanics

■ History: Entropy, a Borrowed Name

In 1948, Claude Shannon published “A Mathematical Theory of Communication” at Bell Labs, and, in order to measure just how much “uncertainty” a message contains, defined $H = -\sum_i p_i \log p_i$ (Equation (2.15) is exactly this expression, in its physical guise). According to a widely circulated anecdote — never formally confirmed by Shannon himself — he originally wanted to call this quantity “uncertainty,” and it was von Neumann who suggested he call it “entropy” instead, on the grounds that the expression has exactly the same form as the entropy that already existed in statistical mechanics (precisely Equation (2.15) of Chapter 1 of this book, traceable back to Boltzmann’s H -function and to Gibbs’s expression given earlier in this chapter) — “since nobody really understands what entropy is anyway, you’ll always have the advantage in an argument.” Whether or not the anecdote is true, the mathematical fact is certain: thermodynamic entropy and information entropy are the very same function, independently discovered in two fields nearly a century apart. E. T. Jaynes was himself a physicist, and it was precisely this “borrowing” that prompted him to ask the question in the other direction: if two fields are using the very same mathematical object, could the entire framework of statistical mechanics originally be derivable from purely information-theoretic principles, with no need to presuppose any physical picture of atoms or molecules at all? What “Jaynes’s Insight,” below, sets out to answer is exactly this question.

Jaynes’s Insight

In 1957, E. T. Jaynes proposed a profound idea: the foundation of statistical mechanics is not microscopic dynamics, but **information theory**.

The core question: Given some macroscopic constraints (such as the average energy), which distribution is most “reasonable,” or least “biased”?

The answer: Among all distributions satisfying the constraints, the one with **maximum entropy** is the most reasonable.

The intuitive reasoning:

- Entropy measures uncertainty, or “missing information”;
- Choosing the maximum-entropy distribution means introducing no additional assumptions beyond the given constraints;
- This is a kind of “Occam’s razor”: introduce no structure that has not been observed.

Mathematical Formulation

Problem setup: Given m constraints:

$$\langle f_k(\mathbf{x}) \rangle = \bar{f}_k, \quad k = 1, 2, \dots, m, \quad (7.1)$$

and the normalization condition:

$$\sum_{\mathbf{x}} P(\mathbf{x}) = 1, \quad (7.2)$$

find the distribution $P(\mathbf{x})$ that maximizes the entropy:

$$\max_P S = - \sum_{\mathbf{x}} P(\mathbf{x}) \ln P(\mathbf{x}). \quad (7.3)$$

Solving via the Method of Lagrange Multipliers

Construct the Lagrangian:

$$\mathcal{F} = - \sum_{\mathbf{x}} P(\mathbf{x}) \ln P(\mathbf{x}) + \lambda_0 \left(1 - \sum_{\mathbf{x}} P(\mathbf{x}) \right) + \sum_{k=1}^m \lambda_k \left(\bar{f}_k - \sum_{\mathbf{x}} P(\mathbf{x}) f_k(\mathbf{x}) \right).$$

Exactly as when Chapter 5 derived the mean-field equations (§6.4’s “preliminary note”), as long as $\mathbf{x}$ takes finitely many discrete values, $P(\mathbf{x})$ is not an abstract “function,” but a collection of independent real numbers $\{P(\mathbf{x})\}_{\mathbf{x}}$ (one real number per state). “Taking a variation with respect to $P(\mathbf{x})$ ” is therefore, for each specific state $\mathbf{x}'$, simply taking the ordinary partial derivative $\partial \mathcal{F} / \partial P(\mathbf{x}')$ with $P(\mathbf{x}')$ treated as an ordinary real variable, with every other $P(\mathbf{x})$ ($\mathbf{x} \neq \mathbf{x}'$) treated as an independent variable unrelated to it — this is exactly the ordinary method of Lagrange multipliers, requiring no calculus of variations. Computing term by term:

- First term: $\frac{\partial}{\partial P(\mathbf{x}')} [-P(\mathbf{x}') \ln P(\mathbf{x}')] = -\ln P(\mathbf{x}') - P(\mathbf{x}') \cdot \frac{1}{P(\mathbf{x}')} = -\ln P(\mathbf{x}') - 1$ (the product rule; only the term $\mathbf{x} = \mathbf{x}'$ in the sum depends on $P(\mathbf{x}')$);
- Second term: $\frac{\partial}{\partial P(\mathbf{x}')} [\lambda_0 (1 - \sum_{\mathbf{x}} P(\mathbf{x}))] = -\lambda_0$;
- Third term: $\frac{\partial}{\partial P(\mathbf{x}')} \left[\sum_k \lambda_k (\bar{f}_k - \sum_{\mathbf{x}} P(\mathbf{x}) f_k(\mathbf{x})) \right] = -\sum_k \lambda_k f_k(\mathbf{x}')$.

Adding the three terms and setting the result to zero (renaming $\mathbf{x}'$ back to $\mathbf{x}$):

$$\frac{\partial \mathcal{F}}{\partial P(\mathbf{x})} = -\ln P(\mathbf{x}) - 1 - \lambda_0 - \sum_{k=1}^m \lambda_k f_k(\mathbf{x}) = 0.$$

Solving:

$$P(\mathbf{x}) = \exp \left(-1 - \lambda_0 - \sum_{k=1}^m \lambda_k f_k(\mathbf{x}) \right).$$

Using the normalization condition $\sum_{\mathbf{x}} P(\mathbf{x}) = 1$ to determine λ_0 : pulling the $\mathbf{x}$ -independent factor $e^{-1-\lambda_0}$ out of the sum,

$$1 = \sum_{\mathbf{x}} P(\mathbf{x}) = \sum_{\mathbf{x}} e^{-1-\lambda_0} \exp \left(- \sum_{k=1}^m \lambda_k f_k(\mathbf{x}) \right) = e^{-1-\lambda_0} \sum_{\mathbf{x}} \exp \left(- \sum_{k=1}^m \lambda_k f_k(\mathbf{x}) \right),$$

multiplying both sides by $e^{1+\lambda_0}$:

$$e^{1+\lambda_0} = \sum_{\mathbf{x}} \exp \left(- \sum_{k=1}^m \lambda_k f_k(\mathbf{x}) \right) \equiv Z.$$

Substituting $e^{-1-\lambda_0} = 1/Z$ back into $P(\mathbf{x}) = e^{-1-\lambda_0} \exp(-\sum_k \lambda_k f_k(\mathbf{x}))$, we therefore get:

$$\boxed{P(\mathbf{x}) = \frac{1}{Z} \exp\left(-\sum_{k=1}^m \lambda_k f_k(\mathbf{x})\right)}, \quad (7.4)$$

where:

$$Z = \sum_{\mathbf{x}} \exp\left(-\sum_{k=1}^m \lambda_k f_k(\mathbf{x})\right).$$

Defining the energy function:

$$E(\mathbf{x}) = \sum_{k=1}^m \lambda_k f_k(\mathbf{x}),$$

we obtain

$$\boxed{P(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})}}. \quad (7.5)$$

This is exactly the Boltzmann distribution!

A Special Case: The Canonical Ensemble

If the only constraint is the average energy:

$$\langle E(\mathbf{x}) \rangle = \bar{E},$$

then the maximum-entropy distribution is

$$P(\mathbf{x}) = \frac{1}{Z} e^{-\beta E(\mathbf{x})},$$

where β is a Lagrange multiplier, determined through the constraint condition:

$$\bar{E} = -\frac{\partial \ln Z}{\partial \beta}.$$

In thermodynamics, $\beta = 1/(k_B T)$ is related to the temperature; but from an information-theoretic point of view, β is simply the Lagrange multiplier enforcing the average-energy constraint.

Philosophical significance: the maximum entropy principle shows that the Boltzmann distribution is not a fundamental assumption at all, but is derived from information-theoretic principles — it is the most “unbiased” distribution consistent with the given macroscopic constraints.

A Biased Die (Jaynes’s Classic Example)

The derivation above is fairly abstract; let us make it completely concrete using the example from Jaynes’s own 1957 paper: for a six-sided die, the probability $P(i)$ ($i = 1, \dots, 6$) of each face landing up is unknown, and all we know is a single constraint, the **average number of points** $\langle X \rangle$; find the maximum-entropy distribution.

Table 7.1: The Maximum-Entropy Distribution of the Die Under Different Constraints $\langle X \rangle$

Constraint $\langle X \rangle$	λ	$P(1)$	$P(2)$	$P(3)$	$P(4)$	$P(5)$	$P(6)$
3.5 (the mean of a fair die)	0.000	0.167	0.167	0.167	0.167	0.167	0.167
4.5	-0.371	0.054	0.079	0.114	0.165	0.240	0.348
5.0	-0.630	0.021	0.038	0.072	0.136	0.255	0.478

By the general result above, $P(i) \propto e^{-\lambda i}$, where λ is determined by the constraint $\sum_i i P(i) = \langle X \rangle$. Taking three different constraint values, and solving numerically for λ (the solution process is a standard one-dimensional root-finding problem, omitted here; only the results are listed):

The first row of Table 7.1 is a good self-check: if the constraint is exactly the mean, 3.5, that a fair die already has, the maximum entropy principle gives $\lambda = 0$, i.e., the **uniform distribution** — entirely reasonable: if nothing is known beyond “the average number of points is 3.5,” and this happens to be exactly the mean that the uniform distribution itself satisfies, there is no reason to think any one face more likely to land up than another, and the maximum entropy principle gives precisely this “no additional assumptions” answer. The second and third rows show how a constraint “distorts” the distribution: once the average is required to be noticeably higher than 3.5 (such as 4.5 or 5.0), the maximum-entropy distribution automatically tilts toward the higher numbers — and the probability is not raised uniformly, but grows **exponentially** with the face value (the probability of a 6 is roughly 6 to 23 times higher than that of a 1), which is exactly the direct manifestation of the exponential form $P(i) \propto e^{-\lambda i}$ that the method of Lagrange multipliers produced. This die is, in fact, a miniature Boltzmann distribution with “energy” $E(i) = i$ and a “temperature” implicitly determined by the constraint.

Reverse-Engineering the “Temperature” of the Two-Level System, Verifying That Independent Paths Converge

Let us now perform a check that captures this section’s central argument more directly. Recall the two-level system of §2.2 ($E_0 = 0, E_1 = \varepsilon = 1$): **starting from the physical derivation**, assuming it is in contact with a heat bath at $k_B T = 1$ ($\beta = 1$), we computed the exact average energy $\langle E \rangle = 0.269$.

Now let us go **in reverse**: pretend we know nothing at all about “temperature” or “heat bath” as physical concepts, and know only a single isolated constraint — “the average energy of this system is 0.269” — and apply the maximum-entropy formula $P(x) \propto e^{-\lambda E(x)}$ just derived in this section, solving numerically for the λ that satisfies the constraint:

Numerically solving gives: $\lambda = 1.000000$.

This number equals *exactly* the $\beta = 1/(k_B T) = 1$ assumed in §2.2! Two entirely different derivation paths — one starting from the physical picture of “the system exchanging energy with a heat bath,” the other proceeding purely from the information-theoretic principle of “given constraints, maximize the entropy” — converge numerically, down to the last digit, on exactly the same distribution. This is precisely what Jaynes wanted to convey: **temperature, the**

partition function, the Boltzmann distribution are perhaps not so much physical properties that a thermodynamic system “must possess,” as they are the optimal solution to the more general logical problem of “making inferences given a certain amount of information”; statistical mechanics is simply the application of this logical problem to the particular case in which the constraint is the average energy.

■ Think and Check

If the die’s constraint were changed to “at least two different moment constraints,” say specifying both $\langle X \rangle$ and $\langle X^2 \rangle$ simultaneously, can we infer the form of the maximum-entropy distribution from the general derivation of §7.1 (m constraints correspond to m Lagrange multipliers)?

Solution: $P(i) \propto \exp(-\lambda_1 i - \lambda_2 i^2)$ — both a linear term and a quadratic term appear in the exponent simultaneously. This form should look familiar: it is exactly the (discrete version of the) form of the Gaussian distribution — for a continuous variable, constraining only the mean and the variance, the maximum-entropy distribution is precisely the normal distribution, which is also the reason the Gaussian distribution is so “special” in statistics: it is the unique unbiased choice under the information state “only the mean and the variance are known.”

7.2 Applications of Maximum Entropy in Machine Learning

Maximum Entropy Models (MaxEnt Models)

In machine learning, the maximum entropy principle can be used to construct a classifier.

Problem setup: given training data $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}$, learn the conditional probability distribution $P(y|\mathbf{x})$.

Constraint condition: the model’s expected features match the data’s empirical features:

$$\sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}) P(y|\mathbf{x}) f_k(\mathbf{x}, y) = \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) f_k(\mathbf{x}, y), \quad (7.6)$$

where $\tilde{P}(\mathbf{x}, y)$ is the empirical distribution of the training data, and $f_k(\mathbf{x}, y)$ is a feature function.

Objective: among all distributions satisfying the constraint, choose the one with maximum conditional entropy:

$$\max_{P(y|\mathbf{x})} H(Y|X) = - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}) P(y|\mathbf{x}) \ln P(y|\mathbf{x}). \quad (7.7)$$

Solution: this problem has a structure similar to the maximum-entropy problem solved in §7.1, but with two differences, requiring us to work through the method of Lagrange multipliers once again: first, what needs to be normalized here is the *conditional* distribution, requiring $\sum_y P(y|\mathbf{x}) = 1$ separately for every $\mathbf{x}$, so each $\mathbf{x}$ needs its own Lagrange multiplier $\nu(\mathbf{x})$; second, every term in the objective (7.7) and the constraint (7.6) carries the empirical marginal

distribution $\tilde{P}(\mathbf{x})$ as a weight. Construct the Lagrangian:

$$\begin{aligned} \mathcal{F} = & - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}) P(y|\mathbf{x}) \ln P(y|\mathbf{x}) + \sum_{\mathbf{x}} \nu(\mathbf{x}) \left(1 - \sum_y P(y|\mathbf{x}) \right) \\ & + \sum_k \lambda_k \left(\sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}) P(y|\mathbf{x}) f_k(\mathbf{x}, y) - \sum_{\mathbf{x}, y} \tilde{P}(\mathbf{x}, y) f_k(\mathbf{x}, y) \right). \end{aligned}$$

By exactly the same reasoning as in §7.1: for a fixed pair $(\mathbf{x}, y) = (\mathbf{x}', y')$, treat $P(y|\mathbf{x})$ as an independent real variable and take the partial derivative. Computing term by term:

- First term: only the term $(\mathbf{x}, y) = (\mathbf{x}', y')$ depends on $P(y'|\mathbf{x}')$; the product rule gives $-\tilde{P}(\mathbf{x}') [\ln P(y'|\mathbf{x}') + 1]$;
- Second term: only the term $\mathbf{x} = \mathbf{x}'$ depends on it, giving $-\nu(\mathbf{x}')$;
- Third term: gives $\tilde{P}(\mathbf{x}') \sum_k \lambda_k f_k(\mathbf{x}', y')$ (the sign convention here is taken opposite to that of §7.1, so that λ_k carries a positive sign in the final result, matching the convention standard in the machine-learning literature for log-linear models; λ_k itself remains a free parameter to be determined, and the sign convention does not affect the physical content).

Adding the three terms and setting the result to zero:

$$-\tilde{P}(\mathbf{x}') [\ln P(y'|\mathbf{x}') + 1] - \nu(\mathbf{x}') + \tilde{P}(\mathbf{x}') \sum_k \lambda_k f_k(\mathbf{x}', y') = 0.$$

Dividing both sides by $-\tilde{P}(\mathbf{x}')$ (assuming $\tilde{P}(\mathbf{x}') > 0$, i.e., this input genuinely occurred in the training set):

$$\ln P(y'|\mathbf{x}') = -1 - \frac{\nu(\mathbf{x}')}{\tilde{P}(\mathbf{x}')} + \sum_k \lambda_k f_k(\mathbf{x}', y').$$

The sum of the first two terms on the right, $-1 - \nu(\mathbf{x}')/\tilde{P}(\mathbf{x}')$, does not depend on y' (for fixed $\mathbf{x}'$, it is simply some undetermined constant), so after exponentiating, $P(y'|\mathbf{x}')$, as a function of y' , is proportional to $\exp(\sum_k \lambda_k f_k(\mathbf{x}', y'))$, with the proportionality constant determined by this constant and ultimately fixed by the normalization condition $\sum_{y'} P(y'|\mathbf{x}') = 1$ (renaming $\mathbf{x}', y'$ back to $\mathbf{x}, y$):

$$\boxed{P(y|\mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp\left(\sum_k \lambda_k f_k(\mathbf{x}, y)\right)}, \quad (7.8)$$

where

$$Z(\mathbf{x}) = \sum_{y'} \exp\left(\sum_k \lambda_k f_k(\mathbf{x}, y')\right).$$

This is exactly the **log-linear model**!

Example: Text Classification

Suppose we predict the category y (such as “sports,” “politics,” etc.) of a document $\mathbf{x}$ (represented as a word-frequency vector). Define the feature function:

$$f_{w,c}(\mathbf{x}, y) = \begin{cases} x_w & \text{if } y = c, \\ 0 & \text{otherwise,} \end{cases}$$

where x_w is the number of occurrences of word w in the document, and c is a category. The maximum entropy model is

$$P(y = c|\mathbf{x}) = \frac{\exp(\sum_w \lambda_{w,c} x_w)}{\sum_{c'} \exp(\sum_w \lambda_{w,c'} x_w)} = \frac{\exp(\boldsymbol{\lambda}_c^T \mathbf{x})}{\sum_{c'} \exp(\boldsymbol{\lambda}_{c'}^T \mathbf{x})}.$$

This is exactly softmax regression!

Conclusion: softmax regression can be viewed as the application of the maximum entropy principle to classification problems.

7.3 The Principle of Minimum Cross-Entropy

The maximum entropy principle has an important generalization: the **principle of minimum relative entropy** (also called the principle of minimum KL divergence).

Problem: given a prior distribution $Q(\mathbf{x})$ and a set of constraints, find the distribution P that minimizes $D_{KL}(P\|Q)$.

■ A Notational Reminder: The Roles of P, Q Are Reversed from Chapter 5

When Chapter 5 discussed variational free energy, P always denoted the “true but computationally intractable target distribution,” and Q the trial distribution searched for within a variational family, used to *approximate* P . This section follows the notational convention standard to the maximum entropy principle, in which P is the *answer being solved for* (i.e., the distribution this section ultimately solves for), and Q is instead the *given, fixed* prior — the roles of P, Q in the two chapters are exactly swapped. This is not a typographical inconsistency, but the result of two independently formed notational traditions (variational inference vs. maximum entropy/minimum cross-entropy) both being preserved, as-is, within this book; when reading, please go by “which distribution is the unknown being solved for,” rather than relying on the letter itself.

Motivation: when we hold prior information, we should not ignore it entirely. Minimizing the KL divergence means staying as close as possible to the prior while still satisfying the new constraints.

Solution: again using the method of Lagrange multipliers, except that this time we minimize $D_{KL}(P\|Q) = \sum_{\mathbf{x}} P(\mathbf{x}) \ln \frac{P(\mathbf{x})}{Q(\mathbf{x})}$ (Q is a given prior distribution, not participating in the optimization). Construct the Lagrangian:

$$\mathcal{F} = \sum_{\mathbf{x}} P(\mathbf{x}) \ln \frac{P(\mathbf{x})}{Q(\mathbf{x})} + \lambda_0 \left(1 - \sum_{\mathbf{x}} P(\mathbf{x})\right) + \sum_k \lambda_k \left(\sum_{\mathbf{x}} P(\mathbf{x}) f_k(\mathbf{x}) - \bar{f}_k\right).$$

Exactly as in §7.1: for fixed $\mathbf{x}'$, treat $P(\mathbf{x}')$ as an independent real variable and differentiate. The first term, by the product rule, gives $\ln \frac{P(\mathbf{x}')}{Q(\mathbf{x}')} + P(\mathbf{x}') \cdot \frac{1}{P(\mathbf{x}')} = \ln \frac{P(\mathbf{x}')}{Q(\mathbf{x}')} + 1$ ($Q(\mathbf{x}')$ is a fixed prior, treated as a constant, and produces no additional term when differentiating with respect

to $P(\mathbf{x}')$; the second term gives $-\lambda_0$; the third term gives $\sum_k \lambda_k f_k(\mathbf{x}')$. Adding the three terms and setting the result to zero:

$$\ln \frac{P(\mathbf{x}')}{Q(\mathbf{x}')} + 1 - \lambda_0 + \sum_k \lambda_k f_k(\mathbf{x}') = 0 \implies \ln \frac{P(\mathbf{x}')}{Q(\mathbf{x}')} = (\lambda_0 - 1) - \sum_k \lambda_k f_k(\mathbf{x}').$$

Exponentiating both sides — $(\lambda_0 - 1)$ does not depend on $\mathbf{x}'$, and becomes a proportionality constant after exponentiating — and renaming $\mathbf{x}'$ back to $\mathbf{x}$:

$$P(\mathbf{x}) = Q(\mathbf{x}) \cdot e^{\lambda_0 - 1} \cdot \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right) \propto Q(\mathbf{x}) \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right).$$

Writing the proportionality constant as $1/Z$ (determined by the normalization $\sum_{\mathbf{x}} P(\mathbf{x}) = 1$, exactly as was done for Equation (7.4)), we obtain the solution of minimum KL divergence, again in exponential-family form:

$$P(\mathbf{x}) = \frac{1}{Z} Q(\mathbf{x}) \exp\left(-\sum_k \lambda_k f_k(\mathbf{x})\right). \quad (7.9)$$

Compared with the maximum-entropy solution (7.4) of §7.1, the only difference is the extra prior factor $Q(\mathbf{x})$ — exactly corresponding to the fact, noted in the derivation, that “the first term treats $Q(\mathbf{x}')$ as a constant”: the role of the prior is simply to multiply, on top of the purely exponential-family weight, an additional “background tint” that does not change with the constraint.

A special case: if the prior is the uniform distribution, $Q(\mathbf{x}) = \text{const}$, we recover the maximum-entropy distribution.

■ Think and Check

Combine this with the earlier die example: suppose we hold one further piece of prior knowledge about the die — say, it is an old, hand-carved die, not especially well made, and we believe a priori that “the 6 face, having more pips drilled into it and therefore being lighter, is naturally slightly more likely to land up” (i.e., the prior $Q(6)$ is slightly larger than for the other faces) — under the same constraint “average number of points is 4.5,” does the minimum-cross-entropy solution give a larger or smaller $P(6)$ compared with assuming a uniform prior (i.e., the pure maximum-entropy solution)?

Solution: Larger. Minimizing $D_{KL}(P\|Q)$ means staying “as close to the prior as possible” while still satisfying the constraint; if the prior is already tilted toward 6, the final solution will superimpose the “tilt contributed by the prior” on top of the “tilt required by the constraint,” making $P(6)$ even more pronounced than in the pure maximum-entropy solution. This is entirely consistent with the logic of §3.4, that “the prior pulls the estimate toward itself” — the principle of minimum cross-entropy can be viewed as generalizing the “prior + likelihood” logic of MAP, from “estimating a parameter” to “choosing an entire distribution.”

Applications in Machine Learning

The information-theoretic interpretation of regularization: in maximum a posteriori estimation (MAP), we have

$$\max_{\boldsymbol{\theta}} [P(\mathcal{D}|\boldsymbol{\theta})P(\boldsymbol{\theta})],$$

which is equivalent to

$$\min_{\boldsymbol{\theta}} \left[\underbrace{-\ln P(\mathcal{D}|\boldsymbol{\theta})}_{\text{data-fit term}} \underbrace{-\ln P(\boldsymbol{\theta})}_{\text{regularization term}} \right]. \quad (7.10)$$

Here, the **regularization term is simply the negative log-prior**, $-\ln P(\boldsymbol{\theta})$, itself; there is no need — and it would not be appropriate — to rewrite it using the KL divergence: $\boldsymbol{\theta}$ is a single *point* in parameter space, whereas the definition of D_{KL} requires both of its arguments to be probability *distributions*, so an expression like $D_{KL}(\boldsymbol{\theta} \| P_{\text{prior}})$ has no literally well-defined mathematical meaning, and even if accompanied by a disclaimer that it is merely “heuristic notation,” it is easily taken at face value in practice — so this book does not use this notation. Equation (7.10) is already the final, precise form; for example, for a Gaussian prior $P(\boldsymbol{\theta}) \propto e^{-\lambda \|\boldsymbol{\theta}\|^2/2}$, substituting directly gives $-\ln P(\boldsymbol{\theta}) = \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 + \text{const}$, i.e., L^2 regularization (ridge regression) — exactly the continuous-parameter version of the “virtual count” example in §3.4, with no need to pass through the KL divergence at all.

An analogy with the first term (also requiring care about its scope of validity): if $-\ln P(\mathcal{D}|\boldsymbol{\theta})$ is taken in expectation over the entire data distribution (rather than summed over a finite training set), i.e., $\mathbb{E}_{\mathbf{x}, y \sim P_{\text{data}}} [-\ln P_{\boldsymbol{\theta}}(y|\mathbf{x})]$, this **population-level** cross-entropy is strictly equal to $H(P_{\text{data}}) + D_{KL}(P_{\text{data}} \| P_{\boldsymbol{\theta}})$ ($H(P_{\text{data}})$ does not depend on $\boldsymbol{\theta}$, and can be absorbed into a constant); what is actually minimized during training is the **empirical** average over a finite dataset, which is an estimate (dependent on the sample size) of this population-level quantity, and the two are strictly equal only in the limit of infinite data.

The physical analogy (valid under the precise meaning above):

- $-\ln P(\mathcal{D}|\boldsymbol{\theta})$: the data-fit term, analogous to “energy”; as the amount of data tends to infinity, it tends to $D_{KL}(P_{\text{data}} \| P_{\boldsymbol{\theta}}) + \text{const}$;
- $-\ln P(\boldsymbol{\theta})$: the regularization term, analogous to an “external field” pulling the parameters toward the region favored by the prior (this is not, strictly speaking, an “entropy penalty” — the term “entropy penalty” refers specifically to the $-\ln Q$ term inside $D_{KL}(Q \| P)$ in variational inference, and the two should not be conflated).

This once again displays the deep connection among information theory, statistical mechanics, and machine learning.

■ Summary

This section re-derived, in the most abstract way possible, depending on no physical intuition whatsoever, the Boltzmann distribution that runs throughout this book — requiring only the single information-theoretic axiom, “among distributions satisfying the constraints, choose the one with maximum entropy.” We used the die example to see how a constraint “distorts”

a uniform distribution into an exponential one, and used the numerical reverse-engineering of the two-level system to verify, with our own eyes, that this information-theoretic path and the physical path of §2.2 converge on exactly the same destination. This also lets us look back over the core argument of the book up to this point: “the system exchanges energy with a heat bath” (§2.2), “searching for a lower bound on the partition function” (Chapter 5), “pursuing maximum unbiasedness under constraints” (this chapter) — three mutually independent starting points, all pointing to the same mathematical object, $P(\mathbf{x}) \propto e^{-E(\mathbf{x})}$; while the step of “giving a probabilistic model a loss function” (Chapter 3) is the bridge that lets this conclusion apply equally to machine learning, rather than being a fourth independent source. This is not a coincidence, but a reflection of the fact that physics, optimization, and information theory share, at bottom, the very same mathematical structure.

Exercises

Foundational

Exercise 7.1: Using the Maximum Entropy Principle to Reverse-Engineer the “Temperature” of Another Two-Level System

In §7.1’s “A Worked Example: Reverse-Engineering the Temperature of the Two-Level System,” a two-level system with energy levels $E_0 = 0, E_1 = \varepsilon = 1$, in contact with a $k_B T = 1$ heat bath, was used to compute the exact $\langle E \rangle = 0.269$, and then, working in reverse, using only the single piece of information “the constraint is $\langle E \rangle = 0.269$,” the maximum entropy principle was verified to give a λ equal to exactly $\beta = 1$. Now consider a different two-level system: energy levels $E_0 = 0, E_1 = \varepsilon = 2$, in contact with a $k_B T = 2$ heat bath.

- Using Chapter 1’s Boltzmann-distribution formula for the two-level system, directly compute the exact $\langle E \rangle$ (to three decimal places);
- Now pretend not to know the physical concept of “temperature” at all, and know only the constraint “the average energy equals the value computed in (a)”; write down the expression for the maximum-entropy distribution $P(x) \propto e^{-\lambda E(x)}$ ($x \in \{0, 1\}, E(0) = 0, E(1) = 2$);
- Substituting the constraint value from (a), numerically solve for λ ;
- Verify whether the λ you computed equals exactly $\beta = 1/(k_B T) = 1/2$, thereby confirming once again, as in the main text, that the “information-theoretic path” and the “physical path” arrive at the same destination.

Exercise 7.2: Verifying the Moment-Matching Constraint with the Simplest Possible Example

When §7.2 derived the maximum-entropy classifier, the constraint condition (7.6) was written fairly abstractly. This problem makes the meaning of this constraint completely concrete,

using a minimal setup with a single input and two classes. Suppose only a single input $\mathbf{x}$ ever occurs in the training set (i.e., the empirical distribution $\tilde{P}(\mathbf{x}) = 1$), among which the proportion with label $y = 1$ is 0.7, and the proportion with $y = 0$ is 0.3. Take the feature function $f(\mathbf{x}, y) = \mathbb{I}[y = 1]$ (i.e., it equals 1 when $y = 1$, and 0 otherwise).

- Write down what the constraint equation (7.6) looks like in this specific setting (hint: since there is only one $\mathbf{x}$, the sum over $\mathbf{x}$ can simply be dropped);
- By Equation (7.8), in the two-class case $P(y = 1|\mathbf{x}) = \frac{e^\lambda}{e^\lambda + e^0}$, that is, the sigmoid function $\sigma(\lambda)$ (recall the definition of sigmoid from Chapter 3). Substitute the constraint from (a) and solve for λ ;
- Verify: does the resulting $P(y = 1|\mathbf{x})$ equal exactly the empirical proportion of $y = 1$ in the training data, 0.7? What does this tell us — in the simplest possible case of a single input and a single indicator feature, into what a remarkably plain statement does the abstract constraint “the model’s expected features match the data’s empirical features” degenerate?

Intermediate

Exercise 7.3: Deriving the Maximum-Entropy Distribution

- Under the constraints $\sum_x P(x) = 1$ and $\sum_x P(x) x = \mu$, find the distribution that maximizes the entropy $H = -\sum_x P(x) \ln P(x)$.
Hint: Use the method of Lagrange multipliers.
- Prove that the result is an exponential distribution (in the continuous case) or a geometric distribution (in the discrete case);
- Under the constraints $\sum_x P(x) = 1$, $\sum_x P(x) x = \mu$, $\sum_x P(x) x^2 = \sigma^2 + \mu^2$, find the maximum-entropy distribution;
- Prove that the result is the Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$;
- In general, given m moment constraints, what is the form of the maximum-entropy distribution?

Exercise 7.4: A Die with a Prior: A Concrete Worked Example of Minimum Cross-Entropy

The “Think and Check” box of §7.3 discussed, qualitatively, the effect on the solution of a prior favoring the 6 face; this problem turns it into a concrete example that can be worked through to the end. Let the die’s prior distribution be

$$Q(1) = Q(2) = Q(3) = Q(4) = Q(5) = 0.15, \quad Q(6) = 0.25,$$

with the constraint still the average number of points, $\langle X \rangle = 4.5$.

- (a) By Equation (7.9), the minimum-cross-entropy solution takes the form $P(i) \propto Q(i)e^{-\lambda i}$; first compute the $\langle X \rangle_Q$ corresponding to $\lambda = 0$ (i.e., $P = Q$, no adjustment at all);
- (b) Compare the $\langle X \rangle_Q$ computed in (a) with the required constraint value 4.5, and use this to determine whether the λ needed to satisfy the constraint should be positive or negative (hint: in $P(i) \propto Q(i)e^{-\lambda i}$, the sign of λ determines whether the exponential factor pushes weight toward the larger or the smaller face values);
- (c) (Numerical computation, a calculator or a short program may help) Numerically solve for the λ satisfying the constraint, and compute the complete $P(1), \dots, P(6)$;
- (d) Compare the result of (c) with the solution in the main text's Table 7.1 for “a uniform prior, the same constraint $\langle X \rangle = 4.5$ ”: which distribution has a larger $P(6)$? Does this match the intuition that “the prior itself already favors 6”?

Challenge

Exercise 7.5: Deriving the Grand Canonical Ensemble from the Maximum Entropy Principle

§7.1 derived the Boltzmann distribution of the canonical ensemble from a single constraint, the “average energy.” There is a more general ensemble in statistical mechanics — the **grand canonical ensemble** — which allows the system to exchange both energy and particle number with the heat bath; in this case, in addition to the average-energy constraint $\langle E \rangle = \bar{E}$, there is an additional average-particle-number constraint $\langle N \rangle = \bar{N}$.

- (a) By analogy with the derivation of §7.1, write down the Lagrangian corresponding to this constrained optimization problem (there are now two Lagrange multipliers, conventionally denoted β and $-\beta\mu$, where μ is called the chemical potential);
- (b) Following the derivation of Equation (7.4), prove that the maximum-entropy distribution takes the form

$$P(\mathbf{x}, N) \propto e^{-\beta[E(\mathbf{x}, N) - \mu N]}.$$

- (c) Look up the literature: the combination $E - \mu N$ appearing in this distribution is usually called the “grand potential” or “effective energy”; in which physical contexts (such as the Fermi level in semiconductor physics, or Bose–Einstein condensation) is it widely used?
- (d) **Open-ended reflection:** does machine learning have any scenario analogous to “the system exchanging particle number with its surroundings” — for instance, models with variable output length (such as a language model generating sentences of indefinite length, or point-cloud or molecular-structure generative models with a variable number of atoms)? Could this analogy inspire a “grand-canonical-ensemble description of variable-length generative models”? Would it be meaningful?

Summary of Part I: The Mathematical Bridge Between Statistical Mechanics and Machine Learning

In this part of the book, we have built a mathematical bridge between statistical mechanics and machine learning. Below, we review the core ideas running through all six chapters.

The Core Correspondence

At the end of Chapter 1, a seed table was planted (Table 2.8): seven rows of statistical-mechanics vocabulary, with the machine-learning column left blank, marked only “where it will be revealed.” We can now check, row by row, whether that promise has been kept: “microstate,” “Boltzmann distribution,” “energy function,” “inverse temperature” were filled in in Chapter 3 (§4.1); “partition function” was defined in Chapter 3, and its computational difficulty revealed in Chapter 4; “free energy” and the first half of “entropy” were filled in in Chapter 5 (§6.3); the second half of “entropy,” together with this chapter’s new addition, the “maximum entropy principle,” are completed in the final rows of the table below. Not a single row of the seed table has been left unfilled, nor was any promise made prematurely — this is exactly why the table below can be stated with confidence, rather than the other way around.

The rightmost column of Table 7.2 marks the nature of each row’s correspondence: “**Definition**” means the two sides are simply the same mathematical operation under two different names (for example, “energy function” and “loss” are both simply two names for “the bare, undetermined quantity inside the exponent,” see the notational reminder in Chapter 3’s §4.1); “**Rigorously proved**” means the row has a complete theorem-proof in the main text, traceable to a specific chapter; “**Analogy**” means the row provides useful intuition and structural similarity, but does not constitute an equation that can be directly substituted and used (for example, “inverse temperature $\leftrightarrow$ regularization parameter”: both control “how concentrated/how spread out” a distribution is, but there is no natural, universally valid formula converting one into the other). **A special note:** the table above renders the “free energy F ” row as “variational free energy $F[Q] \leftrightarrow$ negative ELBO,” rather than adopting the simpler but more misleading “ $F \leftrightarrow -\ln Z$ ” (i.e., “negative log-evidence”) — the latter is strictly valid only when the machine-learning evidence $P(\mathbf{v})$ happens to serve as the partition function of some energy-based model, and is not accurate as a general statement; the strict correspondence that has actually been proved is the inequality, in §6.3, between the variational free energy $F[Q]$ and the negative evidence lower bound $-\text{ELBO}[Q]$ — this is the row that can be safely cited as “rigorously proved.”

Every row of Table 7.2 marked “Definition” or “Rigorously proved” has been rigorously derived, one by one, in the preceding six chapters, and can be traced to a specific source: the correspondence between energy and loss is found in Chapter 4; the computational difficulty of the partition function is found in Chapter 5; the equivalence between variational free energy and the ELBO is found in Chapter 6; the relationship between maximum entropy and the Boltzmann distribution is found in Chapter 7. The handful of rows marked “Analogy” (inverse temperature, the mean-field approximation) are honestly labeled as such, and do not masquerade as equations. This table is therefore not a “quick-reference sheet” that conflates analogy with theorem, but a compressed summary of the book’s entire argument chain, one that honestly marks which entries

Table 7.2: The Core Correspondence Between Statistical Mechanics and Machine Learning — The Complete Version of Chapter 1's Seed Table

Statistical mechanics	Machine learning	Mathematical form	Type
Microstate $\mathbf{x}$	Model configuration / latent variable	Discrete or continuous variable	Definition
Energy function $E(\mathbf{x})$ (bare quantity)	Loss $\mathcal{L}$ (bare quantity, inside the exponent)	Both appear in the exponent of $\propto e^{-(\cdot)}$	Definition
Boltzmann distribution	Probabilistic model	$P(\mathbf{x}) \propto e^{-\beta E(\mathbf{x})}$	Definition
Partition function Z	Normalization constant	$\sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})}$	Definition
Negative log (of the normalized distribution)	Complete loss / log-likelihood	$-\ln P = \beta E + \ln Z$	Definition
Inverse temperature β	Regularization parameter	Controls randomness/fluctuation	Analogy
Entropy S	Entropy / model complexity	$-\sum P \ln P$	Definition
Variational free energy $F[Q]$	Negative evidence bound $-\text{ELBO}[Q]$	$\langle E \rangle_Q - TS[Q]$	Rigorously proved (Chapter 6)
Minimizing $F[Q]$	Maximizing $\text{ELBO}[Q]$	Two statements of the same inequality $F[Q] \geq F_{\text{true}}$	Rigorously proved (Chapter 6)
Mean-field approximation	A special case of variational inference	Factorization assumption	Analogy (Chapter 6) discusses the scope of validity)
Maximum entropy principle + energy constraint	Maximum entropy model / softmax	Maximum entropy under constraints $\Rightarrow$ Boltzmann form	Rigorously proved (Chapter 7, see the axiomatic premise noted in §7.1)

are mathematical conclusions ready to be used directly, and which are merely useful intuitions — this is also what Chapter 1’s empty table has grown into, after five full chapters.

The Boundaries of the Analogy: Where This Language Stops Applying

Table 7.2 honestly marks three different strengths of correspondence — “Definition,” “Rigorously proved,” “Analogy” — but all three strengths share the common premise that *some correspondence still exists*. What needs to be said here is something further: there are several places where the statistical-mechanics language built up in this book does not even rise to the level of an “analogy,” and simply breaks down outright. Knowing where a language stops applying is just as important as knowing what the language itself says.

1. **A loss function need not be a physical energy.** Chapter 4 proved “loss = energy” on the premise that the probabilistic model is genuinely written in Boltzmann form, $P(y|\mathbf{x}, \boldsymbol{\theta}) \propto e^{-\mathcal{L}}$. But in reality, a great many loss functions are directly designed optimization targets (a proxy for some business metric, a weighted sum of several sub-task losses), and do not arise from the construction process of “first write down a probabilistic model, then take the negative log-likelihood.” In this case, the language “loss function $\leftrightarrow$ energy function” can still be used as the language of an *optimization algorithm* — Part III will discuss loss landscapes and optimization dynamics — but it no longer corresponds to a concrete physical system with a well-defined temperature and entropy.
2. **The training process is not a thermal-equilibrium process.** The update rule of stochastic gradient descent, $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_t) + (\text{noise from the mini-batch})$, superficially resembles a particle undergoing noisy descent within an energy landscape (similar to Langevin dynamics). But SGD does not satisfy detailed balance: the structure of its noise is determined by how mini-batches are sampled, and there is no relationship, of the kind found in physical systems (“temperature determines the magnitude of the noise”), between this noise structure and the specific shape of the loss landscape $\mathcal{L}(\boldsymbol{\theta})$; whether the parameters, once training has converged, strictly obey $P(\boldsymbol{\theta}) \propto e^{-\beta \mathcal{L}(\boldsymbol{\theta})}$ is likewise not a theorem this book has proved, but a question still under active research.
3. **Parameter space is not a physical configuration space.** The microstates $\mathbf{x}$ of statistical mechanics can typically be distinguished by physical measurement; the parameters $\boldsymbol{\theta}$ of a neural network, by contrast, can carry a great deal of redundancy — different $\boldsymbol{\theta}_1 \neq \boldsymbol{\theta}_2$ can give exactly the same function, $f_{\boldsymbol{\theta}_1} = f_{\boldsymbol{\theta}_2}$ (for instance, swapping the indices of two neurons within a layer, or scaling one layer’s weights up by a factor of k while scaling the next layer’s weights down by the same factor). This means that “the volume of parameter space” and “the volume of realizable functions” are two different quantities: the partition function $Z = \int e^{-\beta \mathcal{L}(\boldsymbol{\theta})} d\boldsymbol{\theta}$, computed by integrating over parameter space, will count these physically indistinguishable redundant points repeatedly — something that typically does not happen in a genuine physical system (unless the system itself possesses a symmetry, in which case one must additionally divide by the size of the symmetry group).

4. **The thermodynamic limit need not correspond to a network’s $N \rightarrow \infty$.** The reason “ $N \rightarrow \infty$ ” matters so much in physics is that a large number of independent or weakly correlated degrees of freedom cause fluctuations, relative to the mean, to tend to zero. Neural-network theory also extensively discusses the limit of infinite width (such as neural tangent kernel theory), but whether this limit possesses the same “vanishing fluctuations” property as the physical thermodynamic limit depends on the specific network architecture and initialization scheme, and the conclusions of the physical thermodynamic limit cannot simply be transplanted wholesale, taken for granted.
5. **“Temperature” may be nothing more than a mathematical parameter.** The β or “temperature” T appearing in this book, in contexts such as softmax and simulated annealing, is a freely chosen hyperparameter, adjusting only how sharply peaked a probability distribution is. It agrees, with the *genuine* temperature of a physical system — determined by the equipartition theorem and coupling to a heat bath — only in mathematical form (both are written $P \propto e^{-\beta E}$); there is no reason, and no evidence, that the two can be converted into one another numerically or physically.

None of these five points negate any conclusion of the previous six chapters — what has been proved in Chapters 4, 5, 6, and 7 remains true; it is simply that every one of those proofs comes with an explicit scope of applicability. What these five points want to say is: this is where the boundary of this language lies, and beyond that boundary, “statistical mechanics” is nothing more than a convenient metaphor, no longer a source of theorems that can be substituted into and used directly.

More Languages: What Optimization Theory and Information Theory Call the Same Object

Table 7.2 compares only two languages, statistical physics and machine learning. But the handful of mathematical objects that keep recurring throughout the book are, in fact, simultaneously standard objects in **optimization theory** and **information theory** as well, merely under different names. The table below picks out only the rows whose correspondence genuinely holds up under scrutiny — not every row translates equally cleanly into all several languages, and wherever the connection is a stretch (for example, “inverse temperature” has no clean, independent counterpart in optimization), it is better left out than forced in:

This table is not a new proof; it is simply a reminder to the reader that the story this book has told in the single language of “statistical physics” is, in optimization-theory and information-theory textbooks, very often the very same story told with an entirely different vocabulary — if, sometime in the future, the reader opens a textbook on convex optimization or information theory and recognizes one of these rows, that recognition is exactly what this table is for.

Key Insights

1. **The energy–loss equivalence:** the energy function of statistical mechanics corresponds to the loss function (negative log-likelihood) of machine learning. Minimizing energy cor-

Table 7.3: The Same Mathematical Object, Under Its Names in Optimization Theory and Information Theory

Mathematical object	Name in optimization theory	Name in information theory
Energy function $E(\mathbf{x})$	Objective function	Self-information $-\log P(\mathbf{x})$
(The logarithm of the) partition function Z	A convex function, whose derivatives generate moments	Cumulant generating function
Entropy S	An entropy-regularization term	A measure of uncertainty (Shannon entropy)
Free energy F / variational free energy	The objective to be minimized / its upper bound	$-\ln Z$ / negative ELBO
$D_{KL}(Q\ P)$	The gap minimized in variational methods	Relative entropy, a “distance” between two distributions
Maximum entropy principle	Constrained convex optimization (Lagrangian duality)	Minimizing deviation from a prior

responds to maximizing probability — Chapter 4, through the three concrete examples of linear regression, logistic regression, and softmax regression, showed how this equivalence transforms “designing a loss function” into the more fundamental problem of “choosing a noise distribution.”

2. **The universal difficulty of the partition function:** computing the normalization constant (the partition function) is a central challenge, in both physics and machine learning alike. The number of states grows exponentially with the number of variables, a difficulty that cannot be fundamentally solved simply by adding more computing power, and can only be worked around with approximation methods — exactly why Chapter 5 introduces the language of Markov random fields, to systematically organize this difficulty.
3. **The unification of the variational principle:** variational free energy (physics) and the evidence lower bound (learning) are the very same mathematical object. Minimizing free energy is equivalent to maximizing the ELBO, and the two share exactly the same proof, based on the non-negativity of the KL divergence — Chapter 6 gave this proof, and, through the mean-field approximation, showed how variational methods are concretely applied in practice (along with their limitations).
4. **The foundational status of maximum entropy:** the Boltzmann distribution is not an assumption, but is derived from the maximum entropy principle. Chapter 7 showed that

three independent derivation paths — physical intuition (the system exchanging energy with a heat bath), the variational principle (Chapter 6), and information-theoretic constraints (this chapter) — ultimately converge on the very same mathematical object; the loss-function perspective (Chapter 4) then explains why this conclusion can cover any machine-learning model without modification, rather than being a fourth independent source. This provides a shared information-theoretic foundation for statistical mechanics and machine learning.

5. **The multiple roles of temperature:** the parameter “temperature” recurs, again and again, across different contexts, yet always plays the very same mathematical role — controlling the balance between “certainty” and “flexibility”:
 - Physics: controls the intensity of thermal fluctuations;
 - Learning: controls the randomness of a model’s output and the strength of regularization (Chapter 4’s softmax temperature parameter);
 - Optimization: controls the balance between exploration and exploitation.
6. **The energy–entropy trade-off:** a system searches for a balance between minimizing energy (exact fitting) and maximizing entropy (retaining flexibility), a trade-off uniformly captured by the free energy $F = E - TS$ (Chapter 2). In physics, the direction of this trade-off is given by the second law of thermodynamics; in machine learning, this bears a structural resemblance to the bias–variance trade-off — a similar “fitting accuracy $\leftrightarrow$ complexity” structure — but this is only a structural analogy, not a mathematical equivalence: the bias–variance decomposition is an independent, exact decomposition of the expected prediction error, in which the “variance” term is not the same mathematical object as the entropy S here, and this book has not proved that a strict correspondence exists between the two (this is also one of the points the previous subsection, “The Boundaries of the Analogy,” wanted to stress).

Why Do These Connections Matter?

1. **Transferring concepts:** mature tools from physics (phase-transition theory, the renormalization group, response functions, and so on) can be applied to phenomena in machine learning that are not yet well understood.
2. **Algorithmic inspiration:** many modern machine-learning algorithms (Boltzmann machines, variational autoencoders, simulated annealing) come directly from statistical mechanics; understanding their physical origins helps in understanding why the algorithms work, and when they fail.
3. **Deeper understanding:** the physical point of view reveals the underlying structure, limitations, and possibilities behind machine-learning methods, rather than treating them merely as “empirically effective tricks.”

4. **New directions:** the intersection of physics and machine learning is giving rise to new research directions, including phase transitions and critical phenomena in neural networks, the statistical-mechanics theory of deep learning, quantum machine learning, and neural scaling laws.

Further Reflection

The following questions have no standard answer, and are left for the reader to continue thinking about while working through the material that follows:

1. **The boundaries of the analogy:** the “Boundaries of the Analogy” subsection following “The Core Correspondence” listed five places already known to break down (a loss need not be an energy, training is not thermal equilibrium, parameter space has redundancy, the thermodynamic limit need not correspond, temperature may be a purely mathematical parameter). Beyond these five, what other machine-learning phenomena do you think the language of physics currently cannot describe, or can describe only awkwardly?
2. **Causation vs. correlation:** statistical mechanics usually deals with equilibrium states, but machine learning involves learning dynamics (the training process is itself a time evolution). How should this time evolution be understood within a physical framework?
3. **The quantum perspective:** classical statistical mechanics has already established a deep connection with machine learning. What insight might quantum statistical mechanics offer for quantum machine learning?
4. **Practical applications:** how do these theoretical insights translate into practical algorithmic improvements? On which specific problems do physics-inspired methods hold an advantage?

Suggested Reading

Introductory Texts

- **Mehta et al. (2019).** *A High-Bias, Low-Variance Introduction to Machine Learning for Physicists*. arXiv:1803.08823. — A machine-learning tutorial written specifically for physicists, containing extensive statistical-mechanics analogies.
- **MacKay (2003).** *Information Theory, Inference, and Learning Algorithms*. — A classic textbook on information theory and machine learning, with a physical point of view running throughout.
- **Murphy (2012).** *Machine Learning: A Probabilistic Perspective*. — An authoritative textbook on probabilistic machine learning.

Further Reading

- **Carleo et al. (2019).** “Machine Learning and the Physical Sciences.” *Reviews of Modern Physics*. — A review of the intersection of physics and machine learning.
- **Engel & Van den Broeck (2001).** *Statistical Mechanics of Learning*. — Statistical-mechanics methods for learning theory.
- **Opper & Saad (2001).** *Advanced Mean Field Methods: Theory and Practice*. — Applications of mean-field theory in machine learning.

Classic Papers

- **Jaynes (1957).** “Information Theory and Statistical Mechanics.” *Physical Review*. — The founding paper of the maximum entropy principle, see Chapter 7.
- **Landauer (1961).** “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal*. — The physical cost of information processing, see Exercise 2.3.
- **Hinton & Sejnowski (1986).** “Learning and Relearning in Boltzmann Machines.” — The original Boltzmann-machine paper.
- **Amari (1998).** “Natural Gradient Works Efficiently in Learning.” *Neural Computation*. — Natural gradients and information geometry, see Exercise 3.5.
- **Tishby et al. (2000).** “The Information Bottleneck Method.” arXiv:physics/0004057. — The information bottleneck theory, see Exercise 6.4.

Online Resources

- **Stanford CS229:** *Machine Learning* (Andrew Ng) — A classic machine-learning course.
- **Princeton COS 597F:** *Advanced Topics in Machine Learning: Physics for Machine Learning* — A course at the intersection of physics and machine learning.
- **Blog:** *The Science of Deep Learning* (a blog series by Francis Bach and others).

A Look Ahead to the Next Part

In Part II, we will build on these basic concepts to study concrete neural-network models and their physical counterparts:

- **Hopfield networks:** how can the Ising model help us understand associative memory?
- **Boltzmann machines:** how can statistical sampling be used for learning?
- **Restricted Boltzmann Machines (RBMs):** an early success of deep learning.

- **The statistical-mechanics theory of deep learning:** why are deep networks able to generalize?

We will see how the mathematical framework built in Part I applies to understanding real neural networks, and how physical intuition can guide algorithm design.

Appendix A

Quick Reference of Common Notation and Formulas

A.1 Notation for Probability Theory and Mathematical Tools

This section collects two kinds of basic notation used throughout the book, independent of the specific subject of any one chapter — notation from probability theory, and mathematical-tool notation for differentiation, logarithms, and the like. The three sections that follow (notation for graphs and Markov random fields; variational free energy and the KL divergence; the method of Lagrange multipliers) instead correspond, respectively, to the specialized content of Chapters 4, 5, and 6, and are of a different character from this section.

Table A.1: Basic Notation for Probability Theory

Concept	Formula	See
Normalization (discrete)	$P(x) \geq 0, \sum_x P(x) = 1$	§2.1
Normalization (continuous)	$P(x) \geq 0, \int P(x) dx = 1$	Chapters 2 and 3
Joint and marginal probability	$P(x) = \sum_y P(x, y)$	Chapter 2’s “Preliminary Definitions”
Conditional probability	$P(x y) = \frac{P(x, y)}{P(y)} \quad (P(y) > 0)$	§3.2
Independence	$X \perp Y \Leftrightarrow P(x, y) = P(x)P(y)$	§3.2
Conditional independence	$X \perp Y \mid Z \Leftrightarrow P(x, y z) = P(x z)P(y z)$	Chapter 4 (separation on a graph)
Bayes’ theorem	$P(x y) = \frac{P(y x)P(x)}{P(y)}$	Equation (3.10), Chapter 2
Expectation value	$\langle A \rangle = \sum_x P(x)A(x)$	Chapter 1’s “Preliminary Definitions”
Variance	$\text{Var}(A) = \langle A^2 \rangle - \langle A \rangle^2$	Chapter 1’s “Preliminary Definitions”
i.i.d. likelihood	$P(\mathcal{D} \boldsymbol{\theta}) = \prod_n P(y^{(n)} \mathbf{x}^{(n)}, \boldsymbol{\theta})$	§3.2

Table A.2: Mathematical Identities Used Repeatedly Throughout the Book

Tool	Formula	See
Logarithm identities	$\ln(ab) = \ln a + \ln b, \ln(a/b) = \ln a - \ln b$	Used throughout the book
Taylor expansion (single-variable)	$g(a + \delta) = g(a) + \delta g'(a) + \frac{1}{2} \delta^2 g''(a) + \dots$	§2.2
$\ln Z$ generates moments	$\langle E \rangle = -\frac{\partial \ln Z}{\partial \beta}, \text{Var}(E) = \frac{\partial^2 \ln Z}{\partial \beta^2}$	§2.2
Gradient notation	$\nabla_{\boldsymbol{\theta}} L = \left(\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_d} \right)$	Chapters 2 and 3
Hessian notation	$H_{ij} = \frac{\partial^2 L}{\partial \theta_i \partial \theta_j}$	Chapter 2 exercises (Fisher information)
Vector differentiation	$\frac{\partial(\boldsymbol{\theta}^T \mathbf{x})}{\partial \boldsymbol{\theta}} = \mathbf{x}$	§4.2, “Preliminary Rule”

A.2 Notation for Graphs and Markov Random Fields

Table A.3: Basic Notation for Graph Theory and MRFs

Concept	Formula/Definition	See
Graph	$G = (V, E)$, with node set V and edge set E	§5.2
Neighborhood	$N(i) = \{j \in V : (i, j) \in E\}$	§5.2
Clique	$c \subseteq V$, with every pair of nodes in c adjacent	Definition 5.1
Local Markov property	$X_i \perp X_{V \setminus (\{i\} \cup N(i))} \mid X_{N(i)}$	§5.2
Global Markov property	S separates $A, B \Rightarrow X_A \perp X_B \mid X_S$	§5.2
Gibbs factorization	$P(\mathbf{x}) = \frac{1}{Z} \prod_{c \in C} \psi_c(\mathbf{x}_c) = \frac{1}{Z} e^{-E(\mathbf{x})}$	Theorem 5.1 (Hammersley–Clifford)

A usage note: the equivalence in the direction “local $\Rightarrow$ global” does *not* hold unconditionally — it requires the additional condition that the joint distribution be everywhere positive ($P(\mathbf{x}) > 0$); see the discussion of the Moussouris (1974) counterexample in §5.2. Appendix C supplements this with Markov *chains* (the Markov property arranged along time, rather than along a graph) and their convergence theory — a related but distinct concept from the Markov *fields* of this section; please do not conflate the two.

A.3 Key Formulas for Variational Free Energy and the KL Divergence

Every conclusion in the book relating to “variational” methods ultimately comes down to the single inequality below, together with its equality condition:

$$D_{KL}(Q\|P) = \frac{1}{T}F[Q] + \ln Z \geq 0 \implies F[Q] \geq F_{\text{true}} = -T \ln Z,$$

with equality if and only if $Q = P$. Here $D_{KL} \geq 0$ (the Gibbs inequality) is itself derived from Jensen’s inequality; the complete proof is given in the course of proving Theorem 6.1. The corresponding version in the machine-learning context (the ELBO) is:

$$\ln P(\mathbf{v}) = \mathcal{L}[Q] + D_{KL}(Q(\mathbf{h})\|P(\mathbf{h}|\mathbf{v})) \implies \mathcal{L}[Q] \leq \ln P(\mathbf{v}),$$

see Theorem 6.2 for details. Table A.4 summarizes the correspondence between the two languages.

Table A.4: Notational Correspondence Between Variational Free Energy and the ELBO

Statistical physics (Chapter 5, §6.1)	Machine learning (Chapter 5, §6.2)
$F[Q] \geq F_{\text{true}}$	$\mathcal{L}[Q] \leq \ln P(\mathbf{v})$
Minimizing $F[Q]$	Maximizing $\mathcal{L}[Q]$
$Q(\mathbf{x}) \rightarrow P(\mathbf{x})$	$Q(\mathbf{h}) \rightarrow P(\mathbf{h} \mathbf{v})$

A.4 The Method of Lagrange Multipliers

Both Chapter 5 (the mean-field equations) and Chapter 6 (the maximum entropy principle) require finding the extremum of an entropy functional of the form $-\sum_x P(x) \ln P(x)$, subject to a normalization constraint (and sometimes also a moment constraint). Since the state space in this book is always discrete, $P(x)$ is, in essence, simply a finite collection of independent real numbers, and so “taking a variation with respect to $P(\cdot)$ ” is therefore simply the ordinary method of Lagrange multipliers from multivariable calculus, not the calculus of variations, which would require additional tools — see the complete methodological explanation in Chapter 5’s §6.4, “preliminary note.” The general format:

$$\Lambda = \left[-\sum_x P(x) \ln P(x) \right] + \lambda_0 \left(1 - \sum_x P(x) \right) + \sum_k \lambda_k \left(\bar{f}_k - \sum_x P(x) f_k(x) \right),$$

Taking the partial derivative with respect to each $P(x')$ (treated as an independent real variable) and setting it to zero, the resulting extremal distribution must take the exponential-family form $P(x) \propto \exp(-\sum_k \lambda_k f_k(x))$ — see Chapter 6’s §7.1 for the specific derivation.

Appendix B

Common Probability Distributions and Statistics Table

This appendix collects the probability distributions used throughout the book, giving only their definitions, means, and variances, without repeating the derivations — the negative-log-likelihood loss corresponding to each distribution has already been derived, from first principles, in its corresponding section of the main text; this appendix serves only as a lookup reference.

B.1 Common Probability Distributions

Table B.1: Summary of Discrete Distributions

Distribution	Probability Mass Function	Mean / Variance	Appears in
Bernoulli	$P(x) = p^x(1-p)^{1-x}$, $x \in \{0, 1\}$	$\langle x \rangle = p$ $\text{Var}(x) = p(1-p)$	§3.3 , §4.3
Binomial	$P(k) = \mathcal{C}_N^k p^k (1-p)^{N-k}$	$\langle k \rangle = Np$ $\text{Var}(k) = Np(1-p)$	§3.3 (tossing a coin N times)
Categorical	$P(y = k) = \pi_k$, $\sum_k \pi_k = 1$	—	§4.4 (softmax)
Poisson	$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$, $k = 0, 1, 2, \dots$	$\langle k \rangle = \lambda$ $\text{Var}(k) = \lambda$	Chapter 3 exercises (Poisson regression)

■ Why Does Neither Table Include a “Partition Function” Column?

The careful reader may already have noticed: the normalization constant of every distribution in these two tables ($\frac{1}{\sqrt{2\pi\sigma^2}}$, $\frac{1}{2b}$, $\binom{N}{k}$, and so on) plays exactly the role of the reciprocal of the “partition function” Z that Chapter 1 emphasizes again and again — it is simply that, in a probability-theory textbook, this role is not usually given its own name. The Bernoulli distribution is the most direct machine-learning version of Chapter 1’s “two-level system”; the “energy” $E(x) = -\ln p(x)$ of continuous distributions such as the Gaussian and the

Table B.2: Summary of Continuous Distributions

Distribution	Probability Density Function	Mean / Variance	Appears in
Gaussian	$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$	$\langle x \rangle = \mu$ $\text{Var}(x) = \sigma^2$	§4.2 (the noise model)
Laplace	$p(x) = \frac{1}{2b} \exp\left(-\frac{ x-\mu }{b}\right)$	$\langle x \rangle = \mu$ $\text{Var}(x) = 2b^2$	Chapter 3 exercises (L_1 loss)

Laplace is exactly the starting point of the loss-function derivations in Chapter 3. Reading these two tables side by side with Chapter 1’s seed table (Table 2.8) makes it easier to see that the correspondence “distribution $\leftrightarrow$ energy” holds in every single row — discrete or continuous alike.

B.2 General Definitions

Table B.3: General Definitions of Expectation, Variance, and Covariance

Quantity	Definition	See
Expectation	$\langle X \rangle = \sum_x P(x) x$ (or $\int p(x) x dx$)	Chapter 1’s “Preliminary Definitions”
Variance	$\text{Var}(X) = \langle X^2 \rangle - \langle X \rangle^2$	Chapter 1’s “Preliminary Definitions”
Covariance	$\text{Cov}(X, Y) = \langle XY \rangle - \langle X \rangle \langle Y \rangle$	Not separately defined in the main text; usable at any time

■ The Exponential Family: The Hidden Structure Running Through This Table

Every distribution in Table B.1 and Table B.2 can be written in the unified **exponential family** form $P(y|\eta) = h(y) \exp(\eta y - A(\eta))$ — this is exactly what Chapter 3’s challenge exercise, “Generalized Linear Models and the Exponential Family,” asks the reader to verify independently. If you need the formula for some specific distribution while working that exercise, look it up directly in this appendix, rather than flipping back through the main text of each chapter.

Appendix C

Markov Chains and MCMC Basics

Chapter 5’s §5.2 borrowed the definition of a Markov *chain* (arranged along time) to introduce the Markov *field* (arranged along a graph); these are two related but distinct concepts, and what this appendix deals with is the former — not “locality on a graph,” but the question of “where repeated random wandering eventually ends up.” The MCMC and Gibbs sampling mentioned in Chapter 5’s §5.4, together with the exercise “Gibbs Sampling and Image Segmentation” in that chapter, all rely on the results developed here.

C.1 The Stationary Distribution

Consider a process that jumps randomly among a finite set of states $\{x\}$: at time t , being in state x , it jumps to x' with probability $T(x \rightarrow x')$ (T is called the **transition probability**, satisfying $\sum_{x'} T(x \rightarrow x') = 1$). If the state distribution at time t is $P_t(x)$, the distribution at time $t + 1$ is

$$P_{t+1}(x') = \sum_x P_t(x) T(x \rightarrow x'). \quad (\text{C.1})$$

A distribution $\pi(x)$ is called a **stationary distribution** of this chain if, once substituted into Equation (C.1), it no longer changes:

$$\pi(x') = \sum_x \pi(x) T(x \rightarrow x'). \quad (\text{C.2})$$

The core idea of MCMC methods is to use this relation **in reverse**: rather than starting from a transition rule and then asking “which distribution does it settle into,” one instead first specifies the target distribution one wants to sample from (for example, the Boltzmann distribution $P(x) = e^{-\beta E(x)}/Z$), and then *designs* a set of transition rules T such that this target distribution is exactly its stationary distribution. As long as the design is correct, starting from any initial state and repeatedly jumping according to T , after enough jumps the frequency with which each state appears converges to the target distribution — and none of this requires first computing the intractable Z , because Z cancels automatically in the construction of T (this will be seen shortly).

C.2 Detailed Balance

Directly verifying Equation (C.2) is usually cumbersome. A more tractable and more commonly used **sufficient condition** is **detailed balance**:

$$\pi(x) T(x \rightarrow x') = \pi(x') T(x' \rightarrow x), \quad \forall x, x'. \quad (\text{C.3})$$

Intuitively, this says that “the probability flow from x to x' ” exactly equals “the probability flow back from x' to x ” — everywhere in balance, with no net flow anywhere.

■ Verification: Detailed Balance Implies Stationarity

Summing both sides of Equation (C.3) over x :

$$\sum_x \pi(x) T(x \rightarrow x') = \sum_x \pi(x') T(x' \rightarrow x) = \pi(x') \sum_x T(x' \rightarrow x) = \pi(x'),$$

where the last step uses the normalization of the transition probability, $\sum_x T(x' \rightarrow x) = 1$. This is exactly Equation (C.2), i.e., π is a stationary distribution. ■

Note that this is only a *sufficient* condition, not a necessary one — there exist Markov chains with a stationary distribution that does not satisfy detailed balance; but for a chain that does satisfy detailed balance, its stationarity can always be verified directly by algebra, exactly as above, which is precisely why it is so widely used in practice.

C.3 The Metropolis Algorithm

Given a target distribution $\pi(x) = e^{-\beta E(x)}/Z$, the Metropolis algorithm constructs the transition probability as follows: starting from the current state x ,

- (1) propose a candidate state x' using a **symmetric** proposal distribution $g(x \rightarrow x') = g(x' \rightarrow x)$ (for example, “propose flipping any one spin, with equal probability”);
- (2) accept this candidate with **acceptance probability**

$$A(x \rightarrow x') = \min \left(1, \frac{\pi(x')}{\pi(x)} \right) = \min(1, e^{-\beta[E(x') - E(x)]}) \quad (\text{C.4})$$

(otherwise, remain in the original state x).

The key point of Equation (C.4) is this: $\pi(x')/\pi(x) = e^{-\beta E(x')}/e^{-\beta E(x)}$, and the partition function Z cancels in the ratio — this is exactly the fundamental reason MCMC methods can sidestep the difficulty, emphasized repeatedly in Chapter 4, that “ Z is intractable to compute”: **sampling never requires knowing Z ; it requires only knowing the energy difference.**

■ Verification: The Metropolis Transition Satisfies Detailed Balance

Suppose $\pi(x') \geq \pi(x)$ (the other case is entirely analogous). In this case $T(x \rightarrow x') = g(x \rightarrow x') \cdot 1$ (the ratio in Equation (C.4) is ≥ 1 , so the acceptance probability is 1), while

$T(x' \rightarrow x) = g(x' \rightarrow x) \cdot \frac{\pi(x)}{\pi(x')}$. Substituting into the two sides of the detailed-balance condition:

$$\begin{aligned}\pi(x)T(x \rightarrow x') &= \pi(x)g(x \rightarrow x'), \\ \pi(x')T(x' \rightarrow x) &= \pi(x')g(x' \rightarrow x)\frac{\pi(x)}{\pi(x')} = \pi(x)g(x' \rightarrow x).\end{aligned}$$

By the symmetry of g ($g(x \rightarrow x') = g(x' \rightarrow x)$), the two expressions are equal. Detailed balance holds automatically for *any* energy function $E(x)$ — this is exactly the cleverness of the Metropolis algorithm: the acceptance probability is designed *specifically* to make Equation (C.3) an identity. ■

Returning to Chapter 2's Two-Level System

Let us work the Metropolis algorithm through from beginning to end, using the two-level system of Chapter 2's Table 2.4, with $E_0 = 0, E_1 = \varepsilon = 1, k_B T = 1$ (i.e., $\beta = 1$). The exact solution is known to be $\pi(0) = P_0 = 0.731$, $\pi(1) = P_1 = 0.269$. The system has only two states, and the proposal distribution is trivial (“propose jumping to the other state,” $g = 1$, automatically symmetric).

Proposing a jump from state 0 to state 1: $\Delta E = E_1 - E_0 = 1$, acceptance probability $A(0 \rightarrow 1) = \min(1, e^{-1}) = e^{-1} \approx 0.368$.

Proposing a jump from state 1 to state 0: $\Delta E = E_0 - E_1 = -1$, acceptance probability $A(1 \rightarrow 0) = \min(1, e^1) = 1$.

The transition probabilities are therefore $T(0 \rightarrow 1) = 0.368$, $T(0 \rightarrow 0) = 0.632$, $T(1 \rightarrow 0) = 1$, $T(1 \rightarrow 1) = 0$. **Verifying detailed balance:**

$$\pi(0)T(0 \rightarrow 1) = 0.731 \times 0.368 = 0.269, \quad \pi(1)T(1 \rightarrow 0) = 0.269 \times 1 = 0.269,$$

the two are equal, so detailed balance holds. **Verifying stationarity** (substituting directly into Equation (C.2) to check):

$$\begin{aligned}\pi(0)T(0 \rightarrow 0) + \pi(1)T(1 \rightarrow 0) &= 0.731 \times 0.632 + 0.269 \times 1 = 0.462 + 0.269 = 0.731 = \pi(0), \\ \pi(0)T(0 \rightarrow 1) + \pi(1)T(1 \rightarrow 1) &= 0.731 \times 0.368 + 0.269 \times 0 = 0.269 = \pi(1),\end{aligned}$$

matching exactly. In other words: even with no prior knowledge whatsoever that “the exact solution of the two-level system is 0.731, 0.269,” simply repeatedly random-walking between the two states according to the Metropolis rule — staying, most of the time, in the lower-energy state 0, occasionally jumping to the higher-energy state 1 with probability 0.368, and then, once there, almost certainly (probability 1) jumping straight back — the *fraction of time* spent in each state will, over the long run, automatically converge to 0.731 : 0.269, in exact agreement with the exact solution computed in Chapter 2 from the partition function. This is exactly the simplest example — and the one most directly verifiable by hand — of MCMC's core idea: “replace an exact sum with sampling.”

C.4 Gibbs Sampling

The Gibbs sampling that Chapter 5’s exercises ask the reader to implement is a special case of the Metropolis algorithm in the setting of a “multivariable joint distribution.” Given a joint distribution $P(\mathbf{x}) = P(x_1, \dots, x_N)$, Gibbs sampling updates only one variable x_i at each step, **replacing it directly with a new value drawn from the conditional distribution** $P(x_i \mid \mathbf{x}_{-i})$ ($\mathbf{x}_{-i}$ denotes all variables other than x_i , held fixed during this step).

It can be shown that this rule is exactly the Metropolis algorithm with proposal distribution $g(\mathbf{x} \rightarrow \mathbf{x}') = P(x'_i \mid \mathbf{x}_{-i})$ (proposing a change only in the i -th coordinate, with the proposal itself drawn from the conditional distribution), whose acceptance probability is **identically** 1:

$$A(\mathbf{x} \rightarrow \mathbf{x}') = \min \left(1, \frac{\pi(\mathbf{x}') g(\mathbf{x}' \rightarrow \mathbf{x})}{\pi(\mathbf{x}) g(\mathbf{x} \rightarrow \mathbf{x}')} \right) = \min \left(1, \frac{P(x'_i \mid \mathbf{x}_{-i}) P(x_i \mid \mathbf{x}_{-i})}{P(x_i \mid \mathbf{x}_{-i}) P(x'_i \mid \mathbf{x}_{-i})} \right) = 1,$$

where we have used $\pi(\mathbf{x}) = P(\mathbf{x}_{-i})P(x_i \mid \mathbf{x}_{-i})$ ($\mathbf{x}_{-i}$ is unchanged in this step, so $P(\mathbf{x}_{-i})$ cancels on both sides). “Always accept” means that Gibbs sampling requires no accept/reject comparison at all — it requires only the ability to sample from each variable’s conditional distribution — and this is exactly the theoretical basis for the concrete procedure, in Chapter 5’s exercises, of “updating pixel by pixel, sampling according to the conditional probability.”

C.5 Ergodicity

Detailed balance guarantees that the target distribution is *a* stationary distribution, but it does not guarantee that “starting from any initial point and wandering repeatedly, the chain is certain to converge to it” — in principle there could be multiple stationary distributions, or the chain could be “trapped” in one part of the state space, never reaching another part. The condition that rules out these pathological situations is called **ergodicity**, and roughly requires:

- **Irreducibility**: starting from any state, there exists a path with nonzero probability to reach any other state;
- **Aperiodicity**: there is no “metronome-like” restriction under which the chain can only possibly return to some state after a fixed period.

As long as the transition rule satisfies these two conditions (Metropolis and Gibbs sampling generally satisfy them automatically, provided the proposal distribution can potentially reach “anywhere”), the ergodic theorem for Markov chains guarantees: regardless of the initial state, the distribution $P_t(x)$ at time $t \rightarrow \infty$ converges to the unique stationary distribution $\pi(x)$, and the *time average* of the states visited along the way likewise converges to $\pi(x)$. The proof of this theorem lies beyond the scope of this appendix (it belongs to the standard content of Markov-chain theory within probability theory); here we only state the conclusion, and verify its result directly in the two-level-system example above.

■ Summary: How This Answers Chapter 4's Difficulty

Chapter 5's central difficulty is that computing the partition function Z grows exponentially with the number of variables, making exhaustive enumeration impossible. This appendix has shown how MCMC methods sidestep this difficulty — the acceptance probabilities and conditional distributions of Metropolis and Gibbs sampling depend only on *differences* in energy or on *local* conditional distributions, and the global sum $Z = \sum_{\mathbf{x}} e^{-\beta E(\mathbf{x})}$ itself never once appears. The price paid is that one obtains only an approximation, converging over time through repeated sampling, rather than an exact solution reached in a single step — this is the same trade-off, realized in a different way, as Chapter 6's variational methods, which likewise “trade approximation for tractability.”

Bibliography

- [1] M. Mézard, A. Montanari, *Information, Physics, and Computation*, Oxford University Press, 2009.
- [2] E. Ising, “Beitrag zur Theorie des Ferromagnetismus,” *Zeitschrift für Physik*, 31(1):253–258, 1925.
- [3] L. Onsager, “Crystal Statistics. I. A Two-Dimensional Model with an Order-Disorder Transition,” *Physical Review*, 65(3–4):117–149, 1944.
- [4] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, E. Teller, “Equation of State Calculations by Fast Computing Machines,” *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [5] W. K. Hastings, “Monte Carlo Sampling Methods Using Markov Chains and Their Applications,” *Biometrika*, 57(1):97–109, 1970.
- [6] C. H. Bennett, “The Thermodynamics of Computation—a Review,” *International Journal of Theoretical Physics*, 21(12):905–940, 1982.
- [7] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” *IBM Journal of Research and Development*, 5(3):183–191, 1961.
- [8] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, 27(3):379–423, 27(4):623–656, 1948.
- [9] S. Kullback, R. A. Leibler, “On Information and Sufficiency,” *Annals of Mathematical Statistics*, 22(1):79–86, 1951.
- [10] E. T. Jaynes, “Information Theory and Statistical Mechanics,” *Physical Review*, 106(4):620–630, 1957.
- [11] A. L. Berger, S. A. Della Pietra, V. J. Della Pietra, “A Maximum Entropy Approach to Natural Language Processing,” *Computational Linguistics*, 22(1):39–71, 1996.
- [12] N. Tishby, F. C. Pereira, W. Bialek, “The Information Bottleneck Method,” arXiv:physics/0004057, 2000.
- [13] J. M. Hammersley, P. Clifford, “Markov Fields on Finite Graphs and Lattices,” unpublished manuscript, 1971.

- [14] J. Besag, "Spatial Interaction and the Statistical Analysis of Lattice Systems," *Journal of the Royal Statistical Society, Series B*, 36(2):192–236, 1974.
- [15] S. Geman, D. Geman, "Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6):721–741, 1984.
- [16] R. Kindermann, J. L. Snell, *Markov Random Fields and Their Applications*. American Mathematical Society, 1980.
- [17] R. A. Fisher, "On the Mathematical Foundations of Theoretical Statistics," *Philosophical Transactions of the Royal Society A*, 222:309–368, 1922.
- [18] J. A. Nelder, R. W. M. Wedderburn, "Generalized Linear Models," *Journal of the Royal Statistical Society, Series A*, 135(3):370–384, 1972.
- [19] A. P. Dempster, N. M. Laird, D. B. Rubin, "Maximum Likelihood from Incomplete Data via the EM Algorithm," *Journal of the Royal Statistical Society, Series B*, 39(1):1–38, 1977.
- [20] S. Amari, "Natural Gradient Works Efficiently in Learning," *Neural Computation*, 10(2):251–276, 1998.
- [21] J. J. Hopfield, "Neural Networks and Physical Systems with Emergent Collective Computational Abilities," *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982.
- [22] D. H. Ackley, G. E. Hinton, T. J. Sejnowski, "A Learning Algorithm for Boltzmann Machines," *Cognitive Science*, 9(1):147–169, 1985.
- [23] P. Smolensky, "Information Processing in Dynamical Systems: Foundations of Harmony Theory," in D. E. Rumelhart, J. L. McClelland (eds.), *Parallel Distributed Processing*, Vol. 1, Chapter 6, pp. 194–281. MIT Press, 1986.
- [24] G. E. Hinton, T. J. Sejnowski, "Learning and Relearning in Boltzmann Machines," in D. E. Rumelhart, J. L. McClelland (eds.), *Parallel Distributed Processing*, Vol. 1, Chapter 7, pp. 282–317. MIT Press, 1986.
- [25] D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, 323:533–536, 1986.
- [26] R. Salakhutdinov, G. Hinton, "Deep Boltzmann Machines," *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 448–455, 2009.
- [27] M. I. Jordan, Z. Ghahramani, T. S. Jaakkola, L. K. Saul, "An Introduction to Variational Methods for Graphical Models," *Machine Learning*, 37(2):183–233, 1999.
- [28] R. M. Neal, G. E. Hinton, "A View of the EM Algorithm that Justifies Incremental, Sparse, and Other Variants," in M. I. Jordan (ed.), *Learning in Graphical Models*, pp. 355–368. Kluwer Academic Publishers, 1998.

- [29] D. M. Blei, A. Kucukelbir, J. D. McAuliffe, “Variational Inference: A Review for Statisticians,” *Journal of the American Statistical Association*, 112(518):859–877, 2017.
- [30] D. P. Kingma, M. Welling, “Auto-Encoding Variational Bayes,” *International Conference on Learning Representations (ICLR)*, 2014.
- [31] P. Mehta, M. Bukov, C.-H. Wang, A. G. R. Day, C. Richardson, C. K. Fisher, D. J. Schwab, “A High-Bias, Low-Variance Introduction to Machine Learning for Physicists,” *Physics Reports*, 810:1–124, 2019.
- [32] G. Carleo, I. Cirac, K. Cranmer, L. Daudet, M. Schuld, N. Tishby, L. Vogt-Maranto, L. Zdeborová, “Machine Learning and the Physical Sciences,” *Reviews of Modern Physics*, 91:045002, 2019.
- [33] Y. Bahri, J. Kadmon, J. Pennington, S. S. Schoenholz, J. Sohl-Dickstein, S. Ganguli, “Statistical Mechanics of Deep Learning,” *Annual Review of Condensed Matter Physics*, 11:501–528, 2020.
- [34] Y. LeCun, Y. Bengio, G. Hinton, “Deep Learning,” *Nature*, 521:436–444, 2015.
- [35] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [36] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [37] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [38] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*. MIT Press, 2016.
- [39] A. Engel, C. Van den Broeck, *Statistical Mechanics of Learning*. Cambridge University Press, 2001.
- [40] M. Oppen, D. Saad (eds.), *Advanced Mean Field Methods: Theory and Practice*. MIT Press, 2001.